

VIA Platform - Complete Project Documentation & Logic

This document contains the complete source code, architecture, and logic used throughout the VIA platform.

File: `backend\agents\agent\_executor.py`

```py

# backend/agents/agent\_executor.py ? Phase 3: All 9 agents

import asyncio, time

from backend.agents.backend\_agent import backend\_agent

from backend.agents.security\_agent import security\_agent

from backend.agents.devops\_agent import devops\_agent

from backend.agents.ai\_research\_agent import ai\_research\_agent

from backend.agents.architecture\_agent import architecture\_agent

from backend.agents.frontend\_agent import frontend\_agent

from backend.agents.hr\_agent import hr\_agent

from backend.agents.finance\_agent import finance\_agent

from backend.agents.marketing\_agent import marketing\_agent

from backend.agents.presentation\_agent import presentation\_agent

from backend.core.inter\_agent\_bus import InterAgentBus, extract\_summary

from backend.core.logger import logger

AGENT\_REGISTRY = {

    "backend": backend\_agent,

    "security": security\_agent,

    "devops": devops\_agent,

    "ai\_research": ai\_research\_agent,

    "architecture": architecture\_agent,

    "frontend": frontend\_agent,

    "hr": hr\_agent,

    "finance": finance\_agent,

    "marketing": marketing\_agent,

    "presentation": presentation\_agent,

}

```

EXECUTION_ORDER = [
 "architecture", "backend", "security", "devops", "ai_research",
 "frontend", "hr", "finance", "marketing", "presentation"
]

```

```

async def execute_agents(
 agent_names: list,
 task: str,
 ceo_strategy: str = "",
 project_brief: dict = None, # ? NEW
 job_id: str = None,
 ws_manager=None
) -> dict:
 bus = InterAgentBus()
 results = {}
 brief = project_brief or {} # ? NEW

 ordered = sorted(
 agent_names,
 key=lambda n: EXECUTION_ORDER.index(n) if n in EXECUTION_ORDER else 99
)

 for name in ordered:
 if name not in AGENT_REGISTRY:
 logger.warning(f"Agent not found: {name}")
 results[name] = {
 "status": "not_found",
 "error": f"'{name}' not in registry.",
 "execution_time_seconds": 0.0,
 "confidence": 0.0,
 "output": None
 }

```

```
continue
```

```
if ws_manager and job_id:
```

```
 await ws_manager.send_agent_start(job_id, name)
```

```
inter_context = bus.get_context_for(name)
```

```
if inter_context and ws_manager and job_id:
```

```
 deps = [d for d in bus.DEPENDENCIES.get(name, []) if d in bus.get_all()]
```

```
 await ws_manager.send_inter_agent(job_id, str(deps), [name], inter_context[:100])
```

```
? passes project_brief to every agent
```

```
name, result_data = await _run_agent(
```

```
 name,
```

```
 AGENT_REGISTRY[name],
```

```
 task,
```

```
 ceo_strategy,
```

```
 brief, # ? NEW
```

```
 inter_context
```

```
)
```

```
results[name] = result_data
```

```
if result_data["status"] == "success" and result_data.get("output"):
```

```
 summary = extract_summary(result_data["output"], name)
```

```
 bus.deposit(name, summary)
```

```
if ws_manager and job_id:
```

```
 await ws_manager.send_agent_done(
```

```
 job_id, name,
```

```
 result_data["status"],
```

```
 result_data["execution_time_seconds"],
```

```
 result_data["confidence"]
```

```
)
```

```
await asyncio.sleep(7) # 7s gap: safe for Groq (30 RPM) + Gemini (10 RPM)
```

```
return results
```

```
async def _run_agent(
 name: str,
 agent_func,
 task: str,
 ceo_strategy: str,
 project_brief: dict, # ? NEW
 inter_context: str
) -> tuple:

 start = time.time()

 try:
 result = await agent_func(
 task=task,
 project_brief=project_brief, # ? NEW
 ceo_strategy=ceo_strategy,
 inter_context=inter_context
)
 duration = round(time.time() - start, 2)
 confidence = _confidence(result)
 logger.info(f"Agent OK | {name} | {duration}s | conf={confidence}")
 return name, {
 "status": "success",
 "execution_time_seconds": duration,
 "confidence": confidence,
 "output": result
 }
 except Exception as e:
 duration = round(time.time() - start, 2)
 logger.error(f"Agent FAIL | {name} | {duration}s | {e}")
 return name, {
 "status": "failed",
```

```

 "execution_time_seconds": duration,

 "confidence": 0.0,

 "error": str(e),

 "output": None

 }

```

```

def _confidence(result: dict) -> float:

```

```

 if not result or not isinstance(result, dict): return 0.0

 total = sum(len(v) if isinstance(v, (dict, list)) else 1 for v in result.values())

 if total >= 15: return 0.97

 if total >= 10: return 0.92

 if total >= 6: return 0.85

 if total >= 3: return 0.72

 return 0.55

```

```

...

```

```

File: `backend\agents\ai_research_agent.py`

```

```

```py

```

```

# backend/agents/ai_research_agent.py ? Phase 2: reads architecture + backend context

```

```

import json, re

```

```

from backend.core.llm_provider import llm

```

```

from backend.core.logger import logger

```

```

def _extract(text):

```

```

    m = re.search(r"\{.*\}", text, re.DOTALL)

```

```

    if m:

```

```

        try: return json.loads(m.group())

```

```

        except: pass

```

```

    return None

```

```

async def ai_research_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:

```

```

    context_block = f"\nContext from other departments:\n{inter_context}\n" if inter_context else

```

"""

```
prompt = (
    "You are a Chief AI Research Scientist at an MNC-level tech company.\n\n"
    "CEO Strategic Direction: " + ceo_strategy + "\n"
    + context_block +
    "Analyze the business idea and return ONLY valid JSON:\n"
    '        {"department": "AI Research", "model_strategy": {"primary_model": "...", "fallback_model": "...", "deployment": "..."}, '
    '        {"prompt_engineering": {"approach": "...", "optimization": "...", "guardrails": "..."}, '
    '        {"fine_tuning": {"recommended": true, "reasoning": "...", "data_requirements": "..."}, '
    '        {"future_roadmap": [...], "performance_targets": {"latency": "...", "accuracy": "...", "cost": "...}}\n'
    '\n'
    "Business idea: " + task + "\n"
)

logger.info("AI Research Agent | Generating plan...")
raw = await llm.agenerate(prompt)
parsed = _extract(raw)
if parsed:
    parsed["department"] = "AI Research"
    return parsed
return {
    "department": "AI Research",
    "model_strategy": {"primary_model": "llama3 via Ollama.", "fallback_model": "GPT-4o.",
    "deployment": "Local dev, cloud prod."},
    "prompt_engineering": {"approach": "Chain-of-thought + zero-shot.", "optimization": "30% token reduction.", "guardrails": "JSON schema validation."},
    "fine_tuning": {"recommended": False, "reasoning": "Base model sufficient initially.", "data_requirements": "10k+ labeled examples."},
    "future_roadmap": ["Multi-agent reasoning.", "RAG integration.", "A/B model testing."],
    "performance_targets": {"latency": "Sub-3s p95.", "accuracy": ">95% JSON parse.", "cost": "<$0.01/request."}
}
```

...

File: `backend\agents\architecture\_agent.py`

```

```py
backend/agents/architecture_agent.py ? Phase 2: reads backend context

import json, re

from backend.core.llm_provider import llm

from backend.core.logger import logger

def _extract(text):

 m = re.search(r"\{.*\}", text, re.DOTALL)

 if m:

 try: return json.loads(m.group())

 except: pass

 return None

async def architecture_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:

 context_block = f"\nContext from other departments:\n{inter_context}\n" if inter_context else
""

 prompt = (

 "You are a Principal Systems Architect at an MNC-level tech company.\n\n"

 "CEO Strategic Direction: " + ceo_strategy + "\n"

 + context_block +

 "Analyze the business idea and return ONLY valid JSON:\n"

 '{"department":"System
Architecture","design_pattern":{"primary":"...","rationale":"..."},"
'"system_components":["..."],"data_flow":{"ingestion":"...","processing":"...","storage":"...","eg
ress":"..."},"
'"service_boundaries":["..."],"resilience":{"fault_tolerance":"...","disaster_recovery":"...","cir
cuit_breaker":"..."},"
 '"scalability_path":"..."}\n\n'

 "Business idea: " + task + "\n"

)

 logger.info("Architecture Agent | Generating plan...")

 raw = await llm.agenerate(prompt)

 parsed = _extract(raw)

 if parsed:

 parsed["department"] = "System Architecture"

```

```

 return parsed

 return {

 "department": "System Architecture",

 "design_pattern": {"primary": "Layered Monolith with modular boundaries.", "rationale":
"Fastest to production. Enables future microservice extraction."},

 "system_components": ["API Gateway: routing + auth.", "Business Logic: stateless
services.", "Data Layer: repository pattern."],

 "data_flow": {"ingestion": "Client ? HTTPS ? Gateway ? Validation.", "processing":
"Controller ? Service ? Domain.", "storage": "Repository ? ORM ? PostgreSQL + Redis.", "egress":
"Pydantic JSON ? Client."},

 "service_boundaries": ["Auth boundary: isolated AuthService.", "Domain boundary: no direct
DB access."],

 "resilience": {"fault_tolerance": "Graceful degradation, no cascading failures.",
"disaster_recovery": "RPO 1hr, RTO 30min.", "circuit_breaker": "Exponential backoff, max 3
retries."},

 "scalability_path": "MVP monolith ? extract services ? full microservice mesh."

 }

```

```

...

```

```

File: `backend\agents\backend_agent.py`

```

```

```py

```

```

# backend/agents/backend_agent.py ? Phase 2: reads inter-agent context

```

```

import json, re

```

```

from backend.core.llm_provider import llm

```

```

from backend.core.logger import logger

```

```

def _extract(text):

```

```

    m = re.search(r"\{.*\}", text, re.DOTALL)

```

```

    if m:

```

```

        try: return json.loads(m.group())

```

```

        except: pass

```

```

    return None

```

```

async def backend_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,

```



```

inter_context: str = "") -> dict:

    context_block = f"\nContext from other departments:\n{inter_context}\n" if inter_context else ""

    prompt = (

        "You are a Principal Backend Engineer at an MNC-level tech company.\n\n"

        "CEO Strategic Direction: " + ceo_strategy + "\n"

        + context_block +

        "Analyze the business idea and return ONLY valid JSON:\n"

                                                    '{"department":"Backend
Engineering","architecture":"...","database":{"primary":"...","caching":"...","schema_approach":".
.."},"'

        '"api_design":{"style":"...","key_endpoints":["..."],"auth_integration":"..."},"'

        '"services":["..."],"key_recommendations":["..."]}\n\n'

        "Business idea: " + task + "\n"

    )

    logger.info("Backend Agent | Generating plan...")

    raw = await llm.agenerate(prompt)

    parsed = _extract(raw)

    if parsed:

        parsed["department"] = "Backend Engineering"

        return parsed

    return {

        "department": "Backend Engineering",

        "architecture": "FastAPI with async modular service separation.",

        "database": {"primary": "PostgreSQL - ACID compliant.", "caching": "Redis for hot-path.",
"schema_approach": "Normalized relational schema."},

        "api_design": {"style": "REST /api/v1/", "key_endpoints": ["POST /api/v1/users", "GET
/api/v1/items/{id}"], "auth_integration": "JWT middleware."},

        "services": ["UserService", "CoreDomainService", "NotificationService"],

        "key_recommendations": ["Async/await throughout.", "Pydantic validation.", "asyncpg
connection pooling."]

    }

...

## File: `backend\agents\ceo_agent.py`

```py

```

```

backend/agents/ceo_agent.py

import json

import re

from backend.core.llm_provider import llm

from backend.core.config import VALID_DEPARTMENTS, MEMORY_INJECTION_COUNT

from backend.core.logger import logger

def extract_json(text):
 match = re.search(r"\{.*\}", text, re.DOTALL)

 if match:
 try:
 return json.loads(match.group())

 except json.JSONDecodeError:
 pass

 return None

def build_memory(history):
 if not history:
 return "No prior company history."

 lines = []

 for i, r in enumerate(history[:MEMORY_INJECTION_COUNT], 1):
 task = r.get("task", "")
 res = r.get("result", {})

 if isinstance(res, dict):
 strat = res.get("ceo_strategy", {})
 short = strat.get("short_term_strategy", "") if isinstance(strat, dict) else ""
 depts = res.get("selected_departments", [])

 lines.append(f"Run {i}: Task='{task[:80]}' | Strategy='{short[:60]}' | Depts={depts}")

 else:
 lines.append(f"Run {i}: Task='{task[:80]}'")

 return "\n".join(lines)

```

```

async def ceo_agent(task: str, history: list = None) -> dict:

 # history is passed in from main.py which fetches it async before calling this
 memory = build_memory(history or [])

 prompt = (

 "You are the CEO of an AI-powered autonomous MNC-level digital company.\n\n"

 "COMPANY MEMORY:\n" + memory + "\n\n"

 "DEPARTMENTS AVAILABLE:\n"

 "--- TECH DIVISION ---\n"

 "- backend: APIs, databases, services, server-side logic\n"

 "- frontend: React/Vite UI, responsive design, user experience\n"

 "- security: Auth, encryption, threats, compliance, pen testing\n"

 "- devops: Infrastructure, CI/CD, Docker, Kubernetes, cloud scaling\n"

 "- ai_research: LLM strategy, ML models, AI integration, optimization\n"

 "- architecture: System design, microservices, data flow, resilience\n"

 "--- BUSINESS DIVISION ---\n"

 "- hr: Job descriptions, team structure, onboarding, org planning\n"

 "- finance: Cost estimates, ROI analysis, budget breakdown, pricing\n"

 "- marketing: Landing page copy, go-to-market, brand, user acquisition\n\n"

 "Rules:\n"

 "- Select ONLY departments GENUINELY needed for this specific task\n"

 "- A tech product ALWAYS needs at least: backend + frontend\n"

 "- Add security, devops for production apps\n"

 "- Add hr/finance/marketing for business/startup ideas\n"

 "- Return ONLY valid JSON, no extra text:\n"

 "{\n"

 " \"short_term_strategy\": \"0-3 month concrete execution plan\",\n"

 " \"long_term_vision\": \"6-24 month product direction\",\n"

 " \"departments\": [\"dept1\", \"dept2\", ...]\n"

 "}\n\n"

 "Task / Business Idea:\n" + task + "\n"

)

 logger.info(f"CEO Agent | Task: {task[:80]}...")

 raw = await llm.agenerate(prompt)

```

```

parsed = extract_json(raw)

if parsed:

 selected = [d for d in parsed.get("departments", []) if d in VALID_DEPARTMENTS]

 if not selected:

 selected = ["backend"]

 logger.info(f"CEO decision | Departments: {selected}")

 return {

 "short_term_strategy": parsed.get("short_term_strategy", "Begin core development."),
 "long_term_vision": parsed.get("long_term_vision", "Build a scalable product."),
 "departments": selected,
 "_raw_llm_response": raw,
 "_extracted_json": parsed

 }

return {

 "short_term_strategy": "Execute core development immediately.",
 "long_term_vision": "Build a scalable, secure, full-stack product.",
 "departments": ["backend", "frontend"],
 "_raw_llm_response": raw,
 "_extracted_json": {}

}

...

```

```

File: `backend\agents\devops_agent.py`

```

```

```py

```

```

# backend/agents/devops_agent.py ? Phase 2+3: reads backend + architecture context

```

```

import json, re

```

```

from backend.core.llm_provider import llm

```

```

from backend.core.logger import logger

```

```

def _extract(text):

```

```

    m = re.search(r"\{.*\}", text, re.DOTALL)

```

```

    if m:

```

```

try: return json.loads(m.group())

except: pass

return None

```

```

async def devops_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:

    context_block = f"\nContext from other departments:\n{inter_context}\n" if inter_context else
    ""

    prompt = (

        "You are a Principal DevOps Architect at an MNC-level tech company.\n\n"

        "CEO Strategic Direction: " + ceo_strategy + "\n"

        + context_block +

        "Analyze the business idea and return ONLY valid JSON:\n"

        '        {"department": "DevOps & Infrastructure", "infrastructure": {"cloud_provider": "...", "architecture": "...", "containerization": "..."}, '
        '        "ci_cd": {"pipeline_tool": "...", "stages": [...], "deployment_strategy": "..."}, '
        '        "scaling": {"strategy": "...", "auto_scaling": "...", "load_balancing": "..."}, '
        '        "monitoring": {"tools": [...], "alerting": "...", "logging": "...}}\n\n'

        "Business idea: " + task + "\n"

    )

    logger.info("DevOps Agent | Generating plan...")

    raw = await llm.agenerate(prompt)

    parsed = _extract(raw)

    if parsed:

        parsed["department"] = "DevOps & Infrastructure"

        return parsed

    return {

        "department": "DevOps & Infrastructure",

        "infrastructure": {"cloud_provider": "AWS - enterprise grade.", "architecture": "Multi-AZ private subnets.", "containerization": "Docker + ECS/Kubernetes."},

        "ci_cd": {"pipeline_tool": "GitHub Actions.", "stages": ["Lint & Test", "Build Image", "Deploy Staging", "Deploy Production"], "deployment_strategy": "Blue-green zero downtime."},

        "scaling": {"strategy": "Horizontal stateless scaling.", "auto_scaling": "Scale at 70% CPU.", "load_balancing": "AWS ALB health checks."},

        "monitoring": {"tools": ["Prometheus + Grafana", "CloudWatch"], "alerting": "PagerDuty on error rate >1%.", "logging": "ELK Stack 30-day retention."}
    }

```

```

    }
    ...

## File: `backend\agents\finance_agent.py`

```py
backend/agents/finance_agent.py ? VIA Phase 3: Finance Department

import re
import time
import logging
from backend.core.llm_provider import llm

logger = logging.getLogger("AI-Digital-Company")

async def finance_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:

 start = time.time()

 logger.info(f"Finance Agent | Task: {task[:60]}")

 context_block = ""

 if ceo_strategy:
 context_block += f"\nCEO Strategic Direction: {ceo_strategy}\n"

 if inter_context:
 context_block += f"\nContext from other departments:\n{inter_context}\n"

 prompt = f"""You are the Chief Financial Officer (CFO) at a tech MNC called VIA.
{context_block}
A new project has been initiated: {task}

Produce a comprehensive financial plan covering:

1. PROJECT BUDGET BREAKDOWN: Itemized costs (infrastructure, dev tools, licenses, APIs)
2. ROI ANALYSIS: Expected return on investment with 6/12/24 month projections

```

3. COST OPTIMIZATION: Where to cut costs without sacrificing quality
4. REVENUE MODEL: How this project can generate revenue (SaaS, freemium, ads, etc.)
5. BURN RATE: Monthly operational costs estimate
6. FUNDING REQUIREMENTS: How much capital needed and for what milestones
7. FINANCIAL RISKS: Top 5 financial risks and mitigation strategies
8. BREAK-EVEN ANALYSIS: When the project becomes profitable
9. PRICING STRATEGY: Recommended pricing tiers with justification
10. FINANCIAL KPIs: Key financial metrics to track

Use realistic market numbers. Be specific with dollar amounts.

Format as a professional CFO financial report.

"""

```
try:

 output = await llm.agenerate(prompt)
 duration = round(time.time() - start, 2)
 logger.info(f"Finance Agent done | {duration}s")
 return {
 "department": "Finance",
 "status": "success",
 "execution_time_seconds": duration,
 "confidence": 0.90,
 "output": {
 "department": "Finance",
 "full_report": output or "",
 "summary": f"Financial plan for '{task[:80]}' ? covering budget, ROI, pricing, and
risk.",
 "highlights": _extract_highlights(output or ""),
 },
 }

except Exception as e:

 duration = round(time.time() - start, 2)
 logger.error(f"Finance Agent failed | {e}")
 return {
```

```

 "department": "Finance",

 "status": "failed",

 "execution_time_seconds": duration,

 "confidence": 0.0,

 "error": str(e),

 "output": {},

 }

```

```

def _extract_highlights(text: str) -> dict:

 highlights = {}

 amounts = re.findall(r'\$[\d,]+(?:\.\d+)?(?:k|M|m|M)?', text)

 if amounts:

 highlights["mentioned_amounts"] = amounts[:10]

 pcts = re.findall(r'\d+(?:\.\d+)?%', text)

 if pcts:

 highlights["percentages"] = pcts[:8]

 periods = re.findall(r'\d+\s*(?:month|year|week|day)s?', text, re.IGNORECASE)

 if periods:

 highlights["time_periods"] = periods[:6]

 return highlights

'''

```

## File: `backend\agents\frontend\_agent.py`

```

'''py

backend/agents/frontend_agent.py ? VIA Phase 6

#

FIXES vs previous version:

1. _deploy_workflow() ? peaceiris/actions-gh-pages@v4 (not deploy-pages)

2. permissions: contents: write

3. _vite_config() ? correct base path per repo slug

4. _api_js() ? smart BASE_URL via VITE_API_URL env var

5. _is_valid_js() ? rejects hardcoded Render URLs

```



```
6. [NEW] _package_json() ? includes react-toastify, lucide-react, react-icons,
react-hot-toast, date-fns, react-hook-form, clsx so LLM imports never 404
7. [NEW] _build_prompt() ? APPROVED PACKAGES list prevents LLM from importing
anything outside what's installed
8. [NEW] _is_valid_jsx() ? rejects LLM output that imports unapproved packages,
falling back to our safe generated App.jsx
9. [FIX] _app_jsx() ? 5 bugs fixed
10. [FIX] _is_valid_js() ? minimum length raised from 50 to 200 chars
11. [FIX] _index_css() ? replaced @apply directives with plain CSS properties
12. [FIX] _is_valid_js() ? rejects api.js with duplicate BASE_URL declaration
13. [FIX v7] _build_prompt() ? task-specific UI instructions, not generic CRUD
14. [FIX v7] _is_valid_jsx() ? threshold lowered 200?800, rejects generic CRUD
15. [FIX v7] _app_jsx() ? 8 app-type detections (image, recipe, grade, map,
calculator, chart, kanban, todo) with matching UI; generic fallback improved
#
PERMANENT FIXES (v8):
16. [FIX v8] _slug() ? removed 50-char truncation; GitHub supports up to 100 chars.
Truncation caused vite base path to mismatch the actual repo name ? blank page / 404.
The slug here MUST match whatever slug the GitHub-repo-creation agent uses.
17. [FIX v8] _app_jsx() ? domain-specific checks (hospital, game, expense, todo, weather)
now evaluated BEFORE the generic "chart" branch. Previously "expense dashboard" or
"weather dashboard" always rendered the analytics/chart UI because feat["chart"] was
True (matched "dashboard") and was checked first. Priority order is now:
image ? recipe ? grade ? calculator ? kanban ? weather ? hospital ? game ?
expense ? todo ? chart ? generic-fallback
18. [FIX v8] _build_prompt() ? same priority reorder as _app_jsx()
19. [FIX v8] _features() ? added "weather" detection keyword set
20. [FIX v8] _theme() ? added weather theme entry
21. [FIX v8] _app_jsx() ? added full weather UI branch (city search, current
conditions card, 5-day forecast strip, uses /api/v1/weather endpoint)
```

```
import re
```

```
import time
```

```
import logging
```

```

from backend.core.llm_provider import llm

from backend.core.code_writer import extract_code_blocks, save_project_files

logger = logging.getLogger("AI-Digital-Company")

APPROVED_PACKAGES = {

 "react", "react-dom", "react-router-dom", "axios",

 "react-toastify", "react-hot-toast", "lucide-react",

 "date-fns", "react-hook-form", "clsx",

}

async def frontend_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:

 start = time.time()

 logger.info(f"Frontend Agent | Task: {task[:60]}")

 try:

 prompt = _build_prompt(task, ceo_strategy, inter_context)

 llm_output = await llm.agenerate(prompt)

 files = extract_code_blocks(llm_output) if llm_output else {}

 files = _build_all_files(task, files, llm_output or "")

 save_result = save_project_files(task, "frontend", files)

 duration = round(time.time() - start, 2)

 logger.info(f"Frontend Agent done | {save_result['file_count']} files | {duration}s")

 return {

 "department": "Frontend Engineering",

 "status": "success",

 "execution_time_seconds": duration,

 "confidence": 0.91,

 "output": {

 "department": "Frontend Engineering",

 "files_generated": save_result["files_written"],

 "file_count": save_result["file_count"],

```

```

 "project_path": save_result["project_path"],
 "department_path": save_result["department_path"],
 "framework": "React 18 + Vite + Tailwind CSS",
 "deploy_target": "GitHub Pages via GitHub Actions",
 },
}

```

```
except Exception as e:
```

```

 duration = round(time.time() - start, 2)
 logger.error(f"Frontend Agent failed | {str(e)}")
 return {
 "department": "Frontend Engineering",
 "status": "failed",
 "execution_time_seconds": duration,
 "confidence": 0.0,
 "error": str(e),
 "output": {},
 }

```

```
def _title(task: str) -> str:
```

```

 t = re.sub(r"^[^w\s]", " ", task)
 return " ".join(w.capitalize() for w in t.split()[:8])

```

```

FIX v8: _slug() ? removed 50-char truncation.
#
WHY: The vite base path is /<slug>/ and must exactly match the GitHub repo
name that the repo-creation agent uses. If this slug is truncated and the
repo name is not (or vice-versa), every asset request 404s and the page
appears blank. GitHub repo names support up to 100 characters so we cap
there instead. Any change to this function MUST be mirrored in the agent
that creates the GitHub repository.

```

```

```

```
def _slug(task: str) -> str:
```

```
 text = task.lower().strip()
```

```
 text = re.sub(r"^[^w\s-]", "", text)
```

```
 text = re.sub(r"[\s_-]+", "-", text)
```

```
 # GitHub max repo name length is 100 chars; do NOT truncate shorter than that.
```

```
 return text[:100].strip("-") or "via-app"
```

```
def _features(task: str) -> dict:
```

```
 t = task.lower()
```

```
 return {
```

```
 "hospital": any(w in t for w in ["hospital", "appointment", "doctor", "patient",
"medical"]),
```

```
 "game": any(w in t for w in ["game", "gaming", "player", "score", "leaderboard"]),
```

```
 "expense": any(w in t for w in ["expense", "budget", "finance", "spending", "money"]),
```

```
 "todo": any(w in t for w in ["todo", "task", "checklist"]),
```

```
 "employee": any(w in t for w in ["employee", "staff", "hr"]),
```

```
 "inventory": any(w in t for w in ["inventory", "stock", "product", "warehouse"]),
```

```
 "blog": any(w in t for w in ["blog", "article", "post", "cms"]),
```

```
 "booking": any(w in t for w in ["booking", "reservation", "event"]),
```

```
 "auth": any(w in t for w in ["auth", "login", "user", "register"]),
```

```
 # v7 detections
```

```
 "image": any(w in t for w in ["image", "photo", "upload", "grayscale", "picture",
"crop", "filter"]),
```

```
 "recipe": any(w in t for w in ["recipe", "cook", "ingredient", "meal", "food",
"dish"]),
```

```
 # FIX v8: removed standalone "score" (conflicts with game), "tip" (substring of
```

```
 # "multiplayer"), "column" (substring of "columnist"). Use longer anchored phrases.
```

```
 "grade": any(w in t for w in ["grade", "student", "mark", "gpa", "academic", "exam",
"result", "subject", "marks"]),
```

```
 "calculator": any(w in t for w in ["calculator", "calculate", "math", "compute",
"converter", "bmi", "tip calc"]),
```

```
 "chart": any(w in t for w in ["chart", "graph", "analytics", "dashboard", "visuali",
"report", "statistic"]),
```

```
 # FIX v8: "board" was a substring of "dashboard" and "leaderboard", causing false
positives.
```

```
 # Now uses word-boundary regex so "kanban board" matches but "dashboard" does NOT.
```

```

Removed standalone "column" (substring of "columns", "columnist" etc).

"kanban": any(w in t for w in ["kanban", "sprint", "project management"]) or
 bool(re.search(r'\bboard\b', t)),

 "map": any(w in t for w in ["map", "location", "gps", "geograph", "place",
"address"])),

 # FIX v8: added weather detection ? prevents weather apps from rendering as generic
chart/dashboard

 "weather": any(w in t for w in ["weather", "forecast", "temperature", "climate",
"rain", "humidity", "wind", "storm"]),

}

```

```

def _theme(task: str) -> dict:

```

```

 t = task.lower()

 if any(w in t for w in ["hospital", "medical", "doctor", "patient"]):

 return {"primary": "#0ea5e9", "bg": "#f0f9ff", "icon": "?", "score_cls": "text-sky-600"}

 if any(w in t for w in ["game", "gaming"]):

 return {"primary": "#8b5cf6", "bg": "#faf5ff", "icon": "?", "score_cls":
"text-purple-600"}

 if any(w in t for w in ["expense", "budget", "finance", "money"]):

 return {"primary": "#10b981", "bg": "#f0fdf4", "icon": "?", "score_cls":
"text-emerald-600"}

 if any(w in t for w in ["employee", "staff", "hr"]):

 return {"primary": "#f59e0b", "bg": "#fffbeb", "icon": "?", "score_cls": "text-amber-600"}

 if any(w in t for w in ["todo", "task", "checklist"]):

 return {"primary": "#3b82f6", "bg": "#eff6ff", "icon": "?", "score_cls": "text-blue-600"}

 if any(w in t for w in ["blog", "article", "post"]):

 return {"primary": "#ec4899", "bg": "#fdf2f8", "icon": "?", "score_cls": "text-pink-600"}

 if any(w in t for w in ["booking", "reservation", "event"]):

 return {"primary": "#14b8a6", "bg": "#f0fdfa", "icon": "?", "score_cls": "text-teal-600"}

 if any(w in t for w in ["inventory", "stock", "product"]):

 return {"primary": "#f97316", "bg": "#fff7ed", "icon": "?", "score_cls":
"text-orange-600"}

 if any(w in t for w in ["image", "photo", "upload", "grayscale"]):

 return {"primary": "#6366f1", "bg": "#eef2ff", "icon": "??", "score_cls":
"text-indigo-600"}

 if any(w in t for w in ["recipe", "cook", "food", "meal"]):

 return {"primary": "#f59e0b", "bg": "#fffbeb", "icon": "?", "score_cls": "text-amber-600"}

```

```

 if any(w in t for w in ["grade", "student", "academic", "exam"]):
 return {"primary": "#3b82f6", "bg": "#eff6ff", "icon": "?", "score_cls": "text-blue-600"}

 if any(w in t for w in ["calculator", "compute", "bmi", "converter"]):
 return {"primary": "#8b5cf6", "bg": "#faf5ff", "icon": "?", "score_cls":
"text-purple-600"}

 if any(w in t for w in ["chart", "analytics", "dashboard", "report"]):
 return {"primary": "#10b981", "bg": "#f0fdf4", "icon": "?", "score_cls":
"text-emerald-600"}

 if any(w in t for w in ["kanban", "board", "sprint"]):
 return {"primary": "#f97316", "bg": "#fff7ed", "icon": "?", "score_cls":
"text-orange-600"}

FIX v8: weather theme
 if any(w in t for w in ["weather", "forecast", "temperature", "climate"]):
 return {"primary": "#0ea5e9", "bg": "#f0f9ff", "icon": "?", "score_cls": "text-sky-600"}
 return {"primary": "#6366f1", "bg": "#eef2ff", "icon": "?", "score_cls": "text-indigo-600"}

_is_valid_jsx ? lowered threshold + reject generic fallback output (v7, unchanged)

def _is_valid_jsx(code: str) -> bool:
 if not code or len(code.strip()) < 800:
 return False

 bad_imports = [
 "from './Home'", "from './Test'", "from './Result'",
 "from './Pages'", "from './components/", "from './views/",
 "from './screens/", "from './pages/", "from '../components/",
 "import Home from", "import Test from", "import Result from",
 "import Quiz from", "import Question from", "import Score from",
]

 if any(b in code for b in bad_imports):
 return False

 if ("localhost:8000" in code or "127.0.0.1:8000" in code) and "import.meta.env" not in code:
 return False

 for match in re.finditer(r'from\s+["\']([^\\".\/][^\\"']*)[\']', code):

```

```

pkg = match.group(1).split("/")[0]
if pkg.startswith("@"):
 full_scope = "/" .join(match.group(1).split("/")[1:])
 if full_scope not in APPROVED_PACKAGES:
 logger.warning(f"LLM App.jsx rejected ? unapproved import: {match.group(1)}")
 return False
elif pkg not in APPROVED_PACKAGES:
 logger.warning(f"LLM App.jsx rejected ? unapproved import: {pkg}")
 return False

return "return" in code and "useState" in code

```

```

def _is_valid_js(code: str) -> bool:
 if not code or len(code.strip()) < 200:
 return False

 if "localhost:8000" in code and "import.meta.env" not in code:
 return False

 if "onrender.com" in code:
 return False

 if code.count("BASE_URL") > 1:
 return False

 # FIX: Reject api.js that uses axios/BASE_URL without importing/declaring them.
 # The LLM sometimes emits a stub like `export const api = axios.create({baseUrl: BASE_URL})`
 # without the actual import or env-var declaration ? causing ReferenceError at runtime.
 if "axios" in code and "import axios" not in code:
 return False

 if "import.meta.env" not in code:
 return False

 # Must have at least one named export function (getItems, createItem, etc.)
 if not re.search(r'export\s+(?:const|function|async\s+function)\s+\w+', code):
 return False

 return True

```

```

def _build_all_files(task: str, llm_files: dict, raw: str) -> dict:
 f = dict(llm_files)
 name = _title(task)
 repo = _slug(task)
 feat = _features(task)
 th = _theme(task)

 f["package.json"] = _package_json()
 f["vite.config.js"] = _vite_config(repo)
 f["index.html"] = _index_html(name)
 f["tailwind.config.js"] = _tailwind_config()
 f["postcss.config.js"] = _postcss_config()
 f["public/404.html"] = _404_html()
 f[".github/workflows/deploy.yml"] = _deploy_workflow()
 f["src/main.jsx"] = _main_jsx()
 f["src/index.css"] = _index_css(th)
 f["src/api.js"] = _api_js(feat)

 if "src/App.jsx" not in f or len(f.get("src/App.jsx", "").strip()) < 10:
 raise ValueError("LLM failed to generate valid src/App.jsx. Please ensure the prompt instructs the LLM to output the main App component.")

 if "BrowserRouter" in f.get("src/main.jsx", ""):
 f["src/main.jsx"] = f["src/main.jsx"].replace("BrowserRouter", "HashRouter")

 if raw.strip():
 f["llm_raw_output.md"] = f"# LLM Output\n\n{raw}"

 return f

def _deploy_workflow() -> str:
 return """name: Deploy React to GitHub Pages

```



on:

push:

branches: [main]

workflow\_dispatch:

permissions:

contents: write

jobs:

build-and-deploy:

runs-on: ubuntu-latest

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: '20'

- name: Install dependencies

run: npm install

- name: Build

run: npm run build

env:

CI: "false"

VITE\_API\_URL: \${vars.VITE\_API\_URL }

- name: Deploy to GitHub Pages

uses: peaceiris/actions-gh-pages@v4

with:

github\_token: \${secrets.GITHUB\_TOKEN }

```
publish_dir: ./dist
```

```
force_orphan: true
```

```
"""
```

```
def _package_json() -> str:
```

```
 return """{
```

```
 "name": "via-frontend",
```

```
 "private": true,
```

```
 "version": "1.0.0",
```

```
 "type": "module",
```

```
 "scripts": {
```

```
 "dev": "vite",
```

```
 "build": "vite build",
```

```
 "preview": "vite preview"
```

```
 },
```

```
 "dependencies": {
```

```
 "react": "^18.2.0",
```

```
 "react-dom": "^18.2.0",
```

```
 "react-router-dom": "^6.20.0",
```

```
 "axios": "^1.6.0",
```

```
 "react-toastify": "^10.0.5",
```

```
 "react-hot-toast": "^2.4.1",
```

```
 "lucide-react": "^0.383.0",
```

```
 "date-fns": "^3.6.0",
```

```
 "react-hook-form": "^7.51.0",
```

```
 "clsx": "^2.1.0"
```

```
 },
```

```
 "devDependencies": {
```

```
 "@vitejs/plugin-react": "^4.2.0",
```

```
 "autoprefixer": "^10.4.16",
```

```
 "postcss": "^8.4.32",
```

```
 "tailwindcss": "^3.4.0",
```

```
 "vite": "^5.0.0"
```

```
}
}
"""
```

```
def _vite_config(repo: str = "") -> str:
 # Use relative base path to ensure assets load correctly on GitHub Pages
 # regardless of dynamic UUIDs appended to the repository name.
 return f"""import {{ defineConfig }} from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({{
 plugins: [react()],
 base: "./",
 build: {{ outDir: "dist", assetsDir: "assets" }},
 server: {{ port: 3000 }},
}});
"""
```

```
def _index_html(name: str) -> str:
 return f"""<!DOCTYPE html>

<html lang="en">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>{name}</title>
 <link rel="preconnect" href="https://fonts.googleapis.com" />
 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
 <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Plus+Jakarta+
Sans:wght@600;700;800&display=swap" rel="stylesheet" />
 </head>
 <body>
 <div id="root"></div>
```

```

 <script type="module" src="/src/main.jsx"></script>

 </body>

</html>

"""

```

```
def _404_html() -> str:
```

```
 return """<!DOCTYPE html>
```

```
<html>
```

```
 <head>
```

```
 <meta charset="utf-8" />
```

```
 <title>Redirecting...</title>
```

```
 <script>
```

```
 var l = window.location;
```

```
 l.replace(
```

```
 l.protocol + "://" + l.hostname + (l.port ? ":" + l.port : "") +
```

```
 l.pathname.split("/").slice(0, 1).join("/") +
```

```
 "?/" + l.pathname.slice(1).replace(/&/g, "~and~") +
```

```
 (l.search ? "&" + l.search.slice(1).replace(/&/g, "~and~") : "") + l.hash
```

```
);
```

```
 </script>
```

```
 </head>
```

```
 <body>Redirecting...</body>
```

```
</html>
```

```
"""
```

```
def _tailwind_config() -> str:
```

```
 return """/** @type {import('tailwindcss').Config} */
```

```
export default {
```

```
 content: ["../index.html", "./src/**/*.{js,ts,jsx,tsx}"],
```

```
 theme: {
```

```
 extend: {
```

```
 fontFamily: {
```

```

 sans: ["Inter", "sans-serif"],

 display: ["Plus Jakarta Sans", "sans-serif"],

 },

},

},

plugins: [],

};

"""

```

```

def _postcss_config() -> str:

 return """export default {

 plugins: { tailwindcss: {}, autoprefixer: {} },

};

"""

```

```

def _main_jsx() -> str:

 return """import React from "react";

import ReactDOM from "react-dom/client";

import { HashRouter } from "react-router-dom";

import App from "./App.jsx";

import "./index.css";

```

```

ReactDOM.createRoot(document.getElementById("root")).render(

 <React.StrictMode>

 <HashRouter>

 <App />

 </HashRouter>

 </React.StrictMode>

);

"""

```

```

def _index_css(th: dict) -> str:
 p = th["primary"]
 bg = th["bg"]
 return f""@tailwind base;

@tailwind components;
@tailwind utilities;

@layer base {{
 body {{
 font-family: "Inter", sans-serif;
 background-color: {bg};
 color: #111827;
 -webkit-font-smoothing: antialiased;
 }}
 h1, h2, h3 {{ font-family: "Plus Jakarta Sans", sans-serif; }}
}}

@layer components {{
 .btn-primary {{
 background-color: {p};
 color: #ffffff;
 font-weight: 600;
 padding: 0.625rem 1.25rem;
 border-radius: 0.75rem;
 transition: all 0.2s;
 display: inline-block;
 cursor: pointer;
 border: none;
 box-shadow: 0 1px 2px rgba(0,0,0,0.05);
 }}
 .btn-primary:hover {{ opacity: 0.9; }}
 .btn-primary:active {{ transform: scale(0.97); }}
 .btn-primary:disabled {{ opacity: 0.5; cursor: not-allowed; }}

```

```
.btn-secondary {{
 background-color: #ffffff;
 color: #374151;
 font-weight: 500;
 padding: 0.625rem 1.25rem;
 border-radius: 0.75rem;
 border: 1px solid #e5e7eb;
 transition: all 0.2s;
 display: inline-block;
 cursor: pointer;
 box-shadow: 0 1px 2px rgba(0,0,0,0.05);
}}

.btn-secondary:hover {{ background-color: #f9fafb; }}
.btn-secondary:active {{ transform: scale(0.97); }}
```

```
.card {{
 background-color: #ffffff;
 border-radius: 1rem;
 border: 1px solid #f3f4f6;
 box-shadow: 0 1px 3px rgba(0,0,0,0.06);
 padding: 1.5rem;
}}
```

```
.input {{
 width: 100%;
 border: 1px solid #e5e7eb;
 border-radius: 0.75rem;
 padding: 0.625rem 1rem;
 font-size: 0.875rem;
 background-color: #ffffff;
 outline: none;
 transition: all 0.2s;
 color: #111827;
}}
```

```

.input:focus {{
 border-color: {p};

 box-shadow: 0 0 0 3px {p}33;
}}

.input::placeholder {{ color: #9ca3af; }}

.badge-active {{ display: inline-flex; align-items: center; padding: 0.25rem 0.625rem;
border-radius: 9999px; font-size: 0.75rem; font-weight: 500; background-color: #dcfce7; color:
#15803d; }}

.badge-inactive {{ display: inline-flex; align-items: center; padding: 0.25rem 0.625rem;
border-radius: 9999px; font-size: 0.75rem; font-weight: 500; background-color: #f3f4f6; color:
#4b5563; }}

.badge-pending {{ display: inline-flex; align-items: center; padding: 0.25rem 0.625rem;
border-radius: 9999px; font-size: 0.75rem; font-weight: 500; background-color: #fef9c3; color:
#a16207; }}

.badge-done {{ display: inline-flex; align-items: center; padding: 0.25rem 0.625rem;
border-radius: 9999px; font-size: 0.75rem; font-weight: 500; background-color: #dbeafe; color:
#1d4ed8; }}

.badge-scheduled {{ display: inline-flex; align-items: center; padding: 0.25rem 0.625rem;
border-radius: 9999px; font-size: 0.75rem; font-weight: 500; background-color: #dcfce7; color:
#15803d; }}

.badge-confirmed {{ display: inline-flex; align-items: center; padding: 0.25rem 0.625rem;
border-radius: 9999px; font-size: 0.75rem; font-weight: 500; background-color: #dcfce7; color:
#15803d; }}
}}
"""

```

```

def _api_js(feat: dict) -> str:
 auth_fns = """

export const login = (u, p) => api.post("/auth/login", { username: u, password: p });
export const register = (data) => api.post("/auth/register", data);
export const setAuthToken = (token) => {
 if (token) {
 api.defaults.headers.common["Authorization"] = `Bearer ${token}`;
 localStorage.setItem("via_token", token);
 } else {
 delete api.defaults.headers.common["Authorization"];
 localStorage.removeItem("via_token");
 }
}

```



```

};

const saved = localStorage.getItem("via_token");

if (saved) setAuthToken(saved);

"" if feat["auth"] else ""

 return f""import axios from "axios";

const BASE_URL =

import.meta.env.VITE_API_URL ||

(typeof window !== "undefined" &&

(window.location.hostname === "localhost" || window.location.hostname === "127.0.0.1")

? "http://localhost:8000"

: "");

const api = axios.create({{

 baseURL: `${BASE_URL}/api/v1`,

 headers: {{ "Content-Type": "application/json" }},

 timeout: 30000,

}});

api.interceptors.response.use(

 (res) => res,

 (err) => {{

 console.error("API error:", err.response?.status, err.config?.url);

 if (err.response?.status === 401) localStorage.removeItem("via_token");

 return Promise.reject(err);

 }}

);

{auth_fns}

export const getItems = (params = {}) => api.get("/items", {{ params }});

export const getItem = (id) => api.get(`/items/${id}`);

export const createItem = (data) => api.post("/items", data);

export const updateItem = (id, data) => api.put(`/items/${id}`, data);

export const deleteItem = (id) => api.delete(`/items/${id}`);

```

```

export const getStats = () => api.get("/stats");

export default api;

"""

FIX v8: _app_jsx ? corrected priority order for app-type detection.
#
PROBLEM (v7): feat["chart"] was checked BEFORE feat["hospital"], feat["game"],
feat["expense"], feat["todo"], and feat["weather"] (which didn't exist).
Any task containing "dashboard" or "analytics" matched feat["chart"]=True and
always rendered the generic analytics UI, regardless of domain specificity.
Examples that were broken:
#
"create a simple weather dashboard" ? rendered analytics chart UI (WRONG)
"expense dashboard" ? rendered analytics chart UI (WRONG)
"hospital dashboard" ? rendered analytics chart UI (WRONG)
#
FIX: Domain-specific types are checked first. chart/analytics is now the
last resort before the final generic CRUD fallback. New priority order:
#
image ? recipe ? grade ? calculator ? kanban ?
weather (NEW) ? hospital ? game ? expense ? todo ?
chart (last resort) ? generic CRUD fallback

def _app_jsx(app_title: str, task: str, feat: dict, th: dict) -> str:
 p = th["primary"]
 icon = th["icon"]
 score_cls = th["score_cls"]

 # ?? IMAGE / UPLOAD / GRAYSCALE app ??????????????????????????????????????

 if feat["image"]:
 return f'''// src/App.jsx ? Generated by VIA for: {task}

import {{ useState, useRef, useCallback }} from "react";
import {{ toast, ToastContainer }} from "react-toastify";

```

```

import "react-toastify/dist/ReactToastify.css";

import axios from "axios";

const BASE_URL = import.meta.env.VITE_API_URL ||

 (typeof window !== "undefined" && (window.location.hostname === "localhost" ||
window.location.hostname === "127.0.0.1")

 ? "http://localhost:8000" : "");

export default function App() {{

 const [file, setFile] = useState(null);

 const [preview, setPreview] = useState(null);

 const [result, setResult] = useState(null);

 const [loading, setLoading] = useState(false);

 const [dragOver, setDragOver] = useState(false);

 const inputRef = useRef(null);

 const handleFile = useCallback((f) => {{

 if (!f || !f.type.startsWith("image/")) {{ toast.error("Please select an image file"); return;
 }}

 setFile(f);

 setResult(null);

 const reader = new FileReader();

 reader.onload = (e) => setPreview(e.target.result);

 reader.readAsDataURL(f);

 }}, []);

 const handleDrop = (e) => {{

 e.preventDefault(); setDragOver(false);

 handleFile(e.dataTransfer.files[0]);

 }};

 const handleProcess = async () => {{

 if (!file) {{ toast.error("Please select an image first"); return; }}

 setLoading(true);

 try {{

```

```

const fd = new FormData();

fd.append("file", file);

const res = await axios.post(`${BASE_URL}/api/v1/process`, fd, {{
 headers: {{ "Content-Type": "multipart/form-data" }},
 responseType: "blob",
}});

setResult(URL.createObjectURL(res.data));

toast.success("Image processed successfully!");

}} catch (err) {{
 toast.error(err.response?.data?.detail || "Processing failed ? backend may be starting up");
}} finally {{
 setLoading(false);
}}
}};

```

```

const handleDownload = () => {{
 if (!result) return;

 const a = document.createElement("a");

 a.href = result; a.download = "processed_" + (file?.name || "image.png");

 a.click();
}};

```

```

return (
 <div className="min-h-screen" style={{{{{{ backgroundColor: "{th["bg"]}" }}}}}>
 <ToastContainer position="top-right" autoClose={{3000}} />
 <nav className="bg-white border-b border-gray-100 shadow-sm sticky top-0 z-50">
 <div className="max-w-5xl mx-auto px-6 h-16 flex items-center gap-3">
 {icon}
 {app_title}
 </div>
 </nav>
 <main className="max-w-5xl mx-auto px-6 py-10 space-y-8">
 <div className="text-center">
 <h1 className="text-4xl font-bold text-gray-900">{app_title}</h1>

```

```

 <p className="text-gray-500 mt-2">Upload an image and process it instantly</p>
 </div>

 {{/* Upload Zone */}}

 <div
 onDrop={{handleDrop}} onDragOver={{e => {{ e.preventDefault(); setDragOver(true); }}}}}
 onDragLeave={{() => setDragOver(false)}} onClick={{() => inputRef.current?.click()}}
 className={{`card border-2 border-dashed cursor-pointer text-center py-16 transition-all
$ {{
 dragOver ? "border-indigo-400 bg-indigo-50" : "border-gray-200
hover:border-indigo-300"
 }}`}}
 >

 <input ref={{inputRef}} type="file" accept="image/*" className="hidden" onChange={{e =>
handleFile(e.target.files[0])}} />

 <div className="text-5xl mb-4">??</div>

 <p className="text-gray-600 font-medium">Drop an image here or <span style={{{{ color:
"{p}" }}}>browse</p>

 <p className="text-gray-400 text-sm mt-1">PNG, JPG, WEBP, GIF supported</p>
 </div>

 {{/* Preview + Result */}}

 {{(preview || result) && (
 <div className={{`grid gap-6 ${{result ? "grid-cols-1 md:grid-cols-2" : "grid-cols-1
max-w-md mx-auto"}}`}}>

 {{preview && (
 <div className="card space-y-3">

 <h2 className="font-semibold text-gray-700">Original</h2>

 <img src={{preview}} alt="original" className="w-full rounded-xl object-contain
max-h-80" />

 <p className="text-xs text-gray-400">{{file?.name}} · {{(file?.size /
1024).toFixed(1)}} KB</p>
 </div>

)}}

 {{result && (
 <div className="card space-y-3">

 <h2 className="font-semibold text-gray-700">Processed</h2>

 <img src={{result}} alt="processed" className="w-full rounded-xl object-contain

```

```

max-h-80" />

 <button onClick={{handleDownload}} className="btn-secondary w-full text-sm">?
Download</button>

 </div>

)}}

 </div>

)}}

 {{preview && (
 <div className="flex justify-center">
 <button onClick={{handleProcess}} disabled={{loading}} className="btn-primary px-10
py-3 text-base">
 {{loading ? "Processing?" : "Process Image"}}
 </button>
 </div>
)}}
</main>
</div>

);
}}
...

```

```

?? RECIPE / COOK / FOOD app ???
if feat["recipe"]:
 return f'''// src/App.jsx ? Generated by VIA for: {task}

import {{ useState, useEffect }} from "react";
import {{ toast, ToastContainer }} from "react-toastify";
import "react-toastify/dist/ReactToastify.css";
import {{ getItems, createItem, deleteItem }} from "./api.js";

const CATEGORIES = ["Breakfast", "Lunch", "Dinner", "Snack", "Dessert", "Drink"];

const SAMPLE = [
 {{ id: 1, title: "Spaghetti Carbonara", category: "Dinner", time: "25 min", servings: 2,
 ingredients: "200g spaghetti, 100g pancetta, 2 eggs, 50g parmesan, black pepper",

```

```

 steps: "1. Cook pasta. 2. Fry pancetta. 3. Mix eggs + cheese. 4. Combine off heat.",
 status:"active" }},
 {{ id: 2, title: "Avocado Toast", category: "Breakfast", time: "10 min", servings: 1,
 ingredients: "2 slices bread, 1 avocado, lemon juice, salt, chili flakes",
 steps: "1. Toast bread. 2. Mash avocado with lemon. 3. Spread and season.", status:"active"
 }},
];

```

```

export default function App() {{
 const [recipes, setRecipes] = useState(SAMPLE);
 const [search, setSearch] = useState("");
 const [cat, setCat] = useState("All");
 const [selected, setSelected] = useState(null);
 const [showForm, setShowForm] = useState(false);

 const [form, setForm] = useState({{ title:"", category:"Dinner", time:"", servings:2,
 ingredients:"", steps:"" }});
 const [loading, setLoading] = useState(false);

 useEffect(() => {{
 getItems().then(r => {{
 const items = Array.isArray(r.data) ? r.data : r.data?.items ?? [];
 if (items.length) setRecipes(items);
 }}).catch(() => {{}});
 }}, []);

 const filtered = recipes.filter(r =>
 (cat === "All" || r.category === cat) &&
 r.title.toLowerCase().includes(search.toLowerCase())
);

 const handleAdd = async (e) => {{
 e.preventDefault();
 if (!form.title.trim()) {{ toast.error("Recipe name required"); return; }}
 setLoading(true);
 try {{
 const res = await createItem({{ ...form, status: "active" }});

```

```

const item = res.data?.item ?? res.data ?? {{ ...form, id: Date.now() }};

setRecipes(prev => [item, ...prev]);

setShowForm(false);

setForm({{ title:"", category:"Dinner", time:"", servings:2, ingredients:"", steps:"" }});

toast.success("Recipe added!");

}} catch {{ setRecipes(prev => [{{ ...form, id: Date.now() }}, ...prev]); setShowForm(false);
}} finally {{ setLoading(false); }}

}};

```

```

const handleDelete = async (id) => {{

 if (!confirm("Delete this recipe?")) return;

 setRecipes(prev => prev.filter(r => (r.id ?? r._id) !== id));

 if (selected?.id === id) setSelected(null);

 deleteItem(id).catch(() => {{{}}});

 toast.success("Recipe deleted");

}};

```

```

if (selected) return (

 <div className="min-h-screen" style={{{{{{ backgroundColor: "{th["bg"]}" }}}}}>

 <ToastContainer position="top-right" autoClose={{3000}} />

 <nav className="bg-white border-b border-gray-100 shadow-sm sticky top-0 z-50">

 <div className="max-w-3xl mx-auto px-6 h-16 flex items-center gap-3">

 <button onClick={{({}) => setSelected(null)}} className="text-gray-500 hover:text-gray-800 mr-2">? Back</button>

 {icon}

 {app_title}

 </div>

 </nav>

 <main className="max-w-3xl mx-auto px-6 py-10">

 <div className="card space-y-6">

 <div className="flex justify-between items-start">

 <div>

 {{selected.category}}

 <h1 className="text-3xl font-bold text-gray-900 mt-3">{{selected.title}}</h1>

 <div className="flex gap-4 mt-2 text-sm text-gray-500">

```



```

 {{selected.time && ? {{selected.time}}}}

 {{selected.servings && ? {{selected.servings}} servings}}

 </div>

</div>

 <button onClick={{() => handleDelete(selected.id ?? selected._id)}} className="text-xs
px-3 py-1.5 bg-red-50 text-red-600 rounded-lg border border-red-100">Delete</button>

</div>

 {{selected.ingredients && (
 <div>
 <h2 className="font-semibold text-gray-800 mb-2">? Ingredients</h2>
 <div className="bg-gray-50 rounded-xl p-4 text-sm text-gray-700
whitespace-pre-line">{{selected.ingredients}}</div>
 </div>
)}}

 {{selected.steps && (
 <div>
 <h2 className="font-semibold text-gray-800 mb-2">? Instructions</h2>
 <div className="bg-gray-50 rounded-xl p-4 text-sm text-gray-700
whitespace-pre-line">{{selected.steps}}</div>
 </div>
)}}

</div>

</main>

</div>

);

return (
 <div className="min-h-screen" style={{{{ backgroundColor: "{th["bg"]}" }}}}>
 <ToastContainer position="top-right" autoClose={{3000}} />
 <nav className="bg-white border-b border-gray-100 shadow-sm sticky top-0 z-50">
 <div className="max-w-6xl mx-auto px-6 h-16 flex items-center justify-between">
 <div className="flex items-center gap-3">{icon}{app_title}</div>
 <button onClick={{() => setShowForm(true)}} className="btn-primary">+ Add
Recipe</button>
 </div>
 </nav>
 </div>
);

```

```

<main className="max-w-6xl mx-auto px-6 py-8 space-y-6">

 <div className="flex flex-col sm:flex-row gap-3">

 <input className="input max-w-sm" placeholder="Search recipes?" value={{search}}
 onChange={{e => setSearch(e.target.value)}} />

 <div className="flex gap-2 flex-wrap">

 {[["All", ...CATEGORIES].map(c => (

 <button key={{c}} onClick={{() => setCat(c)}}

 className={{`px-3 py-1.5 rounded-full text-sm font-medium border transition-all
 ${{cat === c ? "text-white border-transparent" : "bg-white text-gray-600 border-gray-200
 hover:border-amber-300"}}`}}

 style={{{{ backgroundColor: cat === c ? "{p}" : undefined }}}}>{{c}}</button>

)]]}

 </div>

 </div>

 {{filtered.length === 0 ? (

 <div className="card text-center py-20"><p className="text-5xl mb-4">{icon}</p><p
 className="text-gray-400 text-lg">No recipes found</p><button onClick={{() => setShowForm(true)}}
 className="btn-primary mt-4">+ Add Recipe</button></div>

) : (

 <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">

 {{filtered.map(r => (

 <div key={{r.id ?? r._id}} onClick={{() => setSelected(r)}} className="card
 cursor-pointer hover:shadow-md transition-all hover:-translate-y-0.5 space-y-3">

 <div className="flex justify-between items-start">

 <span className="text-xs font-semibold px-2 py-1 rounded-full bg-amber-100
 text-amber-700">{{r.category}}

 <button onClick={{e => {{ e.stopPropagation(); handleDelete(r.id ?? r._id); }}}
 className="text-xs text-red-400 hover:text-red-600">?</button>

 </div>

 <h3 className="font-bold text-gray-900 text-lg leading-snug">{{r.title}}</h3>

 {{r.ingredients && <p className="text-sm text-gray-500
 line-clamp-2">{{r.ingredients}}</p>}}

 <div className="flex gap-3 text-xs text-gray-400 pt-1">

 {{r.time && ? {{r.time}}}}

 {{r.servings && ? {{r.servings}} servings}}

 </div>

 </div>

)]]}

 </div>

)]]}

 </div>

```

```

)}}
</main>

{{showForm && (
 <div className="fixed inset-0 bg-black/40 z-50 flex items-center justify-center p-4">
 <div className="bg-white rounded-2xl shadow-2xl w-full max-w-lg p-6 space-y-4">
 <div className="flex justify-between items-center"><h2 className="text-xl font-bold text-gray-900">New Recipe</h2><button onClick={{() => setShowForm(false)}} className="text-gray-400 hover:text-gray-600 text-xl">?</button></div>
 <form onSubmit={{handleAdd}} className="space-y-3">
 <input className="input" placeholder="Recipe name *" value={{form.title}} onChange={{e => setForm({{...form, title: e.target.value}})}} required />
 <div className="grid grid-cols-2 gap-3">
 <select className="input" value={{form.category}} onChange={{e => setForm({{...form, category: e.target.value}})}}>
 {{CATEGORIES.map(c => <option key={{c}}>{{c}}</option>)}}
 </select>
 <input className="input" placeholder="Time (e.g. 30 min)" value={{form.time}} onChange={{e => setForm({{...form, time: e.target.value}})}} />
 </div>
 <input className="input" type="number" placeholder="Servings" min="1" value={{form.servings}} onChange={{e => setForm({{...form, servings: parseInt(e.target.value) || 1}})}} />
 <textarea className="input resize-none h-20" placeholder="Ingredients (one per line)" value={{form.ingredients}} onChange={{e => setForm({{...form, ingredients: e.target.value}})}} />
 <textarea className="input resize-none h-20" placeholder="Steps / instructions" value={{form.steps}} onChange={{e => setForm({{...form, steps: e.target.value}})}} />
 <div className="flex gap-3 pt-1"><button type="submit" className="btn-primary" disabled={{loading}}>{{loading ? "Saving?" : "Add Recipe"}}</button><button type="button" onClick={{() => setShowForm(false)}} className="btn-secondary">Cancel</button></div>
 </form>
 </div>
 </div>
)}}
</div>

);
}}
...

```

```

FIX v8: guard with not feat["game"] ? "score" appears in game keywords too.

game is checked first in the branch order below.

if feat["grade"] and not feat["game"]:

 return f'''// src/App.jsx ? Generated by VIA for: {task}

import {{ useState, useEffect }} from "react";

import {{ toast, ToastContainer }} from "react-toastify";

import "react-toastify/dist/ReactToastify.css";

import {{ getItems, createItem, deleteItem }} from "./api.js";

const GRADES = [

 {{ min:90, letter:"A+", color:"text-emerald-600 bg-emerald-50" }},
 {{ min:80, letter:"A", color:"text-emerald-600 bg-emerald-50" }},
 {{ min:70, letter:"B", color:"text-blue-600 bg-blue-50" }},
 {{ min:60, letter:"C", color:"text-amber-600 bg-amber-50" }},
 {{ min:50, letter:"D", color:"text-orange-600 bg-orange-50" }},
 {{ min:0, letter:"F", color:"text-red-600 bg-red-50" }},
];

const gradeInfo = (score) => GRADES.find(g => score >= g.min) || GRADES[GRADES.length-1];

const SAMPLE_STUDENTS = [

 {{ id:1, title:"Alice Johnson", roll:"001", math:92, science:88, english:95, history:79 }},
 {{ id:2, title:"Bob Smith", roll:"002", math:74, science:81, english:68, history:85 }},
 {{ id:3, title:"Carol White", roll:"003", math:56, science:63, english:72, history:60 }},
];

export default function App() {{

 const [students, setStudents] = useState(SAMPLE_STUDENTS);

 const [search, setSearch] = useState("");

 const [showForm, setShowForm] = useState(false);

 const [form, setForm] = useState({{ title:"", roll:"", math:"", science:"", english:"",
history:"" }});

 const [loading, setLoading] = useState(false);

 const [sortBy, setSortBy] = useState("title");

 useEffect(() => {{

```

```

 getItems().then(r => {{
 const items = Array.isArray(r.data) ? r.data : r.data?.items ?? [];
 if (items.length) setStudents(items);
 }}).catch(() => {{}});
}}, []);

const avg = (s) => {{
 const scores = [s.math, s.science, s.english, s.history].filter(x => x !== undefined && x !==
 "");
 return scores.length ? Math.round(scores.reduce((a,b) => a + Number(b), 0) / scores.length) :
 0;
 }};

const filtered = students
 .filter(s => s.title?.toLowerCase().includes(search.toLowerCase()) ||
s.roll?.includes(search))
 .sort((a,b) => sortBy === "avg" ? avg(b) - avg(a) : a.title?.localeCompare(b.title));

const classAvg = students.length ? Math.round(students.reduce((s,st) => s + avg(st), 0) /
students.length) : 0;

const passing = students.filter(s => avg(s) >= 50).length;

const handleAdd = async (e) => {{
 e.preventDefault();
 if (!form.title.trim()) {{ toast.error("Student name required"); return; }}
 setLoading(true);
 try {{
 const res = await createItem({{ ...form, status:"active" }});
 const item = res.data?.item ?? res.data ?? {{ ...form, id: Date.now() }};
 setStudents(prev => [...prev, item]);
 setShowForm(false); setForm({{ title:"", roll:"", math:"", science:"", english:"",
history:"" }});
 toast.success("Student added!");
 }} catch {{ setStudents(prev => [...prev, {{ ...form, id:Date.now() }}]); setShowForm(false);
 }} finally {{ setLoading(false); }}
 }};

```

```

const handleDelete = (id) => {{
 if (!confirm("Remove this student?")) return;
 setStudents(prev => prev.filter(s => (s.id ?? s._id) !== id));
 deleteItem(id).catch(() => {{}});
 toast.success("Student removed");
}};

return (
 <div className="min-h-screen" style={{{{ backgroundColor: "{th["bg"]}" }}}}>
 <ToastContainer position="top-right" autoClose={{3000}} />
 <nav className="bg-white border-b border-gray-100 shadow-sm sticky top-0 z-50">
 <div className="max-w-7xl mx-auto px-6 h-16 flex items-center justify-between">
 <div className="flex items-center gap-3">{icon}{app_title}</div>
 <button onClick={{({}) => setShowForm(true)}} className="btn-primary">+ Add
Student</button>
 </div>
 </nav>
 <main className="max-w-7xl mx-auto px-6 py-8 space-y-6">
 <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
 {{[
 ["Total Students", students.length, "?"],
 ["Passing", passing, "?"],
 ["Failing", students.length - passing, "??"],
 ["Class Average", classAvg + "%", "?"],
]}.map(([label, val, ic]) => (
 <div key={{label}} className="card text-center">
 <p className="text-2xl mb-1">{{ic}}</p>
 <p className="text-2xl font-bold text-gray-900">{{val}}</p>
 <p className="text-xs text-gray-500 mt-1">{{label}}</p>
 </div>
))}}
 </div>
 <div className="flex flex-col sm:flex-row gap-3">
 <input className="input max-w-xs" placeholder="Search by name or roll?" value={{search}}
onChange={{e => setSearch(e.target.value)}} />

```

```

 <select className="input max-w-xs" value={{sortBy}} onChange={{e =>
setSortBy(e.target.value)}}>

 <option value="title">Sort by Name</option>

 <option value="avg">Sort by Average</option>

 </select>

</div>

<div className="card overflow-hidden p-0">

 <table className="w-full text-sm">

 <thead className="bg-gray-50 border-b border-gray-100">

 <tr>

 <th className="px-4 py-3 text-left text-xs font-semibold text-gray-500
uppercase">Roll</th>

 <th className="px-4 py-3 text-left text-xs font-semibold text-gray-500
uppercase">Name</th>

 <th className="px-4 py-3 text-center text-xs font-semibold text-gray-500
uppercase">Math</th>

 <th className="px-4 py-3 text-center text-xs font-semibold text-gray-500
uppercase">Science</th>

 <th className="px-4 py-3 text-center text-xs font-semibold text-gray-500
uppercase">English</th>

 <th className="px-4 py-3 text-center text-xs font-semibold text-gray-500
uppercase">History</th>

 <th className="px-4 py-3 text-center text-xs font-semibold text-gray-500
uppercase">Average</th>

 <th className="px-4 py-3 text-center text-xs font-semibold text-gray-500
uppercase">Grade</th>

 <th className="px-4 py-3 text-right text-xs font-semibold text-gray-500
uppercase">Actions</th>

 </tr>

 </thead>

 <tbody className="divide-y divide-gray-50">

 {{filtered.map(s => {{

 const a = avg(s); const gi = gradeInfo(a);

 return (

 <tr key={{s.id ?? s._id}} className="hover:bg-gray-50/50">

 <td className="px-4 py-3 text-gray-500">{{s.roll || "?"}}</td>

 <td className="px-4 py-3 font-medium text-gray-900">{{s.title}}</td>

 {{{["math", "science", "english", "history"].map(sub => (

 <td key={{sub}} className="px-4 py-3 text-center text-gray-700">{{s[sub] ??
"?"}}</td>

```

```

)}}
 <td className="px-4 py-3 text-center font-bold text-gray-900">{{a}}%</td>

 <td className="px-4 py-3 text-center"><span className={{`text-xs font-bold
px-2 py-1 rounded-full ${{{gi.color}}`}}>{{gi.letter}}</td>

 <td className="px-4 py-3 text-right"><button onClick={{() => handleDelete(s.id
?? s._id)}} className="text-xs px-2 py-1 bg-red-50 text-red-500 rounded-lg border
border-red-100">Remove</button></td>

 </tr>

);

 }}}}
</tbody>

</table>

 <div className="px-4 py-3 bg-gray-50/50 border-t text-xs
text-gray-400">{{filtered.length}} student{{filtered.length !== 1 ? "s" : ""}}</div>

 </div>

</main>

{{showForm && (

 <div className="fixed inset-0 bg-black/40 z-50 flex items-center justify-center p-4">

 <div className="bg-white rounded-2xl shadow-2xl w-full max-w-md p-6 space-y-4">

 <div className="flex justify-between items-center"><h2 className="text-xl
font-bold">Add Student</h2><button onClick={{() => setShowForm(false)}} className="text-gray-400
hover:text-gray-600 text-xl">?</button></div>

 <form onSubmit={{handleAdd}} className="space-y-3">

 <div className="grid grid-cols-2 gap-3">

 <input className="input" placeholder="Full name *" value={{form.title}}
onChange={{e => setForm({{...form, title: e.target.value}})}} required />

 <input className="input" placeholder="Roll number" value={{form.roll}}
onChange={{e => setForm({{...form, roll: e.target.value}})}} />

 </div>

 <div className="grid grid-cols-2 gap-3">

 {{{["math", "science", "english", "history"].map(sub => (

 <input key={{sub}} className="input" type="number" min="0" max="100"

 placeholder={{sub.charAt(0).toUpperCase() + sub.slice(1) + " score"}}

 value={{form[sub]}} onChange={{e => setForm({{...form, [sub]:
e.target.value}})}} />

)}}

 </div>

 <div className="flex gap-3 pt-1"><button type="submit" className="btn-primary"
disabled={{loading}}>{{loading ? "Saving?" : "Add"}}</button><button type="button" onClick={{() =>
setShowForm(false)}} className="btn-secondary">Cancel</button></div>

```



```
</form>

</div>

</div>

)}}

</div>

);

}}

'''

?? CALCULATOR / BMI / CONVERTER / TIP app ??????????????????????????????

if feat["calculator"]:

 return f'''// src/App.jsx ? Generated by VIA for: {task}

import {{ useState }} from "react";

import {{ toast, ToastContainer }} from "react-toastify";

import "react-toastify/dist/ReactToastify.css";

export default function App() {{

 const [display, setDisplay] = useState("0");

 const [prev, setPrev] = useState(null);

 const [op, setOp] = useState(null);

 const [fresh, setFresh] = useState(true);

 const [history, setHistory] = useState([]);

 const press = (val) => {{

 if (fresh) {{ setDisplay(String(val)); setFresh(false); }}

 else setDisplay(display === "0" ? String(val) : display + val);

 }};

 const decimal = () => {{ if (!display.includes(".")) {{ setDisplay(display + "."); setFresh(false); }} }};

 const clear = () => {{ setDisplay("0"); setPrev(null); setOp(null); setFresh(true); }};

 const sign = () => setDisplay(String(-parseFloat(display)));

 const pct = () => setDisplay(String(parseFloat(display) / 100));

 const operate = (nextOp) => {{

 const cur = parseFloat(display);

 if (prev !== null && !fresh) {{
```

```

 const res = calc(prev, cur, op);

 setHistory(h => [...h.slice(-4), `${prev} ${op} ${cur} = ${res}`]);

 setDisplay(String(res)); setPrev(res);
 } else { setPrev(cur); }

 setOp(nextOp); setFresh(true);
});

const calc = (a, b, o) => {
 switch(o) {
 case "+": return Math.round((a+b)*1e10)/1e10;
 case "-": return Math.round((a-b)*1e10)/1e10;
 case "x": return Math.round((a*b)*1e10)/1e10;
 case "÷": return b !== 0 ? Math.round((a/b)*1e10)/1e10 : (toast.error("Cannot divide by
zero"), a);
 default: return b;
 }
};

const equals = () => {
 if (prev === null || op === null) return;

 const cur = parseFloat(display);

 const res = calc(prev, cur, op);

 setHistory(h => [...h.slice(-4), `${prev} ${op} ${cur} = ${res}`]);

 setDisplay(String(res)); setPrev(null); setOp(null); setFresh(true);
};

const BTN = ({ label, onClick, cls="", style={} }) => (
 <button onClick={onClick} style={style}

 className={`h-16 w-full rounded-2xl text-lg font-semibold transition-all active:scale-95
shadow-sm ${cls}`} >

 {label}

 </button>
);

return (
 <div className="min-h-screen flex flex-col items-center justify-center p-4" style={{
backgroundColor: "th["bg"]" }} >

 <ToastContainer position="top-right" autoClose={3000} />

```

```

<h1 className="text-2xl font-bold text-gray-900 mb-6">{app_title}</h1>

<div className="w-full max-w-xs space-y-4">

 {{history.length > 0 && (

 <div className="card p-3 space-y-1">

 {{history.map((h,i) => <p key={{i}} className="text-xs text-gray-400
text-right">{{h}}</p>)}}

 </div>

)}}

 <div className="bg-gray-900 rounded-2xl px-6 py-5 text-right shadow-xl">

 <p className="text-gray-400 text-sm h-5">{{prev !== null ? `${{prev}} ${{op}}` :
""}}</p>

 <p className="text-white text-4xl font-light mt-1 break-all">{{display}}</p>

 </div>

 <div className="grid grid-cols-4 gap-3">

 <BTN label="AC" onClick={{clear}} cls="bg-gray-200 text-gray-800 hover:bg-gray-300" />

 <BTN label="+/-" onClick={{sign}} cls="bg-gray-200 text-gray-800 hover:bg-gray-300" />

 <BTN label="%" onClick={{pct}} cls="bg-gray-200 text-gray-800 hover:bg-gray-300" />

 <BTN label="÷" onClick={{() => operate("÷")}} cls="text-white hover:opacity-90"
style={{{{ backgroundColor: "{p}" }}}}> />

 {{["7","8","9"].map(n => <BTN key={{n}} label={{n}} onClick={{() => press(n)}}
cls="bg-white text-gray-900 hover:bg-gray-50 border border-gray-100" />)}}

 <BTN label="×" onClick={{() => operate("×")}} cls="text-white hover:opacity-90"
style={{{{ backgroundColor: "{p}" }}}}> />

 {{["4","5","6"].map(n => <BTN key={{n}} label={{n}} onClick={{() => press(n)}}
cls="bg-white text-gray-900 hover:bg-gray-50 border border-gray-100" />)}}

 <BTN label="?" onClick={{() => operate("?")}} cls="text-white hover:opacity-90"
style={{{{ backgroundColor: "{p}" }}}}> />

 {{["1","2","3"].map(n => <BTN key={{n}} label={{n}} onClick={{() => press(n)}}
cls="bg-white text-gray-900 hover:bg-gray-50 border border-gray-100" />)}}

 <BTN label="+" onClick={{() => operate("+")}} cls="text-white hover:opacity-90"
style={{{{ backgroundColor: "{p}" }}}}> />

 <BTN label="0" onClick={{() => press("0")}} cls="col-span-2 bg-white text-gray-900
hover:bg-gray-50 border border-gray-100" />

 <BTN label="." onClick={{decimal}} cls="bg-white text-gray-900 hover:bg-gray-50 border
border-gray-100" />

 <BTN label="=" onClick={{equals}} cls="text-white hover:opacity-90" style={{{{
backgroundColor: "{p}" }}}}> />

 </div>

</div>

</div>

```

```

);
}}
'''

?? KANBAN / BOARD app ???
FIX v8: checked AFTER weather. "weather dashboard" no longer hits this
branch because _features() now uses word-boundary regex for "board",
so "dashboard" correctly returns feat["kanban"]=False.
if feat["kanban"] and not feat["weather"]:
 return f'''// src/App.jsx ? Generated by VIA for: {task}

import {{ {useState} }} from "react";
import {{ toast, ToastContainer }} from "react-toastify";
import "react-toastify/dist/ReactToastify.css";

const COLS = ["To Do", "In Progress", "Review", "Done"];
const COL_COLORS = {{
 "To Do": "border-gray-300 bg-gray-50",
 "In Progress": "border-blue-300 bg-blue-50",
 "Review": "border-amber-300 bg-amber-50",
 "Done": "border-green-300 bg-green-50",
}};

const PRIORITY = {{
 High: "bg-red-100 text-red-700",
 Medium: "bg-amber-100 text-amber-700",
 Low: "bg-gray-100 text-gray-600"
}};

const SEED = [
 {{ id:1, title:"Design system setup", col:"Done", priority:"High", assignee:"Alice" }},
 {{ id:2, title:"API integration", col:"In Progress", priority:"High", assignee:"Bob" }},
 {{ id:3, title:"Write unit tests", col:"To Do", priority:"Medium", assignee:"Carol" }},
 {{ id:4, title:"UI review", col:"Review", priority:"Low", assignee:"Dave" }},
];

export default function App() {{
 const [cards, setCards] = useState(SEED);
 const [dragging, setDragging] = useState(null);
 const [over, setOver] = useState(null);

```

```

const [showForm, setShowForm] = useState(false);

const [form, setForm] = useState({{ title:"", priority:"Medium", assignee:"" }});

const addCard = (e) => {{
 e.preventDefault();

 if (!form.title.trim()) {{ toast.error("Title required"); return; }}

 setCards(c => [...c, {{ id: Date.now(), col:"To Do", ...form }}]);

 setForm({{ title:"", priority:"Medium", assignee:"" }}); setShowForm(false);

 toast.success("Card added!");
}};

const deleteCard = (id) => setCards(c => c.filter(x => x.id !== id));

const moveCard = (col) => {{
 if (dragging === null) return;

 setCards(c => c.map(x => x.id === dragging ? {{...x, col}} : x));

 setDragging(null); setOver(null);
}};

return (
 <div className="min-h-screen" style={{{{{{ backgroundColor: "{th["bg"]}" }}}}}>
 <ToastContainer position="top-right" autoClose={{3000}} />
 <nav className="bg-white border-b border-gray-100 shadow-sm sticky top-0 z-50">
 <div className="max-w-full px-6 h-16 flex items-center justify-between">
 <div className="flex items-center gap-3">{icon}{app_title}</div>
 <button onClick={{({}) => setShowForm(true)}} className="btn-primary">+ Add Card</button>
 </div>
 </nav>
 <main className="px-6 py-8 overflow-x-auto">
 <div className="flex gap-5 min-w-max">
 {{COLS.map(col => (
 <div key={{col}}
 onDragOver={{e => {{ e.preventDefault(); setOver(col); }}}}}
 onDrop={{({}) => moveCard(col)}}
 className={{`w-72 rounded-2xl border-2 ${{COL_COLORS[col]} || "border-gray-200
bg-gray-50"}} p-4 space-y-3 transition-all`}}>

```

```

<div className="flex items-center justify-between">

 <h2 className="font-semibold text-gray-800">{{col}}</h2>

 <span className="text-xs bg-white border border-gray-200 text-gray-500 px-2 py-0.5
rounded-full">

 {{cards.filter(c => c.col === col).length}}

</div>

{{cards.filter(c => c.col === col).map(card => (

 <div key={{card.id}} draggable

 onDragStart={{() => setDragging(card.id)}}

 className="bg-white rounded-xl border border-gray-100 shadow-sm p-3 space-y-2
cursor-grab active:cursor-grabbing hover:shadow-md transition-all">

 <div className="flex justify-between items-start gap-2">

 <p className="font-medium text-gray-900 text-sm
leading-snug">{{card.title}}</p>

 <button onClick={{() => deleteCard(card.id)}} className="text-gray-300
hover:text-red-400 text-xs shrink-0">?</button>

 </div>

 <div className="flex items-center justify-between">

 <span className={{`text-xs px-2 py-0.5 rounded-full font-medium
$${PRIORITY[card.priority] || PRIORITY.Medium}}`}}>{{card.priority}}

 {{card.assignee && ?
{{card.assignee}}}}

 </div>

 </div>

)}}

 <button onClick={{() => setShowForm(true)}}

 className="w-full text-sm text-gray-400 hover:text-gray-700 py-2 border-2
border-dashed border-gray-200 rounded-xl hover:border-gray-300 transition-all">

 + Add card

 </button>

</div>

)}}}

</div>

</main>

{{showForm && (

 <div className="fixed inset-0 bg-black/40 z-50 flex items-center justify-center p-4">

 <div className="bg-white rounded-2xl shadow-2xl w-full max-w-sm p-6 space-y-4">

```

```

 <div className="flex justify-between items-center"><h2 className="text-xl font-bold">New Card</h2><button onClick={{() => setShowForm(false)}} className="text-gray-400 text-xl">?</button></div>

 <form onSubmit={{addCard}} className="space-y-3">

 <input className="input" placeholder="Card title *" value={{form.title}}
onChange={{e => setForm({{...form, title:e.target.value}})}} required />

 <select className="input" value={{form.priority}} onChange={{e => setForm({{...form,
priority:e.target.value}})}}>

 <option>High</option><option>Medium</option><option>Low</option>

 </select>

 <input className="input" placeholder="Assignee" value={{form.assignee}} onChange={{e
=> setForm({{...form, assignee:e.target.value}})}} />

 <div className="flex gap-3"><button type="submit"
className="btn-primary">Add</button><button type="button" onClick={{() => setShowForm(false)}}
className="btn-secondary">Cancel</button></div>

 </form>

 </div>

</div>

)}}

</div>

);

}}

'''

```

```
?? WEATHER / FORECAST app (NEW in v8) ?????????????????????????????????
Checked BEFORE feat["chart"] because "weather dashboard" sets both to True.
Without this block, weather apps rendered as a generic analytics dashboard.

if feat["weather"]:
 return f'''// src/App.jsx ? Generated by VIA for: {task}

import {{ useState, useEffect }} from "react";
import {{ toast, ToastContainer }} from "react-toastify";
import "react-toastify/dist/ReactToastify.css";
import axios from "axios";

const BASE_URL = import.meta.env.VITE_API_URL ||

 (typeof window !== "undefined" && (window.location.hostname === "localhost" ||
window.location.hostname === "127.0.0.1"))

? "http://localhost:8000" : "");
```

```

const WEATHER_ICONS = {{
 Clear: "??", Sunny: "??", Clouds: "??", Cloudy: "?",
 Rain: "??", Drizzle: "??", Thunderstorm: "??",
 Snow: "??", Mist: "??", Fog: "??", Haze: "??",
}};

const weatherIcon = (condition) =>
 WEATHER_ICONS[condition] || WEATHER_ICONS[Object.keys(WEATHER_ICONS).find(k
condition?.includes(k))] || "??";

const MOCK_CURRENT = {{
 city: "Hyderabad", country: "IN", temp: 32, feels_like: 35,
 condition: "Partly Cloudy", humidity: 58, wind_speed: 14, visibility: 10,
 sunrise: "6:04 AM", sunset: "6:41 PM", uv_index: 7,
}};

const MOCK_FORECAST = [
 {{ day: "Mon", high: 33, low: 24, condition: "Sunny" }},
 {{ day: "Tue", high: 31, low: 23, condition: "Clouds" }},
 {{ day: "Wed", high: 28, low: 22, condition: "Rain" }},
 {{ day: "Thu", high: 30, low: 23, condition: "Cloudy" }},
 {{ day: "Fri", high: 34, low: 25, condition: "Clear" }},
 {{ day: "Sat", high: 32, low: 24, condition: "Clouds" }},
 {{ day: "Sun", high: 29, low: 22, condition: "Drizzle" }},
];

export default function App() {{
 const [city, setCity] = useState("Hyderabad");
 const [query, setQuery] = useState("Hyderabad");
 const [current, setCurrent] = useState(MOCK_CURRENT);
 const [forecast, setForecast] = useState(MOCK_FORECAST);
 const [loading, setLoading] = useState(false);
 const [unit, setUnit] = useState("C");

 const toF = (c) => Math.round(c * 9/5 + 32);
 const tempDisplay = (c) => unit === "C" ? `${c}°C` : `${toF(c)}°F`;

```



```

const fetchWeather = async (cityName) => {{
 if (!cityName.trim()) {{ toast.error("Enter a city name"); return; }}
 setLoading(true);
 try {{
 const [curRes, foreRes] = await Promise.all([
 axios.get(`${BASE_URL}/api/v1/weather/current?city=${encodeURIComponent(cityName)}`),
 axios.get(`${BASE_URL}/api/v1/weather/forecast?city=${encodeURIComponent(cityName)}`),
]);
 setCurrent(curRes.data);
 setForecast(Array.isArray(foreRes.data) ? foreRes.data : foreRes.data?.forecast ??
MOCK_FORECAST);
 setCity(cityName);
 }} catch (err) {{
 if (err.response?.status === 404) {{
 toast.error(`City "${cityName}" not found`);
 }} else {{
 toast.info("Backend offline ? showing demo data");
 setCurrent({{ ...MOCK_CURRENT, city: cityName }});
 setForecast(MOCK_FORECAST);
 setCity(cityName);
 }}
 }} finally {{
 setLoading(false);
 }}
}};

```

```

const handleSearch = (e) => {{
 e.preventDefault();
 fetchWeather(query);
}};

```

```

const bgGradient = current.condition?.toLowerCase().includes("rain") ||
current.condition?.toLowerCase().includes("thunder")
? "from-slate-700 to-slate-900"

```

```

: current.condition?.toLowerCase().includes("cloud")

? "from-slate-500 to-slate-700"

: "from-sky-400 to-blue-600";

return (

 <div className="min-h-screen" style={{{{ backgroundColor: `${th["bg"]}` }}}}>

 <ToastContainer position="top-right" autoClose={{3000}} />

 {{/* Hero / current weather */}}

 <div className={{`bg-gradient-to-br ${bgGradient} text-white`}}>

 <div className="max-w-2xl mx-auto px-6 pt-10 pb-12">

 {{/* Search bar */}}

 <form onSubmit={{handleSearch}} className="flex gap-2 mb-8">

 <input

 value={{query}} onChange={{e => setQuery(e.target.value)}}

 placeholder="Search city?"

 className="flex-1 bg-white/20 backdrop-blur border border-white/30 rounded-xl px-4
py-3 text-white placeholder-white/60 outline-none focus:bg-white/30 transition-all"

 />

 <button type="submit" disabled={{loading}}

 className="px-5 py-3 bg-white/20 hover:bg-white/30 border border-white/30 rounded-xl
font-semibold transition-all disabled:opacity-50">

 {{loading ? "?" : "?"}}

 </button>

 <button type="button" onClick={{() => setUnit(u => u === "C" ? "F" : "C")}}

 className="px-4 py-3 bg-white/20 hover:bg-white/30 border border-white/30 rounded-xl
font-semibold transition-all">

 °{{unit === "C" ? "F" : "C"}}

 </button>

 </form>

 {{/* Current conditions */}}

 <div className="text-center space-y-2">

 <p className="text-white/70 text-lg">{{current.city}}{{current.country ? `,
${current.country}` : ""}}</p>

 <div className="text-8xl leading-none">{{weatherIcon(current.condition)}}</div>

```

```

<p className="text-7xl font-thin">{{tempDisplay(current.temp)}}</p>
<p className="text-xl text-white/80">{{current.condition}}</p>
<p className="text-white/60">Feels like {{tempDisplay(current.feels_like)}}</p>
</div>

```

```

{{/* Stats row */}}

```

```

<div className="grid grid-cols-3 gap-3 mt-8">
 {{[
 ["? Humidity", `${{current.humidity}}%`],
 ["? Wind", `${{current.wind_speed}} km/h`],
 ["? Visibility", `${{current.visibility}} km`],
]}.map(([label, val]) => (
 <div key={{label}} className="bg-white/15 backdrop-blur rounded-2xl px-4 py-3
text-center">
 <p className="text-white/70 text-xs">{{label}}</p>
 <p className="text-white font-semibold text-lg mt-1">{{val}}</p>
 </div>
))}}
</div>
</div>
</div>

```

```

{{/* 7-day forecast */}}

```

```

<div className="max-w-2xl mx-auto px-6 py-8 space-y-6">
 <h2 className="font-bold text-gray-800 text-lg">7-Day Forecast</h2>
 <div className="grid grid-cols-7 gap-2">
 {{forecast.slice(0,7).map((day, i) => (
 <div key={{i}} className="card text-center py-4 px-2 space-y-2">
 <p className="text-xs font-semibold text-gray-500">{{day.day}}</p>
 <p className="text-2xl">{{weatherIcon(day.condition)}}</p>
 <p className="text-xs font-bold text-gray-800">{{tempDisplay(day.high)}}</p>
 <p className="text-xs text-gray-400">{{tempDisplay(day.low)}}</p>
 </div>
))}}
 </div>
</div>

```

```
{/* Sun & UV details */}}

<div className="grid grid-cols-3 gap-4">

 {[

 ["? Sunrise", current.sunrise || "6:04 AM"],

 ["? Sunset", current.sunset || "6:41 PM"],

 ["?? UV Index", current.uv_index ?? 7],

].map(([label, val]) => (

 <div key={{label}} className="card text-center">

 <p className="text-xs text-gray-500">{{label}}</p>

 <p className="font-bold text-gray-800 mt-1">{{val}}</p>

 </div>

)]}

</div>

</div>

);

}}

...

?? HOSPITAL / APPOINTMENT app ??

if feat["hospital"]:

 empty_msg = "No appointments yet"

 new_btn = "+ Book Appointment"

 form_title = "Book Appointment"

 stat_cards = [("Total Appointments", "stats?.total ?? 0"), ("Scheduled", "stats?.active ?? 0"), ("Departments", "'4+')"]

 extra_fields = ""

 <div className="grid grid-cols-2 gap-4">

 <div><label className="block text-sm font-medium text-gray-700 mb-1">Doctor</label>

 <input className="input" placeholder="Dr. Smith" value={form.doctor_name || ""}

onChange={e => setForm({...form, doctor_name: e.target.value})} /></div>

 <div><label className="block text-sm font-medium text-gray-700

mb-1">Department</label>

 <select className="input" value={form.department || "General"} onChange={e =>

setForm({...form, department: e.target.value})}>
```

<option>General</option><option>Cardiology</option><option>Neurology</option><option>Orthopedics</option><option>Pediatrics</option>

</select></div>

</div>""

extra\_row = "<td className=\"px-5 py-4 text-gray-600\">{item.doctor\_name || '?'}</td><td className=\"px-5 py-4 text-gray-600\">{item.department || 'General'}</td>"

extra\_th = "<th className=\"px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase\">Doctor</th><th className=\"px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase\">Dept</th>"

# ?? GAME / LEADERBOARD app ?????????????????????????????????????????????????????????????

elif feat["game"]:

empty\_msg = "No game sessions yet"

new\_btn = "+ Add Session"

form\_title = "New Game Session"

stat\_cards = [("Total Sessions", "stats?.total ?? 0"), ("Active", "stats?.active ?? 0"), ("Platform", "Multi")]

extra\_fields = f""

<div className="grid grid-cols-2 gap-4">

<div><label className="block text-sm font-medium text-gray-700 mb-1">Game</label>

<input className="input" placeholder="e.g. Valorant" value={{form.game\_name || ""}} onChange={{e => setForm({...form, game\_name: e.target.value})}} /></div>

<div><label className="block text-sm font-medium text-gray-700 mb-1">Score</label>

<input className="input" type="number" placeholder="0" value={{form.score || ""}} onChange={{e => setForm({...form, score: parseInt(e.target.value) || 0})}} /></div>

</div>""

extra\_row = f'<td className="px-5 py-4 text-gray-600">{{item.game\_name || "?"}}</td><td className="px-5 py-4 font-bold {score\_cls}">{{item.score ?? 0}}</td>'

extra\_th = '<th className="px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase">Game</th><th className="px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase">Score</th>'

# ?? EXPENSE / FINANCE app ?????????????????????????????????????????????????????????????

elif feat["expense"]:

empty\_msg = "No expenses yet"

new\_btn = "+ Add Expense"

form\_title = "Add Expense"

stat\_cards = [("Total Expenses", "stats?.total ?? 0"), ("Active", "stats?.active ?? 0"), ("Tracked", "Auto")]

```

extra_fields = ""

<div className="grid grid-cols-2 gap-4">

 <div><label className="block text-sm font-medium text-gray-700 mb-1">Amount
(?)</label>

 <input className="input" type="number" placeholder="0.00" value={form.amount || ""}
onChange={e => setForm({...form, amount: parseFloat(e.target.value) || 0})} /></div>

 <div><label className="block text-sm font-medium text-gray-700 mb-1">Category</label>

 <select className="input" value={form.category || "General"} onChange={e =>
setForm({...form, category: e.target.value})}>

<option>General</option><option>Food</option><option>Transport</option><option>Bills</option><opti
on>Entertainment</option>

 </select></div>

</div>""

 extra_row = "<td className=\"px-5 py-4 text-gray-600\">{item.category ||
'General'}</td><td className=\"px-5 py-4 font-semibold text-emerald-700\">?{(item.amount ||
0).toLocaleString()}</td>"

 extra_th = "<th className=\"px-5 py-3 text-left text-xs font-semibold text-gray-500
uppercase\">Category</th><th className=\"px-5 py-3 text-left text-xs font-semibold text-gray-500
uppercase\">Amount</th>"

?? TODO / TASK app ??

elif feat["todo"]:

 empty_msg = "No tasks yet"

 new_btn = "+ Add Task"

 form_title = "New Task"

 stat_cards = [("Total Tasks", "stats?.total ?? 0"), ("Completed", "stats?.active ?? 0"),
("Pending", "Auto")]

 extra_fields = ""

 <div className="grid grid-cols-2 gap-4">

 <div><label className="block text-sm font-medium text-gray-700 mb-1">Priority</label>

 <select className="input" value={form.priority || "Medium"} onChange={e =>
setForm({...form, priority: e.target.value})}>

 <option>High</option><option>Medium</option><option>Low</option>

 </select></div>

 <div><label className="block text-sm font-medium text-gray-700 mb-1">Due Date</label>

 <input className="input" type="date" value={form.due_date || ""} onChange={e =>
setForm({...form, due_date: e.target.value})} /></div>

 </div>""

 extra_row = "<td className=\"px-5 py-4 text-gray-600\">{item.priority || 'Medium'}</td><td
className=\"px-5 py-4 text-gray-500\">{item.due_date || '?'}</td>"

```

```

 extra_th = "<th className=\"px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase\">Priority</th><th className=\"px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase\">Due</th>"

```

```

?? CHART / ANALYTICS / DASHBOARD (last resort before generic) ??????????

```

```

FIX v8: this block is now AFTER all domain-specific checks.

```

```

"weather dashboard" no longer reaches here because feat["weather"] is True

```

```

and is checked first. Same for "expense dashboard", "hospital analytics", etc.

```

```

elif feat["chart"]:

```

```

 return f'''// src/App.jsx ? Generated by VIA for: {task}

```

```

import {{ useState, useEffect }} from "react";

```

```

import {{ getStats, getItems }} from "./api.js";

```

```

const BAR_COLORS = [{"p}", "#6366f1", "#f59e0b", "#10b981", "#ef4444", "#8b5cf6"];

```

```

function BarChart({{ data }}) {{

```

```

 const max = Math.max(...data.map(d => d.value), 1);

```

```

 return (

```

```

 <div className="space-y-2">

```

```

 {{data.map((d, i) => (

```

```

 <div key={{i}} className="flex items-center gap-3">

```

```

 {{d.label}}

```

```

 <div className="flex-1 bg-gray-100 rounded-full h-6 overflow-hidden">

```

```

 <div className="h-full rounded-full flex items-center justify-end pr-2 transition-all duration-700"

```

```

 style={{{{ width: `${(d.value/max)*100}%`, backgroundColor: BAR_COLORS[i % BAR_COLORS.length] }}}}>

```

```

 {{d.value}}

```

```

 </div>

```

```

 </div>

```

```

 </div>

```

```

)}}

```

```

 </div>

```

```

);

```

```

}}

```

```

export default function App() {{
 const [stats, setStats] = useState({{ total:0, active:0 }});
 const [items, setItems] = useState([]);
 const [loading, setLoading] = useState(true);
 const [period, setPeriod] = useState("week");

 useEffect(() => {{
 Promise.all([
 getStats().then(r => r.data).catch(() => ({{ total: 0, active: 0 }})),
 getItems().then(r => Array.isArray(r.data) ? r.data : r.data?.items ?? []).catch(() => []),
]).then(([s, i]) => {{ setStats(s); setItems(i); setLoading(false); }});
 }}, []);

 const MOCK_TREND = [
 {{ label:"Mon", value:42 }}, {{ label:"Tue", value:68 }}, {{ label:"Wed", value:55 }},
 {{ label:"Thu", value:91 }}, {{ label:"Fri", value:74 }}, {{ label:"Sat", value:39 }}, {{
label:"Sun", value:57 }},
];

 const statusData = ["active","pending","done","inactive"].map(s => ({{
 label: s.charAt(0).toUpperCase() + s.slice(1),
 value: items.filter(i => i.status === s).length,
 })).filter(d => d.value > 0);

 return (
 <div className="min-h-screen" style={{{{ backgroundColor: "{th["bg"]}" }}}}>
 <nav className="bg-white border-b border-gray-100 shadow-sm sticky top-0 z-50">
 <div className="max-w-7xl mx-auto px-6 h-16 flex items-center justify-between">
 <div className="flex items-center gap-3">{icon}{app_title}</div>
 <div className="flex gap-2">
 {["week", "month", "year"].map(p => (
 <button key={p} onClick={{{}} => setPeriod(p)}}
 className={{`px-3 py-1.5 rounded-lg text-sm font-medium transition-all
${{period===p ? "text-white" : "text-gray-500 bg-white border border-gray-200"}}`}}
 style={{{{ backgroundColor: period===p ? "{p}" : undefined }}}}>

```



```

 {{p.charAt(0).toUpperCase()+p.slice(1)}}
 </button>

)}}
 </div>

</div>

</nav>

<main className="max-w-7xl mx-auto px-6 py-8 space-y-6">

 <h1 className="text-2xl font-bold text-gray-900">{app_title}</h1>

 <div className="grid grid-cols-2 md:grid-cols-4 gap-4">

 {[

 ["Total Records", stats.total ?? items.length ?? 0, "?", "blue"],

 ["Active", stats.active ?? items.filter(i=>i.status==="active").length, "?",
"green"],

 ["This Period", Math.floor(Math.random()*40)+10, "?", "purple"],

 ["Growth", "+12%", "?", "amber"],

]}.map(([label, val, ic, color]) => (

 <div key={{label}} className={{`card bg-gradient-to-br ${{
 color==="blue" ? "from-blue-50 to-blue-100/50 border-blue-200" :
 color==="green"? "from-green-50 to-green-100/50 border-green-200" :
 color==="purple"? "from-purple-50 to-purple-100/50 border-purple-200":
 "from-amber-50 to-amber-100/50 border-amber-200"
 }}}}>

 <p className="text-2xl mb-1">{{ic}}</p>

 <p className="text-2xl font-bold text-gray-900">{{loading ? "?" : val}}</p>

 <p className="text-xs text-gray-500 mt-1">{{label}}</p>

 </div>

)}}

 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 gap-6">

 <div className="card space-y-4">

 <h2 className="font-semibold text-gray-800">Activity Trend</h2>

 <BarChart data={{MOCK_TREND}} />

 </div>

 <div className="card space-y-4">

 <h2 className="font-semibold text-gray-800">Status Breakdown</h2>

```

```

 {{statusData.length > 0

 ? <BarChart data={{statusData}} />

 : <p className="text-gray-400 text-sm py-8 text-center">No data yet</p>

 }}

 </div>

</div>

{{items.length > 0 && (

 <div className="card overflow-hidden p-0">

 <div className="px-5 py-4 border-b border-gray-100"><h2 className="font-semibold text-gray-800">Recent Records</h2></div>

 <table className="w-full text-sm">

 <thead className="bg-gray-50"><tr>

 <th className="px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase">Title</th>

 <th className="px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase">Status</th>

 <th className="px-5 py-3 text-left text-xs font-semibold text-gray-500 uppercase">Created</th>

 </tr></thead>

 <tbody className="divide-y divide-gray-50">

 {{items.slice(0,8).map(item => (

 <tr key={{item.id ?? item._id}} className="hover:bg-gray-50/50">

 <td className="px-5 py-3 font-medium text-gray-900">{{item.title}}</td>

 <td className="px-5 py-3">{{item.status || "active"}}</td>

 <td className="px-5 py-3 text-gray-400">{{item.created_at ? new Date(item.created_at).toLocaleDateString() : "?"}}</td>

 </tr>

)}}

 </tbody>

 </table>

 </div>

)}}

</main>

</div>

);

}}

```

'''

```
?? GENERIC CRUD FALLBACK ???

else:

 empty_msg = "No records yet"

 new_btn = "+ Create New"

 form_title = "Create New Record"

 stat_cards = [("Total", "stats?.total ?? 0"), ("Active", "stats?.active ?? 0"), ("System",
'"Online"')]

 extra_fields = ""

 <div><label className="block text-sm font-medium text-gray-700 mb-1">Category</label>

 <input className="input" placeholder="Category" value={form.category || ""}
onChange={e => setForm({...form, category: e.target.value})} /></div>""

 extra_row = "<td className=\"px-5 py-4 text-gray-500 hidden
md:table-cell\">{item.description || '?'}</td>"

 extra_th = "<th className=\"px-5 py-3 text-left text-xs font-semibold text-gray-500
uppercase hidden md:table-cell\">Description</th>"

?? SHARED CRUD SCAFFOLD (hospital / game / expense / todo / generic) ???

stat_cards_jsx = "\n ".join([

 f'<StatCard title="{lbl}" value={{val}} loading={{loading}}
color="{["blue","green","purple"][i % 3]}" />'

 for i, (lbl, val) in enumerate(stat_cards)

])

return f'''// src/App.jsx ? Generated by VIA for: {task}

import {{ useState, useEffect, useCallback }} from "react";

import {{ Routes, Route, Link, useNavigate }} from "react-router-dom";

import {{ toast, ToastContainer }} from "react-toastify";

import "react-toastify/dist/ReactToastify.css";

import {{ getItems, createItem, deleteItem, getStats }} from "../api.js";

const parseItems = (data) => {{

 if (Array.isArray(data)) return data;

 if (data?.items && Array.isArray(data.items)) return data.items;

 if (data?.data && Array.isArray(data.data)) return data.data;

 if (data?.results && Array.isArray(data.results)) return data.results;
```

```

 if (data && typeof data === "object" &&
 (data.id !== undefined || data._id !== undefined)) return [data];

 return [];
 }
});

const THEME = "{p}";

export default function App() {
 return (
 <div className="min-h-screen" style={{{ backgroundColor: "{th["bg"]}" }}}>
 <ToastContainer position="top-right" autoClose={{3000}} hideProgressBar={{false}} />
 <nav className="bg-white border-b border-gray-100 shadow-sm sticky top-0 z-50">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex justify-between items-center h-16">
 <Link to="/" className="flex items-center gap-2">
 {icon}
 {app_title}
 </Link>
 <div className="flex items-center gap-3">
 <Link to="/" className="text-sm text-gray-500 hover:text-gray-900 px-3 py-1.5 rounded-lg hover:bg-gray-50 transition-colors">Dashboard</Link>
 <Link to="/items" className="text-sm text-gray-500 hover:text-gray-900 px-3 py-1.5 rounded-lg hover:bg-gray-50 transition-colors">All Records</Link>
 <Link to="/new" className="btn-primary text-sm py-2 px-4">{new_btn}</Link>
 </div>
 </div>
 </div>
 </nav>
 <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
 <Routes>
 <Route path="/" element={{<HomePage />}} />
 <Route path="/items" element={{<ItemListPage />}} />
 <Route path="/new" element={{<ItemFormPage />}} />
 <Route path="/edit/:id" element={{<ItemFormPage />}} />
 <Route path="*" element={{<NotFoundPage />}} />
 </Routes>
 </main>
 </div>
);
}

```

```

 </Routes>

 </main>

</div>

);

}}

function HomePage() {{

 const [stats, setStats] = useState(null);

 const [loading, setLoading] = useState(true);

 const [items, setItems] = useState([]);

 useEffect(() => {{

 Promise.all([

 getStats().then(r => r.data).catch(() => ({{total: 0, active: 0}})),

 getItems().then(r => parseItems(r.data)).catch(() => []),

]).then(([s, i]) => {{ setStats(s); setItems(i); setLoading(false); }});

 }}, []);

 return (

 <div className="space-y-8">

 <div className="flex items-start justify-between">

 <div>

 <h1 className="text-3xl font-display font-bold text-gray-900">{app_title}</h1>

 <p className="text-gray-500 mt-1 text-sm">Powered by VIA ? Autonomous AI Platform</p>

 </div>

 <Link to="/new" className="btn-primary">{new_btn}</Link>

 </div>

 <div className="grid grid-cols-1 sm:grid-cols-3 gap-5">

 {stat_cards_jsx}

 </div>

 <div className="card">

 <div className="flex items-center justify-between mb-5">

 <h2 className="text-lg font-display font-semibold text-gray-900">Recent Records</h2>

 <Link to="/items" className="text-sm font-medium hover:underline" style={{{{ color:
THEME }}}}>View all ?</Link>

 </div>

 {{loading ? <Spinner /> : items.length === 0 ? (

```

```

 <div className="text-center py-12">
 <p className="text-gray-400 mb-4">{empty_msg}</p>
 <Link to="/new" className="btn-primary inline-block">{new_btn}</Link>
 </div>
) : (
 <div className="space-y-3">
 {{items.slice(0, 5).map(item => (
 <div key={{item.id ?? item._id}} className="flex items-center justify-between p-4 rounded-xl border border-gray-100 hover:bg-gray-50 transition-all">
 <div>
 <p className="font-medium text-gray-900">{{item.title}}</p>
 <p className="text-xs text-gray-400 mt-0.5">{{item.created_at ? new Date(item.created_at).toLocaleDateString() : ""}}</p>
 </div>
 <Badge status={{item.status || "active"}} />
 </div>
)}})
 </div>
)}}
</div>
</div>
);
}}

```

```

function StatCard({{ title, value, loading, color }}) {{
 const cls = {{
 blue: "from-blue-50 to-blue-100/50 border-blue-200 text-blue-700",
 green: "from-green-50 to-green-100/50 border-green-200 text-green-700",
 purple: "from-purple-50 to-purple-100/50 border-purple-200 text-purple-700",
 }};
 return (
 <div className={{`rounded-2xl border bg-gradient-to-br p-6 ${cls[color]} || cls.blue`}}>
 <p className="text-sm font-medium opacity-70">{{title}}</p>
 <p className="text-3xl font-display font-bold mt-2">{{loading ? "..." : value}}</p>
 </div>
)
}

```

```

);
 }}

function ItemListPage() {{
 const [items, setItems] = useState([]);
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState("");
 const [search, setSearch] = useState("");
 const [deleting, setDeleting] = useState(null);

 const load = useCallback(() => {{
 setLoading(true);

 getItems()
 .then(r => setItems(parseItems(r.data)))
 .catch(() => setError("Cannot reach backend. It may be starting up ? wait 30s and retry."))
 .finally(() => setLoading(false));
 }}, []);

 useEffect(() => {{ load(); }}, [load]);

 const handleDelete = async (id) => {{
 if (!confirm("Delete this record?")) return;
 setDeleting(id);

 try {{
 await deleteItem(id);
 toast.success("Deleted successfully!");
 load();
 }} catch {{
 toast.error("Delete failed. Please try again.");
 }} finally {{
 setDeleting(null);
 }}
 }};
}};

```

```

const filtered = items.filter(i => (i.title ||
 "").toLowerCase().includes(search.toLowerCase()));

if (loading) return <Spinner />;

if (error) return (
 <div className="card text-center py-16">
 <p className="text-2xl mb-3">??</p>
 <p className="text-red-500 font-medium mb-4">{{error}}</p>
 <button onClick={{load}} className="btn-primary">Retry</button>
 </div>
);

return (
 <div className="space-y-6">
 <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
 <h1 className="text-2xl font-display font-bold text-gray-900">All Records</h1>
 <div className="flex gap-3">
 <input className="input max-w-xs" placeholder="Search..." value={{search}} onChange={{e
=> setSearch(e.target.value)}} />
 <Link to="/new" className="btn-primary whitespace-nowrap">{new_btn}</Link>
 </div>
 </div>
 {{filtered.length === 0 ? (
 <div className="card text-center py-20">
 <p className="text-5xl mb-4">{icon}</p>
 <p className="text-gray-400 text-lg mb-6">{empty_msg}</p>
 <Link to="/new" className="btn-primary inline-block">{new_btn}</Link>
 </div>
) : (
 <div className="card overflow-hidden p-0">
 <table className="w-full text-sm">
 <thead className="bg-gray-50/80 border-b border-gray-100">
 <tr>
 <th className="px-5 py-3 text-left text-xs font-semibold text-gray-500
uppercase">Title</th>
 {extra_th}
 <th className="px-5 py-3 text-left text-xs font-semibold text-gray-500
uppercase">Status</th>

```



```

 <th className="px-5 py-3 text-right text-xs font-semibold text-gray-500
uppercase">Actions</th>

 </tr>

</thead>

<tbody className="divide-y divide-gray-50">

 {{filtered.map(item => (

 <tr key={{item.id ?? item._id}} className="hover:bg-gray-50/50 transition-colors">

 <td className="px-5 py-4 font-medium text-gray-900">{{item.title}}</td>

 {extra_row}

 <td className="px-5 py-4"><Badge status={{item.status || "active"}} /></td>

 <td className="px-5 py-4">

 <div className="flex justify-end gap-2">

 <Link to={{`/edit/${{item.id ?? item._id}}`} className="text-xs py-1.5 px-3
bg-gray-100 hover:bg-gray-200 text-gray-700 rounded-lg transition-colors">Edit</Link>

 <button onClick={{() => handleDelete(item.id ?? item._id)}}
disabled={{deleting === (item.id ?? item._id)}}

 className="text-xs py-1.5 px-3 bg-red-50 hover:bg-red-100 text-red-600
rounded-lg border border-red-100 transition-colors disabled:opacity-50">

 {{deleting === (item.id ?? item._id) ? "... " : "Delete"}}

 </button>

 </div>

 </td>

 </tr>

)}}

 </tbody>

</table>

<div className="px-5 py-3 bg-gray-50/50 border-t border-gray-100 text-xs text-gray-400">

 {{filtered.length}} record{{filtered.length !== 1 ? "s" : ""}}

</div>

</div>

)}}

</div>

);

}}

```

```

function ItemFormPage() {{

```

```

const [form, setForm] = useState({{ title: "", description: "", status: "active" }});
const [loading, setLoading] = useState(false);
const [error, setError] = useState("");
const navigate = useNavigate();
const submit = async (e) => {{
 e.preventDefault();

 if (!form.title.trim()) {{ setError("Title is required"); return; }}

 setLoading(true); setError("");

 try {{
 await createItem(form);
 navigate("/items");
 toast.success("Saved successfully!");
 }} catch (err) {{
 const msg = err.response?.data?.detail || "Save failed. Backend may be starting up.";
 setError(msg);
 toast.error(msg);
 }} finally {{
 setLoading(false);
 }}
}};

return (
 <div className="max-w-2xl mx-auto">
 <Link to="/items" className="text-sm text-gray-500 hover:text-gray-700 mb-6 inline-block">?
Back</Link>

 <div className="card">
 <h1 className="text-2xl font-display font-bold text-gray-900 mb-6">{{form_title}}</h1>

 {{error && <div className="bg-red-50 border border-red-100 text-red-700 p-4 rounded-xl
mb-5 text-sm">{{error}}</div>}}

 <form onSubmit={{submit}} className="space-y-5">
 <div>
 <label className="block text-sm font-medium text-gray-700 mb-1">Title *</label>

 <input className="input" placeholder="Enter title" value={{form.title}} onChange={{e
=> setForm({{...form, title: e.target.value}})}} required />

 </div>

 {{extra_fields}}

 </div>

```

```

 <label className="block text-sm font-medium text-gray-700 mb-1">Notes</label>

 <textarea className="input resize-none h-24" placeholder="Optional notes..."
value={{form.description || ""}} onChange={{e => setForm({...form, description:
e.target.value}})} />

 </div>

 <div>

 <label className="block text-sm font-medium text-gray-700 mb-1">Status</label>

 <select className="input" value={{form.status}} onChange={{e => setForm({...form,
status: e.target.value}})}>

 <option value="active">Active</option><option value="inactive">Inactive</option>

 <option value="pending">Pending</option><option value="done">Done</option>

 </select>

 </div>

 <div className="flex gap-3 pt-2">

 <button type="submit" className="btn-primary" disabled={{loading}}>{{loading ?
"Saving..." : "Save"}}</button>

 <Link to="/items" className="btn-secondary">Cancel</Link>

 </div>

</form>

</div>

</div>

);
}}

```

```

function Badge({{ status }}) {{
 const m = {{ active: "badge-active", inactive: "badge-inactive", pending: "badge-pending", done:
"badge-done", scheduled: "badge-scheduled", confirmed: "badge-confirmed" }};

 return {{status}};
}}

```

```

function Spinner() {{
 return (
 <div className="flex flex-col items-center justify-center py-24 gap-4">

 <div className="w-10 h-10 border-4 border-gray-200 rounded-full animate-spin" style={{{{
borderTopColor: THEME }}}}> />

 <p className="text-sm text-gray-400">Loading...</p>

 </div>
)
}

```

```

);
}}

function NotFoundPage() {{
 return (
 <div className="card text-center py-24">
 <p className="text-8xl font-display font-bold text-gray-100">404</p>
 <p className="text-gray-400 mt-4 mb-6">Page not found</p>
 <Link to="/" className="btn-primary inline-block">Go Home</Link>
 </div>
);
}}

'''

FIX v8: _build_prompt ? same priority reorder as _app_jsx().
Domain-specific hints come before the generic "chart/analytics" hint so
"create a simple weather dashboard" gets the weather UI hint, not the
analytics/chart hint.

def _build_prompt(task: str, ceo_strategy: str = "", inter_context: str = "") -> str:
 strategy_block = f"\nCEO Strategic Direction: {ceo_strategy}\n" if ceo_strategy else ""
 context_block = f"\nContext from other departments:\n{inter_context}\n" if inter_context else ""

 t = task.lower()

 # ?? Detect app type ? domain-specific checks first, generic last ????????
 if any(w in t for w in ["image", "photo", "upload", "grayscale", "picture", "crop",
"filter"]):
 ui_hint = ""
 UI REQUIREMENTS ? IMAGE/FILE APP:
 - File drag-and-drop upload zone (large, prominent, dashed border)
 - Image preview (original) shown immediately after selection
 - "Process" button triggers POST /api/v1/process with multipart/form-data

```

- Processed result shown side-by-side with original
- Download button for the result
- Show file name and size below preview
- NO generic form/table ? this is a media processing app"""

```
elif any(w in t for w in ["recipe", "cook", "ingredient", "meal", "food", "dish"]):
```

```
 ui_hint = """"UI REQUIREMENTS ? RECIPE APP:
```

- Recipe cards grid (not a table) ? each card shows name, category badge, prep time, servings
- Category filter buttons (Breakfast, Lunch, Dinner, Snack, Dessert)
- Click a card to open detail view with full ingredients list and step-by-step instructions
- Add Recipe modal with fields: name, category, prep time, servings, ingredients (textarea), steps (textarea)
- NO generic Title/Description/Status form ? fields must match recipes"""

```
elif any(w in t for w in ["game", "gaming", "player", "score", "leaderboard"]):
```

```
 ui_hint = """"UI REQUIREMENTS ? GAME/LEADERBOARD APP:
```

- Leaderboard table sorted by score descending with rank numbers (??? for top 3)
- Columns: Rank, Player name, Game, Score, Date
- Add Session form: player name, game name, score (number input), platform
- Top 3 players shown as podium cards at the top
- Score highlighted in bold with color based on rank"""

```
elif any(w in t for w in ["grade", "student", "mark", "gpa", "academic", "exam", "result", "subject", "marks"]):
```

```
 ui_hint = """"UI REQUIREMENTS ? GRADE/STUDENT TRACKER:
```

- Table with columns: Roll No, Name, Subject scores (Math, Science, English, History), Average %, Grade letter
- Grade letter computed from average (A+?90, A?80, B?70, C?60, D?50, F<50) shown as colored badge
- Stats row: total students, class average, passing count, failing count
- Add Student modal with subject score inputs (0-100 number fields)
- Sort by name or by average descending
- NO generic Title/Description/Status ? use student-specific fields"""

```
elif any(w in t for w in ["calculator", "calculate", "math", "compute", "converter", "bmi", "tip calc"]):
```

```
 ui_hint = """"UI REQUIREMENTS ? CALCULATOR APP:
```

- Full calculator UI: digit buttons 0-9, operators (+, -, ×, ÷), decimal, equals, clear, +/-, %
- Large display showing current value and pending operation
- Calculation history log (last 5 operations)
- Buttons in a 4-column grid, styled with color-coded operator buttons
- NO form, NO table, NO API calls needed ? pure client-side logic""

```
elif any(w in t for w in ["kanban", "sprint"]) or (re.search(r'\bboard\b', t) and "weather"
not in t and "dashboard" not in t):
```

```
ui_hint = ""UI REQUIREMENTS ? KANBAN BOARD:
```

- 4 columns: To Do, In Progress, Review, Done
- Cards inside each column showing title, priority badge (High/Medium/Low), assignee
- Drag-and-drop cards between columns (use HTML5 draggable + onDrop)
- Add Card button opens modal: title, priority select, assignee input
- Card count badge on each column header
- Color-coded column headers
- NO generic list/table view""

```
FIX v8: weather BEFORE chart ? "weather dashboard" must get this hint, not the analytics
hint
```

```
elif any(w in t for w in ["weather", "forecast", "temperature", "climate", "rain", "humidity",
"wind", "storm"]):
```

```
ui_hint = ""UI REQUIREMENTS ? WEATHER APP:
```

- City search bar at the top (text input + search button + °C/°F toggle)
- Hero section with gradient background showing: city name, large weather emoji, temperature, condition, feels-like
- Stats strip below hero: humidity %, wind speed km/h, visibility km
- 7-day forecast as a horizontal strip of day cards (day name, emoji, high/low temps)
- Sunrise, sunset, UV index info cards
- API calls: GET /api/v1/weather/current?city=X and GET /api/v1/weather/forecast?city=X
- Show demo/mock data when backend is offline
- NO generic form/table ? this is purely a weather display app""

```
elif any(w in t for w in ["hospital", "appointment", "doctor", "patient", "medical"]):
```

```
ui_hint = ""UI REQUIREMENTS ? HOSPITAL/APPOINTMENT APP:
```

- Appointment list with Doctor name, Department, Date/Time, Status columns
- Book Appointment form: patient name, doctor, department (dropdown), date, time, notes

- Department filter (All, General, Cardiology, Neurology, Orthopedics, Pediatrics)
- Status badges: Scheduled (green), Pending (yellow), Cancelled (red)
- Stats: total appointments, scheduled today, active doctors count"""

```
elif any(w in t for w in ["expense", "budget", "finance", "spending", "money"]):
```

```
 ui_hint = """UI REQUIREMENTS ? EXPENSE/FINANCE APP:
```

- Expense list with Category, Amount, Date, Notes columns
- Add Expense form: description, amount (number), category (Food/Transport/Bills/Entertainment/Other), date
- Category filter tabs
- Total spent summary card and breakdown by category
- Amount displayed with currency symbol (? or \$)
- Color-coded categories"""

```
elif any(w in t for w in ["todo", "task", "checklist"]):
```

```
 ui_hint = """UI REQUIREMENTS ? TODO/TASK APP:
```

- Task list with checkboxes to mark complete (strikethrough on done)
- Priority levels (High/Medium/Low) shown as colored badges
- Filter tabs: All, Active, Completed
- Add Task: title, priority select, due date, optional notes
- Stats: total, completed, pending counts
- Completed tasks visually distinct (grayed out, strikethrough)"""

```
FIX v8: chart/analytics is now LAST ? only reached if no domain type matched
```

```
elif any(w in t for w in ["chart", "graph", "analytics", "dashboard", "visuali", "report", "statistic"]):
```

```
 ui_hint = """UI REQUIREMENTS ? ANALYTICS DASHBOARD:
```

- KPI metric cards at top (4 cards: Total, Active, This Period, Growth)
- Bar chart built with divs/CSS (no external chart library) showing weekly trend
- Status breakdown chart (another bar chart)
- Recent records table below charts
- Period filter buttons (Week / Month / Year)
- All charts built with plain CSS bar charts using percentage widths ? NO recharts or chart.js"""

```
else:
```

```
ui_hint = f"""UI REQUIREMENTS ? CUSTOM APP for: {task}
```

- Build the UI that DIRECTLY matches this task ? not a generic form
- Identify the core entities from the task and make columns/fields match them exactly
- Use appropriate layout: cards for browseable content, table for structured data, wizard for multi-step
- The app should look purpose-built for "{task}", not like a generic CRUD template
- Include relevant domain-specific fields, actions, and terminology"""

```
return f"""You are a World-Class Frontend Engineer and UI/UX Designer. Build a COMPLETE, PRODUCTION-READY React frontend.
```

CRITICAL DESIGN REQUIREMENT:

The UI must be **\*\*STUNNING, MODERN, AND PREMIUM\*\***. DO NOT build a basic, generic white page with simple tables and boring forms.

- Use rich aesthetics: vibrant modern color palettes, glassmorphism, subtle gradients, and dark modes.
- Implement a beautiful layout with a sidebar or modern top navigation, hero sections, and metric cards.
- Use micro-animations (hover states, transitions) using Tailwind utility classes (`transition-all duration-300 hover:scale-105`).
- Ensure all components look polished like a real SaaS product.

```
??
```

```
TASK: {task}
```

```
??
```

```
{strategy_block}{context_block}
```

```
{ui_hint}
```

OUTPUT FORMAT ? each file EXACTLY like this (no backticks, no markdown):

```
=== FILE: src/App.jsx ===
```

```
// complete code here
```

```
=== END ===
```

FILES TO GENERATE: src/App.jsx

APPROVED PACKAGES ? import ONLY from this list (build will crash on anything else):

```
react, react-dom, react-router-dom, axios,
```



react-toastify, react-hot-toast, react-icons, lucide-react,  
date-fns, react-hook-form, clsx

#### TECHNICAL RULES:

- React 18 hooks only (useState, useEffect, useCallback, useRef)
- HashRouter ? required for GitHub Pages (NOT BrowserRouter)
- API base: `const BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:8000"`
- Tailwind CSS for all styling ? no inline style objects except for dynamic colors
- NEVER import packages outside the approved list
- NEVER hardcode Render, Railway, or any deployment URLs
- CRITICAL DATA SAFETY: The backend may return a 404 JSON object (`{{"detail": "Not Found"}}`) if starting up. ALWAYS parse API list responses safely before using `.map()`. Never set a list state to a raw object.

Example: `const safeList = Array.isArray(r.data) ? r.data : (r.data?.items || []);`

- `src/App.jsx` must be at least 150 lines ? build a real, complete UI
- CRITICAL ICON RULE: You MUST ONLY use ``lucide-react`` for icons. DO NOT use ``react-icons`` as it causes build errors.
- CRITICAL ROUTING RULE: You MUST use ``useNavigate`` from ``react-router-dom`` to change pages. NEVER use ``window.location.href`` or ``window.location.replace`` as it breaks GitHub Pages deployment.
- CRITICAL API RULE: You MUST import and use the functions provided in ``./api.js`` for all API calls (e.g. ``import { login, register,.getItems, createItem } from './api';``). DO NOT import or use ``axios`` directly in ``App.jsx``!

#### CRITICAL - SINGLE FILE ONLY:

- `src/App.jsx` MUST be completely self-contained.
- DO NOT import any local components (e.g. ``import Header from './components/Header'``).
- DO NOT import local pages (e.g. ``import Home from './pages/Home'``).
- You MUST define all your sub-components (Header, Footer, Modals, Pages) directly inside ``src/App.jsx``.
- If you use ``import ... from './'`` or ``import ... from '../'``, the build WILL FAIL.

#### QUALITY BAR:

- The UI must visually match the task ? someone reading the task should recognise the app
- Include realistic sample/seed data so the UI is not empty on first load
- Every interactive element must work (buttons, forms, filters)

""

...

```
File: `backend\agents\hr_agent.py`
```

```
```py
```

```
# backend/agents/hr_agent.py ? VIA Phase 3: Human Resources Department
```

```
import time
```

```
import logging
```

```
from backend.core.llm_provider import llm
```

```
logger = logging.getLogger("AI-Digital-Company")
```

```
async def hr_agent(task: str, ceo_strategy: str = "", project_brief: dict = None, inter_context: str = "") -> dict:
```

```
    start = time.time()
```

```
    logger.info(f"HR Agent | Task: {task[:60]}")
```

```
    context_block = ""
```

```
    if ceo_strategy:
```

```
        context_block += f"\nCEO Strategic Direction: {ceo_strategy}\n"
```

```
    if inter_context:
```

```
        context_block += f"\nContext from other departments:\n{inter_context}\n"
```

```
    prompt = f"""You are the Head of Human Resources at a top-tier tech MNC called VIA.
```

```
{context_block}
```

```
A new project has been assigned: {task}
```

```
Your job is to produce a complete HR plan for this project covering:
```

1. TEAM STRUCTURE: List all roles needed (job titles, seniority levels, team size)
2. JOB DESCRIPTIONS: Write 2-3 key job descriptions with responsibilities and requirements
3. ONBOARDING PLAN: 30/60/90 day onboarding plan for new hires
4. CULTURE & VALUES: Team culture guidelines and working principles
5. PERFORMANCE METRICS: KPIs and success metrics for the team
6. HIRING TIMELINE: Phased hiring plan with estimated dates

7. SALARY BUDGET: Approximate salary ranges per role (USD/year)

8. TEAM POLICIES: Remote work, meetings, code review, communication norms

Be specific, professional, and realistic. Format as a structured HR document.

Tailor everything specifically to the project described.

"""

try:

output = await llm.agenerate(prompt)

duration = round(time.time() - start, 2)

sections = _parse_hr_output(output or "")

logger.info(f"HR Agent done | {duration}s")

return {

 "department": "Human Resources",

 "status": "success",

 "execution_time_seconds": duration,

 "confidence": 0.91,

 "output": {

 "department": "Human Resources",

 "team_structure": sections.get("team_structure", ""),

 "job_descriptions": sections.get("job_descriptions", ""),

 "onboarding_plan": sections.get("onboarding_plan", ""),

 "culture_values": sections.get("culture_values", ""),

 "performance_metrics": sections.get("performance_metrics", ""),

 "hiring_timeline": sections.get("hiring_timeline", ""),

 "salary_budget": sections.get("salary_budget", ""),

 "team_policies": sections.get("team_policies", ""),

 "full_report": output or "",

 "summary": f"HR plan for '{task[:80]}' ? covering team structure, hiring, onboarding, and culture.",

 },

}

except Exception as e:

duration = round(time.time() - start, 2)

```

logger.error(f"HR Agent failed | {e}")

return {
    "department": "Human Resources",
    "status": "failed",
    "execution_time_seconds": duration,
    "confidence": 0.0,
    "error": str(e),
    "output": {},
}

```

```

def _parse_hr_output(text: str) -> dict:
    sections = {}
    keywords = {
        "team_structure": ["team structure", "roles", "team size"],
        "job_descriptions": ["job description", "responsibilities", "requirements"],
        "onboarding_plan": ["onboarding", "30/60/90", "day plan"],
        "culture_values": ["culture", "values", "principles"],
        "performance_metrics": ["kpi", "metrics", "performance"],
        "hiring_timeline": ["timeline", "hiring plan", "phases"],
        "salary_budget": ["salary", "budget", "compensation"],
        "team_policies": ["policy", "policies", "remote", "communication"],
    }

    lines = text.split("\n")
    current = "general"
    buckets = {k: [] for k in keywords}
    buckets["general"] = []
    for line in lines:
        ll = line.lower()
        for key, kws in keywords.items():
            if any(kw in ll for kw in kws):
                current = key
                break
        buckets.get(current, buckets["general"]).append(line)

```

```

    for key in keywords:

        sections[key] = "\n".join(buckets[key]).strip()

    return sections

'''

## File: `backend\agents\marketing_agent.py`

'''py

# backend/agents/marketing_agent.py ? VIA Phase 3: Marketing Department

import time

import logging

from backend.core.llm_provider import llm

logger = logging.getLogger("AI-Digital-Company")

async def marketing_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:

    start = time.time()

    logger.info(f"Marketing Agent | Task: {task[:60]}")

    context_block = ""

    if ceo_strategy:

        context_block += f"\nCEO Strategic Direction: {ceo_strategy}\n"

    if inter_context:

        context_block += f"\nContext from other departments:\n{inter_context}\n"

    prompt = f"""You are the Chief Marketing Officer (CMO) at a high-growth tech MNC called VIA.
{context_block}

A new product/project has been launched: {task}

Create a complete go-to-market strategy covering:

1. PRODUCT POSITIONING: Unique value proposition and market positioning statement

```

2. TARGET AUDIENCE: Detailed buyer personas (3 personas with demographics, pain points, goals)
3. BRAND IDENTITY: Brand name ideas, tagline options, tone of voice, brand colors
4. GO-TO-MARKET STRATEGY: Launch plan with phases and channels
5. CONTENT STRATEGY: Blog topics, social media plan, video content ideas
6. LANDING PAGE COPY: Hero headline, subheading, feature bullets, CTA text
7. SEO STRATEGY: Target keywords, content pillars, technical SEO priorities
8. GROWTH CHANNELS: Top 5 acquisition channels with expected CAC and conversion rates
9. COMPETITIVE ANALYSIS: 3 main competitors, their weaknesses, our differentiators
10. LAUNCH CAMPAIGN: 90-day marketing campaign plan with budget allocation
11. SUCCESS METRICS: Marketing KPIs, OKRs, and measurement framework

Be creative, data-driven, and specific. Write like a top-tier marketing strategist.

"""

```
try:
    output = await llm.agenerate(prompt)
    duration = round(time.time() - start, 2)
    logger.info(f"Marketing Agent done | {duration}s")
    return {
        "department": "Marketing",
        "status": "success",
        "execution_time_seconds": duration,
        "confidence": 0.92,
        "output": {
            "department": "Marketing",
            "full_report": output or "",
            "summary": f"Go-to-market strategy for '{task[:80]}' ? positioning, personas,
campaigns, and growth.",
            "landing_page_copy": _extract_landing_copy(output or ""),
            "key_channels": _extract_channels(output or ""),
        },
    }

except Exception as e:
    duration = round(time.time() - start, 2)
```

```

logger.error(f"Marketing Agent failed | {e}")

return {
    "department": "Marketing",
    "status": "failed",
    "execution_time_seconds": duration,
    "confidence": 0.0,
    "error": str(e),
    "output": {},
}

```

```

def _extract_landing_copy(text: str) -> dict:
    copy = {}
    lines = text.split("\n")
    for i, line in enumerate(lines):
        ll = line.lower()
        if "headline" in ll or "hero" in ll:
            if i + 1 < len(lines):
                copy["headline"] = lines[i + 1].strip()
        if "tagline" in ll or "subheading" in ll:
            if i + 1 < len(lines):
                copy["tagline"] = lines[i + 1].strip()
        if "cta" in ll or "call to action" in ll:
            if i + 1 < len(lines):
                copy["cta"] = lines[i + 1].strip()
    return copy

```

```

def _extract_channels(text: str) -> list:
    channels = []
    channel_keywords = [
        "SEO", "SEM", "Google Ads", "LinkedIn", "Twitter", "Instagram",
        "Facebook", "TikTok", "YouTube", "Email", "Content Marketing",
        "Influencer", "Affiliate", "Product Hunt", "Reddit", "Discord",

```

```
"Developer Community", "Open Source", "Webinar", "Podcast"

]

tl = text.lower()

for ch in channel_keywords:

    if ch.lower() in tl:

        channels.append(ch)

return channels[:8]

...


## File: `backend\agents\presentation_agent.py`

```py
backend/agents/presentation_agent.py ? VIA Phase 4: PowerPoint Generator

import time

import logging

import uuid

from pathlib import Path

logger = logging.getLogger("AI-Digital-Company")

PROJECTS_BASE = Path("projects")

?? Lazy color helpers (pptx is optional) ??

All pptx imports are deferred so the module can load without python-pptx.

def _rgb(r, g, b):

 """Create an RGBColor ? only call inside functions that truly need pptx."""

 from pptx.dml.color import RGBColor

 return RGBColor(r, g, b)

def _get_colors():

 """Return the VIA colour palette dict. Called lazily inside generate_pptx."""

 return {
```



```

"DARK_BG": _rgb(1, 1, 8),
"DARK_CARD": _rgb(10, 10, 30),
"CYAN": _rgb(0, 212, 255),
"VIOLET": _rgb(124, 58, 237),
"GREEN": _rgb(0, 255, 157),
"MAGENTA": _rgb(255, 0, 110),
"WHITE": _rgb(255, 255, 255),
"GRAY": _rgb(123, 163, 192),
"DARK_GRAY": _rgb(42, 74, 96),
"PINK": _rgb(255, 77, 148),
"PURPLE": _rgb(192, 132, 252),
}

```

```

def _blank_slide(prs):
 return prs.slides.add_slide(prs.slide_layouts[6])

```

```

def _set_bg(slide, color):
 slide.background.fill.solid()
 slide.background.fill.fore_color.rgb = color

```

```

def _add_rect(slide, left, top, width, height, color):
 from pptx.util import Inches
 sh = slide.shapes.add_shape(1, Inches(left), Inches(top), Inches(width), Inches(height))
 sh.fill.solid()
 sh.fill.fore_color.rgb = color
 sh.line.fill.background()
 return sh

```

```

def _add_text(slide, text, left, top, width, height, size, color, bold=False):
 from pptx.util import Inches, Pt

```

```

tx = slide.shapes.add_textbox(Inches(left), Inches(top), Inches(width), Inches(height))
tf = tx.text_frame
tf.word_wrap = True
p = tf.paragraphs[0]
p.text = str(text)[:500]
p.font.size = Pt(size)
p.font.color.rgb = color
p.font.bold = bold
p.font.name = "Calibri"
return tx

```

```

def _cover_slide(prs, task, job_id, C):

```

```

 slide = _blank_slide(prs)
 _set_bg(slide, C["DARK_BG"])
 _add_rect(slide, 0, 0, 16, 2.4, C["DARK_CARD"])
 _add_rect(slide, 0, 0, 0.18, 9, C["CYAN"])
 _add_text(slide, "VIA", 0.5, 0.25, 4, 1.4, 52, C["CYAN"], bold=True)
 _add_text(slide, "AUTONOMOUS AI DIGITAL TEAM · PHASE 4 MNC ENGINE", 0.5, 1.6, 14, 0.6, 10,
C["DARK_GRAY"])
 _add_text(slide, task[:120], 0.5, 3.0, 15, 2.0, 28, C["WHITE"], bold=True)
 _add_text(slide, f"Job ID: {job_id} · Generated by VIA", 0.5, 8.3, 15, 0.5, 10,
C["DARK_GRAY"])

```

```

def _section_slide(prs, title, bullets, accent, C):

```

```

 from pptx.util import Inches, Pt
 slide = _blank_slide(prs)
 _set_bg(slide, C["DARK_BG"])
 _add_rect(slide, 0, 0, 16, 1.3, C["DARK_CARD"])
 _add_rect(slide, 0, 0, 0.18, 9, accent)
 _add_text(slide, title, 0.4, 0.2, 15, 0.9, 22, accent, bold=True)
 tx = slide.shapes.add_textbox(Inches(0.5), Inches(1.6), Inches(15), Inches(7.0))
 tf = tx.text_frame
 tf.word_wrap = True

```

```

for i, b in enumerate(bullets[:10]):
 p = tf.paragraphs[0] if i == 0 else tf.add_paragraph()
 p.text = f"? {str(b)[:200]}"
 p.font.size = Pt(15)
 p.font.color.rgb = C["WHITE"] if i == 0 else C["GRAY"]
 p.font.name = "Calibri"
 p.space_after = Pt(6)

```

```

def _dept_meta(C):
 """Return department metadata ? called lazily inside generate_pptx."""
 return {
 "backend": ("?? Backend Engineering", C["CYAN"]),
 "frontend": ("? Frontend Engineering", C["VIOLET"]),
 "security": ("? Security Architecture", C["MAGENTA"]),
 "devops": ("? DevOps & Deployment", C["GREEN"]),
 "ai_research": ("? AI Research", C["PURPLE"]),
 "architecture": ("? Solutions Architecture", C["GREEN"]),
 "hr": ("? Human Resources", C["PINK"]),
 "finance": ("? Finance & ROI", C["GREEN"]),
 "marketing": ("? Marketing & GTM", C["PURPLE"]),
 }

```

```

def generate_pptx(task: str, job_id: str, ceo_strategy: dict, departments: dict) -> str:
 from pptx import Presentation
 from pptx.util import Inches

 # Initialize colors lazily
 C = _get_colors()
 DEPT_META = _dept_meta(C)

 prs = Presentation()
 prs.slide_width = Inches(16)

```

```

prs.slide_height = Inches(9)

Cover

_cover_slide(prs, task, job_id, C)

CEO Strategy

strat = ceo_strategy or {}

_section_slide(prs, "? CEO Strategy", [
 f"Short-Term (0-3 months): {strat.get('short_term_strategy', 'N/A')[:200]}",
 f"Long-Term Vision (6-24 months): {strat.get('long_term_vision', 'N/A')[:200]}",
 "Execution: 9 autonomous AI agents assigned to all departments",
 "Pipeline: GitHub auto-push ? Render auto-deploy",
], C["MAGENTA"], C)

Department slides

for dept, data in departments.items():
 if data.get("status") != "success":
 continue

 meta = DEPT_META.get(dept)
 if not meta:
 continue

 title, color = meta

 output = data.get("output", {})

 bullets = [f"Confidence: {round(data.get('confidence', 0)*100)}% . Time: {data.get('execution_time_seconds', 0)}s"]

 for k, v in output.items():
 if k in ("department", "department_path"):
 continue

 if isinstance(v, str) and len(v) > 10:
 bullets.append(f"{k.replace('_', ' ').title(): {v[:160]}}")

 elif isinstance(v, list) and v:
 bullets.append(f"{k.replace('_', ' ').title(): {' , '.join(str(x)[:50] for x in v[:4])}}")

 _section_slide(prs, title, bullets, color, C)

```

```
Timeline
```

```
_section_slide(prs, "? Execution Timeline", [
 "Week 1-2: Architecture design + DB schema",
 "Week 3-6: Backend API + Security hardening",
 "Week 7-9: Frontend build + Integration tests",
 "Week 10-11: DevOps setup + CI/CD pipeline",
 "Week 12: Staging deployment + QA",
 "Month 3+: Production launch + Marketing",
], C["GREEN"], C)
```

```
Next Steps
```

```
n = len([d for d in departments.values() if d.get("status") == "success"])
_section_slide(prs, "? Next Steps", [
 f"Review outputs from {n} departments",
 "Merge into unified technical specification",
 "Approve budget & headcount (see Finance slide)",
 "Set up Render account + link GitHub repository",
 "Schedule engineering kickoff meeting",
 "Launch marketing campaign (see Marketing slide)",
], C["CYAN"], C)
```

```
Save
```

```
out_dir = PROJECTS_BASE / f"pptx_{job_id}"
out_dir.mkdir(parents=True, exist_ok=True)
safe = "".join(c if c.isalnum() or c in " _-" else "_" for c in task[:40]).strip()
out_path = out_dir / f"VIA_{safe}.pptx"
prs.save(str(out_path))
logger.info(f"PPTX saved | {out_path} | {len(prs.slides)} slides")
return str(out_path)
```

```
async def presentation_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:
```

```
 """Standard agent signature ? generates a presentation summary report."""
```

```
 start = time.time()
```

```

logger.info(f"Presentation Agent | {task[:60]}")

try:

 from backend.core.llm_provider import llm

 prompt = f"""You are VIA's Presentation Director. Create a concise executive presentation
outline.

Task: {task}

CEO Strategy: {ceo_strategy}

{f'Context: {inter_context[:500]}' if inter_context else ''}

Create a 6-slide presentation outline with:

1. Cover slide (title + tagline)
2. Problem Statement
3. Solution Architecture
4. Key Features (5 bullets)
5. Go-to-Market Timeline
6. Next Steps & Call to Action

Format each slide as: SLIDE N: Title | Content"""

 output = await llm.agenerate(prompt)

 dur = round(time.time() - start, 2)

 return {

 "department": "Presentation",

 "presentation_outline": output or "",

 "slide_count": 6,

 "summary": f"Executive presentation outline for '{task[:60]}' ? 6 slides covering
problem, solution, features, timeline.",

 }

except Exception as e:

 dur = round(time.time() - start, 2)

 logger.error(f"Presentation Agent failed | {e}")

 return {

 "department": "Presentation",

 "error": str(e),

 "summary": "Presentation generation failed.",

```

```
}
```

```
...
```

```
File: `backend\agents\security_agent.py`
```

```
```py
```

```
# backend/agents/security_agent.py ? Phase 2: reads backend context
```

```
import json, re
```

```
from backend.core.llm_provider import llm
```

```
from backend.core.logger import logger
```

```
def _extract(text):
```

```
    m = re.search(r"\{.*\}", text, re.DOTALL)
```

```
    if m:
```

```
        try: return json.loads(m.group())
```

```
        except: pass
```

```
    return None
```

```
async def security_agent(task: str, ceo_strategy: str = "", project_brief: dict = None,
inter_context: str = "") -> dict:
```

```
    """ context_block = f"\nContext from other departments:\n{inter_context}\n" if inter_context else
    """
```

```
    prompt = (
```

```
        "You are a Chief Security Officer at an MNC-level tech company.\n\n"
```

```
        "CEO Strategic Direction: " + ceo_strategy + "\n"
```

```
        + context_block +
```

```
        "Analyze the business idea and return ONLY valid JSON:\n"
```

```
        '{"department":"Security Engineering","threat_model":{"top_threats":["..."],"attack_surfaces":["..."]},' +
```

```
        '"authentication":{"strategy":"...","token_management":"...","mfa":"..."},' +
```

```
        '"encryption":{"in_transit":"...","at_rest":"...","key_management":"..."},' +
```

```
        '"risk_mitigation":["..."],"compliance":["..."]}\n\n'
```

```
        "Business idea: " + task + "\n"
```

```
)
```

```
    logger.info("Security Agent | Generating plan...")
```

```

raw = await llm.agenerate(prompt)

parsed = _extract(raw)

if parsed:

    parsed["department"] = "Security Engineering"

    return parsed

return {

    "department": "Security Engineering",

    "threat_model": {"top_threats": ["Unauthorized access", "Data exfiltration", "Injection attacks"], "attack_surfaces": ["Public API", "Auth layer", "DB"]},

    "authentication": {"strategy": "JWT access tokens (15min) + refresh tokens (7d).", "token_management": "Redis revocation list.", "mfa": "TOTP for admin accounts."},

    "encryption": {"in_transit": "TLS 1.3 + HSTS.", "at_rest": "AES-256 + bcrypt."},

    "key_management": "Env secrets, 90-day rotation."},

    "risk_mitigation": ["Rate limiting 100 req/min.", "Parameterized queries only.", "CORS restricted origins."],

    "compliance": ["OWASP Top 10", "GDPR"]

}

```

```

...

```

```

## File: `backend\agents\__init__.py`

```

```

```py

```

```

...

```

```

File: `backend\auth\auth.py`

```

```

```py

```

```

from datetime import datetime, timedelta

from typing import Optional

from fastapi import Depends, HTTPException, status

from fastapi.security import OAuth2PasswordBearer

from jose import JWTError, jwt

from pydantic import BaseModel, EmailStr

import hashlib

```



```

import bcrypt

from backend.core.config import JWT_SECRET_KEY, JWT_ALGORITHM, JWT_EXPIRE_MINUTES
from backend.core.logger import logger
from backend.database.db import get_user_by_email, create_user

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")

class Token(BaseModel):
    access_token: str
    token_type: str

class UserCreate(BaseModel):
    email: str
    password: str

# ----- PASSWORD HASHING (direct bcrypt ? passlib bypass) -----

def hash_password(p: str) -> str:
    """
    Supports unlimited password length safely.
    Step 1: SHA256 normalize (ensures < 72 bytes)
    Step 2: bcrypt hash directly
    """
    sha = hashlib.sha256(p.encode("utf-8")).hexdigest()[:72]
    return bcrypt.hashpw(sha.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(plain: str, hashed: str) -> bool:
    sha = hashlib.sha256(plain.encode("utf-8")).hexdigest()[:72]
    try:

```

```

        return bcrypt.checkpw(sha.encode("utf-8"), hashed.encode("utf-8"))

except Exception:

    return False

# -----

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()

    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=JWT_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})

    return jwt.encode(to_encode, JWT_SECRET_KEY, algorithm=JWT_ALGORITHM)

async def get_current_user(token: str = Depends(oauth2_scheme)):
    exc = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or expired token.",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:
        payload = jwt.decode(token, JWT_SECRET_KEY, algorithms=[JWT_ALGORITHM])
        email: str = payload.get("sub")

        if email is None:
            raise exc

    except JWTError:
        raise exc

    user = await get_user_by_email(email)

    if not user:

```

```
raise exc
```

```
if not user.get("is_active"):  
    raise HTTPException(  
        status_code=403,  
        detail="User account disabled."  
    )
```

```
return user
```

```
async def get_current_active_user(current_user: dict = Depends(get_current_user)):  
    return current_user
```

```
async def authenticate_user(email, password):  
    user = await get_user_by_email(email)  
  
    if not user or not verify_password(password, user["hashed_password"]):  
        return None  
  
    return user
```

```
async def register_user(email, password):  
    if await get_user_by_email(email):  
        raise HTTPException(  
            status_code=400,  
            detail="Email already registered."  
        )  
  
    await create_user(email, hash_password(password))  
  
    logger.info(f"User registered: {email}")
```

```

    return {
        "email": email,
        "is_active": True,
        "is_verified": False
    }
}

```

```

## File: `backend\auth\__init__.py`

```

```

```py

```

```


```

```

File: `backend\core\chat_engine.py`

```

```

```py

```

```

# backend/core/chat_engine.py ? VIA Phase 5: Conversational Chat Engine

```

```

# Uses Gemini first, falls back to Groq if Gemini hits rate limits

```

```

import logging

```

```

import os

```

```

import httpx

```

```

from backend.core.llm_provider import llm

```

```

logger = logging.getLogger("AI-Digital-Company")

```

```

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")

```

```

GEMINI_URL

```

```

"https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent"

```

```

=

```

```

SYSTEM_PROMPT = """You are VIA, a highly capable, versatile, and friendly AI assistant.

```

```

You are an expert in software engineering, data analysis, creative writing, and problem-solving.
You provide clear, concise, and accurate information, and you write high-quality, efficient, and
well-documented code.

```

Capabilities:

- Software development: FastAPI, React, Python, JavaScript, Docker, and beyond.
- Analysis & Reasoning: Explaining complex topics, debugging, and providing strategic advice.
- Creativity: Assisting with writing, brainstorming, and content generation.
- Action-Oriented: You can build and deploy applications. If a user asks to "Build me a [project]", take the initiative to design and scaffold the solution.

Guidelines:

- Be helpful, neutral, and encouraging.
- Use structured formatting (Markdown, bullet points, code blocks).
- Always specify the language when providing code blocks.
- Keep responses concise unless asked for depth.
- If you don't know something, admit it; never hallucinate facts.
- Use emojis sparingly to maintain a professional yet approachable tone.
- When asked to build something, guide the user through the process and explain your technical choices.

```
def _build_gemini_contents(message: str, history: list) -> list:
    """Build contents array for Gemini API."""
    contents = []
    if history:
        for msg in history[-10:]:
            role = msg.get("role", "user")
            content = msg.get("message", msg.get("content", ""))
            role = "model" if role in ("assistant", "via") else "user"
            if content:
                contents.append({"role": role, "parts": [{"text": content}]})
        contents.append({"role": "user", "parts": [{"text": message}]})
    return contents
```

```
def _build_groq_messages(message: str, history: list) -> list:
    """Build messages array for Groq API."""
```

```

messages = [{"role": "system", "content": SYSTEM_PROMPT}]

if history:
    for msg in history[-10:]:
        role = msg.get("role", "user")
        content = msg.get("message", msg.get("content", ""))
        role = "assistant" if role in ("assistant", "via") else "user"
        if content:
            messages.append({"role": role, "content": content})
messages.append({"role": "user", "content": message})

return messages

```

```

async def _try_gemini(message: str, history: list) -> str | None:
    """Try Gemini API ? returns None if fails."""
    if not GEMINI_API_KEY:
        return None
    try:
        contents = _build_gemini_contents(message, history)
        async with httpx.AsyncClient(timeout=20.0) as client:
            response = await client.post(
                f"{GEMINI_URL}?key={GEMINI_API_KEY}",
                json={
                    "system_instruction": {"parts": [{"text": SYSTEM_PROMPT}]},
                    "contents": contents,
                    "generationConfig": {"maxOutputTokens": 4096, "temperature": 0.7},
                },
            )
            response.raise_for_status()
        text = response.json()["candidates"][0]["content"]["parts"][0]["text"].strip()
        if text:
            logger.info(f"Chat response | Gemini | {len(text)} chars")
            return text
    except Exception as e:
        logger.warning(f"Gemini failed, trying Groq: {e}")

```

```
return None
```

```
async def _try_groq(message: str, history: list) -> str | None:

    """Try Groq via llm_provider ? returns None if fails."""

    try:

        messages = _build_groq_messages(message, history)

        response = await llm.achat(messages)

        if response and response.strip():

            logger.info(f"Chat response | Groq | {len(response)} chars")

            return response.strip()

    except Exception as e:

        logger.warning(f"Groq failed: {e}")

    return None
```

```
async def chat(message: str, history: list = None) -> str:

    """

    Generate a conversational response from VIA.

    Tries Gemini first, falls back to Groq automatically.

    """

    history = history or []

    # Try Gemini first

    response = await _try_gemini(message, history)

    if response:

        return response

    # Fallback to Groq

    response = await _try_groq(message, history)

    if response:

        return response

    # Both failed
```

```

    return (
        "I'm having a moment ? please try again in a few seconds! "
        "If you'd like me to build something, just say "
        "'Build me a [your idea]' ?"
    )

```

```

## File: `backend\core\code_runner.py`

```

```

```py

```

```

backend/core/code_runner.py ? Phase 4

```

```

import os

```

```

import sys

```

```

import time

```

```

import subprocess

```

```

import py_compile

```

```

import logging

```

```

logger = logging.getLogger("AI-Digital-Company")

```

```

def strip_markdown_fences(content: str) -> str:

```

```

 import re

```

```

 content = content.strip()

```

```

 fence_pattern = re.compile(r"^```[a-zA-Z]*\n(?:.*?)```s*$", re.DOTALL)

```

```

 match = fence_pattern.match(content)

```

```

 if match:

```

```

 return match.group(1).strip()

```

```

 if content.startswith("```"):

```

```

 lines = content.splitlines()

```

```

 lines = lines[1:]

```

```

 if lines and lines[-1].strip() == "```":

```

```

 lines = lines[:-1]

```



```
 return "\n".join(lines).strip()

 return content
```

```
?? 1. SYNTAX CHECKER ???
```

```
def check_syntax(file_path: str) -> dict:

 try:

 py_compile.compile(file_path, doraise=True)

 return {"file": file_path, "passed": True, "error": None}

 except py_compile.PyCompileError as e:

 return {"file": file_path, "passed": False, "error": str(e)}

def check_all_syntax(department_path: str) -> dict:

 results = []

 passed = 0

 failed = 0

 for root, dirs, files in os.walk(department_path):

 dirs[:] = [d for d in dirs if d != "__pycache__"]

 for fname in files:

 if fname.endswith(".py"):

 full_path = os.path.join(root, fname)

 result = check_syntax(full_path)

 results.append(result)

 if result["passed"]:

 passed += 1

 else:

 failed += 1

 logger.warning(f"Syntax error | {full_path} | {result['error']}")

 return {

 "total_files": passed + failed,
```

```

 "passed": passed,
 "failed": failed,
 "all_passed": failed == 0,
 "results": results
}

```

# ?? 2. REQUIREMENTS ? skip local install, Render handles it ??????????????????

```
def install_requirements(department_path: str) -> dict:
```

```
 """
```

```
 Skips local pip install completely.
```

```
 Reason: Windows venv causes Fatal Python error: init_import_site
```

```
 when subprocess pip runs inside uvicorn's async loop.
```

```
 Render installs requirements.txt automatically on deploy ? no local install needed.
```

```
 """
```

```
 req_path = os.path.join(department_path, "requirements.txt")
```

```
 if not os.path.exists(req_path):
```

```
 return {
```

```
 "found": False,
```

```
 "installed": False,
```

```
 "output": "No requirements.txt found",
```

```
 "error": None
```

```
 }
```

```
 logger.info(f"Requirements found | {req_path} | Render will install on deploy")
```

```
 return {
```

```
 "found": True,
```

```
 "installed": True,
```

```
 "output": "requirements.txt found ? Render will install on deploy.",
```

```
 "error": None
```

```
 }
```

```
?? 3. IMPORT TESTER ???
```

```
def test_imports(department_path: str) -> dict:

 results = []

 passed = 0

 failed = 0

 testable = ["main.py", "models.py", "services.py", "auth.py", "security.py"]

 for fname in testable:

 full_path = os.path.join(department_path, fname)

 if not os.path.exists(full_path):

 continue

 # Use forward slashes for subprocess compatibility on Windows

 safe_path = full_path.replace("\\", "/")

 try:

 result = subprocess.run(

 [sys.executable, "-c",

 f"import importlib.util; "

 f"spec=importlib.util.spec_from_file_location('mod',r'{full_path}'); "

 f"mod=importlib.util.module_from_spec(spec)",

 capture_output=True,

 text=True,

 timeout=15,

 cwd=department_path

)

 if result.returncode == 0:

 passed += 1

 results.append({"file": fname, "importable": True, "error": None})

 else:

 failed += 1

 results.append({"file": fname, "importable": False, "error": result.stderr[:200]})

 except subprocess.TimeoutExpired:

 failed += 1
```

```

 results.append({"file": fname, "importable": False, "error": "Timed out"})

except Exception as e:

 failed += 1

 results.append({"file": fname, "importable": False, "error": str(e)})

return {

 "tested": passed + failed,

 "passed": passed,

 "failed": failed,

 "all_passed": failed == 0,

 "results": results

}

```

# ?? 4. MASTER RUNNER ?????????????????????????????????????????????????????????????????

```

def run_phase4_checks(task: str, department: str, department_path: str) -> dict:

 start = time.time()

 logger.info(f"Phase 4 checks starting | {department} | {department_path}")

 if not os.path.exists(department_path):

 return {

 "department": department,

 "phase4_ran": False,

 "error": f"Department path not found: {department_path}"

 }

 syntax_report = check_all_syntax(department_path)

 requirements_report = install_requirements(department_path)

 import_report = test_imports(department_path)

 duration = round(time.time() - start, 2)

 overall_passed = (

 syntax_report["all_passed"] and

```

```

 (not requirements_report["found"] or requirements_report["installed"])
)

 logger.info(
 f"Phase 4 done | {department} | "
 f"Syntax: {'OK' if syntax_report['all_passed'] else 'FAIL'} | "
 f"Requirements: {'OK' if requirements_report['installed'] else 'SKIP'} | "
 f"Duration: {duration}s"
)

 return {
 "department": department,
 "phase4_ran": True,
 "overall_passed": overall_passed,
 "duration_seconds": duration,
 "syntax_check": syntax_report,
 "requirements_install": requirements_report,
 "import_test": import_report,
 "summary": _build_summary(syntax_report, requirements_report, import_report)
 }

```

```

def _build_summary(syntax: dict, requirements: dict, imports: dict) -> str:
 lines = []
 if syntax["all_passed"]:
 lines.append(f"[OK] Syntax: {syntax['passed']} files clean")
 else:
 lines.append(f"[FAIL] Syntax errors in {syntax['failed']} file(s)")
 for r in syntax["results"]:
 if not r["passed"]:
 lines.append(f" -> {os.path.basename(r['file'])}: {r['error']}")

 if not requirements["found"]:
 lines.append("[SKIP] No requirements.txt")

```

```

elif requirements["installed"]:
 lines.append("[OK] Requirements ready for Render deploy")
else:
 lines.append(f"[FAIL] Requirements issue: {requirements['error']}")

if imports["tested"] == 0:
 lines.append("[SKIP] No files to import-test")
elif imports["all_passed"]:
 lines.append(f"[OK] Imports: {imports['passed']} files OK")
else:
 lines.append(f"[WARN] Import issues in {imports['failed']} file(s)")

return " | ".join(lines)
```

## File: `backend\core\code_writer.py`

```py
backend/core/code_writer.py ? Phase 3
Handles LLM code block extraction and project file saving

import os
import re
import logging
from datetime import datetime

logger = logging.getLogger("AI-Digital-Company")

Base directory where all generated projects are saved
PROJECTS_BASE_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "projects")

def _strip_fences(content: str) -> str:
 """

```

Strip markdown code fences from content before saving to disk.

Fixes SyntaxError on line 1 when LLM wraps output in ```python ... ```

```
"""
```

```
content = content.strip()
```

```
Full fence block: ```python\n...\n```
```

```
match = re.match(r"^```[a-zA-Z]*\n(.*?)```s*$", content, re.DOTALL)
```

```
if match:
```

```
 return match.group(1).strip()
```

```
Partial ? starts with ``` but no closing fence
```

```
if content.startswith("```"):
```

```
 lines = content.splitlines()
```

```
 lines = lines[1:] # remove opening ```python line
```

```
 if lines and lines[-1].strip() == "```":
```

```
 lines = lines[:-1] # remove closing ``` line
```

```
 return "\n".join(lines).strip()
```

```
return content
```

```
def extract_code_blocks(llm_output: str) -> dict:
```

```
"""
```

Parse LLM output that follows this format:

```
=== FILE: main.py ===
```

```
code here
```

```
=== END ===
```

Returns a dict of { "filename": "file content" }

Falls back to markdown ``` blocks if === format not found.

```
"""
```

```
files = {}
```

```
?? Primary parser: === FILE: name === ... === END === ????????????????????
```

```
pattern = r"=== FILE:\s*(.+?)\s*===\s*\n(.*?)=== END ==="
```

```
matches = re.findall(pattern, llm_output, re.DOTALL)
```

```

if matches:
 for filename, content in matches:
 filename = filename.strip()
 content = content.strip()
 if filename and content:
 # Strip fences in case LLM nested them inside === FILE === blocks
 if filename.endswith(".py"):
 content = _strip_fences(content)
 files[filename] = content
 logger.info(f"Extracted file | {filename} | {len(content)} chars")
 return files

?? Fallback parser: ```python / ```bash / ``` blocks ??????????????????
fallback_pattern = r"```(?:python|bash|sql|yaml|json|txt|md|)?\n(?:.*?)```"
fallback_matches = re.findall(fallback_pattern, llm_output, re.DOTALL)

if fallback_matches:
 logger.warning("Primary file format not found ? using markdown code block fallback")
 for i, content in enumerate(fallback_matches):
 content = content.strip()
 if content:
 # Try to detect filename from first line comment
 first_line = content.split("\n")[0]
 if first_line.startswith("#") and "." in first_line:
 detected = first_line.lstrip("# ").split("?")[0].strip()
 if detected.endswith(".py") or detected.endswith(".txt") or
detected.endswith(".md"):
 files[detected] = content
 continue
 files[f"file_{i+1}.py"] = content

return files

```



```

def save_project_files(task: str, department: str, files: dict) -> dict:
 """
 Save all generated files to:

 projects/<task_slug>/<department>/

 Strips markdown fences from .py files before writing to disk.
 """
 task_slug = _slugify(task)
 timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
 project_folder = f"{task_slug}_{timestamp}"

 project_path = os.path.abspath(os.path.join(PROJECTS_BASE_DIR, project_folder))
 department_path = os.path.join(project_path, department)

 os.makedirs(department_path, exist_ok=True)
 logger.info(f"Saving files | {department_path}")

 files_written = []
 errors = []

 for filename, content in files.items():
 try:
 # Strip fences from Python files before saving
 if filename.endswith(".py"):
 content = _strip_fences(content)

 file_path = os.path.join(department_path, filename)
 os.makedirs(os.path.dirname(file_path), exist_ok=True)

 with open(file_path, "w", encoding="utf-8") as f:
 f.write(content)

 files_written.append(filename)
 logger.info(f"Saved | {filename}")

```

```

 except Exception as e:

 logger.error(f"Failed to save {filename} | {str(e)}")

 errors.append({"file": filename, "error": str(e)})

 logger.info(

 f"Save complete | {department} | "

 f"{len(files_written)} saved | {len(errors)} errors"

)

 return {

 "project_path": project_path,

 "department_path": department_path,

 "files_written": files_written,

 "file_count": len(files_written),

 "errors": errors,

 }

def _slugify(text: str) -> str:

 text = text.lower().strip()

 text = re.sub(r"^[^w\s-]", "", text)

 text = re.sub(r"[\s_-]+", "_", text)

 text = re.sub(r"^[^_]+|[_]+$", "", text)

 return text[:60]

...

File: `backend\core\config.py`

```py
# backend/core/config.py ? Phase 6

import os

from dotenv import load_dotenv

load_dotenv()

```

```
MODEL_NAME = os.getenv("MODEL_NAME", "llama3:latest")

OLLAMA_URL = os.getenv("OLLAMA_URL", "http://localhost:11434/api/generate")

REQUEST_TIMEOUT = int(os.getenv("REQUEST_TIMEOUT", "120"))

LLM_MAX_RETRIES = int(os.getenv("LLM_MAX_RETRIES", "2"))

LLM_RETRY_DELAY = int(os.getenv("LLM_RETRY_DELAY", "2"))

# Groq cloud LLM (alternative to local Ollama)

USE_GROQ = os.getenv("USE_GROQ", "false").lower() == "true"

GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")

GROQ_MODEL = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")

POSTGRES_HOST = os.getenv("POSTGRES_HOST", "localhost")

POSTGRES_PORT = int(os.getenv("POSTGRES_PORT", "5432"))

POSTGRES_DB = os.getenv("POSTGRES_DB", "ai_digital_team")

POSTGRES_USER = os.getenv("POSTGRES_USER", "ai_admin")

POSTGRES_PASSWORD = os.getenv("POSTGRES_PASSWORD", "")

DATABASE_URL = os.getenv("DATABASE_URL", f"postgresql://{POSTGRES_USER}:{POSTGRES_PASSWORD}@{POSTGRES_HOST}:{POSTGRES_PORT}/{POSTGRES_DB}")

REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")

CELERY_BROKER_URL = os.getenv("CELERY_BROKER_URL", "redis://localhost:6379/0")

CELERY_RESULT_URL = os.getenv("CELERY_RESULT_URL", "redis://localhost:6379/1")

JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY", "change-this")

JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")

JWT_EXPIRE_MINUTES = int(os.getenv("JWT_ACCESS_TOKEN_EXPIRE_MINUTES", "30"))

RATE_LIMIT_PER_MINUTE = int(os.getenv("RATE_LIMIT_PER_MINUTE", "20"))

APP_ENV = os.getenv("APP_ENV", "development")

APP_VERSION = os.getenv("APP_VERSION", "6.0.0")

# SMTP Configuration

SMTP_HOST = os.getenv("SMTP_HOST", "smtp.gmail.com")

SMTP_PORT = int(os.getenv("SMTP_PORT", "587"))
```

```

SMTP_USER                = os.getenv("SMTP_USER", "")
SMTP_PASSWORD            = os.getenv("SMTP_PASSWORD", "")
FROM_EMAIL               = os.getenv("FROM_EMAIL", "noreply@via-platform.local")
MEMORY_INJECTION_COUNT   = int(os.getenv("MEMORY_INJECTION_COUNT", "3"))
COMPLEX_TASK_THRESHOLD   = int(os.getenv("COMPLEX_TASK_THRESHOLD", "80"))

VALID_DEPARTMENTS = {
    # Tech division
    "backend", "security", "devops", "ai_research", "architecture", "frontend",
    # Business division (Phase 3)
    "hr", "finance", "marketing",
    # Special
    "presentation",
}

...

## File: `backend\core\fullstack_builder.py`

```py
backend/core/fullstack_builder.py ? VIA Phase 3
Generates a complete deployable FastAPI backend based on task complexity
#
FIXES:
1. render.yaml fromDatabase.name matches what render_deployer.py creates
2. requirements.txt includes aiosqlite for SQLite fallback
3. database.py handles both postgresql:// and sqlite:// correctly
4. models.py amount/category fields for expense apps
5. Root / endpoint always present in generated main.py
6. Base.metadata.create_all() inside startup event ? never crashes at import
7. No duplicate 'from fastapi import' statements
8. TABLE PREFIX per app ? each app gets unique table names derived from its slug
e.g. expense tracker ? expense_tracker_items (not "items")

```

```
This allows ALL apps to safely share ONE PostgreSQL database with zero conflicts.

9. RESOURCE NAME detection ? routes use domain name (books, expenses, etc.)

so backend always matches what the LLM generates in api.js

10. ALIAS ROUTES ? both /resource and /api/v1/resource always work
```

```
import re
```

```
import logging
```

```
logger = logging.getLogger("AI-Digital-Company")
```

```
?? Complexity Detection ???
```

```
def detect_app_type(task: str) -> str:
```

```
 t = task.lower()
```

```
 db_signals = [
```

```
 "database", "store", "save", "persist", "crud", "users", "login",
```

```
 "register", "auth", "profile", "history", "records", "data",
```

```
 "postgresql", "mysql", "sqlite", "mongodb", "supabase",
```

```
 "transaction", "transactions", "tracker", "tracking", "management",
```

```
 "system", "dashboard", "analytics", "portfolio",
```

```
]
```

```
 backend_signals = [
```

```
 "api", "backend", "server", "fastapi", "endpoint", "rest",
```

```
 "quiz", "score", "leaderboard", "submit", "fetch", "real-time",
```

```
 "test", "iq", "exam", "assessment", "game", "track"
```

```
]
```

```
Only pure static/brochure sites with NO interactivity should be frontend-only
```

```
frontend_signals = [
```

```
 "landing page", "static page", "brochure", "one page website",
```

```
 "simple page", "showcase website"
```

```
]
```

```
DB signals always win ? check these FIRST before frontend classification
```

```
has_db = any(w in t for w in db_signals)
```

```
if has_db:
```

```
 return "fullstack_db"
```

```
Then check if it's a pure static frontend
```

```
if any(w in t for w in frontend_signals):
```

```
 return "frontend"
```

```
has_backend = any(w in t for w in backend_signals)
```

```
if has_backend:
```

```
 return "fullstack"
```

```
return "frontend"
```

```
?? Resource Name Detection ??
```

```
def _detect_resource_name(task: str) -> str:
```

```
 """
```

```
 Detects the domain resource name from the task description.
```

```
 Used as the API route path so the backend matches what the LLM generates in api.js.
```

```
e.g. "library book management" ? "books"
```

```
 "expense tracker" ? "expenses"
```

```
 "hospital appointment" ? "appointments"
```

```
 """
```

```
t = task.lower()
```

```
if any(w in t for w in ["book", "library", "isbn"]):
```

```
 return "books"
```

```
if any(w in t for w in ["expense", "budget", "finance", "money", "transaction"]):
```

```
 return "expenses"
```

```
if any(w in t for w in ["appointment", "doctor", "hospital", "patient"]):
```

```
 return "appointments"
```

```

if any(w in t for w in ["donor", "blood", "donation"]):
 return "donors"

if any(w in t for w in ["workout", "exercise", "fitness"]):
 return "workouts"

if any(w in t for w in ["student", "course", "class", "school", "grade"]):
 return "students"

if any(w in t for w in ["product", "inventory", "stock", "shop", "store"]):
 return "products"

if any(w in t for w in ["employee", "staff", "hr", "payroll"]):
 return "employees"

if any(w in t for w in ["task", "todo", "checklist"]):
 return "tasks"

if any(w in t for w in ["player", "team", "cricket", "football", "sport", "tournament"]):
 return "players"

if any(w in t for w in ["recipe", "food", "meal", "diet"]):
 return "recipes"

if any(w in t for w in ["event", "ticket", "conference"]):
 return "events"

if any(w in t for w in ["note", "journal", "diary"]):
 return "notes"

if any(w in t for w in ["contact", "address", "phone"]):
 return "contacts"

if any(w in t for w in ["order", "cart", "purchase", "checkout"]):
 return "orders"

if any(w in t for w in ["user", "member", "profile"]):
 return "users"

return "items" # safe fallback

```

# ?? Alias Route Generator ?????????????????????????????????????????????????????????????

```
def _alias_routes(resource: str, use_db: bool) -> str:
```

```
 """
```

```
 Generates alias routes so BOTH formats work regardless of what the LLM picks in api.js:
```

/api/v1/{resource}    ? most common LLM output  
/{resource}            ? short form LLM sometimes generates

If resource == "items", aliases are skipped (original routes already cover it).

"""

```
if resource == "items":
```

```
 return ""
```

```
if use_db:
```

```
 return f'''
```

```
?? Route aliases: /api/v1/{resource} and /{resource} mirror /api/v1/items ????
```

```
@app.get("/api/v1/{resource}")
```

```
def alias_get_{resource}(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
```

```
 return get_items(skip=skip, limit=limit, db=db)
```

```
@app.post("/api/v1/{resource}")
```

```
def alias_create_{resource}(payload: dict, db: Session = Depends(get_db)):
```

```
 return create_item(payload=payload, db=db)
```

```
@app.get("/api/v1/{resource}/{item_id}")
```

```
def alias_get_{resource}_one(item_id: int, db: Session = Depends(get_db)):
```

```
 return get_item(item_id=item_id, db=db)
```

```
@app.put("/api/v1/{resource}/{item_id}")
```

```
def alias_update_{resource}(item_id: int, payload: dict, db: Session = Depends(get_db)):
```

```
 return update_item(item_id=item_id, payload=payload, db=db)
```

```
@app.delete("/api/v1/{resource}/{item_id}")
```

```
def alias_delete_{resource}(item_id: int, db: Session = Depends(get_db)):
```

```
 return delete_item(item_id=item_id, db=db)
```

```
@app.get("/{resource}")
```

```
def alias_get_{resource}_short(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
```

```
 return get_items(skip=skip, limit=limit, db=db)
```



```

@app.post("/{resource}")

def alias_create_{resource}_short(payload: dict, db: Session = Depends(get_db)):

 return create_item(payload=payload, db=db)

@app.get("/{resource}/{item_id}")

def alias_get_{resource}_short_one(item_id: int, db: Session = Depends(get_db)):

 return get_item(item_id=item_id, db=db)

@app.put("/{resource}/{item_id}")

def alias_update_{resource}_short(item_id: int, payload: dict, db: Session = Depends(get_db)):

 return update_item(item_id=item_id, payload=payload, db=db)

@app.delete("/{resource}/{item_id}")

def alias_delete_{resource}_short(item_id: int, db: Session = Depends(get_db)):

 return delete_item(item_id=item_id, db=db)
'''

 else:

 return f'''

?? Route aliases: /api/v1/{resource} and /{resource} mirror /api/v1/items ???

@app.get("/api/v1/{resource}")

def alias_get_{resource}():

 return get_items()

@app.post("/api/v1/{resource}")

def alias_create_{resource}(item: dict):

 return create_item(item=item)

@app.get("/api/v1/{resource}/{item_id}")

def alias_get_{resource}_one(item_id: int):

 return get_item(item_id=item_id)

@app.put("/api/v1/{resource}/{item_id}")

def alias_update_{resource}(item_id: int, update: dict):

```

```

 return update_item(item_id=item_id, update=update)

@app.delete("/api/v1/{resource}/{item_id}")
def alias_delete_{resource}(item_id: int):
 return delete_item(item_id=item_id)

@app.get("/{resource}")
def alias_get_{resource}_short():
 return get_items()

@app.post("/{resource}")
def alias_create_{resource}_short(item: dict):
 return create_item(item=item)

@app.get("/{resource}/{item_id}")
def alias_get_{resource}_short_one(item_id: int):
 return get_item(item_id=item_id)

@app.put("/{resource}/{item_id}")
def alias_update_{resource}_short(item_id: int, update: dict):
 return update_item(item_id=item_id, update=update)

@app.delete("/{resource}/{item_id}")
def alias_delete_{resource}_short(item_id: int):
 return delete_item(item_id=item_id)
'''

?? FastAPI Backend Generator ???

from backend.core.llm_provider import llm

import json

async def _generate_schema_via_llm(task: str) -> list:

```

```

prompt = (
 "You are a database architect. Based on this app idea: '" + task + "'\n"
 "Return ONLY a JSON array of database fields needed for the main item. "
 "Each field MUST be a dictionary with 'name' (snake_case) and "
 "'type' (one of String, Integer, Float, DateTime). "
 "DO NOT include 'id' or 'created_at' (they are automatically added).\n"
 "Keep it simple (maximum 4-6 fields). Return ONLY valid JSON, no markdown.\n"
 "Example output: [{\"name\": \"title\", \"type\": \"String\"}, {\"name\": \"status\", \"type\": \"String\"}]"
)

raw = await llm.agenerate(prompt)
m = re.search(r"\[.*\]", raw, re.DOTALL)
if m:
 try:
 schema = json.loads(m.group())
 if isinstance(schema, list) and all(isinstance(x, dict) for x in schema):
 return schema
 except: pass
raise ValueError("LLM returned invalid schema format")

async def generate_backend_files(task: str, app_type: str) -> dict:
 if app_type == "frontend":
 return {}

 schema = []
 if app_type == "fullstack_db":
 schema = await _generate_schema_via_llm(task)

 files = {}
 slug = _slugify(task)
 title = _title(task)
 table_prefix = slug.replace("-", "_")[:40]
 resource = _detect_resource_name(task)

 files["main.py"] = _generate_main_py(task, title, app_type, table_prefix, resource,

```



```
"""
```

```
 items_data_var = ""
```

```
 extra_endpoints = ''
```

```
def _item_to_dict(item):
```

```
 """Serialize any SQLAlchemy Item to a plain dict, preserving types."""
```

```
 result = {}
```

```
 for col in Item.__table__.columns:
```

```
 val = getattr(item, col.name)
```

```
 result[col.name] = str(val) if hasattr(val, "isoformat") else val
```

```
 return result
```

```
@app.get("/api/v1/items")
```

```
def get_items(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
```

```
 items = db.query(Item).offset(skip).limit(limit).all()
```

```
 return {"items": [_item_to_dict(i) for i in items], "total": db.query(Item).count()}
```

```
@app.get("/api/v1/items/{item_id}")
```

```
def get_item(item_id: int, db: Session = Depends(get_db)):
```

```
 item = db.query(Item).filter(Item.id == item_id).first()
```

```
 if not item:
```

```
 raise HTTPException(status_code=404, detail="Item not found")
```

```
 return _item_to_dict(item)
```

```
@app.post("/api/v1/items")
```

```
def create_item(payload: dict, db: Session = Depends(get_db)):
```

```
 writable = {c.name for c in Item.__table__.columns if c.name not in ("id", "created_at")}
```

```
 item = Item()
```

```
 for key, val in payload.items():
```

```
 if key in writable:
```

```
 setattr(item, key, val)
```

```
 # Ensure title always has a value if it exists in writable
```

```
 if not getattr(item, "title", None) and "title" in writable:
```

```
 item.title = payload.get("title", "Untitled")
```

```
 db.add(item)
```

```
db.commit()

db.refresh(item)

return _item_to_dict(item)
```

```
@app.put("/api/v1/items/{item_id}")
```

```
def update_item(item_id: int, payload: dict, db: Session = Depends(get_db)):

 item = db.query(Item).filter(Item.id == item_id).first()

 if not item:

 raise HTTPException(status_code=404, detail="Item not found")

 writable = {c.name for c in Item.__table__.columns if c.name not in ("id", "created_at")}

 for key, val in payload.items():

 if key in writable:

 setattr(item, key, val)

 db.commit()

 db.refresh(item)

 return _item_to_dict(item)
```

```
@app.delete("/api/v1/items/{item_id}")
```

```
def delete_item(item_id: int, db: Session = Depends(get_db)):

 item = db.query(Item).filter(Item.id == item_id).first()

 if not item:

 raise HTTPException(status_code=404, detail="Item not found")

 db.delete(item)

 db.commit()

 return {"deleted": item_id}
```

```
@app.get("/api/v1/stats")
```

```
def get_stats(db: Session = Depends(get_db)):

 from sqlalchemy import func as sqlfunc

 total = db.query(sqlfunc.count(Item.id)).scalar()

 cols = [c.name for c in Item.__table__.columns]

 active = 0

 if "status" in cols:

 active = db.query(sqlfunc.count(Item.id)).filter(Item.status == "active").scalar()
```

```

 return {"total": total, "active": active}
'''

 extra_endpoints += _alias_routes(resource, use_db)
else:

 db_import = ""

 startup_event = ""

 items_data_var = 'items_data = []'

 extra_endpoints = ''

@app.get("/api/v1/items")

def get_items():

 return {"items": items_data, "total": len(items_data)}

@app.post("/api/v1/items")

def create_item(item: dict):

 item["id"] = len(items_data) + 1

 items_data.append(item)

 return item

@app.get("/api/v1/items/{item_id}")

def get_item(item_id: int):

 item = next((i for i in items_data if i["id"] == item_id), None)

 if not item:

 raise HTTPException(status_code=404, detail="Item not found")

 return item

@app.put("/api/v1/items/{item_id}")

def update_item(item_id: int, update: dict):

 for i, item in enumerate(items_data):

 if item["id"] == item_id:

 items_data[i].update(update)

 return items_data[i]

 raise HTTPException(status_code=404, detail="Item not found")

@app.delete("/api/v1/items/{item_id}")

```

```

def delete_item(item_id: int):

 global items_data

 items_data = [i for i in items_data if i["id"] != item_id]

 return {"deleted": item_id}

@app.get("/api/v1/stats")

def get_stats():

 return {"total": len(items_data), "active": sum(1 for i in items_data if i.get("status", "")
== "active")}]

'''

 extra_endpoints += _alias_routes(resource, use_db)

fastapi_import = (

 "from fastapi import FastAPI, HTTPException, Depends"

 if use_db

 else "from fastapi import FastAPI, HTTPException"

)

return f'''# main.py ? Generated by VIA for: {task}

{fastapi_import}

from fastapi.middleware.cors import CORSMiddleware

{db_import}

import os

app = FastAPI(

 title="{title}",

 description="Generated by VIA ? Autonomous AI Digital Team",

 version="1.0.0"

)

app.add_middleware(

 CORSMiddleware,

 allow_origins=["*"],

 allow_credentials=True,

 allow_methods=["*"],

```



```

 allow_headers=["*"],
)

{startup_event}

{items_data_var}

@app.get("/")

def root():

 return {"app": "{title}", "status": "running", "docs": "/docs", "api": "/api/v1/items"}}

@app.get("/health")

def health():

 return {"status": "healthy"}}

{extra_endpoints}

if __name__ == "__main__":

 import uvicorn

 port = int(os.getenv("PORT", 8000))

 uvicorn.run("main:app", host="0.0.0.0", port=port, reload=False)

'''

def _generate_requirements(app_type: str) -> str:

 base = """fastapi==0.115.0

uvicorn[standard]==0.29.0

pydantic==2.10.6

python-dotenv==1.0.1

httpx==0.27.0

"""

 if app_type == "fullstack_db":

 base += """sqlalchemy==2.0.29

psycopg2-binary==2.9.9

alembic==1.13.1

aiosqlite==0.20.0

```

```
python-multipart==0.0.9
```

```
passlib[bcrypt]==1.7.4
```

```
PyJWT==2.8.0
```

```
python-jose[cryptography]==3.3.0
```

```
"""
```

```
 return base
```

```
def _generate_render_yaml(slug: str, app_type: str) -> str:
```

```
 db_env = """
```

```
 - key: DATABASE_URL
```

```
 sync: false""" if app_type == "fullstack_db" else ""
```

```
 return f"""services:
```

```
- type: web
```

```
 name: {slug[:50]}
```

```
 env: python
```

```
 plan: free
```

```
 buildCommand: pip install -r requirements.txt
```

```
 startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
```

```
 envVars:
```

```
 - key: APP_ENV
```

```
 value: production{db_env}
```

```
"""
```

```
def _generate_gitignore() -> str:
```

```
 return '''
```

```
Environments
```

```
.env
```

```
.venv
```

```
env/
```

```
venv/
```

```
ENV/
```

env.bak/

venv.bak/

# Byte-compiled / optimized / DLL files

\_\_pycache\_\_/

\*.py[cod]

\*\$py.class

'''

\_LLM\_BACKEND\_PROMPT = """You are a Principal Backend Engineer.

Task: {task}

App Type: {app\_type}

Generate the full backend code for this app using FastAPI and SQLAlchemy (if app\_type is fullstack\_db).

Required files:

- main.py (FastAPI application with all necessary endpoints)

CRITICAL RULES:

- DO NOT generate `database.py`. Assume it already exists and provides `Base`, `engine`, and `get\_db`. Import them like: `from database import Base, engine, get\_db`.

- When defining SQLAlchemy models in Python, DO NOT use Pydantic types (like EmailStr) inside Column(). You MUST use SQLAlchemy types (like String, Integer). Example: `email = Column(String, unique=True)`.

- Use `from sqlalchemy import Column, String, Integer` etc.

- CRITICAL AUTH RULE: The frontend will send LOGIN and REGISTER requests as standard JSON (`application/json`) to `/api/v1/auth/login` and `/api/v1/auth/register`. DO NOT use `OAuth2PasswordRequestForm` (which requires form-data). Accept JSON via standard Pydantic models (e.g. `class LoginRequest(BaseModel): email: str, password: str`).

- CRITICAL ROOT ROUTE RULE: main.py MUST always include a root GET "/" route that returns JSON status. Example:

```
@app.get("/")
```

```
def root():
```

```
 return {"status": "running", "docs": "/docs"}
```

- CRITICAL HEALTH ROUTE RULE: main.py MUST always include a GET "/health" route:

```
@app.get("/health")
```

```
def health():
```

```
 return {"status": "healthy"}
```

Use standard markdown code blocks, e.g.,

```
```python
# main.py

...

```

"""

def _extract_llm_files(raw: str) -> dict:

 import re

 files = {}

 # Find all code blocks

 pattern = re.compile(r'```[a-zA-Z]*\n(.*?)```', re.DOTALL)

 for i, match in enumerate(pattern.finditer(raw)):

 content = match.group(1).strip()

 # Try to find the filename in the first line

 first_line = content.split('\n')[0].strip()

 filename = None

 if first_line.startswith('#') and '.' in first_line:

 filename = first_line.strip('# ').split()[0]

 elif "FastAPI(" in content or "from fastapi" in content:

 filename = "main.py"

 elif "sqlalchemy" in content and "declarative_base" in content:

 filename = "database.py"

 elif "sqlalchemy" in content and "Column" in content:

 filename = "models.py"

 elif "uvicorn" in content and "fastapi" in content:

 filename = "requirements.txt"

 else:

 filename = f"file_{i}.txt"

 if filename:

 files[filename] = content
```

```

Also look outside codeblocks for explicit markers like "***main.py**"

return files

def _validate_llm_backend(files: dict) -> bool:

 if "main.py" not in files:

 logger.warning("Validation failed: main.py not in files. Files found: " +
 str(list(files.keys()))))

 return False

 if "FastAPI" not in files["main.py"]:

 logger.warning("Validation failed: FastAPI not in main.py")

 return False

 return True

async def generate_backend_files_llm(task: str, app_type: str) -> dict:

 from backend.core.llm_provider import llm

 if app_type == "frontend":

 return {}

 logger.info(f"LLM Backend Generator | task={task[:60]} | app_type={app_type}")

 prompt = _LLM_BACKEND_PROMPT.format(task=task, app_type=app_type)

 raw = await llm.agenerate(prompt)

 files = _extract_llm_files(raw)

 if not _validate_llm_backend(files):

 raise ValueError("LLM Backend Generator failed validation. Prompt should be updated to
 ensure valid FastAPI output.")

 if "main.py" in files:

 files["main.py"] = files["main.py"].replace("sqlite+aiosqlite:///", "sqlite:///")

 files["main.py"] = files["main.py"].replace("create_async_engine", "create_engine")

 files["main.py"] = files["main.py"].replace("from sqlalchemy.ext.asyncio import", "# from
 sqlalchemy.ext.asyncio import")

 files["main.py"] = files["main.py"].replace("@app.on_startup()",
 '@app.on_event("startup")')

```

```

 files["main.py"] = files["main.py"].replace("async def on_startup():", "def
on_startup():")

 # Inject root route if LLM forgot to include it

 if '@app.get("/")' not in files["main.py"] and "@app.get('/')\" not in files["main.py"]:

 inject = (

 '\n\n@app.get("/")\n'

 'def root():\n'

 ' return {"status": "running", "docs": "/docs", "health": "/health"}\n'

 '\n@app.get("/health")\n'

 'def health():\n'

 ' return {"status": "healthy"}\n'

)

 # Insert after the last middleware/CORS setup, before first route

 import_end = files["main.py"].find('\napp.add_middleware')

 if import_end == -1:

 import_end = files["main.py"].find('\n@app.')

 if import_end > 0:

 files["main.py"] = files["main.py"][:import_end] + inject +
files["main.py"][import_end:]

 else:

 files["main.py"] = files["main.py"] + inject

 logger.info("Auto-injected root / and /health routes into generated main.py")

slug = _slugify(task)

files["requirements.txt"] = _generate_requirements(app_type)

files.setdefault("render.yaml", _generate_render_yaml(slug, app_type))

files.setdefault(".gitignore", _generate_gitignore())

files.setdefault(".python-version", "3.11.0\n")

if app_type == "fullstack_db":

 files["database.py"] = _generate_database_py()

if "requirements.txt" in files and app_type == "fullstack_db":

 if "psycopg2" not in files["requirements.txt"]:

 files["requirements.txt"] += "\npsycopg2-binary\n"

```

```

 logger.info(f"LLM Backend Generator | SUCCESS | {len(files)} files generated")

 return files

def _generate_database_py() -> str:
 return '''# database.py ? SQLAlchemy setup

import os

from sqlalchemy import create_engine

from sqlalchemy.ext.declarative import declarative_base

from sqlalchemy.orm import sessionmaker

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./app.db")

Normalize URL scheme for SQLAlchemy compatibility

if DATABASE_URL.startswith("postgres://"):

 DATABASE_URL = DATABASE_URL.replace("postgres://", "postgresql://", 1)

SQLite needs connect_args; PostgreSQL does not

connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}

engine = create_engine(DATABASE_URL, connect_args=connect_args)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():

 db = SessionLocal()

 try:

 yield db

 finally:

 db.close()

'''

```

[illegible]



```
def _slugify(text: str) -> str:
 text = text.lower().strip()
 text = re.sub(r"^\w\s-", "", text)
 text = re.sub(r"[\s_-]+", "-", text)
 return text[:50].strip("-") or "via-app"
```

```
def _title(text: str) -> str:
 text = re.sub(r"^\w\s", " ", text)
 return " ".join(w.capitalize() for w in text.split()[:6])
...

```

```
File: `backend\core\github_pusher.py`

```

```
```py

```

```
# backend/core/github_pusher.py

```

```
#

```

```
# ROOT CAUSE OF BLANK WHITE SCREEN:

```

```
# SKIP_PATTERNS had ".git" which matches ".github/workflows/deploy.yml"

```

```
# so the workflow file was NEVER pushed to GitHub.

```

```
# The old workflow (from a previous push) ran instead ? it deployed

```

```
# raw source files, not the built dist/ ? causing blank white screen.

```

```
#

```

```
# THE ONLY CHANGE: ".git" ? "/.git" in SKIP_PATTERNS

```

```
# "/.git" only matches the actual .git folder, NOT .github directories.

```

```
import os

```

```
import re

```

```
import base64

```

```
import requests

```

```
import logging

```

```
import time

```

```
from datetime import datetime

```

```
from requests.adapters import HTTPAdapter

from urllib3.util.retry import Retry


logger = logging.getLogger("AI-Digital-Company")


# FIX: "/.git" not ".git" ? .github/workflows/deploy.yml was being silently skipped
SKIP_PATTERNS = ["__pycache__", ".pyc", ".pyo", ".pyd", "/.git"]


CURRENT_PACKAGE_JSON = """{
  "name": "via-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0",
    "axios": "^1.6.0",
    "react-toastify": "^10.0.5",
    "react-hot-toast": "^2.4.1",
    "react-icons": "^5.0.1",
    "lucide-react": "^0.383.0",
    "date-fns": "^3.6.0",
    "react-hook-form": "^7.51.0",
    "clsx": "^2.1.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.0",
    "autoprefixer": "^10.4.16",
```

```

"postcss":            "^8.4.32",
"tailwindcss":        "^3.4.0",
"vite":               "^5.0.0"
}
}
"""

```

```

def _make_session() -> requests.Session:
    session = requests.Session()
    retry = Retry(total=5, backoff_factor=2, status_forcelist=[500, 502, 503, 504],
                    allowed_methods=["GET", "POST", "PUT", "PATCH", "DELETE"])
    adapter = HTTPAdapter(max_retries=retry)
    session.mount("https://", adapter)
    session.mount("http://", adapter)
    return session

```

```

class GitHubPusher:
    def __init__(self):
        self.token    = os.getenv("GITHUB_TOKEN", "")
        self.username = os.getenv("GITHUB_USERNAME", "")
        self.headers = {
            "Authorization": f"token {self.token}",
            "Accept":        "application/vnd.github+json",
            "Content-Type":  "application/json",
        }

    def _ok(self) -> bool:
        return bool(self.token and self.username)

    def _session(self) -> requests.Session:
        s = _make_session()
        s.headers.update(self.headers)

```

```
return s
```

```
def _create_repo(self, name: str, desc: str) -> dict:
```

```
    try:
```

```
        r = self._session().post(
            "https://api.github.com/user/repos",
            json={"name": name, "description": desc, "private": False, "auto_init": True},
            timeout=60,
        )
        if r.status_code == 201:
            return {"success": True, "url": r.json()["html_url"]}
        if r.status_code == 422:
            return {"success": True, "url": f"https://github.com/{self.username}/{name}"}
        return {"success": False, "error": r.json().get("message", f"HTTP {r.status_code}")}
    except Exception as e:
        return {"success": False, "error": str(e)}
```

```
def _get_head_sha(self, repo: str) -> str:
```

```
    for branch in ["main", "master"]:
```

```
        try:
```

```
            r = self._session().get(
```

```
f"https://api.github.com/repos/{self.username}/{repo}/git/refs/heads/{branch}",
```

```
            timeout=60,
```

```
        )
```

```
        if r.status_code == 200:
```

```
            return r.json().get("object", {}).get("sha", "")
```

```
    except Exception as e:
```

```
        logger.warning(f"Get HEAD SHA error on {branch}: {e}")
```

```
    return ""
```

```
def _create_branch(self, repo: str, branch_name: str, sha: str):
```

```
    try:
```

```
        r = self._session().post(
```

```
f"https://api.github.com/repos/{self.username}/{repo}/git/refs",
```

```

        json={"ref": f"refs/heads/{branch_name}", "sha": sha},
        timeout=30,
    )

    if r.status_code == 201:

        logger.info(f"Created branch {branch_name} | {repo}")

    elif r.status_code == 422:

        logger.info(f"Branch {branch_name} already exists | {repo}")

except Exception as e:

    logger.warning(f"Branch create failed: {e}")


def _enable_pages(self, repo: str):

    try:

        time.sleep(5)

        session = self._session()

        # Try to enable Pages with GitHub Actions as the build source

        r = session.post(

            f"https://api.github.com/repos/{self.username}/{repo}/pages",

            json={"build_type": "workflow"},

            timeout=30,

        )

        if r.status_code in (201, 409):

            logger.info(f"GitHub Pages enabled (Actions source) | {repo}")

            return

        # If already enabled, update it to use Actions source

        if r.status_code in (422,):

            r2 = session.put(

                f"https://api.github.com/repos/{self.username}/{repo}/pages",

                json={"build_type": "workflow"},

                timeout=30,

            )

            if r2.status_code in (200, 204):

                logger.info(f"GitHub Pages updated to Actions source | {repo}")

                return

            logger.warning(f"Pages enable response {r.status_code} | {repo} |
body={r.text[:200]}")

```

```
except Exception as e:
```

```
    logger.warning(f"Pages enable failed: {e}")
```

```
def _set_repo_variable(self, repo: str, name: str, value: str):
```

```
    try:
```

```
        r = self._session().post(
```

```
            f"https://api.github.com/repos/{self.username}/{repo}/actions/variables",
```

```
            json={"name": name, "value": value},
```

```
            timeout=30,
```

```
        )
```

```
        if r.status_code in (201, 204):
```

```
            logger.info(f"Repo variable created | {name}={value} | {repo}")
```

```
            return
```

```
        if r.status_code in (409, 422):
```

```
            r2 = self._session().patch(
```

```
                f"https://api.github.com/repos/{self.username}/{repo}/actions/variables/{name}",
```

```
                json={"name": name, "value": value},
```

```
                timeout=30,
```

```
            )
```

```
            if r2.status_code in (200, 201, 204):
```

```
                logger.info(f"Repo variable updated | {name}={value} | {repo}")
```

```
    except Exception as e:
```

```
        logger.warning(f"Repo variable set failed: {e}")
```

```
def _trigger_workflow(self, repo: str):
```

```
    try:
```

```
        session = self._session()
```

```
        r = session.get(
```

```
            f"https://api.github.com/repos/{self.username}/{repo}/git/refs/heads/main",
```

```
            timeout=30,
```

```
        )
```

```
        if r.status_code != 200:
```

```
            return
```

```

head_sha = r.json().get("object", {}).get("sha", "")

if not head_sha:

    return

readme_r = session.get(

    f"https://api.github.com/repos/{self.username}/{repo}/contents/README.md",

    timeout=30,

)

timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

if readme_r.status_code == 200:

    current_content = base64.b64decode(readme_r.json()["content"]).decode("utf-8")

    file_sha = readme_r.json()["sha"]

    if "<!-- VIA deploy trigger:" in current_content:

        new_content = re.sub(

            r'<!-- VIA deploy trigger:.*?-->',

            f'<!-- VIA deploy trigger: {timestamp} -->',

            current_content

        )

    else:

        new_content = current_content.rstrip() + f"\n\n<!-- VIA deploy trigger:

{timestamp} -->\n"

    put_r = session.put(

        f"https://api.github.com/repos/{self.username}/{repo}/contents/README.md",

        json={

            "message": f"VIA: trigger deploy [{timestamp}]",

            "content": base64.b64encode(new_content.encode()).decode(),

            "sha": file_sha,

        },

        timeout=30,

    )

    if put_r.status_code in (200, 201):

        logger.info(f"Workflow triggered via commit | {repo}")

    else:

        new_content = f"# {repo}\n\nGenerated by VIA.\n\n<!-- VIA deploy trigger:

```

```
{timestamp} -->\n"
```

```
        put_r = session.put(
            f"https://api.github.com/repos/{self.username}/{repo}/contents/README.md",
            json={
                "message": f"VIA: trigger deploy [{timestamp}]",
                "content": base64.b64encode(new_content.encode()).decode(),
            },
            timeout=30,
        )
        if put_r.status_code in (200, 201):
            logger.info(f"Workflow triggered via new README commit | {repo}")
except Exception as e:
    logger.warning(f"Workflow trigger error: {e}")
```

```
def _push_all(self, repo: str, files: dict, message: str) -> dict:
```

```
    head_sha = self._get_head_sha(repo)
```

```
    if not head_sha:
```

```
        logger.info(f"Repo not initialized yet. Manually initializing {repo}...")
```

```
    try:
```

```
        r = self._session().put(
            f"https://api.github.com/repos/{self.username}/{repo}/contents/README.md",
            json={
                "message": "Initial commit",
                "content": base64.b64encode(b"# VIA App\n").decode()
            },
            timeout=30,
        )
```

```
        if r.status_code in (200, 201):
```

```
            head_sha = r.json().get("commit", {}).get("sha", "")
```

```
            logger.info(f"Manual initialization successful. SHA: {head_sha}")
```

```
except Exception as e:
```

```
    logger.warning(f"Manual init failed: {e}")
```



```

# Fallback retry just in case it was created concurrently
if not head_sha:
    for attempt in range(5):
        time.sleep(2)

        head_sha = self._get_head_sha(repo)

        if head_sha:
            break

        logger.info(f"Waiting for concurrent repo init... attempt {attempt+1}/5")

if not head_sha:
    logger.error(f"HEAD SHA not found after manual init for repo: {repo}")
    return {"success": False, "error": "Cannot get HEAD SHA - repo initialization failed"}

session = self._session()
tree_items = []
for path, content in files.items():
    for attempt in range(3):
        try:
            br = session.post(
                f"https://api.github.com/repos/{self.username}/{repo}/git/blobs",
                json={"content": base64.b64encode(content.encode("utf-8")).decode(),
"encoding": "base64"},
                timeout=60,
            )

            if br.status_code == 201:
                tree_items.append({"path": path, "mode": "100644", "type": "blob", "sha":
br.json()["sha"]})

                break

        except Exception:
            if attempt < 2:
                time.sleep(3)

if not tree_items:
    return {"success": False, "error": "No blobs created"}

```

```

pushed_paths = [t["path"] for t in tree_items]

workflow_ok = any(".github/workflows" in p for p in pushed_paths)

logger.info(f"Blobs created | {len(tree_items)} files | deploy.yml_included={workflow_ok}
| {repo}")

try:

    cr = session.get(

        f"https://api.github.com/repos/{self.username}/{repo}/git/commits/{head_sha}",

        timeout=60,

    )

    base_tree = cr.json()["tree"]["sha"]

    tr = session.post(

        f"https://api.github.com/repos/{self.username}/{repo}/git/trees",

        json={"base_tree": base_tree, "tree": tree_items},

        timeout=60,

    )

    new_tree = tr.json()["sha"]

    co = session.post(

        f"https://api.github.com/repos/{self.username}/{repo}/git/commits",

        json={"message": message, "tree": new_tree, "parents": [head_sha]},

        timeout=60,

    )

    new_commit = co.json()["sha"]

    session.patch(

        f"https://api.github.com/repos/{self.username}/{repo}/git/refs/heads/main",

        json={"sha": new_commit, "force": True},

        timeout=60,

    )

except Exception as e:

    return {"success": False, "error": str(e)}

logger.info(f"Single-commit push OK | {len(pushed_paths)} files | {repo}")

return {"success": True, "pushed": pushed_paths, "commit": new_commit}

def push_project(self, task: str, dept_path: str, repo_name: str = "", extra_files: dict =

```

None) -> dict:

```
    if not self._ok():
        return {"phase5_ran": False, "error": "GITHUB_TOKEN or GITHUB_USERNAME not set"}

    if not os.path.exists(dept_path):
        return {"phase5_ran": False, "error": f"Path not found: {dept_path}"}

    if not repo_name:
        repo_name = _slugify(task)

    repo = self._create_repo(repo_name, f"Generated by VIA: {task[:100]}")
    if not repo["success"]:
        return {"phase5_ran": False, "error": repo.get("error")}

    repo_url = repo["url"]

    files = {}
    for root, dirs, fnames in os.walk(dept_path):
        dirs[:] = [d for d in dirs if not any(s in ("/" + d) for s in SKIP_PATTERNS)]
        for fname in fnames:
            full = os.path.join(root, fname)
            rel = os.path.relpath(full, dept_path).replace("\\", "/")
            if any(s in ("/" + rel) for s in SKIP_PATTERNS):
                continue
            try:
                with open(full, "r", encoding="utf-8", errors="ignore") as f:
                    files[rel] = f.read()
            except Exception as e:
                logger.error(f"Read {rel}: {e}")

    if extra_files:
        files.update(extra_files)

    if "README.md" not in files:
```

```

files["README.md"] = _fallback_readme(task, repo_name)

files["package.json"] = CURRENT_PACKAGE_JSON

logger.info(f"Phase 5 | package.json locked to current version | {repo_name}")

for vite_cfg in ["vite.config.js", "vite.config.ts"]:
    if vite_cfg in files:
        files[vite_cfg] = _patch_vite_config(files[vite_cfg], repo_name)
        break
    else:
        files["vite.config.js"] = _default_vite_config(repo_name)

files = _fix_api_exports(files)
files = _fix_nested_routers(files)

if ".github/workflows/deploy.yml" not in files:
    logger.warning(f"deploy.yml missing ? injecting directly | {repo_name}")
    files[".github/workflows/deploy.yml"] = _deploy_workflow()

    logger.info(f"Phase 5 | {len(files)} files | workflow={'YES' if
'.github/workflows/deploy.yml' in files else 'MISSING'} | {repo_name}")

result = self._push_all(
    repo_name, files,
    f"VIA generated project ? {datetime.now().strftime('%Y-%m-%d %H:%M')}"
)

if result["success"]:
    pushed = result.get("pushed", list(files.keys()))
    commit_sha = result.get("commit")
    placeholder = f"https://{repo_name}.onrender.com"

    self._enable_pages(repo_name)
    self._set_repo_variable(repo_name, "VITE_API_URL", placeholder)
    self._trigger_workflow(repo_name)

```

```
    return {  
        "phase5_ran":    True,  
        "success":       True,  
        "repo_url":      repo_url,  
        "repo_name":     repo_name,  
        "files_pushed":  pushed,  
        "render_url":    placeholder,  
    }
```

```
    return {"phase5_ran": False, "error": result.get("error", "Push failed")}
```

```
def _deploy_workflow() -> str:
```

```
    return """name: Deploy React to GitHub Pages
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  workflow_dispatch:
```

```
permissions:
```

```
  contents: read
```

```
  pages: write
```

```
  id-token: write
```

```
concurrency:
```

```
  group: pages
```

```
  cancel-in-progress: true
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- name: Checkout
uses: actions/checkout@v4
- name: Setup Node.js
uses: actions/setup-node@v4
with:
 - node-version: '20'
 - cache: 'npm'
- name: Install dependencies
run: npm install
- name: Build
run: npm run build
env:
 - CI: "false"
 - VITE_API_URL: \${vars.VITE_API_URL }
- name: Setup Pages
uses: actions/configure-pages@v5
- name: Upload artifact
uses: actions/upload-pages-artifact@v3
with:
 - path: ./dist

deploy:

environment:

name: github-pages

url: \${steps.deployment.outputs.page_url }

runs-on: ubuntu-latest

needs: build

steps:

- name: Deploy to GitHub Pages

```
id: deployment
uses: actions/deploy-pages@v4
```

```
"""
```

```
def _patch_vite_config(content: str, repo_name: str = "") -> str:
    correct_base = f"/{repo_name}/" if repo_name else "/"
    existing_base = re.search(r'base\s*:\s*"([^"]+)"', content)
    if existing_base:
        b = existing_base.group(1)
        if b not in (".", "/", ""):
            return content
        return re.sub(r'base\s*:\s*"([^"]+)"', f'base: "{correct_base}"', content,
count=1)
    if "defineConfig" in content:
        return re.sub(r'(defineConfig\s*\(\s*\{)', f'\1\n    base: "{correct_base}"', content,
count=1)
    return _default_vite_config(repo_name) + "\n// Original:\n" + content
```

```
def _default_vite_config(repo_name: str = "") -> str:
    base = f"/{repo_name}/" if repo_name else "/"
    return f'''import {{ defineConfig }} from "vite";
import react from "@vitejs/plugin-react";
```

```
export default defineConfig({{
    plugins: [react()],
    base: "{base}",
    build: {{ outDir: "dist", assetsDir: "assets" }},
    server: {{ port: 3000 }},
}});
'''
```

```
def _fix_nested_routers(files: dict) -> dict:
```

```

    main_key = next((k for k in files if k.endswith("main.jsx") or k.endswith("main.js") or
k.endswith("main.tsx")), None)

    app_key = next((k for k in files if k.endswith("App.jsx") or k.endswith("App.js") or
k.endswith("App.tsx")), None)

    if not main_key or not app_key:

        return files

    main_content = files[main_key]
    app_content = files[app_key]

    has_router_main = "<BrowserRouter" in main_content or "<HashRouter" in main_content
    has_router_app = "<BrowserRouter" in app_content or "<HashRouter" in app_content

    if has_router_main and has_router_app:

        logger.info("Auto-fixing nested React routers in App.jsx")

        app_content = re.sub(r'<BrowserRouter[^>]*>', '<div className="app-wrapper">',
app_content)

        app_content = app_content.replace('</BrowserRouter>', '</div>')

        app_content = re.sub(r'<HashRouter[^>]*>', '<div className="app-wrapper">', app_content)

        app_content = app_content.replace('</HashRouter>', '</div>')

        files[app_key] = app_content

    return files

def _fix_api_exports(files: dict) -> dict:

    api_key = next((k for k in files if k.endswith("api.js")), None)

    if not api_key:

        return files

    api_content = files[api_key]

    # ?? Safety net: if the base api.js is fundamentally broken, replace it ??

    # The LLM sometimes emits a stub that references axios/BASE_URL without

    # importing/declaring them. Appending export stubs to that file still

```



```

# crashes at runtime with "ReferenceError: axios is not defined".

if ("import axios" not in api_content or "import.meta.env" not in api_content):
    logger.warning(f"api.js is missing axios import or BASE_URL ? replacing with safe
fallback")
    api_content = '''import axios from "axios";

const BASE_URL =
    import.meta.env.VITE_API_URL ||
    (typeof window !== "undefined" &&
    (window.location.hostname === "localhost" || window.location.hostname === "127.0.0.1")
    ? "http://localhost:8000"
    : "");

const api = axios.create({
    baseURL: `${BASE_URL}/api/v1`,
    headers: { "Content-Type": "application/json" },
    timeout: 30000,
});

api.interceptors.response.use(
    (res) => res,
    (err) => {
        console.error("API error:", err.response?.status, err.config?.url);
        return Promise.reject(err);
    }
);

export const getItems    = (params = {}) => api.get("/items",      { params });
export const getItem     = (id)         => api.get(`/items/${id}`);
export const createItem  = (data)        => api.post("/items",    data);
export const updateItem  = (id, data)    => api.put(`/items/${id}`, data);
export const deleteItem  = (id)          => api.delete(`/items/${id}`);
export const getStats    = ()            => api.get("/stats");

export default api;

```

```

'''

    files[api_key] = api_content

    return files

pattern      = re.compile(r'import\s*\{([^\}]+\)}\s*from\s*["\'](?:\.\\\.?/*)*api(?:\.js)?["\']',
re.DOTALL)

needed      = set()

for path, content in files.items():

    if not path.endswith(("jsx", ".js", ".tsx")) or path == api_key:

        continue

    for match in pattern.finditer(content):

        names = [n.strip().split(" as ")[0].strip() for n in match.group(1).split(",")]

        needed.update(filter(None, names))

if not needed:

    return files

existing      = set(re.findall(r'export\s+(?:const|function|async\s+function)\s+(\w+)',
api_content))

missing      = needed - existing

# FIX: BASE_URL is a variable declaration, not an export ? removing it

# from missing prevents duplicate declaration build crash in GitHub Actions

missing.discard("BASE_URL")

if not missing:

    return files

logger.info(f"Auto-adding missing exports: {missing}")

base_url = "${import.meta.env.VITE_API_URL || ''}"

stubs      = "\n\n// Auto-generated missing exports by VIA\n"

def _pl(w):

    if not w: return "items"

```

```

    if w.endswith("s"): return w

    if w.endswith("y"): return w[:-1] + "ies"

    return w + "s"

for name in sorted(missing):

    if name in ("getStats", "getStatistics"):

        stubs += f'export const {name} = async () => {{ const r = await
fetch(`{base_url}/api/v1/stats`); if (!r.ok) throw new Error("{name} failed"); return r.json();
}};\n'

    elif name.startswith("get"):

        res = _pl(name[3:].lower())

        stubs += f'export const {name} = async (p) => {{ const q = p ? "?" + new
URLSearchParams(p) : ""; const r = await fetch(`{base_url}/api/v1/{res}${{q}}`); if (!r.ok) throw
new Error("{name} failed"); return r.json(); }};\n'

    elif name.startswith("create") or name.startswith("add"):

        res = _pl(name[6:].lower() if name.startswith("create") else name[3:].lower())

        stubs += f'export const {name} = async (d) => {{ const r = await
fetch(`{base_url}/api/v1/{res}`,
{{method:"POST",headers:{{"Content-Type":"application/json"}},body:JSON.stringify(d)}}); if
(!r.ok) throw new Error("{name} failed"); return r.json(); }};\n'

    elif name.startswith("update") or name.startswith("edit"):

        res = _pl(name[6:].lower() if name.startswith("update") else name[4:].lower())

        stubs += f'export const {name} = async (id,d) => {{ const r = await
fetch(`{base_url}/api/v1/{res}/${{id}}`,
{{method:"PUT",headers:{{"Content-Type":"application/json"}},body:JSON.stringify(d)}}); if (!r.ok)
throw new Error("{name} failed"); return r.json(); }};\n'

    elif name.startswith("delete") or name.startswith("remove"):

        res = _pl(name[6:].lower())

        stubs += f'export const {name} = async (id) => {{ const r = await
fetch(`{base_url}/api/v1/{res}/${{id}}`, {{method:"DELETE"}}); if (!r.ok) throw new Error("{name}
failed"); return r.json(); }};\n'

    else:

        res = _pl(name.lower())

        stubs += f'export const {name} = async () => {{ const r = await
fetch(`{base_url}/api/v1/{res}`); if (!r.ok) throw new Error("{name} failed"); return r.json();
}};\n'

files[api_key] = api_content.rstrip() + stubs

return files

```

```

def _slugify(text: str) -> str:
    import uuid

    text = text.lower().strip()

    text = re.sub(r"[\w\s-]", "", text)

    text = re.sub(r"[\s_-]+", "-", text)

    base = text[:80].strip("-") or "via-app"

    return f"{base}-{uuid.uuid4().hex[:6]}"


def _fallback_readme(task: str, repo: str) -> str:
    now = datetime.now().strftime('%Y-%m-%d %H:%M')

    return (f"# {repo}\n> Generated by **VIA**\n\n## Task\n{task}\n\n"
           f"### Frontend\nGitHub Pages\n\n## Backend\nRender\n\n"
           f"---\n*Generated by VIA on {now}*\n")


github_pusher = GitHubPusher()
...


## File: `backend\core\groq_provider.py`


```py
import os

import logging

import asyncio

from groq import Groq

logger = logging.getLogger("AI-Digital-Company")

class GroqProvider:
 def __init__(self):
 self.api_key = os.getenv("GROQ_API_KEY")

 self.model = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")

 self.client = Groq(api_key=self.api_key)

```

```
logger.info(f"GroqProvider ready | model={self.model}")
```

```
async def generate(self, prompt: str, timeout: int = 120) -> str:
```

```
 max_attempts = 3
```

```
 for attempt in range(1, max_attempts + 1):
```

```
 try:
```

```
 loop = asyncio.get_event_loop()
```

```
 response = await loop.run_in_executor(
```

```
 None,
```

```
 lambda: self.client.chat.completions.create(
```

```
 model=self.model,
```

```
 messages=[{"role": "user", "content": prompt}],
```

```
 max_tokens=4096,
```

```
 temperature=0.7,
```

```
)
```

```
)
```

```
 result = response.choices[0].message.content
```

```
 logger.info(f"Groq ok | attempt={attempt}")
```

```
 return result
```

```
 except Exception as e:
```

```
 logger.warning(f"Groq error attempt {attempt}: {e}")
```

```
 if attempt == max_attempts:
```

```
 logger.warning("Groq failed all attempts -- using fallback")
```

```
 return ""
```

```
 await asyncio.sleep(2 ** attempt)
```

```
 return ""
```

```
...
```

```
File: `backend\core\hierarchy.py`
```

```
```py
```

```
# backend/core/hierarchy.py
```

```
ORG_CHART = {
```

```
    "CEO": {"title": "Chief Executive Officer", "role": "Strategic decision engine. Analyzes  
tasks, selects departments, sets company direction.", "reports_to": None, "manages":
```

```

["backend","security","devops","ai_research","architecture"]},

    "backend":          {"title": "Backend Engineering Department",      "role": "APIs, databases,
services, backend architecture.", "reports_to": "CEO", "manages": []},

    "security":         {"title": "Security Department",                "role": "Threat modeling, auth,
encryption, compliance.",      "reports_to": "CEO", "manages": []},

    "devops":           {"title": "DevOps & Infrastructure Department",  "role": "Infrastructure,
CI/CD, scaling, monitoring.",  "reports_to": "CEO", "manages": []},

    "ai_research":      {"title": "AI Research Department",              "role": "LLM strategy, model
optimization, AI roadmap.",    "reports_to": "CEO", "manages": []},

    "architecture":    {"title": "System Architecture Department",      "role": "System design, data
flow, resilience strategy.",   "reports_to": "CEO", "manages": []}

}

```

```

def get_active_structure(selected):

    return {

        "CEO": {"title":  ORG_CHART["CEO"]["title"], "role":  ORG_CHART["CEO"]["role"],
"active_departments": selected},

        "departments": {d: {"title":  ORG_CHART[d]["title"], "role":  ORG_CHART[d]["role"],
"reports_to": "CEO"} for d in selected if d in ORG_CHART}

    }

```

```

def get_full_chart(): return ORG_CHART

```

```

...

```

```

## File: `backend\core\intent_detector.py`

```

```

```py

```

```

backend/core/intent_detector.py ? VIA Phase 5: 3-Mode Intent Detection

```

```

Classifies user messages into: 'chat', 'build', or 'analyze'

```

```

import logging

```

```

logger = logging.getLogger("AI-Digital-Company")

```

```

def detect_intent(message: str) -> str:

```

```

 """

```

Detect user intent from their message.

Returns: 'chat', 'build', or 'analyze'

"""

```
msg = message.lower().strip()
```

```
Very short messages are almost always chat
```

```
if len(msg) < 8:
```

```
 return "chat"
```

```
?? Strong BUILD signals ???
```

```
strong_build = [
```

```
 "build me", "create me", "make me", "develop a", "develop me",
```

```
 "build a", "create a", "make a", "generate a", "generate me",
```

```
 "i want to build", "i need an app", "build an app", "create an app",
```

```
 "make an app", "build an application", "create an application",
```

```
 "deploy a", "deploy an", "launch a", "launch an",
```

```
 "start a project", "write an app", "code me", "code a",
```

```
 "i want an app", "i want a website", "i want a platform",
```

```
 "build this", "create this", "make this",
```

```
]
```

```
if any(phrase in msg for phrase in strong_build):
```

```
 logger.info(f"Intent: BUILD (strong signal) | {msg[:60]}")
```

```
 return "build"
```

```
?? Strong ANALYZE signals ???
```

```
strong_analyze = [
```

```
 "analyze my", "analyze the", "analyze this",
```

```
 "give me a business plan", "create a business plan",
```

```
 "security audit", "security review", "security analysis",
```

```
 "give me a plan", "create a plan", "make a plan",
```

```
 "what tech stack", "recommend a tech stack",
```

```
 "create a hiring plan", "hiring plan for",
```

```
 "evaluate my", "assess my", "review my",
```

```
]
```

```
if any(phrase in msg for phrase in strong_analyze):

 logger.info(f"Intent: ANALYZE (strong signal) | {msg[:60]}")

 return "analyze"

?? Keyword scoring ??

build_keywords = [

 "build", "create", "make", "develop", "generate", "code",

 "application", "app", "website", "platform", "system",

 "tool", "dashboard", "api", "backend", "frontend",

 "deploy", "launch", "project", "write an app",

 "todo", "tracker", "management", "portal",

]

analyze_keywords = [

 "analyze", "analysis", "plan", "strategy", "review",

 "assess", "evaluate", "recommend", "suggest", "advise",

 "business plan", "roadmap", "architecture", "design",

 "security audit", "tech stack", "budget", "hiring",

 "competitive", "market research", "feasibility",

 "risk assessment", "cost analysis", "roi",

]

build_score = sum(1 for kw in build_keywords if kw in msg)

analyze_score = sum(1 for kw in analyze_keywords if kw in msg)

Need at least 2 keyword hits to avoid false positives

if build_score >= 2 and build_score > analyze_score:

 logger.info(f"Intent: BUILD (score={build_score}) | {msg[:60]}")

 return "build"

if analyze_score >= 2 and analyze_score > build_score:

 logger.info(f"Intent: ANALYZE (score={analyze_score}) | {msg[:60]}")

 return "analyze"
```



```

 # Default: chat mode

 logger.info(f"Intent: CHAT (default) | {msg[:60]}")

 return "chat"

'''

File: `backend\core\inter_agent_bus.py`

'''py
backend/core/inter_agent_bus.py

Phase 2: Inter-Agent Communication Bus

#

Enables departments to share context with each other BEFORE final output.

Example flow:

- backend finishes ? shares API design summary ? security reads it
- security finishes ? shares threat model ? devops reads it
- architecture finishes ? shares system design ? backend refines
#

This makes the final output coherent, not just isolated reports.

from .logger import logger

class InterAgentBus:
 """
 Shared context store for agent-to-agent communication.
 Agents deposit summaries, other agents read them before generating output.
 """

 # Defines which agents read whose output
 # Format: {consumer: [producers it depends on]}
 DEPENDENCIES = {
 "security": ["backend"],
 "devops": ["backend", "architecture"],
 }

```

```

 "ai_research": ["architecture", "backend"],
 "architecture": ["backend"],
 "backend": []
}

```

```

def __init__(self):

```

```

 self._context: dict[str, str] = {}

```

```

def deposit(self, agent: str, summary: str):

```

```

 """Agent deposits a context summary after completing."""

```

```

 self._context[agent] = summary

```

```

 logger.info(f"InterAgentBus | deposit from: {agent} ({len(summary)} chars)")

```

```

def get_context_for(self, agent: str) -> str:

```

```

 """

```

```

 Returns relevant context from upstream agents.

```

```

 Only includes agents this agent depends on.

```

```

 """

```

```

 deps = self.DEPENDENCIES.get(agent, [])

```

```

 parts = []

```

```

 for dep in deps:

```

```

 if dep in self._context:

```

```

 parts.append(f"[{dep.upper()} context]: {self._context[dep]}")

```

```

 if not parts:

```

```

 return ""

```

```

 result = "\n".join(parts)

```

```

 logger.info(f"InterAgentBus | {agent} received context from: {[d for d in deps if d in self._context]}")

```

```

 return result

```

```

def has_context(self, agent: str) -> bool:

```

```

 return any(dep in self._context for dep in self.DEPENDENCIES.get(agent, []))

```

```

def get_all(self) -> dict:

```

```

 return dict(self._context)

```

```

def clear(self):
 self._context.clear()

def extract_summary(output: dict, agent_name: str) -> str:
 """
 Extract a concise summary from an agent's output for inter-agent sharing.
 """
 if not output or not isinstance(output, dict):
 return ""

 summary_parts = []

 if agent_name == "backend":
 arch = output.get("architecture", "")
 db = output.get("database", {})
 api = output.get("api_design", {})

 if arch: summary_parts.append(f"Architecture: {str(arch)[:100]}")
 if isinstance(db, dict): summary_parts.append(f"DB: {db.get('primary', '')[:80]}")
 if isinstance(api, dict): summary_parts.append(f"API style: {api.get('style', '')[:80]}")

 elif agent_name == "security":
 threats = output.get("threat_model", {})
 auth = output.get("authentication", {})

 if isinstance(threats, dict): summary_parts.append(f"Top threats: {str(threats.get('top_threats', ''))[:100]}")
 if isinstance(auth, dict): summary_parts.append(f"Auth: {auth.get('strategy', '')[:80]}")

 elif agent_name == "architecture":
 pattern = output.get("design_pattern", {})
 flow = output.get("data_flow", {})

 if isinstance(pattern, dict): summary_parts.append(f"Pattern: {pattern.get('primary', '')[:100]}")
 if isinstance(flow, dict): summary_parts.append(f"Flow: {flow.get('ingestion', '')[:80]}")

```

```

elif agent_name == "devops":

 infra = output.get("infrastructure", {})

 cicd = output.get("ci_cd", {})

 if isinstance(infra, dict):
 summary_parts.append(f"Cloud: {infra.get('cloud_provider', '')[:80]}")

 if isinstance(cicd, dict):
 summary_parts.append(f"CI/CD: {cicd.get('pipeline_tool', '')[:80]}")

elif agent_name == "ai_research":

 model = output.get("model_strategy", {})

 if isinstance(model, dict):
 summary_parts.append(f"Model: {model.get('primary_model', '')[:100]}")

return " | ".join(summary_parts) if summary_parts else str(output)[:200]

...

```

## File: `backend\core\llm\_provider.py`

```

```py
# backend/core/llm_provider.py ? Groq + Gemini + Ollama support
# Strategy: Groq for all planning/logic agents (fast, 30 RPM free)
#           Gemini for frontend agent only (quality UI code)

import asyncio, requests, time, os

from .config import MODEL_NAME, OLLAMA_URL, REQUEST_TIMEOUT, LLM_MAX_RETRIES, LLM_RETRY_DELAY
from .logger import logger

USE_GROQ = os.getenv("USE_GROQ", "false").lower() == "true"
GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")
GROQ_MODEL = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")
GROQ_MAX_TOK = int(os.getenv("GROQ_MAX_TOKENS", "4000"))
GROQ_TEMP = float(os.getenv("GROQ_TEMPERATURE", "0.7"))
GROQ_URL = "https://api.groq.com/openai/v1/chat/completions"

USE_ANTHROPIC = os.getenv("USE_ANTHROPIC", "false").strip().lower() == "true"

```

```

ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY", "").strip()

ANTHROPIC_MODEL    = os.getenv("ANTHROPIC_MODEL", "claude-3-5-sonnet-20240620").strip()

ANTHROPIC_URL      = "https://api.anthropic.com/v1/messages"


USE_GEMINI         = os.getenv("USE_GEMINI", "false").strip().lower() == "true"

GEMINI_API_KEY     = os.getenv("GEMINI_API_KEY", "").strip()

GEMINI_MODEL       = os.getenv("GEMINI_MODEL", "gemini-2.5-flash").strip()

GEMINI_URL         = "https://generativelanguage.googleapis.com/v1beta/openai/chat/completions"


import itertools

_groq_keys = [v for k, v in os.environ.items() if k.startswith("GROQ_API_KEY") and v.strip()]

if not _groq_keys and GROQ_API_KEY:
    _groq_keys = [GROQ_API_KEY]

if not _groq_keys:
    _groq_keys = [""]

key_iterator = itertools.cycle(_groq_keys)


class LLMPProvider:
    def __init__(self):
        self.model      = MODEL_NAME

        self.url        = OLLAMA_URL

        self.timeout     = REQUEST_TIMEOUT

        self.max_retries = LLM_MAX_RETRIES

        self.retry_delay = LLM_RETRY_DELAY

        self.use_groq    = USE_GROQ

        self.use_anthropic = USE_ANTHROPIC

        self.use_gemini  = USE_GEMINI


        if self.use_gemini:
            logger.info(f"LLM Provider | Gemini | model={GEMINI_MODEL}")

        elif self.use_anthropic:
            logger.info(f"LLM Provider | Anthropic | model={ANTHROPIC_MODEL}")

        elif self.use_groq:

```

```

        logger.info(f"LLM Provider | Groq | model={GROQ_MODEL}")
    else:
        logger.info(f"LLM Provider | Ollama | model={self.model}")

def _generate_groq(self, prompt: str) -> str:
    attempt = 0
    while attempt <= self.max_retries:
        try:
            start = time.time()
            current_key = next(key_iterator)
            r = requests.post(
                GROQ_URL,
                headers={
                    "Authorization": f"Bearer {current_key}",
                    "Content-Type": "application/json",
                },
                json={
                    "model": GROQ_MODEL,
                    "messages": [{"role": "user", "content": prompt}],
                    "max_tokens": GROQ_MAX_TOK,
                    "temperature": GROQ_TEMP,
                },
                timeout=60,
            )
            r.raise_for_status()
            content = r.json()["choices"][0]["message"]["content"]
            logger.info(f"LLM ok | Groq | attempt={attempt+1} | {round(time.time()-start,2)}s")
            return content
        except Exception as e:
            attempt += 1
            logger.warning(f"Groq error attempt {attempt}: {e}")
            if attempt <= self.max_retries:
                delay = self.retry_delay * (2 ** (attempt - 1))
                if hasattr(e, "response") and e.response is not None and

```

```

e.response.status_code == 429:

    delay = max(delay, 5) # Minimum 5s backoff for 429

    time.sleep(delay)

return ""

def _generate_ollama(self, prompt: str) -> str:

    attempt = 0

    while attempt <= self.max_retries:

        try:

            start = time.time()

            r = requests.post(

                self.url,

                json={"model": self.model, "prompt": prompt, "stream": False},

                timeout=self.timeout,

            )

            r.raise_for_status()

            logger.info(f"LLM ok | Ollama | attempt={attempt+1} | {round(time.time()-start,2)}s")

            return r.json().get("response", "")

        except Exception as e:

            attempt += 1

            logger.warning(f"LLM error attempt {attempt}: {e}")

            if attempt <= self.max_retries:

                time.sleep(self.retry_delay)

    return ""

def _generate_anthropic(self, prompt: str) -> str:

    attempt = 0

    while attempt <= self.max_retries:

        try:

            start = time.time()

            r = requests.post(

                ANTHROPIC_URL,

                headers={

                    "x-api-key": ANTHROPIC_API_KEY,

```

```

        "anthropic-version": "2023-06-01",
        "content-type": "application/json",
    },
    json={
        "model": ANTHROPIC_MODEL,
        "max_tokens": 8192,
        "messages": [{"role": "user", "content": prompt}],
    },
    timeout=120,
)
r.raise_for_status()
content = r.json()["content"][0]["text"]
        logger.info(f"LLM ok | Anthropic | attempt={attempt+1} |
{round(time.time()-start,2)}s")
    return content
except Exception as e:
    attempt += 1
    logger.warning(f"Anthropic error attempt {attempt}: {e}")
    if attempt <= self.max_retries:
        delay = self.retry_delay * (2 ** (attempt - 1))
        if hasattr(e, "response") and e.response is not None and
e.response.status_code == 429:
            delay = max(delay, 5)
        time.sleep(delay)
    return ""

def _generate_gemini(self, prompt: str) -> str:
    attempt = 0
    max_attempts = 5 # increased to handle rate limits
    while attempt <= max_attempts:
        try:
            start = time.time()
            r = requests.post(
                GEMINI_URL,
                headers={

```



```

        "Authorization": f"Bearer {GEMINI_API_KEY}",
        "Content-Type": "application/json",
    },
    json={
        "model": GEMINI_MODEL,
        "messages": [{"role": "user", "content": prompt}],
    },
    timeout=120,
)
r.raise_for_status()
content = r.json()["choices"][0]["message"]["content"]
        logger.info(f"LLM ok | Gemini | attempt={attempt+1} |
{round(time.time()-start,2)}s")
    return content
except Exception as e:
    attempt += 1
    logger.warning(f"Gemini error attempt {attempt}: {e}")
    if attempt <= max_attempts:
        if "429" in str(e):
            logger.warning("Gemini Rate Limit (429) hit. Waiting 30 seconds...")
            time.sleep(30)
        else:
            time.sleep(15)
return ""

```

```

def _generate_sync(self, prompt: str) -> str:
    if self.use_gemini:
        return self._generate_gemini(prompt)
    if self.use_anthropic:
        return self._generate_anthropic(prompt)
    if self.use_groq:
        return self._generate_groq(prompt)
    return self._generate_ollama(prompt)

```

```

def generate(self, prompt: str) -> str:

```

```
return self._generate_sync(prompt)
```

```
async def agenerate(self, prompt: str) -> str:
```

```
    return await asyncio.to_thread(self._generate_sync, prompt)
```

```
def _chat_groq(self, messages: list) -> str:
```

```
    attempt = 0
```

```
    while attempt <= self.max_retries:
```

```
        try:
```

```
            start = time.time()
```

```
            current_key = next(key_iterator)
```

```
            r = requests.post(
```

```
                GROQ_URL,
```

```
                headers={
```

```
                    "Authorization": f"Bearer {current_key}",
```

```
                    "Content-Type": "application/json",
```

```
                },
```

```
                json={
```

```
                    "model": GROQ_MODEL,
```

```
                    "messages": messages,
```

```
                    "max_tokens": GROQ_MAX_TOK,
```

```
                    "temperature": GROQ_TEMP,
```

```
                },
```

```
                timeout=60,
```

```
            )
```

```
            r.raise_for_status()
```

```
            content = r.json()["choices"][0]["message"]["content"]
```

```
            logger.info(f"LLM ok | Groq Chat | attempt={attempt+1} | {round(time.time()-start,2)}s")
```

```
            return content
```

```
        except Exception as e:
```

```
            attempt += 1
```

```
            logger.warning(f"Groq Chat error attempt {attempt}: {e}")
```

```
            if attempt <= self.max_retries:
```

```
                delay = self.retry_delay * (2 ** (attempt - 1))
```

```

        if hasattr(e, "response") and e.response is not None and
e.response.status_code == 429:

            delay = max(delay, 5) # Minimum 5s backoff for 429

            time.sleep(delay)

    return ""

def _chat_ollama(self, messages: list) -> str:
    chat_url = self.url.replace("/api/generate", "/api/chat")
    attempt = 0
    while attempt <= self.max_retries:
        try:
            start = time.time()
            r = requests.post(
                chat_url,
                json={"model": self.model, "messages": messages, "stream": False},
                timeout=self.timeout,
            )
            r.raise_for_status()
            logger.info(f"LLM ok | Ollama Chat | attempt={attempt+1} |
{round(time.time()-start,2)}s")
            return r.json().get("message", {}).get("content", "")
        except Exception as e:
            attempt += 1
            logger.warning(f"Ollama Chat error attempt {attempt}: {e}")
            if attempt <= self.max_retries:
                time.sleep(self.retry_delay)
    return ""

def _chat_anthropic(self, messages: list) -> str:
    # Anthropic doesn't support 'system' messages in the messages array the same way,
    # it requires a separate 'system' parameter. We must extract it.
    system_text = ""
    anthropic_msgs = []
    for m in messages:
        if m["role"] == "system":

```

```

        system_text += m["content"] + "\n"

    else:

        anthropic_msgs.append({"role": m["role"], "content": m["content"]})

attempt = 0
while attempt <= self.max_retries:

    try:

        start = time.time()

        payload = {

            "model": ANTHROPIC_MODEL,

            "max_tokens": 8192,

            "messages": anthropic_msgs,

        }

        if system_text:

            payload["system"] = system_text.strip()

        r = requests.post(

            ANTHROPIC_URL,

            headers={

                "x-api-key": ANTHROPIC_API_KEY,

                "anthropic-version": "2023-06-01",

                "content-type": "application/json",

            },

            json=payload,

            timeout=120,

        )

        r.raise_for_status()

        content = r.json()["content"][0]["text"]

        logger.info(f"LLM ok | Anthropic Chat | attempt={attempt+1} | {round(time.time()-start,2)}s")

        return content

    except Exception as e:

        attempt += 1

        logger.warning(f"Anthropic Chat error attempt {attempt}: {e}")

        if attempt <= self.max_retries:

```

```

        delay = self.retry_delay * (2 ** (attempt - 1))

        if hasattr(e, "response") and e.response is not None and
e.response.status_code == 429:

            delay = max(delay, 5)

            time.sleep(delay)

    return ""

def _chat_gemini(self, messages: list) -> str:

    attempt = 0

    max_attempts = 5

    while attempt <= max_attempts:

        try:

            start = time.time()

            r = requests.post(

                GEMINI_URL,

                headers={

                    "Authorization": f"Bearer {GEMINI_API_KEY}",

                    "Content-Type": "application/json",

                },

                json={

                    "model": GEMINI_MODEL,

                    "messages": messages,

                },

                timeout=120,

            )

            r.raise_for_status()

            content = r.json()["choices"][0]["message"]["content"]

            logger.info(f"LLM chat ok | Gemini | attempt={attempt+1} |

{round(time.time()-start,2)}s")

            return content

        except Exception as e:

            attempt += 1

            logger.warning(f"Gemini chat error attempt {attempt}: {e}")

            if attempt <= max_attempts:

                if "429" in str(e):

```

```

        logger.warning("Gemini Rate Limit (429) hit. Waiting 30 seconds...")

        time.sleep(30)

    else:

        time.sleep(15)

    return ""

def _chat_sync(self, messages: list) -> str:

    if self.use_gemini:

        return self._chat_gemini(messages)

    if self.use_anthropic:

        return self._chat_anthropic(messages)

    if self.use_groq:

        return self._chat_groq(messages)

    return self._chat_ollama(messages)

async def achat(self, messages: list) -> str:

    return await asyncio.to_thread(self._chat_sync, messages)

# ?????????????????????????????????????????????????????????????????????????????????????
# Dedicated Gemini provider ? ALWAYS uses Gemini regardless of env flags.
# Used by frontend_agent.py only (for quality React/UI code generation).
# All other agents use the main `llm` instance (Groq ? fast, 30 RPM free tier).
# ?????????????????????????????????????????????????????????????????????????????????????

class GeminiLLMProvider:

    """Always uses Gemini API. Used exclusively by the frontend agent."""

    def _generate(self, prompt: str) -> str:

        attempt = 0

        max_attempts = 5

        while attempt <= max_attempts:

            try:

                start = time.time()

                r = requests.post(

```

```

        GEMINI_URL,

        headers={

            "Authorization": f"Bearer {GEMINI_API_KEY}",

            "Content-Type": "application/json",

        },

        json={

            "model": GEMINI_MODEL,

            "messages": [{"role": "user", "content": prompt}],

        },

        timeout=120,

    )

    r.raise_for_status()

    content = r.json()["choices"][0]["message"]["content"]

    logger.info(f"LLM ok | Gemini-Frontend | attempt={attempt+1} | {round(time.time()-start,2)}s")

    return content

except Exception as e:

    attempt += 1

    logger.warning(f"Gemini-Frontend error attempt {attempt}: {e}")

    if attempt <= max_attempts:

        if "429" in str(e):

            logger.warning("Gemini rate limit hit ? waiting 30s before retry...")

            time.sleep(30)

        else:

            time.sleep(10)

    return ""

def _chat(self, messages: list) -> str:

    attempt = 0

    max_attempts = 5

    while attempt <= max_attempts:

        try:

            start = time.time()

            r = requests.post(

                GEMINI_URL,

```

```

        headers={
            "Authorization": f"Bearer {GEMINI_API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": GEMINI_MODEL,
            "messages": messages,
        },
        timeout=120,
    )
    r.raise_for_status()
    content = r.json()["choices"][0]["message"]["content"]
    logger.info(f"LLM chat ok | Gemini-Frontend | attempt={attempt+1} | {round(time.time()-start,2)}s")
    return content
except Exception as e:
    attempt += 1
    logger.warning(f"Gemini-Frontend chat error attempt {attempt}: {e}")
    if attempt <= max_attempts:
        if "429" in str(e):
            logger.warning("Gemini rate limit hit ? waiting 30s before retry...")
            time.sleep(30)
        else:
            time.sleep(10)
    return ""

def generate(self, prompt: str) -> str:
    return self._generate(prompt)

async def agenerate(self, prompt: str) -> str:
    return await asyncio.to_thread(self._generate, prompt)

async def achat(self, messages: list) -> str:
    return await asyncio.to_thread(self._chat, messages)

```



```
# ??? Singletons ?????????????????????????????????????????????????????????????????
# llm          ? used by all agents (Gemini ? high limits)
# llm_gemini ? used by frontend agent only (Gemini ? quality UI)
# ?????????????????????????????????????????????????????????????????????????????
```

```
llm          = LLMProvider()
llm_gemini = GeminiLLMProvider() if (USE_GEMINI and GEMINI_API_KEY) else llm
...
```

```
## File: `backend\core\logger.py`
```

```
```py
backend/core/logger.py

import logging, os

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("AI-Digital-Company")

logger.setLevel(logging.INFO)

fmt = logging.Formatter("%(asctime)s | %(levelname)s | %(name)s | %(message)s")
ch = logging.StreamHandler(); ch.setFormatter(fmt)
fh = logging.FileHandler("logs/app.log"); fh.setFormatter(fmt)

if not logger.handlers:
 logger.addHandler(ch); logger.addHandler(fh)

...
```

```
File: `backend\core\meeting_engine.py`
```

```
```py
# backend/core/meeting_engine.py ? VIA Phase 3: Agent Meeting Room

import asyncio

import time

import logging
```

```
from backend.core.llm_provider import llm

logger = logging.getLogger("AI-Digital-Company")

AGENT_PERSONAS = {

    "ceo": {

        "name": "Alex Chen",

        "title": "CEO & Visionary",

        "emoji": "?",

        "personality": "Strategic, decisive, inspiring. Speaks in big-picture terms. Uses phrases like 'This aligns with our vision', 'Let's move fast on this', 'What's the bottleneck?',"

        "color": "m2"

    },

    "backend": {

        "name": "Priya Sharma",

        "title": "Lead Backend Engineer",

        "emoji": "??",

        "personality": "Technical, precise, pragmatic. Talks about APIs, databases, performance. Uses phrases like 'We need to consider scalability', 'The API contract should be', 'I'll wire up the endpoints'",

        "color": "c"

    },

    "frontend": {

        "name": "Marcus Liu",

        "title": "Senior Frontend Engineer",

        "emoji": "?",

        "personality": "Creative, user-focused, detail-oriented. Talks about UX, components, responsiveness. Uses phrases like 'The user journey needs to', 'I'll build a clean interface', 'Let me sketch the component tree'",

        "color": "v2"

    },

    "security": {

        "name": "Zara Ahmed",

        "title": "Security Architect",

        "emoji": "?",

        "personality": "Cautious, thorough, risk-aware. Always thinking about threats. Uses phrases like 'We need to threat model this', 'Authentication must be', 'What about injection attacks?'"

    }

}
```

```
    "color": "y"
  },
  "devops": {
    "name": "Carlos Rivera",
    "title": "DevOps Lead",
    "emoji": "?",
    "personality": "Practical, efficiency-driven, automation-obsessed. Uses phrases like 'I'll set up the CI/CD pipeline', 'Container this with Docker', 'Zero-downtime deployment is critical'",
    "color": "g"
  },
  "ai_research": {
    "name": "Dr. Aisha Patel",
    "title": "AI Research Director",
    "emoji": "?",
    "personality": "Analytical, curious, forward-thinking. Talks about models, data, ML pipelines. Uses phrases like 'The model architecture should', 'Training data quality matters', 'Let me run some benchmarks'",
    "color": "v"
  },
  "architecture": {
    "name": "James O'Brien",
    "title": "Solutions Architect",
    "emoji": "?",
    "personality": "Systematic, design-focused, pattern-aware. Talks about system design and scalability. Uses phrases like 'The microservices boundary should be', 'Event-driven architecture fits here', 'Let me draw the data flow'",
    "color": "o"
  },
  "hr": {
    "name": "Sophie Kim",
    "title": "HR Director",
    "emoji": "?",
    "personality": "Empathetic, people-focused, organized. Talks about team dynamics and culture. Uses phrases like 'We need to hire for this gap', 'Team morale is critical', 'Let me draft the onboarding plan'",
    "color": "m"
  },
  "finance": {
```

```

        "name": "David Okafor",

        "title": "CFO",

        "emoji": "?",

        "personality": "Numbers-driven, risk-conscious, ROI-focused. Uses phrases like 'What's the burn rate?', 'We need to optimize costs', 'The break-even point is'",

        "color": "g2"

    },

    "marketing": {

        "name": "Luna Rodriguez",

        "title": "CMO",

        "emoji": "?",

        "personality": "Creative, data-driven, growth-obsessed. Talks about positioning, GTM, growth channels. Uses phrases like 'This is our differentiator', 'Target persona is', 'I see a viral loop here'",

        "color": "c2"

    },

}

```

```

async def generate_meeting(task: str, departments: list, ceo_strategy: str) -> list:

```

```

    """

```

```

    Generate a realistic agent meeting transcript.

```

```

    Returns list of message dicts: {agent, name, emoji, message, timestamp, color}

```

```

    """

```

```

    messages = []

```

```

    t = time.time()

```

```

    def msg(agent_key: str, text: str):

```

```

        persona = AGENT_PERSONAS.get(agent_key, AGENT_PERSONAS["ceo"])

```

```

        messages.append({

```

```

            "agent": agent_key,

```

```

            "name": persona["name"],

```

```

            "title": persona["title"],

```

```

            "emoji": persona["emoji"],

```

```

            "message": text,

```

```

            "timestamp": round(time.time() - t, 2),

```

```
        "color": persona["color"],
    })
```

```
# Phase 1: CEO opens the meeting
```

```
prompt_ceo_open = f"""You are Alex Chen, CEO of VIA (an autonomous AI company).
```

```
You're opening a team meeting about this project: {task}
```

```
Strategy decided: {ceo_strategy}
```

```
Write a brief (3-4 sentences) energetic CEO opening statement to kick off the meeting.
```

```
Be specific about the task, mention the strategy, and motivate the team.
```

```
No quotes, no labels, just the speech text directly."""
```

```
ceo_opening = await llm.agenerate(prompt_ceo_open)
```

```
msg("ceo", ceo_opening.strip() if ceo_opening else f"Team, we've just received a critical  
task: {task}. Our strategy is clear ? {ceo_strategy}. Let's execute fast and smart. I need every  
department to align on their deliverables today.")
```

```
# Phase 2: Departments respond
```

```
active_depts = [d for d in departments if d in AGENT_PERSONAS and d != "ceo"]
```

```
for dept in active_depts[:5]: # Limit to 5 departments for concise meeting
```

```
    persona = AGENT_PERSONAS[dept]
```

```
    prompt_dept = f"""You are {persona['name']], {persona['title']] at VIA.
```

```
    Personality: {persona['personality']}
```

```
The CEO just opened a meeting about: {task}
```

```
CEO Strategy: {ceo_strategy}
```

```
Write your department's response (2-3 sentences). What will YOUR team specifically do?
```

```
Mention your key action items or concerns.
```

```
No quotes, no labels, just speak naturally as {persona['name']}. """
```

```
dept_response = await llm.agenerate(prompt_dept)
```

```
if dept_response:
```

```
msg(dept, dept_response.strip())
```

```
# Phase 3: Cross-department interaction (security asks a question)
```

```
if "security" in departments and "backend" in departments:
```

```
    prompt_security_q = f"""You are Zara Ahmed, Security Architect at VIA.
```

```
The backend team just said they'll build APIs for: {task}
```

Ask ONE sharp security question to the backend engineer about authentication or data protection.

Keep it to 1-2 sentences. Be direct and technical."""

```
sec_q = await llm.agenerate(prompt_security_q)
```

```
if sec_q:
```

```
    msg("security", sec_q.strip())
```

```
# Backend responds to security
```

```
    prompt_backend_ans = f"""You are Priya Sharma, Lead Backend Engineer at VIA.
```

```
The security architect just raised a security concern about authentication for: {task}
```

Give a confident 1-2 sentence technical answer about how you'll handle security in the backend."""

```
be_ans = await llm.agenerate(prompt_backend_ans)
```

```
if be_ans:
```

```
    msg("backend", be_ans.strip())
```

```
# Phase 4: CEO closes with action items
```

```
dept_names = [AGENT_PERSONAS[d]["name"] for d in active_depts[:4]]
```

```
prompt_ceo_close = f"""You are Alex Chen, CEO of VIA.
```

```
The team meeting about "{task}" is wrapping up.
```

Write a crisp 2-3 sentence closing statement that:

1. Assigns clear ownership (mention 1-2 team members by name: {' '.join(dept_names)})
2. Sets urgency / deadline
3. Motivates the team

No quotes, no labels, just the closing speech."""

```
ceo_close = await llm.agenerate(prompt_ceo_close)

msg("ceo", ceo_close.strip() if ceo_close else f"Excellent discussion everyone. {dept_names[0]
if dept_names else 'Team'}, you have the backend. Let's ship this within the sprint ? no excuses.
VIA moves fast!")

return messages
```

async def generate_meeting_fast(task: str, departments: list, ceo_strategy: str) -> list:

"""

Faster version: generates a pre-scripted meeting without LLM calls for each message.

Uses a single LLM call to generate the whole transcript.

"""

```
dept_personas = []
```

```
for d in departments:
```

```
    if d in AGENT_PERSONAS:
```

```
        p = AGENT_PERSONAS[d]
```

```
        dept_personas.append(f"- {p['name']} ({p['title']}): {p['personality'][:80]}")
```

```
personas_text = "\n".join(dept_personas)
```

```
prompt = f"""You are writing a realistic team meeting transcript for VIA's autonomous AI
company.
```

```
PROJECT: {task}
```

```
CEO STRATEGY: {ceo_strategy}
```

```
TEAM MEMBERS:
```

```
{personas_text}
```

Write a natural meeting transcript with 8-12 exchanges. Format EXACTLY like this (one per line):

SPEAKER_KEY|Message text here

Valid speaker keys: ceo, backend, frontend, security, devops, ai_research, architecture, hr,

finance, marketing

Rules:

- CEO opens and closes the meeting
- Each department mentions their specific deliverable
- Include 1-2 cross-department questions/debates
- Keep each message to 2-3 sentences max
- Make it sound like a real Slack/Teams standup
- Only use speaker keys from the Valid list above

"""

try:

```
raw = await llm.agenerate(prompt)
```

```
messages = []
```

```
t = time.time()
```

```
if raw:
```

```
    for line in raw.strip().split("\n"):
```

```
        line = line.strip()
```

```
        if "|" in line:
```

```
            parts = line.split("|", 1)
```

```
            if len(parts) == 2:
```

```
                agent_key = parts[0].strip().lower()
```

```
                text = parts[1].strip()
```

```
                if agent_key in AGENT_PERSONAS and text:
```

```
                    persona = AGENT_PERSONAS[agent_key]
```

```
                    messages.append({
```

```
                        "agent": agent_key,
```

```
                        "name": persona["name"],
```

```
                        "title": persona["title"],
```

```
                        "emoji": persona["emoji"],
```

```
                        "message": text,
```

```
                        "timestamp": round(time.time() - t, 3),
```

```
                        "color": persona["color"],
```



```
    })
```

```
    if not messages:
```

```
        # Fallback: simple scripted meeting
```

```
        messages = _scripted_fallback(task, departments, ceo_strategy)
```

```
    return messages
```

```
except Exception as e:
```

```
    logger.error(f"Meeting generation failed: {e}")
```

```
    return _scripted_fallback(task, departments, ceo_strategy)
```

```
def _scripted_fallback(task: str, departments: list, ceo_strategy: str) -> list:
```

```
    """Pre-written fallback meeting if LLM fails."""
```

```
    t = time.time()
```

```
    messages = []
```

```
def msg(agent_key, text):
```

```
    persona = AGENT_PERSONAS.get(agent_key, AGENT_PERSONAS["ceo"])
```

```
    messages.append({
```

```
        "agent": agent_key,
```

```
        "name": persona["name"],
```

```
        "title": persona["title"],
```

```
        "emoji": persona["emoji"],
```

```
        "message": text,
```

```
        "timestamp": round(time.time() - t, 3),
```

```
        "color": persona["color"],
```

```
    })
```

```
    msg("ceo", f"Team, we have a new priority: '{task[:100]}'. Strategy: {ceo_strategy[:120]}.  
Everyone align on your deliverables ? we move today.")
```

```
dept_msgs = {
```

```
    "backend": "I'll architect the FastAPI backend with PostgreSQL. Estimating 3 endpoints in
```

the first sprint. Database schema will be ready by EOD.",

"frontend": "I'll build the React UI with mobile-first design. Component library is already set up. We'll have a working prototype in 48 hours.",

"security": "I need to threat-model this before we ship. JWT auth, input validation, and rate limiting are non-negotiable. I'll deliver the security spec today.",

"devops": "CI/CD pipeline will be live by tomorrow. Docker containers, GitHub Actions, and Render deployment are all ready to configure.",

"ai_research": "I'll evaluate which LLM fits this use case best. Fine-tuning vs. prompt engineering decision will be made after running benchmarks tonight.",

"architecture": "System design doc is drafted. I'm recommending a microservices boundary between auth and core services. Review it before we start coding.",

"hr": "I'll define the team roles and have job descriptions ready. Onboarding plan for any new hires follows next week.",

"finance": "Initial budget estimate is being prepared. I'll have the ROI projection and cost breakdown in your inbox by 5 PM.",

"marketing": "GTM strategy is already forming. I'm thinking Product Hunt launch + LinkedIn campaign. Personas and messaging framework by end of week.",

}

for dept in departments:

if dept in dept_msgs and dept != "ceo":

msg(dept, dept_msgs[dept])

msg("ceo", "Outstanding. Everyone has clear ownership. Ship fast, ship clean. VIA doesn't miss deadlines ? let's make history with this one.")

return messages

...

File: `backend\core\memory\_store.py`

```py

# backend/core/memory\_store.py ? Phase 3: Agent Memory (DB-mode agnostic)

import json

import logging

from backend.database.db import (

save\_agent\_mem, get\_agent\_mem, get\_all\_mem,

```

 save_meeting_db, get_meeting_db, get_recent_meetings_db
)

logger = logging.getLogger("AI-Digital-Company")

async def save_agent_memory(agent: str, task: str, output_summary: str, confidence: float):
 """Save a memory entry for an agent after task completion."""
 try:
 await save_agent_mem(agent, task, output_summary, confidence)
 logger.info(f"Memory saved | agent={agent}")
 except Exception as e:
 logger.warning(f"Memory save failed | {e}")

async def get_agent_memory(agent: str, limit: int = 5) -> list:
 """Retrieve recent memories for an agent."""
 try:
 rows = await get_agent_mem(agent, limit)
 return [
 {
 "task": dict(r)["task"],
 "output_summary": dict(r)["output_summary"],
 "confidence": dict(r)["confidence"],
 "timestamp": str(dict(r).get("created_at", "")),
 }
 for r in (rows or [])
]
 except Exception as e:
 logger.warning(f"Memory fetch failed | {e}")
 return []

async def get_all_memories(limit: int = 50) -> list:

```

```
"""Retrieve all recent agent memories across all agents."""
```

```
try:
```

```
 rows = await get_all_mem(limit)
```

```
 return [
```

```
 {
```

```
 "agent": dict(r)["agent_name"],
```

```
 "task": dict(r)["task"],
```

```
 "output_summary": dict(r)["output_summary"],
```

```
 "confidence": dict(r)["confidence"],
```

```
 "timestamp": str(dict(r).get("created_at", "")),
```

```
 }
```

```
 for r in (rows or [])
```

```
]
```

```
except Exception as e:
```

```
 logger.warning(f"All memories fetch failed | {e}")
```

```
 return []
```

```
def build_memory_context(memories: list) -> str:
```

```
 """Format memory list into an LLM-injectable context string."""
```

```
 if not memories:
```

```
 return "No prior experience with this type of task."
```

```
 lines = []
```

```
 for i, m in enumerate(memories, 1):
```

```
 lines.append(
```

```
 f"Memory {i}: Task='{m['task'][:80]}' | "
```

```
 f"Result='{m['output_summary'][:120]}' | "
```

```
 f"Confidence={m['confidence']}"
```

```
)
```

```
 return "\n".join(lines)
```

```
async def save_meeting_transcript(job_id: str, task: str, messages: list):
```

```
 """Save a meeting transcript to the database."""
```

```

try:
 await save_meeting_db(job_id, task, json.dumps(messages), len(messages))
 logger.info(f"Meeting saved | job={job_id} | {len(messages)} messages")
except Exception as e:
 logger.warning(f"Meeting save failed | {e}")

```

```

async def get_meeting(job_id: str) -> dict:

```

```

 """Retrieve a meeting transcript by job_id."""

```

```

try:
 row = await get_meeting_db(job_id)
 if not row:
 return {}
 row = dict(row)
 return {
 "job_id": row["job_id"],
 "task": row["task"],
 "messages": json.loads(row["transcript"]) if row.get("transcript") else [],
 "message_count": row["message_count"],
 "timestamp": str(row.get("created_at", "")),
 }
except Exception as e:
 logger.warning(f"Meeting fetch failed | {e}")
 return {}

```

```

async def get_recent_meetings(limit: int = 10) -> list:

```

```

 """Retrieve recent meeting summaries."""

```

```

try:
 rows = await get_recent_meetings_db(limit)
 return [
 {
 "job_id": dict(r)["job_id"],
 "task": dict(r)["task"],

```

```

 "message_count": dict(r)["message_count"],
 "timestamp": str(dict(r).get("created_at", "")),
 }
 for r in (rows or [])
]
except Exception as e:
 logger.warning(f"Recent meetings fetch failed | {e}")
 return []

```

```

...

```

```

File: `backend\core\render_deployer.py`

```

```

```py

```

```

# backend/core/render_deployer.py

```

```

#

```

```

# FIX 1: live_url comes from Render API response, not constructed from svc_name.

```

```

# FIX 2: _redeploy() sends full env var set so nothing gets wiped.

```

```

# FIX 3: Uses RENDER_DATABASE_URL from .env ? never auto-provisions a DB.

```

```

#         Table isolation handled per-app by fullstack_builder.py table prefixes.

```

```

# FIX 4: SQLite fallback uses sqlite:/// not sqlite+aiosqlite:/// ? the

```

```

#         database.py normalizer expects sqlite://, not the async driver prefix.

```

```

# FIX 5: deploy() now returns the corrected live_url so the caller can set

```

```

#         VITE_API_URL to the real URL before triggering the GitHub workflow.

```

```

# FIX 6: autoDeploy set to "no" ? VIA controls deploys explicitly via API.

```

```

#         Prevents ALL previous Render services from redeploying every time

```

```

#         a new app is built (was: "yes" caused every connected repo push to

```

```

#         trigger reddeploys across all services).

```

```

# FIX 7: _redeploy() now does exact name match ? prevents partial name match

```

```

#         from accidentally redeploying a wrong existing service.

```

```

# FIX 8: _redeploy() always returns live_url even if deploy trigger POST fails ?

```

```

#         service exists so URL is known; UI should always show backend link.

```

```

import os

```

```

import re

import time

import requests

from requests.adapters import HTTPAdapter

from urllib3.util.retry import Retry

import logging


logger = logging.getLogger("AI-Digital-Company")


RENDER_API = "https://api.render.com/v1"


class RenderDeployer:

    def __init__(self):

        self.api_key = os.getenv("RENDER_API_KEY", "")

        self.headers = {

            "Authorization": f"Bearer {self.api_key}",

            "Content-Type": "application/json",

        }

        self.session = requests.Session()

        retries = Retry(total=5, backoff_factor=1, status_forcelist=[500, 502, 503, 504])

        self.session.mount("https://", HTTPAdapter(max_retries=retries))


    def _ok(self) -> bool:

        return bool(self.api_key)


    def _owner_id(self) -> str:

        try:

            r = self.session.get(f"{RENDER_API}/owners", headers=self.headers, timeout=15)

            if r.status_code == 200:

                owners = r.json()

                if owners:

                    return owners[0].get("owner", {}).get("id", "")

```

```
except Exception as e:

    logger.error(f"Render owner error: {e}")

return ""
```

```
def _service_name(self, raw: str) -> str:

    name = raw.lower().strip()

    name = re.sub(r"^[a-z0-9-]", "-", name)

    name = re.sub(r"-+", "-", name)

    name = name.strip("-")

    return name[:63] or "via-app"
```

```
def _extract_live_url(self, service: dict) -> str:

    """FIX 1: Extract real URL from Render API response. Never construct it."""

    url = (

        service.get("serviceDetails", {}).get("url", "")

        or service.get("url", "")

    )

    if url:

        if not url.startswith("http"):

            url = f"https://{url}"

            return url.rstrip("/")

    svc_name = service.get("name", "")

    if svc_name:

        return f"https://{svc_name}.onrender.com"

    return ""
```

```
def _normalise_db_url(self, url: str) -> str:

    """Normalize all postgres URL variants to postgresql:// for SQLAlchemy."""

    url = url.replace("postgresql+asyncpg://", "postgresql://")

    url = url.replace("postgres://", "postgresql://")

    return url
```

```
def _db_url(self) -> str:

    """
```


Always use RENDER_DATABASE_URL from .env.

Table isolation handled by per-app __tablename__ prefixes in models.py.

All apps share one DB safely ? no table conflicts.

```
"""
```

```
url = os.getenv("RENDER_DATABASE_URL", "")
```

```
if url:
```

```
        logger.info("Render | Using RENDER_DATABASE_URL from .env (shared DB, isolated  
tables)")
```

```
        return self._normalise_db_url(url)
```

```
logger.warning("Render | RENDER_DATABASE_URL not set in .env!")
```

```
return ""
```

```
def _build_env_vars(self, svc_name: str, db_url: str) -> list:
```

```
# FIX 4: SQLite fallback must use sqlite:/// not sqlite+aiosqlite:///
```

```
# database.py's normalizer handles sqlite:// prefix, not the async variant.
```

```
sqlite_fallback = "sqlite:///./app.db"
```

```
return [
```

```
    {"key": "APP_ENV",          "value": "production"},
```

```
    {"key": "APP_NAME",        "value": svc_name},
```

```
    {"key": "JWT_SECRET_KEY", "generateValue": True},
```

```
    {"key": "DATABASE_URL",    "value": db_url if db_url else sqlite_fallback},
```

```
]
```

```
def _create(self, repo_url: str, repo_name: str, task: str) -> dict:
```

```
owner_id = self._owner_id()
```

```
if not owner_id:
```

```
        return {"success": False, "error": "Cannot get Render owner ID ? check  
RENDER_API_KEY"}
```

```
svc_name = self._service_name(repo_name)
```

```
db_url    = self._db_url()
```

```
env_vars = self._build_env_vars(svc_name, db_url)
```

```
if not db_url:
```

```
        logger.warning(f"Render | No DATABASE_URL ? app will use SQLite fallback |  
{svc_name}")
```

```

payload = {
    "type":      "web_service",
    "name":      svc_name,
    "ownerId":   owner_id,
    "repo":      repo_url,
    "branch":    "main",
    # FIX 6: "no" instead of "yes" ? VIA triggers deploys explicitly via
    # POST /services/{id}/deploys. "yes" caused ALL services to redeploy
    # whenever any connected GitHub repo received a push, because Render
    # watched every repo in the account with autoDeploy enabled.
    "autoDeploy": "no",
    "serviceDetails": {
        "env":    "python",
        "plan":   "free",
        "region": "oregon",
        "envVars": env_vars,
        "envSpecificDetails": {
            "buildCommand": "pip install -r requirements.txt",
            "startCommand": "uvicorn main:app --host 0.0.0.0 --port $PORT",
        },
    },
}

```

```

try:

```

```

    r = self.session.post(f"{RENDER_API}/services", json=payload, headers=self.headers,
timeout=60)

```

```

if r.status_code in (200, 201):

```

```

    data = r.json()

```

```

    service = data.get("service", data)

```

```

    svc_id = service.get("id", "")

```

```

    live_url = self._extract_live_url(service)

```

```

    if not live_url:

```

```

        live_url = f"https://{svc_name}.onrender.com"

```

```

dash_url = f"https://dashboard.render.com/web/{svc_id}"

logger.info(f"Render service created | {live_url}")

return {
    "success": True,
    "service_id": svc_id,
    "live_url": live_url,
    "svc_name": service.get("name", svc_name),
    "dash_url": dash_url,
    "db_ok": bool(db_url),
}

if r.status_code == 400:

    msg = r.json().get("message", "")

    if "already" in msg.lower() or "exists" in msg.lower():
        logger.info(f"Render | Service already exists ? redeploying | {svc_name}")
        return self._redploy(svc_name, db_url)

    return {"success": False, "error": f"Render 400: {msg}"}

    return {"success": False, "error": f"Render {r.status_code}: {r.json().get('message',
''))}"}

except Exception as e:

    logger.error(f"Render create error: {e}")

    return {"success": False, "error": str(e)}

def _redploy(self, svc_name: str, db_url: str = "") -> dict:
    """
    FIX 2: Redeploy sends full env vars so nothing gets wiped.
    FIX 7: Exact name match ? Render's ?name= filter is a partial/prefix match,
        so it can return multiple services. Old code took r.json()[0] which
        could be a completely different service. Now we filter by exact name
        before deploying so only THIS app's service is ever touched.
    FIX 8: Always return live_url even if deploy trigger POST fails ? the
        service exists so the URL is known. UI must always show backend link.
    """

```

```

try:
    r = self.session.get(f"{RENDER_API}/services?name={svc_name}", headers=self.headers,
timeout=15)

    if r.status_code == 200 and r.json():

        services = r.json()

        # FIX 7: exact name match ? never deploy a service with a similar name

        svc      = None

        svc_id = None

        for entry in services:

            candidate = entry.get("service", entry)

            if candidate.get("name", "") == svc_name:

                svc      = candidate

                svc_id = svc.get("id", "")

                break

        if not svc_id:

            logger.warning(f"Render | No exact match for service name '{svc_name}' ?
skipping redeploy")

            return {"success": False, "error": f"No exact service match for '{svc_name}'"}

        if db_url:

            full_env_vars = self._build_env_vars(svc_name, db_url)

            requests.put(

                f"{RENDER_API}/services/{svc_id}/env-vars",

                json=full_env_vars, headers=self.headers, timeout=15,

            )

            logger.info(f"Render | Env vars updated on existing service | {svc_name}")

        # FIX 8: Extract live_url before deploy trigger so we always have it

        live_url = self._extract_live_url(svc)

        if not live_url:

            live_url = f"https://{svc_name}.onrender.com"

        dr = self.session.post(

```

```

        f"{RENDER_API}/services/{svc_id}/deploys",
        json={"clearCache": "do_not_clear"},
        headers=self.headers, timeout=15,
    )

    if dr.status_code in (200, 201):
        logger.info(f"Render redeployed | {live_url}")
    else:
        # FIX 8: Deploy trigger failed but service exists ? still return
        # live_url so the UI always shows the backend link correctly
        logger.warning(f"Render | Deploy trigger returned {dr.status_code} ? service
exists at {live_url}")

    return {
        "success": True,
        "live_url": live_url,
        "svc_name": svc.get("name", svc_name),
        "dash_url": f"https://dashboard.render.com/web/{svc_id}",
        "redeployed": True,
        "db_ok": bool(db_url),
    }

except Exception as e:
    logger.error(f"Render redeploy error: {e}")

return {"success": False, "error": "Could not redeploy existing service"}


def deploy(self, task: str, repo_url: str, repo_name: str) -> dict:
    if not self._ok():
        return {"phase6_ran": False, "error": "RENDER_API_KEY not set."}

    if not repo_url:
        return {"phase6_ran": False, "error": "No repo_url ? Phase 5 must complete first"}

    logger.info(f"Phase 6 | Deploying to Render | {repo_url}")
    result = self._create(repo_url, repo_name, task)

    if result["success"]:
        live = result["live_url"]

```

```

logger.info(f"Phase 6 OK | {live}")

return {

    "phase6_ran":    True,

    "success":       True,

    "live_url":      live,

    "service_name":  result.get("svc_name", ""),

    "dashboard_url": result.get("dash_url", ""),

    "redeployed":    result.get("redeployed", False),

    "db_injected":   result.get("db_ok", False),

    "note":          f"Live in 3-5 mins. Check: {result.get('dash_url', '')}",

}

```

```

return {"phase6_ran": True, "success": False, "error": result.get("error", "Unknown Render
error")}]

```

```

render_deployer = RenderDeployer()

```

```

```

```

```

File: `backend\core\scaling_engine.py`

```

```

```py

```

```

# backend/core/scaling_engine.py

```

```

from .config import VALID_DEPARTMENTS, COMPLEX_TASK_THRESHOLD

```

```

from .logger import logger

```

```

# Departments that are ESSENTIAL for building a full-stack app.

```

```

# HR, Finance, Marketing, AI Research, Architecture, DevOps just waste

```

```

# API credits and add 60+ minutes of delay without contributing code.

```

```

CORE_BUILD_DEPARTMENTS = ["backend", "frontend"]

```

```

def analyze_complexity(task):

```

```

    wc = len(task.split())

```

```

    hits = sum(1 for s in ["enterprise", "large scale", "distributed", "microservices", "high
availability", "real-time", "millions", "global", "compliance", "regulated", "mission critical", "fault
tolerant", "zero downtime", "multi-region"] if s in task.lower())

```

```
    return "complex" if wc > COMPLEX_TASK_THRESHOLD or hits >= 3 else "moderate" if wc > 30 or hits >= 1 else "simple"
```

```
def autonomous_scale(task, ceo_departments):  
  
    # Always restrict to only backend + frontend for build tasks.  
  
    # The CEO tends to hallucinate unnecessary business agents (HR, Finance,  
  
    # Marketing) which add 60+ minutes and produce zero code.  
  
    reason = "Locked to core build agents (backend + frontend) for speed."  
  
    logger.info(f"Scaling: CORE_ONLY | {reason}")  
  
    return CORE_BUILD_DEPARTMENTS, False, reason
```

```
...
```

```
## File: `backend\core\task_router.py`
```

```
```py
```

```
backend/core/task_router.py
```

```
Smart Task Router ? detects task type and routes correctly
```

```
?? Task Type Constants ??
```

```
TECH_MODE = "tech"

RESEARCH_MODE = "research"

BUSINESS_MODE = "business"

CREATIVE_MODE = "creative"

MATH_MODE = "math"

GENERAL_MODE = "general"
```

```
?? Keyword Maps ??
```

```
TECH_KEYWORDS = [

 "build", "create", "develop", "code", "program", "api",

 "backend", "frontend", "database", "deploy", "docker",

 "server", "app", "application", "website", "system",

 "microservice", "endpoint", "rest", "graphql", "fastapi",

 "django", "flask", "react", "node", "python", "javascript",

 "implement", "design a system", "architecture for",
```

```

 "authentication system", "login system", "crud"
]

RESEARCH_KEYWORDS = [
 "explain", "what is", "what are", "how does", "why is",
 "summarize", "summary", "describe", "define", "tell me about",
 "teach me", "help me understand", "notes on", "study",
 "difference between", "compare", "history of", "explain me",
 "what do you know about", "give me information"
]

BUSINESS_KEYWORDS = [
 "business plan", "marketing", "strategy", "startup",
 "revenue", "profit", "customer", "market research",
 "business model", "pitch", "investor", "sales",
 "branding", "product launch", "go to market"
]

CREATIVE_KEYWORDS = [
 "write a story", "poem", "script", "creative",
 "fiction", "novel", "blog post", "article",
 "write about", "generate a story", "essay"
]

MATH_KEYWORDS = [
 "solve", "calculate", "equation", "math",
 "algebra", "geometry", "calculus", "statistics",
 "probability", "formula", "compute", "integral",
 "derivative", "matrix", "theorem"
]

def detect_task_type(task: str) -> str:
 """

```



Detect what type of task the user is asking for.

Returns one of: tech, research, business, creative, math, general

"""

```
task_lower = task.lower().strip()
```

```
for keyword in TECH_KEYWORDS:
```

```
 if keyword in task_lower:
```

```
 return TECH_MODE
```

```
for keyword in MATH_KEYWORDS:
```

```
 if keyword in task_lower:
```

```
 return MATH_MODE
```

```
for keyword in BUSINESS_KEYWORDS:
```

```
 if keyword in task_lower:
```

```
 return BUSINESS_MODE
```

```
for keyword in CREATIVE_KEYWORDS:
```

```
 if keyword in task_lower:
```

```
 return CREATIVE_MODE
```

```
for keyword in RESEARCH_KEYWORDS:
```

```
 if keyword in task_lower:
```

```
 return RESEARCH_MODE
```

```
return GENERAL_MODE
```

```
def get_mode_prompt(task_type: str, task: str) -> str:
```

"""

Returns simple, direct prompts that work well with Ollama local models.

Keep prompts short and direct ? Ollama handles these best.

"""

```
if task_type == MATH_MODE:
```

```

 return f"Solve this step by step and show all working: {task}"

elif task_type == RESEARCH_MODE:

 return f"Explain this clearly with examples: {task}"

elif task_type == BUSINESS_MODE:

 return f"Write a detailed business response for: {task}"

elif task_type == CREATIVE_MODE:

 return f"Write creative content for: {task}"

else:

 return f"Answer this helpfully and clearly: {task}"

def is_tech_task(task: str) -> bool:

 """Quick check ? is this a tech/coding task?"""

 return detect_task_type(task) == TECH_MODE

...

File: `backend\core\tracer.py`


```py
# backend/core/tracer.py ? Phase 2: includes inter-agent trace events

from datetime import datetime


class ExecutionTracer:

    def __init__(self):

        self.trace = []; self.start_time = ""; self.end_time = ""

    def start(self, task: str):

        self.start_time = datetime.now().isoformat()

        self.trace = []

```

```

self._r("ORCHESTRATOR", "Task received", {"task": task})

def add_memory_injection(self, count: int):
    self._r("CEO_AGENT", "Company memory injected", {"records_injected": count})

def add_ceo_decision(self, short_term, long_term, departments):
    self._r("CEO_AGENT", "Strategic decision made", {
        "short_term_strategy": short_term,
        "long_term_vision": long_term,
        "departments_selected": departments
    })

def add_inter_agent(self, from_agent: str, to_agents: list, summary: str):
    self._r("INTER_AGENT_BUS", f"Context: {from_agent} ? {to_agents}", {
        "from": from_agent, "to": to_agents, "context_summary": summary[:150]
    })

def add_scaling_decision(self, original, expanded, reason):
    self._r("SCALING_ENGINE", "Autonomous scaling triggered", {
        "original": original, "expanded": expanded, "reason": reason
    })

def add_agent_result(self, name, status, duration, confidence):
    self._r(f"{name.upper()}_AGENT", f"Execution {status}", {
        "status": status, "duration_seconds": duration, "confidence": confidence
    })

def finish(self, total_duration, success, failed):
    self.end_time = datetime.now().isoformat()
    self._r("ORCHESTRATOR", "Execution complete", {
        "started_at": self.start_time, "finished_at": self.end_time,
        "total_duration_seconds": total_duration,
        "agents_succeeded": success, "agents_failed": failed
    })

```

```

def get_trace(self): return self.trace

def _r(self, actor, event, data):
    self.trace.append({"timestamp": datetime.now().isoformat(), "actor": actor, "event":
event, "data": data})

...

## File: `backend\core\ws_manager.py`

```py
backend/core/ws_manager.py
Phase 2: WebSocket Manager
Manages all active WebSocket connections per job_id
Broadcasts real-time streaming events to connected clients

import asyncio
import json
from datetime import datetime
from fastapi import WebSocket
from .logger import logger

class ConnectionManager:
 """
 Manages WebSocket connections grouped by job_id.
 Multiple clients can subscribe to the same job.
 """

 def __init__(self):
 # {job_id: [WebSocket, ...]}
 self._connections: dict[str, list[WebSocket]] = {}

 async def connect(self, job_id: str, websocket: WebSocket):

```

```

await websocket.accept()

if job_id not in self._connections:
 self._connections[job_id] = []
self._connections[job_id].append(websocket)

logger.info(f"WS connected | job={job_id} | total={len(self._connections[job_id])}")

def disconnect(self, job_id: str, websocket: WebSocket):
 if job_id in self._connections:
 try:
 self._connections[job_id].remove(websocket)
 except ValueError:
 pass

 if not self._connections[job_id]:
 del self._connections[job_id]

 logger.info(f"WS disconnected | job={job_id}")

async def broadcast(self, job_id: str, event: str, data: dict):
 """Send a structured event to all clients subscribed to a job."""
 if job_id not in self._connections:
 return

 message = json.dumps({
 "event": event,
 "data": data,
 "timestamp": datetime.now().isoformat(),
 "job_id": job_id
 })

 dead = []
 for ws in self._connections[job_id]:
 try:
 await ws.send_text(message)
 except Exception:
 dead.append(ws)

```

```
for ws in dead:
```

```
 self.disconnect(job_id, ws)
```

```
async def send_step(self, job_id: str, step: str, message: str, data: dict = None):
```

```
 """Helper to send a pipeline step event."""
```

```
 await self.broadcast(job_id, "pipeline_step", {
```

```
 "step": step,
```

```
 "message": message,
```

```
 "details": data or {}
```

```
 })
```

```
async def send_agent_start(self, job_id: str, agent: str):
```

```
 await self.broadcast(job_id, "agent_start", {"agent": agent})
```

```
 async def send_agent_done(self, job_id: str, agent: str, status: str, duration: float,
confidence: float):
```

```
 await self.broadcast(job_id, "agent_done", {
```

```
 "agent": agent,
```

```
 "status": status,
```

```
 "duration": duration,
```

```
 "confidence": confidence
```

```
 })
```

```
 async def send_ceo_decision(self, job_id: str, short_term: str, long_term: str, departments:
list):
```

```
 await self.broadcast(job_id, "ceo_decision", {
```

```
 "short_term_strategy": short_term,
```

```
 "long_term_vision": long_term,
```

```
 "departments": departments
```

```
 })
```

```
 async def send_inter_agent(self, job_id: str, from_agent: str, to_agents: list, summary: str):
```

```
 await self.broadcast(job_id, "inter_agent_context", {
```

```
 "from": from_agent,
```

```
 "to": to_agents,

 "summary": summary[:200]

 })
```

```
 async def send_complete(self, job_id: str, result: dict):

 await self.broadcast(job_id, "complete", result)

 async def send_error(self, job_id: str, error: str):

 await self.broadcast(job_id, "error", {"error": error})
```

```
Singleton instance
```

```
ws_manager = ConnectionManager()
```

```
...
```

```
File: `backend\core\__init__.py`
```

```
```py
```

```
...
```

```
## File: `backend\database\db.py`
```

```
```py
```

```
backend/database/db.py ? Phase 6: PostgreSQL + aiosqlite fallback
```

```
Tries asyncpg (PostgreSQL) first; falls back to aiosqlite if PG is unreachable.
```

```
import json
```

```
import os
```

```
import aiosqlite
```

```
from backend.core.config import POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER,
POSTGRES_PASSWORD
```

```
from backend.core.logger import logger
```

```
_pool = None

_sqlite_path = None

_using_sqlite = False
```

```
async def get_pool():

 global _pool

 if _pool is None:

 raise RuntimeError("DB pool not initialized.")

 return _pool
```

```
async def init_db():

 global _pool, _sqlite_path, _using_sqlite

 # Try PostgreSQL first

 try:

 import asyncpg

 _pool = await asyncpg.create_pool(

 host=POSTGRES_HOST, port=POSTGRES_PORT, database=POSTGRES_DB,

 user=POSTGRES_USER, password=POSTGRES_PASSWORD,

 min_size=2, max_size=10, command_timeout=60

)

 _using_sqlite = False

 logger.info("PostgreSQL pool initialized.")

 await _create_tables_pg()

 return

 except Exception as e:

 logger.warning(f"PostgreSQL unavailable ({e}). Falling back to SQLite.")

 # Fallback: SQLite

 _sqlite_path = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "..",

 "via_local.db")

 _using_sqlite = True

 _pool = True # sentinel so get_pool() doesn't crash
```



```
async def close_db():

 global _pool, _using_sqlite

 if _pool and not _using_sqlite:

 try:

 await _pool.close()

 logger.info("DB pool closed.")

 except Exception:

 pass

??? Helper: get a sqlite connection ??

async def _sq():

 """Get an aiomysql connection."""

 return await aiomysql.connect(_sqlite_path)

??? Table Creation ??

async def _create_tables_pg():

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute("""CREATE TABLE IF NOT EXISTS company_history (

 id SERIAL PRIMARY KEY, task TEXT NOT NULL, result TEXT NOT NULL,

 timestamp TIMESTAMPTZ DEFAULT NOW())""")

 await c.execute("""CREATE TABLE IF NOT EXISTS execution_stats (

 id SERIAL PRIMARY KEY, task TEXT NOT NULL,

 total_agents INT NOT NULL, successful_agents INT NOT NULL,

 failed_agents INT NOT NULL, total_duration REAL NOT NULL,

 timestamp TIMESTAMPTZ DEFAULT NOW())""")
```

```
await c.execute("""CREATE TABLE IF NOT EXISTS decision_audit (
 id SERIAL PRIMARY KEY, task TEXT NOT NULL,
 raw_llm_response TEXT, extracted_json TEXT,
 final_departments TEXT, execution_timeline TEXT,
 timestamp TIMESTAMPTZ DEFAULT NOW()""")

await c.execute("""CREATE TABLE IF NOT EXISTS users (
 id SERIAL PRIMARY KEY, email TEXT UNIQUE NOT NULL,
 hashed_password TEXT NOT NULL, is_active BOOLEAN DEFAULT TRUE,
 is_verified BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMPTZ DEFAULT NOW()""")

await c.execute("""CREATE TABLE IF NOT EXISTS auth_codes (
 id SERIAL PRIMARY KEY, email TEXT NOT NULL,
 code TEXT NOT NULL, code_type TEXT NOT NULL,
 expires_at TIMESTAMPTZ NOT NULL,
 created_at TIMESTAMPTZ DEFAULT NOW()""")

await c.execute("""CREATE TABLE IF NOT EXISTS async_jobs (
 id TEXT PRIMARY KEY, task TEXT NOT NULL, status TEXT NOT NULL,
 result TEXT, error TEXT, created_at TIMESTAMPTZ DEFAULT NOW(),
 updated_at TIMESTAMPTZ DEFAULT NOW()""")

await c.execute("""CREATE TABLE IF NOT EXISTS agent_memory (
 id SERIAL PRIMARY KEY, agent_name TEXT NOT NULL, task TEXT NOT NULL,
 output_summary TEXT, confidence REAL DEFAULT 0.0,
 created_at TIMESTAMPTZ DEFAULT NOW()""")

await c.execute("""CREATE TABLE IF NOT EXISTS meetings (
 id SERIAL PRIMARY KEY, job_id TEXT UNIQUE NOT NULL, task TEXT NOT NULL,
 transcript TEXT, message_count INT DEFAULT 0,
 created_at TIMESTAMPTZ DEFAULT NOW()""")

await c.execute("""CREATE TABLE IF NOT EXISTS chat_history (
 id SERIAL PRIMARY KEY, username TEXT NOT NULL, role TEXT NOT NULL,
 message TEXT NOT NULL, intent TEXT DEFAULT 'chat',
 created_at TIMESTAMPTZ DEFAULT NOW()""")

logger.info("All PostgreSQL tables ready.")
```

```
async def _create_tables_sqlite():
```

```
 async with aiosqlite.connect(_sqlite_path) as db:
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS company_history (
 id INTEGER PRIMARY KEY AUTOINCREMENT, task TEXT NOT NULL, result TEXT NOT NULL,
 timestamp TEXT DEFAULT (datetime('now'))""")
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS execution_stats (
 id INTEGER PRIMARY KEY AUTOINCREMENT, task TEXT NOT NULL,
 total_agents INTEGER NOT NULL, successful_agents INTEGER NOT NULL,
 failed_agents INTEGER NOT NULL, total_duration REAL NOT NULL,
 timestamp TEXT DEFAULT (datetime('now'))""")
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS decision_audit (
 id INTEGER PRIMARY KEY AUTOINCREMENT, task TEXT NOT NULL,
 raw_llm_response TEXT, extracted_json TEXT,
 final_departments TEXT, execution_timeline TEXT,
 timestamp TEXT DEFAULT (datetime('now'))""")
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS users (
 id INTEGER PRIMARY KEY AUTOINCREMENT, email TEXT UNIQUE NOT NULL,
 hashed_password TEXT NOT NULL, is_active INTEGER DEFAULT 1,
 is_verified INTEGER DEFAULT 0,
 created_at TEXT DEFAULT (datetime('now'))""")
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS auth_codes (
 id INTEGER PRIMARY KEY AUTOINCREMENT, email TEXT NOT NULL,
 code TEXT NOT NULL, code_type TEXT NOT NULL,
 expires_at TEXT NOT NULL,
 created_at TEXT DEFAULT (datetime('now'))""")
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS async_jobs (
 id TEXT PRIMARY KEY, task TEXT NOT NULL, status TEXT NOT NULL,
 result TEXT, error TEXT, created_at TEXT DEFAULT (datetime('now')),
 updated_at TEXT DEFAULT (datetime('now'))""")
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS agent_memory (
 id INTEGER PRIMARY KEY AUTOINCREMENT, agent_name TEXT NOT NULL, task TEXT NOT NULL,
 output_summary TEXT, confidence REAL DEFAULT 0.0,
 created_at TEXT DEFAULT (datetime('now'))""")
```

```
 await db.execute("""CREATE TABLE IF NOT EXISTS meetings (
```

```

 id INTEGER PRIMARY KEY AUTOINCREMENT, job_id TEXT UNIQUE NOT NULL, task TEXT NOT NULL,
 transcript TEXT, message_count INTEGER DEFAULT 0,
 created_at TEXT DEFAULT (datetime('now')))"")
 await db.execute("""CREATE TABLE IF NOT EXISTS chat_history (
 id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT NOT NULL, role TEXT NOT NULL,
 message TEXT NOT NULL, intent TEXT DEFAULT 'chat',
 created_at TEXT DEFAULT (datetime('now')))"")
 await db.commit()

logger.info("All SQLite tables ready.")

??? Core DB Operations ???

async def save_record(task, result):
 if _using_sqlite:
 async with aiosqlite.connect(_sqlite_path) as db:
 await db.execute("INSERT INTO company_history (task, result) VALUES (?, ?)", (task,
result))
 await db.commit()
 else:
 pool = await get_pool()
 async with pool.acquire() as c:
 await c.execute("INSERT INTO company_history (task, result) VALUES ($1, $2)", task,
result)

async def save_execution_stat(task, total, success, failed, duration):
 if _using_sqlite:
 async with aiosqlite.connect(_sqlite_path) as db:
 await db.execute(
 "INSERT INTO execution_stats (task, total_agents, successful_agents,
failed_agents, total_duration) VALUES (?, ?, ?, ?, ?)",
 (task, total, success, failed, duration))
 await db.commit()
 else:
 pool = await get_pool()

```

```

 async with pool.acquire() as c:

 await c.execute(

 "INSERT INTO execution_stats (task, total_agents, successful_agents,
failed_agents, total_duration) VALUES ($1,$2,$3,$4,$5)",

 task, total, success, failed, duration)

async def save_audit_record(task, raw_llm, extracted, departments, timeline):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute(

 "INSERT INTO decision_audit (task, raw_llm_response, extracted_json,
final_departments, execution_timeline) VALUES (?, ?, ?, ?, ?)",

 (task, raw_llm, json.dumps(extracted), json.dumps(departments),
json.dumps(timeline)))

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute(

 "INSERT INTO decision_audit (task, raw_llm_response, extracted_json,
final_departments, execution_timeline) VALUES ($1,$2,$3,$4,$5)",

 task, raw_llm, json.dumps(extracted), json.dumps(departments),
json.dumps(timeline))

async def get_recent_history(limit=10):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 db.row_factory = aiosqlite.Row

 cursor = await db.execute("SELECT task, result, timestamp FROM company_history ORDER
BY id DESC LIMIT ?", (limit,))

 rows = await cursor.fetchall()

 return [{"task": r["task"], "result": json.loads(r["result"]) if r["result"] else {},
"timestamp": str(r["timestamp"])} for r in rows]

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

```

```
 rows = await c.fetch("SELECT task, result, timestamp FROM company_history ORDER BY id
DESC LIMIT $1", limit)
```

```
 return [{"task": r["task"], "result": json.loads(r["result"]) if r["result"] else {},
"timestamp": str(r["timestamp"])} for r in rows]
```

```
async def get_system_health():
```

```
 if _using_sqlite:
```

```
 async with aiosqlite.connect(_sqlite_path) as db:
```

```
 cursor = await db.execute("""SELECT COUNT(*) AS runs, COALESCE(SUM(total_agents),0) AS
ta,
```

```
 COALESCE(SUM(successful_agents),0) AS ts, COALESCE(SUM(failed_agents),0) AS tf,
```

```
 COALESCE(AVG(total_duration),0) AS avg_d, COALESCE(MAX(total_duration),0) AS
max_d,
```

```
 COALESCE(MIN(total_duration),0) AS min_d FROM execution_stats""")
```

```
 r = await cursor.fetchone()
```

```
 ta = r[1] or 0; tf = r[3] or 0
```

```
 return {
```

```
 "total_runs": r[0], "total_agents_executed": ta,
```

```
 "total_successful": r[2], "total_failed": tf,
```

```
 "failure_rate_percent": round(tf/ta*100, 2) if ta else 0.0,
```

```
 "avg_duration_seconds": round(float(r[4] or 0), 2),
```

```
 "max_duration_seconds": round(float(r[5] or 0), 2),
```

```
 "min_duration_seconds": round(float(r[6] or 0), 2)
```

```
 }
```

```
 else:
```

```
 pool = await get_pool()
```

```
 async with pool.acquire() as c:
```

```
 row = await c.fetchrow("""SELECT COUNT(*) AS runs, COALESCE(SUM(total_agents),0) AS
ta,
```

```
 COALESCE(SUM(successful_agents),0) AS ts, COALESCE(SUM(failed_agents),0) AS tf,
```

```
 COALESCE(AVG(total_duration),0) AS avg_d, COALESCE(MAX(total_duration),0) AS
max_d,
```

```
 COALESCE(MIN(total_duration),0) AS min_d FROM execution_stats""")
```

```
 ta = row["ta"] or 0; tf = row["tf"] or 0
```

```
 return {
```

```
 "total_runs": row["runs"], "total_agents_executed": ta,
```

```

 "total_successful": row["ts"], "total_failed": tf,
 "failure_rate_percent": round(tf/ta*100, 2) if ta else 0.0,
 "avg_duration_seconds": round(float(row["avg_d"]), 2),
 "max_duration_seconds": round(float(row["max_d"]), 2),
 "min_duration_seconds": round(float(row["min_d"]), 2)
 }
}

```

```

async def get_company_status():

```

```

 if _using_sqlite:

```

```

 async with aiosqlite.connect(_sqlite_path) as db:

```

```

 cursor = await db.execute("""SELECT COUNT(*) AS runs, COALESCE(AVG(total_duration),0)
AS avg_d,

```

```

 COALESCE(CAST(SUM(failed_agents) AS FLOAT)/MAX(SUM(total_agents),1),0) AS
fail_ratio

```

```

 FROM execution_stats""")

```

```

 stats = await cursor.fetchone()

```

```

 cursor2 = await db.execute("SELECT result FROM company_history ORDER BY id DESC LIMIT
50")

```

```

 rows = await cursor2.fetchall()

```

```

 dept_counts = {}

```

```

 for row in rows:

```

```

 try:

```

```

 res = json.loads(row[0])

```

```

 for d in res.get("selected_departments", []):

```

```

 dept_counts[d] = dept_counts.get(d, 0) + 1

```

```

 except Exception: pass

```

```

 most_active = max(dept_counts, key=dept_counts.get) if dept_counts else "N/A"

```

```

 return {

```

```

 "total_executions": stats[0],

```

```

 "failure_rate_percent": round((stats[2] or 0)*100, 2),

```

```

 "average_response_time_seconds": round(float(stats[1] or 0), 2),

```

```

 "most_active_department": most_active,

```

```

 "department_activity": dept_counts

```

```

 }

```

```

 else:

```

```

pool = await get_pool()

async with pool.acquire() as c:

 stats = await c.fetchrow("""SELECT COUNT(*) AS runs, COALESCE(AVG(total_duration),0)
AS avg_d,
 COALESCE(CAST(SUM(failed_agents) AS FLOAT)/NULLIF(SUM(total_agents),0),0) AS
fail_ratio
 FROM execution_stats""")

 rows = await c.fetch("SELECT result FROM company_history ORDER BY id DESC LIMIT 50")

 dept_counts = {}

 for row in rows:

 try:

 res = json.loads(row["result"])

 for d in res.get("selected_departments", []):

 dept_counts[d] = dept_counts.get(d, 0) + 1

 except Exception: pass

 most_active = max(dept_counts, key=dept_counts.get) if dept_counts else "N/A"

 return {

 "total_executions": stats["runs"],

 "failure_rate_percent": round((stats["fail_ratio"] or 0)*100, 2),

 "average_response_time_seconds": round(float(stats["avg_d"]), 2),

 "most_active_department": most_active,

 "department_activity": dept_counts

 }

```

```

async def create_job(job_id: str, task: str):
 if _using_sqlite:
 async with aiosqlite.connect(_sqlite_path) as db:
 await db.execute("INSERT INTO async_jobs (id, task, status) VALUES (?, ?, 'pending')",
 (job_id, task))
 await db.commit()
 else:
 pool = await get_pool()
 async with pool.acquire() as c:

```



```
 await c.execute("INSERT INTO async_jobs (id, task, status) VALUES ($1,$2,'pending')",
job_id, task)
```

```
async def update_job(job_id: str, status: str, result: str = None, error: str = None):
```

```
 if _using_sqlite:
```

```
 async with aiosqlite.connect(_sqlite_path) as db:
```

```
 await db.execute(
```

```
 "UPDATE async_jobs SET status=?, result=?, error=?, updated_at=datetime('now')
WHERE id=?",
```

```
 (status, result, error, job_id))
```

```
 await db.commit()
```

```
 else:
```

```
 pool = await get_pool()
```

```
 async with pool.acquire() as c:
```

```
 await c.execute(
```

```
 "UPDATE async_jobs SET status=$1, result=$2, error=$3, updated_at=NOW() WHERE
id=$4",
```

```
 status, result, error, job_id)
```

```
async def get_job(job_id: str):
```

```
 if _using_sqlite:
```

```
 async with aiosqlite.connect(_sqlite_path) as db:
```

```
 db.row_factory = aiosqlite.Row
```

```
 cursor = await db.execute("SELECT id, task, status, result, error, created_at,
updated_at FROM async_jobs WHERE id=?", (job_id,))
```

```
 row = await cursor.fetchone()
```

```
 if not row: return None
```

```
 res = dict(row)
```

```
 if res.get("result"):
```

```
 try: res["result"] = json.loads(res["result"])
```

```
 except Exception: pass
```

```
 res["created_at"] = str(res["created_at"])
```

```
 res["updated_at"] = str(res["updated_at"])
```

```
 return res
```

```

else:

 pool = await get_pool()

 async with pool.acquire() as c:

 row = await c.fetchrow("SELECT id, task, status, result, error, created_at, updated_at
FROM async_jobs WHERE id=$1", job_id)

 if not row: return None

 res = dict(row)

 if res.get("result"):

 try: res["result"] = json.loads(res["result"])

 except Exception: pass

 res["created_at"] = str(res["created_at"])

 res["updated_at"] = str(res["updated_at"])

 return res

??? User Operations ???

async def get_user_by_email(email):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 db.row_factory = aiosqlite.Row

 cursor = await db.execute("SELECT id, email, hashed_password, is_active, is_verified
FROM users WHERE email=?", (email,))

 row = await cursor.fetchone()

 return dict(row) if row else None

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 row = await c.fetchrow("SELECT id, email, hashed_password, is_active, is_verified FROM
users WHERE email=$1", email)

 return dict(row) if row else None

Keep old function name as alias so existing code doesn't break

async def get_user_by_username(username):

 return await get_user_by_email(username)

```

```

async def create_user(email, hashed_password):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute("INSERT INTO users (email, hashed_password) VALUES (?,?)", (email,
hashed_password))

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute("INSERT INTO users (email, hashed_password) VALUES ($1,$2)", email,
hashed_password)

async def verify_user_email(email):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute("UPDATE users SET is_verified=1 WHERE email=?", (email,))

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute("UPDATE users SET is_verified=TRUE WHERE email=$1", email)

async def update_user_password(email, hashed_password):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute("UPDATE users SET hashed_password=? WHERE email=?", (hashed_password,
email))

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute("UPDATE users SET hashed_password=$1 WHERE email=$2", hashed_password,

```

email)

```
async def save_auth_code(email, code, code_type, expires_at):

 """Save a verification or reset code to the database."""

 from datetime import datetime as _dt

 # Normalize expires_at to datetime object for PG compatibility
 if isinstance(expires_at, str):

 try:

 expires_dt = _dt.strptime(expires_at, '%Y-%m-%d %H:%M:%S')

 except Exception:

 expires_dt = expires_at

 else:

 expires_dt = expires_at

 # Clear old codes of this type first

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute("DELETE FROM auth_codes WHERE email=? AND code_type=?", (email,
code_type))

 await db.execute("INSERT INTO auth_codes (email, code, code_type, expires_at) VALUES
(?,?,?,?)",

 (email, code, code_type, expires_at)) # SQLite takes string fine

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute("DELETE FROM auth_codes WHERE email=$1 AND code_type=$2", email,
code_type)

 await c.execute("INSERT INTO auth_codes (email, code, code_type, expires_at) VALUES
($1,$2,$3,$4)",

 email, code, code_type, expires_dt) # PG needs datetime object

async def get_auth_code(email, code, code_type):

 """Get a valid (non-expired) auth code for an email."""
```

```

if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 db.row_factory = aiosqlite.Row

 cursor = await db.execute(

 "SELECT id, email, code, code_type, expires_at FROM auth_codes WHERE email=? AND
code=? AND code_type=? AND expires_at > datetime('now')",

 (email, code, code_type))

 row = await cursor.fetchone()

 return dict(row) if row else None

else:

 pool = await get_pool()

 async with pool.acquire() as c:

 row = await c.fetchrow(

 "SELECT id, email, code, code_type, expires_at FROM auth_codes WHERE email=$1 AND
code=$2 AND code_type=$3 AND expires_at > NOW()",

 email, code, code_type)

 return dict(row) if row else None

async def delete_auth_code(email, code_type):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute("DELETE FROM auth_codes WHERE email=? AND code_type=?", (email,
code_type))

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute("DELETE FROM auth_codes WHERE email=$1 AND code_type=$2", email,
code_type)

??? Agent Memory ???

async def save_agent_mem(agent: str, task: str, output_summary: str, confidence: float):

 if _using_sqlite:

```

```

 async with aiosqlite.connect(_sqlite_path) as db:
 await db.execute(
 "INSERT INTO agent_memory (agent_name, task, output_summary, confidence) VALUES
(?,?,?,?)",
 (agent, task, output_summary, confidence))
 await db.commit()
 else:
 pool = await get_pool()
 async with pool.acquire() as c:
 await c.execute(
 "INSERT INTO agent_memory (agent_name, task, output_summary, confidence) VALUES
($1,$2,$3,$4)",
 agent, task, output_summary, confidence)

```

```

async def get_agent_mem(agent: str, limit: int = 5):
 if _using_sqlite:
 async with aiosqlite.connect(_sqlite_path) as db:
 db.row_factory = aiosqlite.Row
 cursor = await db.execute(
 "SELECT task, output_summary, confidence, created_at FROM agent_memory WHERE
agent_name=? ORDER BY id DESC LIMIT ?",
 (agent, limit))
 return await cursor.fetchall()
 else:
 pool = await get_pool()
 async with pool.acquire() as c:
 return await c.fetch(
 "SELECT task, output_summary, confidence, created_at FROM agent_memory WHERE
agent_name=$1 ORDER BY id DESC LIMIT $2",
 agent, limit)

```

```

async def get_all_mem(limit: int = 50):
 if _using_sqlite:
 async with aiosqlite.connect(_sqlite_path) as db:

```

```

async def save_meeting_db(job_id: str, task: str, transcript: str, message_count: int):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute(

 """INSERT INTO meetings (job_id, task, transcript, message_count) VALUES (?, ?, ?, ?)

 ON CONFLICT(job_id) DO UPDATE SET transcript=excluded.transcript,
message_count=excluded.message_count""",

 (job_id, task, transcript, message_count))

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute(

 """INSERT INTO meetings (job_id, task, transcript, message_count)

 VALUES ($1, $2, $3, $4)

 ON CONFLICT (job_id) DO UPDATE

 SET transcript=$3, message_count=$4""",

 job_id, task, transcript, message_count)

```

```

async def get_meeting_db(job_id: str):
 if _using_sqlite:
 async with aiosqlite.connect(_sqlite_path) as db:
 db.row_factory = aiosqlite.Row
 cursor = await db.execute(
 "SELECT job_id, task, transcript, message_count, created_at FROM meetings WHERE
job_id=?",
 (job_id,))
 row = await cursor.fetchone()
 return dict(row) if row else None
 else:
 pool = await get_pool()
 async with pool.acquire() as c:
 row = await c.fetchrow(
 "SELECT job_id, task, transcript, message_count, created_at FROM meetings WHERE
job_id=$1",
 job_id)
 return dict(row) if row else None

async def get_recent_meetings_db(limit: int = 10):
 if _using_sqlite:
 async with aiosqlite.connect(_sqlite_path) as db:
 db.row_factory = aiosqlite.Row
 cursor = await db.execute(
 "SELECT job_id, task, message_count, created_at FROM meetings ORDER BY id DESC
LIMIT ?",
 (limit,))
 return await cursor.fetchall()
 else:
 pool = await get_pool()
 async with pool.acquire() as c:
 return await c.fetch(
 "SELECT job_id, task, message_count, created_at FROM meetings ORDER BY id DESC
LIMIT $1",

```



```
limit)
```

```
??? Chat History ???
```

```
async def save_chat_message(username: str, role: str, message: str, intent: str = "chat"):
```

```
 if _using_sqlite:
```

```
 async with aiosqlite.connect(_sqlite_path) as db:
```

```
 await db.execute(
```

```
 "INSERT INTO chat_history (username, role, message, intent) VALUES (?, ?, ?, ?)",
```

```
 (username, role, message, intent))
```

```
 await db.commit()
```

```
 else:
```

```
 pool = await get_pool()
```

```
 async with pool.acquire() as c:
```

```
 await c.execute(
```

```
 "INSERT INTO chat_history (username, role, message, intent) VALUES ($1, $2, $3, $4)",
```

```
 username, role, message, intent)
```

```
async def get_chat_history(username: str, limit: int = 50):
```

```
 if _using_sqlite:
```

```
 async with aiosqlite.connect(_sqlite_path) as db:
```

```
 db.row_factory = aiosqlite.Row
```

```
 cursor = await db.execute(
```

```
 "SELECT role, message, intent, created_at FROM chat_history WHERE username=? ORDER
BY id DESC LIMIT ?",
```

```
 (username, limit))
```

```
 rows = await cursor.fetchall()
```

```
 return [
```

```
 {"role": r["role"], "message": r["message"], "intent": r["intent"],
```

```
 "timestamp": str(r["created_at"])}
```

```
 for r in reversed(rows or [])
```

```
]
```

```
 else:
```

```

 pool = await get_pool()

 async with pool.acquire() as c:

 rows = await c.fetch(

 "SELECT role, message, intent, created_at FROM chat_history WHERE username=$1
ORDER BY id DESC LIMIT $2",

 username, limit)

 return [

 {"role": r["role"], "message": r["message"], "intent": r["intent"],

 "timestamp": str(r["created_at"])}

 for r in reversed(rows or [])

]

```

```

async def clear_chat_history(username: str):

 if _using_sqlite:

 async with aiosqlite.connect(_sqlite_path) as db:

 await db.execute("DELETE FROM chat_history WHERE username=?", (username,))

 await db.commit()

 else:

 pool = await get_pool()

 async with pool.acquire() as c:

 await c.execute("DELETE FROM chat_history WHERE username=$1", username)

 ...

```

## File: `backend\database\\_\_init\_\_.py`

```

```py

```

```

...

```

File: `backend\Dockerfile`

```

```dockerfile

```

```

...

```

```
File: `backend\main.py`
```

```
```py
```

```
# backend/main.py ? Phase 6: Enterprise Engine + Chat Mode
```

```
#
```

```
# FIXES APPLIED:
```

```
#   FIX 1 ? _is_repo_healthy() now does a real HTTP GET to GitHub Pages URL.
```

```
#           GitHub API status="built" does NOT mean 200 OK. Only an actual
```

```
#           HTTP 200 from gollavinaykumar1.github.io/{repo}/ confirms live.
```

```
#   FIX 2 ? _fix_single_repo() no longer uses time.sleep(5) before dispatch.
```

```
#           It delegates to github_pusher._trigger_workflow() which uses the
```

```
#           proper _workflow_exists() loop + 30-second initial wait.
```

```
#   FIX 3 ? Cooldown guard: repos attempted within the last 30 minutes are
```

```
#           skipped entirely, breaking the infinite fix loop on every restart.
```

```
#   FIX 4 ? Non-VIA repos (my-portfolio, .github, etc.) are explicitly skipped.
```

```
import json, time, uuid, os, asyncio, base64, random, string
```

```
import requests as _requests
```

```
from datetime import datetime, timedelta
```

```
from fastapi import FastAPI, HTTPException, Depends, WebSocket, WebSocketDisconnect
```

```
from fastapi.responses import FileResponse
```

```
from fastapi.staticfiles import StaticFiles
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
from starlette.middleware.base import BaseHTTPMiddleware
```

```
from pydantic import BaseModel, Field
```

```
from typing import Optional, List
```

```
from fastapi.security import OAuth2PasswordRequestForm
```

```
from backend.agents.ceo_agent import ceo_agent
```

```
from backend.agents.agent_executor import execute_agents
```

```
from backend.core.tracer import ExecutionTracer
```

```
from backend.core.scaling_engine import autonomous_scale
```

```
from backend.core.hierarchy import get_active_structure, get_full_chart
```

```
from backend.core.ws_manager import ws_manager

from backend.core.inter_agent_bus import InterAgentBus

from backend.core.logger import logger

from backend.core.config import APP_VERSION

from backend.core.memory_store import save_agent_memory, get_agent_memory, get_all_memories

from backend.core.fullstack_builder import detect_app_type, generate_backend_files_llm

from backend.core.intent_detector import detect_intent

from backend.core import chat_engine

from backend.database.db import (
    init_db, close_db,
    save_record, save_execution_stat, save_audit_record,
    get_recent_history, get_system_health, get_company_status,
    create_job, update_job, get_job,
    save_chat_message, get_chat_history, clear_chat_history,
    save_auth_code, get_auth_code, delete_auth_code,
    verify_user_email, update_user_password, get_user_by_email
)

from backend.auth.auth import (
    Token, UserCreate, get_current_active_user,
    authenticate_user, register_user, create_access_token
)

from backend.middleware.rate_limiter import rate_limit_middleware

from backend.routers.meeting_router import router as meeting_router

from backend.routers.template_router import router as template_router

from backend.routers.filebrowser_router import router as filebrowser_router

from backend.core.github_pusher import github_pusher

from backend.core.render_deployer import render_deployer

from backend.utils.email_sender import send_verification_email, send_reset_password_email

app = FastAPI(
    title="VIA ? Autonomous AI Digital Team | Phase 6 Enterprise",
    version=APP_VERSION,
)
```



```
DEPLOY_YML = """name: Deploy React to GitHub Pages
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  workflow_dispatch:
```

```
permissions:
```

```
  contents: write
```

```
jobs:
```

```
  build-and-deploy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout
```

```
        uses: actions/checkout@v4
```

```
      - name: Setup Node.js
```

```
        uses: actions/setup-node@v4
```

```
        with:
```

```
          node-version: '20'
```

```
      - name: Install dependencies
```

```
        run: npm install
```

```
      - name: Build
```

```
        run: npm run build
```

```
        env:
```

```
          CI: "false"
```

```
          VITE_API_URL: ${ vars.VITE_API_URL }
```

```
      - name: Deploy to GitHub Pages
```

```
        uses: peaceiris/actions-gh-pages@v4
```

```

with:

    github_token: ${ secrets.GITHUB_TOKEN }

    publish_dir: ./dist

    force_orphan: true

"""

# ?? FIX 3: File-backed cooldown store (survives restarts) ??????????????????
# Tracks last fix attempt per repo (epoch float).
# Repos attempted within COOLDOWN_SECONDS are skipped on next startup.
# CHANGE 1: replaced in-memory dict with file-backed version so cooldowns
#           survive VIA restarts ? the root cause of all repos redeploying
#           every time VIA restarted (in-memory {} was wiped on each restart).
COOLDOWN_SECONDS = 1800 # 30 minutes

_COOLDOWN_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), ".fix_cooldown.json")

def _load_cooldown() -> dict:
    try:
        if os.path.exists(_COOLDOWN_FILE):
            with open(_COOLDOWN_FILE, "r") as f:
                return json.load(f)
    except Exception:
        pass
    return {}

def _save_cooldown(data: dict):
    try:
        with open(_COOLDOWN_FILE, "w") as f:
            json.dump(data, f)
    except Exception:
        pass

_fix_cooldown: dict = _load_cooldown()

# ?? FIX 4: Repos that should NEVER be touched by the fix loop ??????????????????

```

```
_SKIP_REPO_NAMES = {  
    "my-portfolio",  
    ".github",  
}
```

```
def _should_skip_repo(repo_name: str) -> bool:  
    """Return True for repos that are not VIA-generated apps."""  
    if repo_name in _SKIP_REPO_NAMES:  
        return True  
  
    # Skip repos with no dash in name (likely personal repos, not VIA slugs)  
    # VIA always slugifies task ? always contains dashes  
  
    return False
```

```
def _is_on_cooldown(repo_name: str) -> bool:  
    """FIX 3: Return True if this repo was attempted within the last 30 minutes."""  
    last = _fix_cooldown.get(repo_name)  
  
    if last is None:  
        return False  
  
    return (time.time() - last) < COOLDOWN_SECONDS
```

```
def _mark_fix_attempted(repo_name: str):  
    """FIX 3: Record the current timestamp and persist so restarts don't wipe it."""  
    _fix_cooldown[repo_name] = time.time()  
    _save_cooldown(_fix_cooldown) # CHANGE 2: persist to disk so restart doesn't reset cooldown
```

```
def _is_repo_healthy(repo: str, username: str, headers: dict) -> bool:  
    """  
  
    FIX 1: Real health check ? 3 layers:  
  
    Layer 1: GitHub API confirms Pages is enabled and status="built"  
  
    Layer 2: deploy.yml exists in the repo
```


Layer 3: Actual HTTP GET to the Pages URL returns 200 (THE KEY FIX)

GitHub API can show status="built" while the site still returns 404.

Only a real HTTP 200 from the live URL confirms the app is truly healthy.

"""

try:

```
s = _requests.Session()
```

```
# Layer 1 ? GitHub Pages API
```

```
r = s.get(
    f"https://api.github.com/repos/{username}/{repo}/pages",
    headers=headers, timeout=15
)
```

```
if r.status_code != 200:
```

```
    return False
```

```
pages_data = r.json()
```

```
if pages_data.get("status") not in ("built", "building"):
```

```
    return False
```

```
# Layer 2 ? deploy.yml must exist
```

```
r2 = s.get(
```

```
f"https://api.github.com/repos/{username}/{repo}/contents/.github/workflows/deploy.yml",
    headers=headers, timeout=15
)
```

```
if r2.status_code != 200:
```

```
    return False
```

```
# Layer 3 ? FIX 1: Actual HTTP GET to the Pages URL
```

```
# This is what was missing ? GitHub API lies about "built" status.
```

```
pages_url = f"https://{username}.github.io/{repo}/"
```

```
try:
```

```
    live_check = s.get(pages_url, timeout=10, allow_redirects=True)
```

```
    if live_check.status_code != 200:
```

```
        logger.debug(f"Health check | Pages URL returned {live_check.status_code} |
```

```

{repo}")

        return False

    except Exception:

        # If the HTTP GET itself fails (timeout, DNS), treat as unhealthy

        return False

    logger.info(f"Startup fix | repo healthy, skipping | {repo}")

    return True

except Exception:

    return False

def _fix_single_repo(repo: str, username: str, headers: dict):
    """
    FIX 1: _is_repo_healthy() now does real HTTP GET ? broken repos can't hide.
    FIX 2: Workflow is triggered via github_pusher._trigger_workflow() which:
        - Waits 30s initial (not 5s)
        - Uses _workflow_exists() loop to confirm GitHub indexed the file
        - Retries up to 6 times with increasing back-off
    FIX 3: Cooldown check at top ? skip repos fixed in the last 30 minutes.
    FIX 4: Skip non-VIA repos entirely.
    FIX 5: Patch src/api.js directly in the repo to add any missing exports
            that App.jsx imports ? fixes "X is not exported by src/api.js" build errors.
    """
    if _should_skip_repo(repo):

        return

    if _is_on_cooldown(repo):

        logger.info(f"Startup fix | cooldown active, skipping | {repo}")

        return

    if _is_repo_healthy(repo, username, headers):

        return

```

```

_mark_fix_attempted(repo)

try:

    s = _requests.Session()

    # Push deploy.yml

    r = s.get(

f"https://api.github.com/repos/{username}/{repo}/contents/.github/workflows/deploy.yml",

        headers=headers, timeout=30

    )

    sha = r.json().get("sha") if r.status_code == 200 else None

    payload = {

        "message": "Add deploy.yml via VIA auto-fix",

        "content": base64.b64encode(DEPLOY_YML.encode()).decode(),

    }

    if sha:

        payload["sha"] = sha

    s.put(

f"https://api.github.com/repos/{username}/{repo}/contents/.github/workflows/deploy.yml",

        headers=headers, json=payload, timeout=30

    )


# Enable GitHub Pages ? FIX: use gh-pages branch source, not workflow mode.
# peaceiris/actions-gh-pages pushes built files to the gh-pages branch.
# Using {"build_type": "workflow"} here is incompatible and causes 404s.
r_pages = s.post(

    f"https://api.github.com/repos/{username}/{repo}/pages",

    headers=headers,

    json={"source": {"branch": "gh-pages", "path": "/"}} ,

    timeout=30,

)

if r_pages.status_code not in (201, 409):

```

```
# Pages may already exist with wrong source ? update it
```

```
s.put(  
    f"https://api.github.com/repos/{username}/{repo}/pages",  
    headers=headers,  
    json={"source": {"branch": "gh-pages", "path": "/"}},  
    timeout=30,  
)
```

```
# Set VITE_API_URL
```

```
render_url = f"https://{repo}.onrender.com"
```

```
r2 = s.post(  
    f"https://api.github.com/repos/{username}/{repo}/actions/variables",  
    headers=headers, json={"name": "VITE_API_URL", "value": render_url}, timeout=30  
)
```

```
if r2.status_code not in (201, 204):
```

```
s.patch(  
    f"https://api.github.com/repos/{username}/{repo}/actions/variables/VITE_API_URL",  
    headers=headers, json={"name": "VITE_API_URL", "value": render_url}, timeout=30  
)
```

```
# FIX 5: Patch src/api.js to add any missing exports App.jsx needs
```

```
_fix_repo_api_js(repo, username, headers, s)
```

```
# FIX 2: Use github_pusher._trigger_workflow() ? proper 30s wait + retry loop
```

```
github_pusher._trigger_workflow(repo)
```

```
logger.info(f"Startup fix | repo fixed | {repo}")
```

```
except Exception as e:
```

```
    logger.warning(f"Startup fix | repo skipped (error) | {repo} | {e}")
```

```
def _fix_repo_api_js(repo: str, username: str, headers: dict, s):
```

```
    """
```

FIX 5: Fetch src/App.jsx and src/api.js from GitHub, check for missing exports, patch api.js with stubs, and push the fix back.

This resolves: 'getStats is not exported by src/api.js, imported by src/App.jsx' without needing a full re-push of all files.

```
"""
```

```
import re as _re
```

```
try:
```

```
    # Fetch App.jsx to find what it imports from api.js
```

```
    app_r = s.get(
```

```
        f"https://api.github.com/repos/{username}/{repo}/contents/src/App.jsx",
```

```
        headers=headers, timeout=15
```

```
    )
```

```
    if app_r.status_code != 200:
```

```
        return # No App.jsx ? nothing to fix
```

```
    app_content = base64.b64decode(app_r.json()["content"]).decode("utf-8", errors="ignore")
```

```
    # Fetch current api.js
```

```
    api_r = s.get(
```

```
        f"https://api.github.com/repos/{username}/{repo}/contents/src/api.js",
```

```
        headers=headers, timeout=15
```

```
    )
```

```
    if api_r.status_code != 200:
```

```
        return # No api.js ? can't patch
```

```
    api_data = api_r.json()
```

```
    api_content = base64.b64decode(api_data["content"]).decode("utf-8", errors="ignore")
```

```
    api_sha = api_data["sha"]
```

```
    # Find what App.jsx imports from api.js
```

```
    import_pattern = _re.compile(
```

```
        r'import\s*\{([^\}]+\)}\s*from\s*["\'](?:\.\/)*api(?:\.js)?["\']'
```

```

)
needed = set()
for match in import_pattern.finditer(app_content):
    names = [n.strip().split(" as ")[0].strip() for n in match.group(1).split(",")]
    needed.update(n for n in names if n)

if not needed:
    return

# Find what api.js already exports
existing = set(_re.findall(r'export\s+(?:const|function|async function)\s+(\w+)',
api_content))

missing = needed - existing

if not missing:
    return

logger.info(f"Startup fix | api.js missing exports {missing} ? patching | {repo}")

# Detect the API base URL pattern
base_match = _re.search(r'(API_URL|API_BASE|BASE_URL)\s*=\s*[^\n;]+', api_content)
if base_match:
    url_expr = "${" + base_match.group(1) + "}"
else:
    url_expr = "${import.meta.env.VITE_API_URL || ''}"

stubs = "\n\n// Auto-patched missing exports by VIA startup fix\n"
for name in sorted(missing):
    if name == "getStats" or name.startswith("get"):
        resource = name[3:].lower() if name != "getStats" else "stats"
        stubs += f"""export const {name} = async () => {{
const r = await fetch(`${url_expr}/api/v1/{resource}`);
if (!r.ok) throw new Error('{name} failed');
return r.json();
}};\n"""

```

```

        elif name.startswith("create"):

            resource = name[6:].lower()

            stubs += f"""export const {name} = async (data) => {{
const r = await fetch(`${url_expr}/api/v1/{resource}`, {{
    method: 'POST', headers: {{ 'Content-Type': 'application/json' }},
    body: JSON.stringify(data),
}}});
if (!r.ok) throw new Error('{name} failed');
return r.json();
}};\n"""

        elif name.startswith("update"):

            resource = name[6:].lower()

            stubs += f"""export const {name} = async (id, data) => {{
const r = await fetch(`${url_expr}/api/v1/{resource}/${{id}}`, {{
    method: 'PUT', headers: {{ 'Content-Type': 'application/json' }},
    body: JSON.stringify(data),
}}});
if (!r.ok) throw new Error('{name} failed');
return r.json();
}};\n"""

        elif name.startswith("delete"):

            resource = name[6:].lower()

            stubs += f"""export const {name} = async (id) => {{
const r = await fetch(`${url_expr}/api/v1/{resource}/${{id}}`, {{
    method: 'DELETE',
}}});
if (!r.ok) throw new Error('{name} failed');
return r.json();
}};\n"""

        else:

            stubs += f"""export const {name} = async (...args) => {{
const r = await fetch(`${url_expr}/api/v1/{name.lower()}`);
if (!r.ok) throw new Error('{name} failed');
return r.json();
}

```

```
}};\n"""
```

```
patched = api_content + stubs
```

```
push_r = s.put(
```

```
    f"https://api.github.com/repos/{username}/{repo}/contents/src/api.js",
```

```
    headers=headers,
```

```
    json={
```

```
        "message": "fix: add missing api.js exports (VIA auto-fix)",
```

```
        "content": base64.b64encode(patched.encode()).decode(),
```

```
        "sha": api_sha,
```

```
    },
```

```
    timeout=30
```

```
)
```

```
if push_r.status_code in (200, 201):
```

```
    logger.info(f"Startup fix | api.js patched OK | {repo}")
```

```
else:
```

```
    logger.warning(f"Startup fix | api.js patch failed {push_r.status_code} | {repo}")
```

```
except Exception as e:
```

```
    logger.warning(f"Startup fix | api.js patch error | {repo} | {e}")
```

```
async def _fix_all_repos():
```

```
    await asyncio.sleep(5)
```

```
    token = os.getenv("GITHUB_TOKEN", "")
```

```
    username = os.getenv("GITHUB_USERNAME", "")
```

```
    if not token or not username:
```

```
        logger.warning("Startup fix | GITHUB_TOKEN or GITHUB_USERNAME not set ? skipping")
```

```
    return
```

```
headers = {
```

```
    "Authorization": f"token {token}",
```

```
    "Accept": "application/vnd.github+json",
```

```
    "Content-Type": "application/json",
```



```
repos, page = [], 1

try:

    while True:

        r = _requests.get(

            f"https://api.github.com/user/repos?per_page=100&page={page}",

            headers=headers, timeout=30

        )

        data = r.json()

        if not data:

            break

        repos.extend([d["name"] for d in data if not d["private"]])

        if len(data) < 100:

            break

        page += 1

except Exception as e:

    logger.warning(f"Startup fix | Could not fetch repo list: {e}")

    return


logger.info(f"Startup fix | {len(repos)} repos found ? checking health...")

for repo in repos:

    await asyncio.to_thread(_fix_single_repo, repo, username, headers)

    await asyncio.sleep(2)

logger.info("Startup fix | All repos checked!")
```

? App Lifecycle ??

```
p.on_event("startup")

nc def on_startup():

    await init_db()

logger.info(f"VIA Phase 6 v{APP_VERSION} started ? Enterprise Engine online.")
```

```
@app.on_event("shutdown")
async def on_shutdown():
    await close_db()


# ?? Helpers ??????????????????????????????????????????????????????????????????????


def _extract_frontend_path(dept_output: dict):
    fe = dept_output.get("frontend", {})
    if not fe or fe.get("status") != "success":
        return None

    outer = fe.get("output", {})
    inner = outer.get("output", outer)
    path = inner.get("department_path") or inner.get("project_path")

    if path:
        logger.info(f"Frontend path found | {path}")
    else:
        logger.warning(f"Frontend path NOT found | outer={list(outer.keys())} | inner={list(inner.keys())}")

    return path


@app.websocket("/ws/{job_id}")
async def websocket_endpoint(websocket: WebSocket, job_id: str):
    await ws_manager.connect(job_id, websocket)

    try:
        while True:
            await websocket.receive_text()
    except WebSocketDisconnect:
        ws_manager.disconnect(job_id, websocket)


@app.post("/auth/register")
async def register(user: UserCreate):
    import re

    email = user.email.strip().lower()

    # Basic email validation
```

```

if not re.match(r'^^[^@]+@[^@]+\.[^@]+$', email):
    raise HTTPException(status_code=400, detail="Invalid email address.")

result = await register_user(email, user.password)

# Generate 6-digit verification code, valid for 15 minutes
code = ''.join(random.choices(string.digits, k=6))
expires_at = (datetime.utcnow() + timedelta(minutes=15)).strftime('%Y-%m-%d %H:%M:%S')
await save_auth_code(email, code, 'verify', expires_at)

# Log the code and send email
logger.info(f"[EMAIL VERIFICATION] Code for {email}: {code} (expires: {expires_at} UTC)")
await send_verification_email(email, code)

    return {"message": "Registration successful. Please check your email for the 6-digit
verification code.",
        "email": email, "requires_verification": True}

@app.post("/auth/verify-email")
async def verify_email(payload: dict):
    email = payload.get("email", "").strip().lower()
    code = payload.get("code", "").strip()
    if not email or not code:
        raise HTTPException(status_code=400, detail="Email and code are required.")

    record = await get_auth_code(email, code, 'verify')
    if not record:
        raise HTTPException(status_code=400, detail="Invalid or expired verification code.")

    await verify_user_email(email)
    await delete_auth_code(email, 'verify')
    logger.info(f"Email verified: {email}")
    return {"message": "Email verified successfully! You can now log in."}

```

```

@app.post("/auth/login", response_model=Token)

async def login(form_data: OAuth2PasswordRequestForm = Depends()):

    email = form_data.username.strip().lower() # OAuth2 form sends as 'username'

    user = await authenticate_user(email, form_data.password)

    if not user:

        raise HTTPException(status_code=401, detail="Incorrect email or password.")

    if not user.get("is_verified"):

        raise HTTPException(status_code=403, detail="Please verify your email first. Check your
inbox for the verification code.")

    token = create_access_token({"sub": user["email"]})

    logger.info(f"Login: {user['email']}")

    return {"access_token": token, "token_type": "bearer"}


@app.post("/auth/forgot-password")

async def forgot_password(payload: dict):

    email = payload.get("email", "").strip().lower()

    if not email:

        raise HTTPException(status_code=400, detail="Email is required.")

    user = await get_user_by_email(email)

    # Always return success to avoid revealing if email exists (security)

    if user:

        code = ''.join(random.choices(string.digits, k=6))

        expires_at = (datetime.utcnow() + timedelta(minutes=15)).strftime('%Y-%m-%d %H:%M:%S')

        await save_auth_code(email, code, 'reset', expires_at)

        logger.info(f"[PASSWORD RESET] Code for {email}: {code} (expires: {expires_at} UTC)")

        await send_reset_password_email(email, code)

    return {"message": "If an account with that email exists, a 6-digit reset code has been sent
to your email."}


@app.post("/auth/reset-password")

```

```

async def reset_password(payload: dict):

    email = payload.get("email", "").strip().lower()

    code = payload.get("code", "").strip()

    new_password = payload.get("new_password", "")

    if not email or not code or not new_password:

        raise HTTPException(status_code=400, detail="Email, code and new_password are required.")

    if len(new_password) < 6:

        raise HTTPException(status_code=400, detail="Password must be at least 6 characters.")

    record = await get_auth_code(email, code, 'reset')

    if not record:

        raise HTTPException(status_code=400, detail="Invalid or expired reset code.")

    from backend.auth.auth import hash_password

    await update_user_password(email, hash_password(new_password))

    await delete_auth_code(email, 'reset')

    logger.info(f"Password reset successful: {email}")

    return {"message": "Password reset successful! You can now log in with your new password."}

@app.post("/start-company/")

async def start_company(request: TaskRequest, current_user: dict = Depends(get_current_active_user)):

    task = request.task.strip()

    job_id = str(uuid.uuid4())

    tracer = ExecutionTracer()

    tracer.start(task)

    t_start = time.time()

    logger.info(f"Task [{current_user['email']}]: {task[:80]}")

    history = await get_recent_history(limit=3)

    ceo = await ceo_agent(task, history=history)

    raw_llm = ceo.get("_raw_llm_response", "")

    extracted = ceo.get("_extracted_json", {})

    short_term = ceo.get("short_term_strategy", "")

    ceo_depts = ceo.get("departments", ["backend"])

    final_depts, _, _ = autonomous_scale(task, ceo_depts)

```

```

dept_output = await execute_agents(final_depts, task, ceo_strategy=short_term, job_id=job_id,
ws_manager=ws_manager)

total_dur = round(time.time() - t_start, 2)

success = sum(1 for v in dept_output.values() if v.get("status") == "success")

failed = sum(1 for v in dept_output.values() if v.get("status") == "failed")

for name, data in dept_output.items():

    tracer.add_agent_result(name, data.get("status", "unknown"),
data.get("execution_time_seconds", 0), data.get("confidence", 0.0))

    if data.get("status") == "success":

        summary = str(data.get("output", {}).get("summary", task[:120]))

        await save_agent_memory(name, task, summary, data.get("confidence", 0.8))

tracer.finish(total_dur, success, failed)

result = {"job_id": job_id, "task": task, "requested_by": current_user["email"],

        "ceo_strategy": {"short_term_strategy": short_term, "long_term_vision":
ceo.get("long_term_vision", "")}},

        "selected_departments": final_depts, "departments": dept_output}

await save_record(task, json.dumps(result))

await save_execution_stat(task, len(final_depts), success, failed, total_dur)

await save_audit_record(task, raw_llm, extracted, final_depts, tracer.get_trace())

return result


@app.post("/feedback/")

async def submit_feedback(request: FeedbackRequest, current_user: dict =
Depends(get_current_active_user)):

    revised_task = f"{request.task}\n\n--- REVISION FEEDBACK FROM USER
---\n{request.feedback}\nPlease address the above feedback specifically in your output."

    job_id = str(uuid.uuid4())

    t_start = time.time()

    dept_output = await execute_agents(request.departments, revised_task,
ceo_strategy=f"Incorporate user feedback: {request.feedback[:100]}", job_id=job_id,
ws_manager=ws_manager)

    total_dur = round(time.time() - t_start, 2)

    success = sum(1 for v in dept_output.values() if v.get("status") == "success")

    return {"job_id": job_id, "original_job_id": request.job_id, "feedback": request.feedback,

        "revised_task": revised_task, "departments": dept_output,

        "total_duration_seconds": total_dur, "successful_revisions": success}


@app.post("/deploy/")

```

```

async def full_deploy(request: DeployRequest, current_user: dict) =
Depends(get_current_active_user)):

    task = request.task.strip()

    job_id = str(uuid.uuid4())

    t_start = time.time()

    app_type = detect_app_type(task)

    logger.info(f"DEPLOY pipeline | user={current_user['email']} | app_type={app_type} |
task={task[:80]}")

    history = await get_recent_history(limit=3)

    ceo = await ceo_agent(task, history=history)

    short_term = ceo.get("short_term_strategy", "")

    ceo_depts = ceo.get("departments", ["backend", "frontend"])

    if "frontend" not in ceo_depts:
        ceo_depts.append("frontend")

    final_depts, _, _ = autonomous_scale(task, ceo_depts)

    dept_output = await execute_agents(final_depts, task, ceo_strategy=short_term, job_id=job_id,
ws_manager=ws_manager)

    total_dur = round(time.time() - t_start, 2)

    success = sum(1 for v in dept_output.values() if v.get("status") == "success")

    for name, data in dept_output.items():
        if data.get("status") == "success":
            summary = str(data.get("output", {}).get("summary", task[:120]))

            await save_agent_memory(name, task, summary, data.get("confidence", 0.8))

    backend_files = await generate_backend_files_llm(task, app_type)

    logger.info(f"Backend files | app_type={app_type} | count={len(backend_files)}")

    phase5 = {"phase5_ran": False, "note": "push_to_github=false"}

    dept_path = _extract_frontend_path(dept_output)

    if request.push_to_github and dept_path:

        logger.info(f"Phase 5 | Pushing | path={dept_path} | extra_files={len(backend_files)}")

```

```

        phase5 = await asyncio.to_thread(
            github_pusher.push_project, task, dept_path, "", backend_files
        )
elif request.push_to_github:
    phase5 = {"phase5_ran": False, "error": "No deployable frontend files found"}

phase6 = {"phase6_ran": False, "note": "deploy_to_render=false"}
repo_url = phase5.get("repo_url", "")
repo_name = phase5.get("repo_name", "")

if request.deploy_to_render and repo_url and app_type != "frontend":
    logger.info(f"Phase 6 | Deploying to Render | repo={repo_url}")
    phase6 = await asyncio.to_thread(render_deployer.deploy, task, repo_url, repo_name)
elif request.deploy_to_render and app_type == "frontend":
    phase6 = {"phase6_ran": False, "note": "Frontend-only ? no backend needed",
              "github_pages_url": f"https://{github_pusher.username}.github.io/{repo_name}/"}
elif request.deploy_to_render and not repo_url:
    phase6 = {"phase6_ran": False, "error": "GitHub push must succeed before Render deploy"}

    github_pages_url = f"https://{github_pusher.username}.github.io/{repo_name}/" if repo_name
else ""

render_url = phase6.get("live_url", "")

# FIX: Correct VITE_API_URL to real Render URL (not guessed), then re-trigger workflow
if render_url and repo_name:
    try:
        await asyncio.to_thread(
            github_pusher._set_repo_variable, repo_name, "VITE_API_URL", render_url
        )
        logger.info(f"VITE_API_URL corrected to real Render URL | {render_url}")
        await asyncio.to_thread(github_pusher._trigger_workflow, repo_name)
        logger.info(f"Workflow re-triggered with correct VITE_API_URL | {repo_name}")
        # Mark repo as healthy so startup fix skips it next restart
        _mark_fix_attempted(repo_name)
    except Exception as e:

```



```
logger.warning(f"VITE_API_URL correction failed: {e}")
```

```
result = {
    "job_id": job_id, "task": task, "app_type": app_type,
    "requested_by": current_user["email"],
    "ceo_strategy": {"short_term_strategy": short_term},
    "selected_departments": final_depts, "departments": dept_output,
    "execution_summary": {"total_duration_seconds": total_dur, "successful": success},
    "github": phase5, "render": phase6,
    "live_urls": {
        "frontend": github_pages_url,
        "backend": render_url,
        "api_docs": f"{render_url}/docs" if render_url else "",
    },
}

await save_record(task, json.dumps(result))

return result
```

```
@app.post("/chat/")
```

```
async def chat_endpoint(request: ChatRequest, current_user: dict) =
Depends(get_current_active_user)):
```

```
    message = request.message.strip()
```

```
    username = current_user["email"]
```

```
    t_start = time.time()
```

```
    intent = detect_intent(message)
```

```
    logger.info(f"Chat | user={username} | intent={intent} | msg={message[:80]}")
```

```
    try:
```

```
        await save_chat_message(username, "user", message, intent)
```

```
    except Exception as e:
```

```
        logger.warning(f"Chat history save failed: {e}")
```

```
    if intent == "build":
```

```
        try:
```

```

task = message

job_id = str(uuid.uuid4())

app_type = detect_app_type(task)


history = await get_recent_history(limit=3)

ceo = await ceo_agent(task, history=history)

short_term = ceo.get("short_term_strategy", "")

ceo_depts = ceo.get("departments", ["backend", "frontend"])

if "frontend" not in ceo_depts:
    ceo_depts.append("frontend")

final_depts, _, _ = autonomous_scale(task, ceo_depts)


dept_output = await execute_agents(
    final_depts, task, ceo_strategy=short_term,
    job_id=job_id, ws_manager=ws_manager
)

total_dur = round(time.time() - t_start, 2)

success = sum(1 for v in dept_output.values() if v.get("status") == "success")


for name, data in dept_output.items():
    if data.get("status") == "success":
        summary = str(data.get("output", {}).get("summary", task[:120]))
        await save_agent_memory(name, task, summary, data.get("confidence", 0.8))


backend_files = await generate_backend_files_llm(task, app_type)


phase5 = {"phase5_ran": False}

dept_path = _extract_frontend_path(dept_output)

if dept_path:
    phase5 = await asyncio.to_thread(
        github_pusher.push_project, task, dept_path, "", backend_files
    )


phase6 = {"phase6_ran": False}

```

```

repo_url = phase5.get("repo_url", "")
repo_name = phase5.get("repo_name", "")
if repo_url and app_type != "frontend":
    phase6 = await asyncio.to_thread(
        render_deployer.deploy, task, repo_url, repo_name
    )

    github_pages_url = f"https://{github_pusher.username}.github.io/{repo_name}/" if
repo_name else ""

    render_url = phase6.get("live_url", "")

# FIX: Correct VITE_API_URL to real Render URL, re-trigger workflow
if render_url and repo_name:
    try:
        await asyncio.to_thread(
            github_pusher._set_repo_variable, repo_name, "VITE_API_URL", render_url
        )
        logger.info(f"VITE_API_URL corrected to real Render URL | {render_url}")
        await asyncio.to_thread(github_pusher._trigger_workflow, repo_name)
        logger.info(f"Workflow re-triggered with correct VITE_API_URL | {repo_name}")
        # Mark as recently fixed so startup loop skips it
        _mark_fix_attempted(repo_name)
    except Exception as e:
        logger.warning(f"VITE_API_URL correction failed: {e}")

via_msg = f"? **Build Complete!**\n\n"
via_msg += f"? **App Type:** {app_type}\n"
via_msg += f"? **Duration:** {total_dur}s\n"
via_msg += f"? **Agents:** {success}/{len(final_depts)} successful\n\n"

if github_pages_url:
    via_msg += f"? **Live URLs:**\n"
    via_msg += f"- ?? Frontend: {github_pages_url}\n"
    via_msg += f"  *(Note: GitHub Pages takes 1-2 minutes to go live. If you see a
404, please wait a minute and refresh)*\n"

```

```

if render_url:

    via_msg += f"- ? Backend: {render_url}\n"

    via_msg += f"- ? API Docs: {render_url}/docs\n"

if not github_pages_url and not render_url:

    via_msg += "?? Deployment didn't produce live URLs. Check GitHub/Render config.\n"

try:

    await save_chat_message(username, "assistant", via_msg, "build")
except Exception:

    pass

await save_record(task, json.dumps({"job_id": job_id, "app_type": app_type}))

return {

    "response": via_msg,

    "intent": "build",

    "mode": "build",

    "job_id": job_id,

    "app_type": app_type,

    "departments": final_depts,

    "dept_results": {k: v.get("status") for k, v in dept_output.items()},

    "live_urls": {

        "frontend": github_pages_url,

        "backend": render_url,

        "api_docs": f"{render_url}/docs" if render_url else "",

    },

    "duration_seconds": total_dur,

}

except Exception as e:

    logger.error(f"Build mode error: {e}")

    error_msg = f"? Build encountered an error: {str(e)}\n\nPlease try again or use the /deploy/ endpoint directly."

    try:

        await save_chat_message(username, "assistant", error_msg, "build")

```

```

except Exception:

    pass

    return {"response": error_msg, "intent": "build", "mode": "build", "error": str(e)}

elif intent == "analyze":

    try:

        task = message

        job_id = str(uuid.uuid4())

        history = await get_recent_history(limit=3)

        ceo = await ceo_agent(task, history=history)

        short_term = ceo.get("short_term_strategy", "")

        ceo_depts = ceo.get("departments", ["backend"])

        final_depts, _, _ = autonomous_scale(task, ceo_depts)

        dept_output = await execute_agents(

            final_depts, task, ceo_strategy=short_term,

            job_id=job_id, ws_manager=ws_manager

        )

        total_dur = round(time.time() - t_start, 2)

        success = sum(1 for v in dept_output.values() if v.get("status") == "success")

        for name, data in dept_output.items():

            if data.get("status") == "success":

                summary = str(data.get("output", {}).get("summary", task[:120]))

                await save_agent_memory(name, task, summary, data.get("confidence", 0.8))

        via_msg = f"? **Analysis Complete!**\n\n"

        via_msg += f"? **Strategy:** {short_term[:200]}\n\n"

        via_msg += f"**Departments consulted:** {' '.join(final_depts)}\n"

        via_msg += f"? **Duration:** {total_dur}s | ? {success}/{len(final_depts)}

successful\n\n"

        for name, data in dept_output.items():

            if data.get("status") == "success":

```

```

        out = data.get("output", {})

        dept_name = out.get("department", name)

        dept_summary = out.get("summary", out.get("full_report", str(out))[:300])

        via_msg += f"### ? {dept_name}\n{dept_summary[:500]}\n\n"

    try:

        await save_chat_message(username, "assistant", via_msg, "analyze")

    except Exception:

        pass

    await save_record(task, json.dumps({"job_id": job_id, "mode": "analyze"}))

    return {

        "response": via_msg,

        "intent": "analyze",

        "mode": "analyze",

        "job_id": job_id,

        "departments": final_depts,

        "dept_results": dept_output,

        "duration_seconds": total_dur,

    }

except Exception as e:

    logger.error(f"Analyze mode error: {e}")

    error_msg = f"? Analysis encountered an error: {str(e)}"

    return {"response": error_msg, "intent": "analyze", "mode": "analyze", "error":
str(e)}

else:

    db_history = []

    try:

        db_history = await get_chat_history(username, limit=20)

    except Exception:

        pass

    history = request.history or db_history

```

```

response = await chat_engine.chat(message, history)

try:
    await save_chat_message(username, "assistant", response, "chat")
except Exception:
    pass

return {
    "response": response,
    "intent": "chat",
    "mode": "chat",
    "duration_seconds": round(time.time() - t_start, 2),
}

```

```
@app.get("/chat/history/")
```

```

async def chat_history_endpoint(current_user: dict = Depends(get_current_active_user)):
    history = await get_chat_history(current_user["email"], limit=100)
    return {"history": history, "total": len(history)}

```

```
@app.delete("/chat/history/")
```

```

async def clear_chat_history_endpoint(current_user: dict = Depends(get_current_active_user)):
    await clear_chat_history(current_user["email"])
    return {"status": "cleared"}

```

```
@app.post("/jobs/submit/")
```

```

async def submit_job(request: JobRequest, current_user: dict = Depends(get_current_active_user)):
    from backend.tasks.orchestration_task import run_orchestration

    if run_orchestration is None:
        raise HTTPException(status_code=503, detail="Background jobs require Redis + Celery. Use /start-company/ or /chat/ for synchronous execution.")

    job_id = str(uuid.uuid4())
    await create_job(job_id, request.task)

```

```

run_orchestration.apply_async(args=[job_id, request.task, current_user["email"]])

return {"job_id": job_id, "status": "pending"}


@app.get("/jobs/{job_id}/")

async def get_job_status(job_id: str, current_user: dict = Depends(get_current_active_user)):

    job = await get_job(job_id)

    if not job: raise HTTPException(status_code=404)

    return job


@app.get("/company-history/")

async def company_history(current_user: dict = Depends(get_current_active_user)):

    return {"recent_history": await get_recent_history(limit=10)}


@app.get("/system-health/")

async def system_health(current_user: dict = Depends(get_current_active_user)):

    return {"status": "operational", "metrics": await get_system_health()}


@app.get("/company-status/")

async def company_status(current_user: dict = Depends(get_current_active_user)):

    return {"status": "operational", "dashboard": await get_company_status()}


@app.get("/org-chart/")

async def org_chart(current_user: dict = Depends(get_current_active_user)):

    return get_full_chart()


@app.get("/agent-memory/")

async def agent_memory(current_user: dict = Depends(get_current_active_user)):

    memories = await get_all_memories(limit=50)

    return {"memories": memories, "total": len(memories)}


@app.get("/agent-memory/{agent_name}/")

async def agent_memory_by_name(agent_name: str, current_user: dict = Depends(get_current_active_user)):

    memories = await get_agent_memory(agent_name, limit=10)

    return {"agent": agent_name, "memories": memories}

```



```

_ROOT_DIR    = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

_INDEX_HTML  = os.path.join(_ROOT_DIR, "index.html")

_CSS_FILE    = os.path.join(_ROOT_DIR, "via-chat.css")


@app.get("/", include_in_schema=False)

def root():

    if os.path.exists(_INDEX_HTML):

        return FileResponse(

            _INDEX_HTML,

            media_type="text/html",

            headers={"Cache-Control": "no-cache, no-store, must-revalidate"}

        )

    return {"app": "VIA", "version": APP_VERSION}


@app.get("/via-chat.css", include_in_schema=False)

def serve_css():

    if os.path.exists(_CSS_FILE):

        return FileResponse(

            _CSS_FILE,

            media_type="text/css",

            headers={"Cache-Control": "no-cache, no-store, must-revalidate"}

        )

    return ""


@app.get("/company_logo.png", include_in_schema=False)

@app.get("/company_logo.jpg", include_in_schema=False)

def serve_logo():

    for fname, mime in [("company_logo.png", "image/png"), ("company_logo.jpg", "image/jpeg")]:

        p = os.path.join(_ROOT_DIR, fname)

        if os.path.exists(p):

            return FileResponse(

                p,

```

```

        media_type=mime,
        headers={
            "Cache-Control": "no-cache, no-store, must-revalidate",
            "Pragma": "no-cache",
            "Expires": "0"
        }
    )

    return {"error": "logo not found"}

@app.get("/login_bg.png", include_in_schema=False)
@app.get("/login_bg.jpg", include_in_schema=False)
def serve_login_bg_permanent():
    for fname, mime in [("login_bg.png", "image/png"), ("login_bg.jpg", "image/jpeg")]:
        p = os.path.join(_ROOT_DIR, fname)
        if os.path.exists(p):
            return FileResponse(
                p,
                media_type=mime,
                headers={
                    "Cache-Control": "no-cache, no-store, must-revalidate",
                    "Pragma": "no-cache",
                    "Expires": "0"
                }
            )

    return {"error": "background not found"}

@app.get("/api/info")
def api_info():
    return {"app": "VIA", "version": APP_VERSION, "phase": "6", "agents": 10,
            "app_types": ["frontend", "fullstack", "fullstack_db"],
            "modes": ["chat", "build", "analyze"]}

@app.get("/health")
def health():

```

```
return {"status": "healthy", "version": APP_VERSION, "phase": "6"}
```

```
...
```

```
## File: `backend\middleware\rate_limiter.py`
```

```
```py
```

```
backend/middleware/rate_limiter.py
```

```
import time
```

```
from collections import defaultdict
```

```
from fastapi import Request, HTTPException, status
```

```
from backend.core.config import RATE_LIMIT_PER_MINUTE
```

```
from backend.core.logger import logger
```

```
_log: dict = defaultdict(list)
```

```
from fastapi.responses import JSONResponse
```

```
async def rate_limit_middleware(request: Request, call_next):
```

```
 skip = {
```

```
 "/", "/docs", "/openapi.json", "/redoc", "/health",
```

```
 "/via-chat.css", "/api/info",
```

```
 "/login_bg.png", "/login_bg.jpg",
```

```
 "/company_logo.png", "/company_logo.jpg",
```

```
 "/favicon.ico",
```

```
 }
```

```
 if request.url.path in skip or request.url.path.startswith("/ws"):
```

```
 return await call_next(request)
```

```
 ip = request.client.host if request.client else "unknown"
```

```
 now = time.time()
```

```
 _log[ip] = [t for t in _log[ip] if now - t < 60]
```

```
 if len(_log[ip]) >= RATE_LIMIT_PER_MINUTE:
```

```
 logger.warning(f"Rate limit exceeded | IP: {ip}")
```

```
 return JSONResponse(
```

```
 status_code=status.HTTP_429_TOO_MANY_REQUESTS,
```

```
 content={"detail": f"Rate limit exceeded. Max {RATE_LIMIT_PER_MINUTE} req/min."}
)
 _log[ip].append(now)
 return await call_next(request)
```

```
...
```

```
File: `backend\middleware\__init__.py`
```

```
```py
```

```
...
```

```
## File: `backend\routers\filebrowser_router.py`
```

```
```py
```

```
backend/routers/filebrowser_router.py ? VIA Phase 3: Project File Browser
```

```
import os
```

```
import json
```

```
import zipfile
```

```
import io
```

```
from pathlib import Path
```

```
from fastapi import APIRouter, Depends, HTTPException
```

```
from fastapi.responses import Response, JSONResponse
```

```
from backend.auth.auth import get_current_active_user
```

```
router = APIRouter(prefix="/files", tags=["File Browser"])
```

```
Projects are saved here by agents
```

```
PROJECTS_BASE = Path("projects")
```

```
ALLOWED_EXTENSIONS = {
```

```
 ".py", ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".json",
```

```
 ".md", ".txt", ".yaml", ".yml", ".toml", ".env", ".sh", ".sql",
 ".Dockerfile", ".gitignore", ".env.example", ""
}
```

```
SYNTAX_MAP = {
 ".py": "python",
 ".js": "javascript",
 ".ts": "typescript",
 ".jsx": "jsx",
 ".tsx": "tsx",
 ".html": "html",
 ".css": "css",
 ".json": "json",
 ".md": "markdown",
 ".yaml": "yaml",
 ".yml": "yaml",
 ".sh": "bash",
 ".sql": "sql",
 ".toml": "toml",
}
```

```
def _is_safe_path(base: Path, target: Path) -> bool:
 """Prevent path traversal attacks."""

 try:
 target.resolve().relative_to(base.resolve())
 return True
 except ValueError:
 return False
```

```
def _build_tree(path: Path, base: Path) -> dict:
 """Recursively build a file tree dict."""

 if not path.exists():
```

```

 return {}

node = {
 "name": path.name,
 "path": str(path.relative_to(base)),
 "type": "directory" if path.is_dir() else "file",
}

if path.is_dir():
 children = []
 try:
 for child in sorted(path.iterdir()):
 if child.name.startswith(".") and child.name not in (".env.example",
".gitignore"):
 continue
 if child.name in ("__pycache__", "node_modules", ".git", ".venv", "venv"):
 continue
 children.append(_build_tree(child, base))
 except PermissionError:
 pass
 node["children"] = children
 node["child_count"] = len(children)
else:
 node["size_bytes"] = path.stat().st_size
 node["extension"] = path.suffix.lower()
 node["language"] = SYNTAX_MAP.get(path.suffix.lower(), "text")

return node

```

```

@router.get("/projects/")
async def list_projects(current_user: dict = Depends(get_current_active_user)):
 """List all generated projects."""
 if not PROJECTS_BASE.exists():
 return {"projects": [], "total": 0}

```

```

projects = []

for d in sorted(PROJECTS_BASE.iterdir(), reverse=True):

 if d.is_dir():

 # Count files recursively

 file_count = sum(1 for _ in d.rglob("*") if _.is_file())

 size = sum(f.stat().st_size for f in d.rglob("*") if f.is_file())

 projects.append({

 "name": d.name,

 "path": str(d.relative_to(PROJECTS_BASE)),

 "file_count": file_count,

 "size_bytes": size,

 "modified": d.stat().st_mtime,

 })

return {"projects": projects, "total": len(projects)}

```

```

@router.get("/projects/{project_name}/tree/")

async def get_project_tree(

 project_name: str,

 current_user: dict = Depends(get_current_active_user)

):

 """Get file tree for a project."""

 project_path = PROJECTS_BASE / project_name

 if not project_path.exists():

 raise HTTPException(404, f"Project '{project_name}' not found")

 if not _is_safe_path(PROJECTS_BASE, project_path):

 raise HTTPException(403, "Access denied")

 tree = _build_tree(project_path, PROJECTS_BASE)

 return {"project": project_name, "tree": tree}

```

```

@router.get("/projects/{project_name}/read/")

async def read_file(
 project_name: str,
 file_path: str,
 current_user: dict = Depends(get_current_active_user)
):
 """Read a specific file from a project."""
 project_base = PROJECTS_BASE / project_name
 full_path = (project_base / file_path).resolve()

 if not _is_safe_path(PROJECTS_BASE, full_path):
 raise HTTPException(403, "Access denied")

 if not full_path.exists():
 raise HTTPException(404, "File not found")

 if full_path.suffix.lower() not in ALLOWED_EXTENSIONS and full_path.suffix != "":
 raise HTTPException(400, "File type not viewable")

 try:
 size = full_path.stat().st_size
 if size > 500_000: # 500KB max
 raise HTTPException(413, "File too large to display")

 content = full_path.read_text(encoding="utf-8", errors="replace")
 return {
 "file": file_path,
 "content": content,
 "language": SYNTAX_MAP.get(full_path.suffix.lower(), "text"),
 "size_bytes": size,
 "lines": content.count("\n") + 1,
 }
 except UnicodeDecodeError:
 raise HTTPException(400, "Binary file cannot be displayed")

```



```

@router.get("/projects/{project_name}/download/")

async def download_project(
 project_name: str,
 current_user: dict = Depends(get_current_active_user)
):
 """Download an entire project as a ZIP file."""
 project_path = PROJECTS_BASE / project_name
 if not project_path.exists():
 raise HTTPException(404, "Project not found")
 if not _is_safe_path(PROJECTS_BASE, project_path):
 raise HTTPException(403, "Access denied")

 buf = io.BytesIO()
 with zipfile.ZipFile(buf, "w", zipfile.ZIP_DEFLATED) as zf:
 for file in project_path.rglob("*"):
 if file.is_file():
 if any(p in file.parts for p in ("__pycache__", "node_modules", ".git")):
 continue
 arcname = file.relative_to(project_path)
 zf.write(file, arcname)

 buf.seek(0)
 return Response(
 content=buf.read(),
 media_type="application/zip",
 headers={"Content-Disposition": f"attachment; filename={project_name}.zip"}
)

```

```

@router.delete("/projects/{project_name}/")

async def delete_project(
 project_name: str,

```

```

 current_user: dict = Depends(get_current_active_user)
):
 """Delete a generated project."""
 import shutil

 project_path = PROJECTS_BASE / project_name

 if not project_path.exists():
 raise HTTPException(404, "Project not found")

 if not _is_safe_path(PROJECTS_BASE, project_path):
 raise HTTPException(403, "Access denied")

 shutil.rmtree(project_path)

 return {"deleted": True, "project": project_name}

...

File: `backend\routers\meeting_router.py`

```py
# backend/routers/meeting_router.py ? VIA Phase 3: Meeting Room API

import uuid

from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel, Field

from backend.auth.auth import get_current_active_user
from backend.core.meeting_engine import generate_meeting_fast, AGENT_PERSONAS
from backend.core.memory_store import save_meeting_transcript, get_meeting, get_recent_meetings

router = APIRouter(prefix="/meetings", tags=["Meetings"])

class MeetingRequest(BaseModel):
    task: str = Field(..., min_length=5, max_length=2000)
    departments: list = Field(default=["ceo", "backend", "frontend"])

```

```
ceo_strategy: str = Field(default="Execute with speed and precision.")
```

```
@router.post("/generate/")
```

```
async def generate_meeting_endpoint(
```

```
    req: MeetingRequest,
```

```
    current_user: dict = Depends(get_current_active_user)
```

```
):
```

```
    """Generate a real-time agent meeting transcript for a given task."""
```

```
    job_id = str(uuid.uuid4())
```

```
    # Ensure ceo is always first
```

```
    depts = req.departments
```

```
    if "ceo" not in depts:
```

```
        depts = ["ceo"] + depts
```

```
    messages = await generate_meeting_fast(req.task, depts, req.ceo_strategy)
```

```
    # Save to DB
```

```
    await save_meeting_transcript(job_id, req.task, messages)
```

```
    return {
```

```
        "meeting_id": job_id,
```

```
        "task": req.task,
```

```
        "departments": depts,
```

```
        "messages": messages,
```

```
        "message_count": len(messages),
```

```
    }
```

```
# ?? IMPORTANT: Specific routes BEFORE dynamic /{meeting_id}/ ??????????????????
```

```
@router.get("/personas/all")
```

```
async def get_personas(current_user: dict = Depends(get_current_active_user)):
```

```
"""Get all agent personas and their details."""
```

```
return {  
    "personas": {  
        k: {  
            "name": v["name"],  
            "title": v["title"],  
            "emoji": v["emoji"],  
            "color": v["color"],  
        }  
        for k, v in AGENT_PERSONAS.items()  
    }  
}
```

```
@router.get("/")
```

```
async def list_meetings(current_user: dict = Depends(get_current_active_user)):
```

```
    """List recent meetings."""
```

```
    meetings = await get_recent_meetings(limit=20)
```

```
    return {"meetings": meetings, "total": len(meetings)}
```

```
@router.get("/{meeting_id}/")
```

```
async def get_meeting_endpoint(  
    meeting_id: str,  
    current_user: dict = Depends(get_current_active_user)
```

```
):
```

```
    """Retrieve a saved meeting transcript."""
```

```
    meeting = await get_meeting(meeting_id)
```

```
    if not meeting:
```

```
        raise HTTPException(status_code=404, detail="Meeting not found")
```

```
    return meeting
```

```
...
```

```
## File: `backend\routers\template_router.py`
```

```
```py
```

```
backend/routers/template_router.py ? VIA Phase 3: Smart Task Templates
```

```
from fastapi import APIRouter, Depends
```

```
from backend.auth.auth import get_current_active_user
```

```
router = APIRouter(prefix="/templates", tags=["Templates"])
```

```
TASK_TEMPLATES = [
```

```
{
```

```
 "id": "saas-app",
```

```
 "category": "Product",
```

```
 "icon": "?",
```

```
 "title": "SaaS Web Application",
```

```
 "description": "Full-stack SaaS with auth, subscription billing, dashboard, and API",
```

```
 "task": "Build a complete SaaS web application with user authentication, subscription
billing with Stripe, user dashboard, REST API, admin panel, and PostgreSQL database. Include email
verification, password reset, and role-based access control.",
```

```
 "departments": ["backend", "frontend", "security", "devops"],
```

```
 "estimated_time": "8-12 min",
```

```
 "complexity": "High",
```

```
 "color": "c",
```

```
},
```

```
{
```

```
 "id": "ecommerce",
```

```
 "category": "Product",
```

```
 "icon": "?",
```

```
 "title": "E-Commerce Platform",
```

```
 "description": "Online store with products, cart, checkout, and order management",
```

```
 "task": "Build a complete e-commerce platform with product catalog, shopping cart, secure
checkout with payment integration, order management system, inventory tracking, customer accounts,
and admin dashboard with analytics.",
```

```
 "departments": ["backend", "frontend", "security", "devops", "marketing"],
```

```
 "estimated_time": "10-15 min",
```

```
 "complexity": "High",
 "color": "v2",
 },
 {
 "id": "ai-chatbot",
 "category": "AI",
 "icon": "?",
 "title": "AI Chatbot Platform",
 "description": "Intelligent chatbot with custom training, multi-channel deployment",
 "task": "Build an AI-powered chatbot platform with custom knowledge base training, multi-channel deployment (web widget, WhatsApp, Slack), conversation history, analytics dashboard, and admin interface for bot configuration.",
 "departments": ["backend", "frontend", "ai_research", "devops"],
 "estimated_time": "8-12 min",
 "complexity": "High",
 "color": "m2",
 },
 {
 "id": "task-manager",
 "category": "Productivity",
 "icon": "?",
 "title": "Project Task Manager",
 "description": "Kanban-style task manager with teams, deadlines, and notifications",
 "task": "Build a project management tool with kanban boards, task assignment, deadline tracking, team collaboration, file attachments, comment threads, email notifications, and productivity analytics dashboard.",
 "departments": ["backend", "frontend", "devops"],
 "estimated_time": "5-8 min",
 "complexity": "Medium",
 "color": "g",
 },
 {
 "id": "blog-cms",
 "category": "Content",
 "icon": "?",
 "title": "Blog & CMS Platform",
```

```

 "description": "Full-featured blog with CMS, SEO optimization, and analytics",
 "task": "Build a blog and content management system with rich text editor, SEO optimization tools, image management, categories and tags, comments system, newsletter integration, social sharing, and traffic analytics.",
 "departments": ["backend", "frontend", "devops", "marketing"],
 "estimated_time": "5-8 min",
 "complexity": "Medium",
 "color": "y",
 },
 {
 "id": "hospital-mgmt",
 "category": "Healthcare",
 "icon": "?",
 "title": "Hospital Management System",
 "description": "Patient records, appointments, billing, and doctor portal",
 "task": "Build a hospital management system with patient registration, appointment scheduling, doctor portal, medical records management, prescription tracking, billing and insurance processing, and department dashboards.",
 "departments": ["backend", "frontend", "security", "devops"],
 "estimated_time": "10-14 min",
 "complexity": "High",
 "color": "m",
 },
 {
 "id": "fintech-app",
 "category": "Finance",
 "icon": "?",
 "title": "FinTech Mobile App",
 "description": "Digital wallet, transfers, spending analytics, and budgeting",
 "task": "Build a fintech application with digital wallet, peer-to-peer transfers, spending analytics, budget planning, bill payments, transaction history, fraud detection alerts, and multi-currency support.",
 "departments": ["backend", "frontend", "security", "ai_research", "devops"],
 "estimated_time": "12-18 min",
 "complexity": "Very High",
 "color": "g2",
 },

```

```
{
 "id": "social-network",
 "category": "Social",
 "icon": "?",
 "title": "Social Network Platform",
 "description": "User profiles, posts, followers, messaging, and feed algorithm",
 "task": "Build a social networking platform with user profiles, post sharing with media, follow/unfollow system, algorithmic feed, direct messaging, notifications, hashtags, trending topics, and content moderation tools.",
 "departments": ["backend", "frontend", "ai_research", "devops", "security"],
 "estimated_time": "12-18 min",
 "complexity": "Very High",
 "color": "v",
},
{
 "id": "inventory-system",
 "category": "Business",
 "icon": "?",
 "title": "Inventory Management",
 "description": "Stock tracking, suppliers, purchase orders, and warehouse management",
 "task": "Build an inventory management system with real-time stock tracking, supplier management, purchase order processing, barcode scanning support, low stock alerts, warehouse location mapping, and comprehensive reporting.",
 "departments": ["backend", "frontend", "devops"],
 "estimated_time": "6-9 min",
 "complexity": "Medium",
 "color": "o",
},
{
 "id": "learning-platform",
 "category": "Education",
 "icon": "?",
 "title": "Online Learning Platform",
 "description": "Course creation, video lessons, quizzes, and student progress tracking",
 "task": "Build an online learning platform with course creation tools, video lesson hosting, interactive quizzes, student progress tracking, certificate generation, instructor dashboard, payment processing, and discussion forums.",
```



```

 "departments": ["backend", "frontend", "ai_research", "devops", "marketing"],
 "estimated_time": "10-14 min",
 "complexity": "High",
 "color": "c",
 },
 {
 "id": "restaurant-app",
 "category": "Food",
 "icon": "??",
 "title": "Restaurant Management App",
 "description": "Table booking, digital menu, orders, kitchen display, and billing",
 "task": "Build a restaurant management application with table reservation system, digital menu with photos, order management, kitchen display system, bill splitting, loyalty program, delivery tracking, and staff management portal.",
 "departments": ["backend", "frontend", "devops"],
 "estimated_time": "6-10 min",
 "complexity": "Medium",
 "color": "m2",
 },
 {
 "id": "real-estate",
 "category": "Property",
 "icon": "?",
 "title": "Real Estate Platform",
 "description": "Property listings, virtual tours, mortgage calculator, and agent portal",
 "task": "Build a real estate platform with property listings with rich media, virtual tour integration, advanced search and filters, mortgage calculator, agent profiles and chat, saved properties, and market analytics dashboard.",
 "departments": ["backend", "frontend", "devops", "marketing"],
 "estimated_time": "8-12 min",
 "complexity": "High",
 "color": "g",
 },
]

```

```

@router.get("/")

async def list_templates(current_user: dict = Depends(get_current_active_user)):

 """Get all available task templates."""

 return {

 "templates": TASK_TEMPLATES,

 "total": len(TASK_TEMPLATES),

 "categories": list({t["category"] for t in TASK_TEMPLATES}),

 }

@router.get("/{template_id}/")

async def get_template(template_id: str, current_user: dict = Depends(get_current_active_user)):

 """Get a specific task template by ID."""

 for t in TASK_TEMPLATES:

 if t["id"] == template_id:

 return t

 from fastapi import HTTPException

 raise HTTPException(status_code=404, detail="Template not found")

@router.get("/category/{category}/")

async def get_templates_by_category(category: str, current_user: dict = Depends(get_current_active_user)):

 """Get templates filtered by category."""

 filtered = [t for t in TASK_TEMPLATES if t["category"].lower() == category.lower()]

 return {"templates": filtered, "total": len(filtered)}

...

File: `backend\routers\__init__.py`

```py
# backend/routers/__init__.py

...

```

```
## File: `backend\tasks\celery_app.py`

```py

backend/tasks/celery_app.py

Phase 2: Celery async task queue using Redis as broker
Handles long-running AI orchestration jobs in background
Gracefully degrades if Redis/Celery is not available

import logging

logger = logging.getLogger("AI-Digital-Company")

try:

 from celery import Celery

 from backend.core.config import CELERY_BROKER_URL, CELERY_RESULT_URL

 celery_app = Celery(

 "ai_digital_team",

 broker=CELERY_BROKER_URL,

 backend=CELERY_RESULT_URL,

 include=["backend.tasks.orchestration_task"]

)

 celery_app.conf.update(

 task_serializer="json",

 result_serializer="json",

 accept_content=["json"],

 timezone="UTC",

 enable_utc=True,

 task_track_started=True,

 task_acks_late=True,

 worker_prefetch_multiplier=1,

 task_soft_time_limit=600,
```

```

 task_time_limit=720,

 result_expires=3600,

)

 CELERY_AVAILABLE = True

 logger.info("Celery configured successfully.")

except Exception as e:

 logger.warning(f"Celery not available ({e}). Background jobs will be disabled.")

 celery_app = None

 CELERY_AVAILABLE = False

...

File: `backend\tasks\orchestration_task.py`

```py
# backend/tasks/orchestration_task.py

# Phase 2: Background orchestration via Celery

# Runs the full CEO ? agents pipeline asynchronously

# Job status tracked in DB; results retrievable via GET /jobs/{job_id}

# Gracefully handles missing Celery/Redis

import asyncio, json, time, logging

from backend.tasks.celery_app import celery_app, CELERY_AVAILABLE

logger = logging.getLogger("AI-Digital-Company")

def _make_task():

    """Create the Celery task only if Celery is available."""

    if not CELERY_AVAILABLE or celery_app is None:

        return None

    @celery_app.task(bind=True, name="orchestration_task")

```

```

def run_orchestration(self, job_id: str, task: str, username: str):
    """
    Celery task: full AI orchestration pipeline.
    Runs asynchronously ? client polls /jobs/{job_id} for result.
    """
    logger.info(f"Celery task started | job={job_id} | user={username}")
    try:
        result = asyncio.run(_async_pipeline(job_id, task, username))
        return result
    except Exception as e:
        logger.error(f"Celery task failed | job={job_id} | {e}")
        asyncio.run(_mark_failed(job_id, str(e)))
        raise

return run_orchestration

```

```

run_orchestration = _make_task()

```

```

async def _async_pipeline(job_id: str, task: str, username: str) -> dict:
    """Full async orchestration pipeline ? same as /start-company/ but backgrounded."""
    from backend.agents.ceo_agent import ceo_agent
    from backend.agents.agent_executor import execute_agents
    from backend.core.tracer import ExecutionTracer
    from backend.core.scaling_engine import autonomous_scale
    from backend.core.hierarchy import get_active_structure
    from backend.database.db import (
        init_db, save_record, save_execution_stat,
        save_audit_record, update_job, get_recent_history
    )

    await init_db()
    await update_job(job_id, "running")

```

```

tracer = ExecutionTracer()

tracer.start(task)

t_start = time.time()


ceo = await ceo_agent(task)

short_term = ceo.get("short_term_strategy", "Execute core development.")
long_term = ceo.get("long_term_vision", "Build a scalable product.")
ceo_depts = ceo.get("departments", ["backend", "frontend"])
raw_llm = ceo.get("_raw_llm_response", "")
extracted = ceo.get("_extracted_json", {})


tracer.add_memory_injection(3)

tracer.add_ceo_decision(short_term, long_term, ceo_depts)


final_depts, was_scaled, scale_reason = autonomous_scale(task, ceo_depts)
if was_scaled:
    tracer.add_scaling_decision(ceo_depts, final_depts, scale_reason)


dept_output = await execute_agents(final_depts, task, ceo_strategy=short_term)
total_dur = round(time.time() - t_start, 2)


success = sum(1 for v in dept_output.values() if v.get("status") == "success")
failed = sum(1 for v in dept_output.values() if v.get("status") == "failed")


for name, data in dept_output.items():
    tracer.add_agent_result(name, data.get("status", "unknown"),
data.get("execution_time_seconds", 0), data.get("confidence", 0.0))

tracer.finish(total_dur, success, failed)


result = {
    "job_id": job_id,
    "task": task,
    "requested_by": username,
    "ceo_strategy": {"short_term_strategy": short_term, "long_term_vision": long_term},
    "selected_departments": final_depts,

```

```

        "autonomous_scaling": {"triggered": was_scaled, "reason": scale_reason if was_scaled else
        "No scaling required."},

        "execution_summary": {"total_agents": len(final_depts), "successful": success, "failed":
        failed, "total_duration_seconds": total_dur},

        "reporting_structure": get_active_structure(final_depts),

        "departments": dept_output,

        "execution_trace": tracer.get_trace()

    }

```

```

result_json = json.dumps(result)

await save_record(task, result_json)

await save_execution_stat(task, len(final_depts), success, failed, total_dur)

await save_audit_record(task, raw_llm, extracted, final_depts, tracer.get_trace())

await update_job(job_id, "completed", result_json)


logger.info(f"Celery task done | job={job_id} | {total_dur}s")

return result

```

```

async def _mark_failed(job_id: str, error: str):

    from backend.database.db import init_db, update_job

    await init_db()

    await update_job(job_id, "failed", error=error)

```

```

...

```

```

## File: `backend\tasks\__init__.py`

```

```

```py

```

```

...

```

```

File: `backend\utils\email_sender.py`

```

```

```py

```

```

import smtplib

from email.mime.text import MIMEText

from email.mime.multipart import MIMEMultipart

from backend.core.config import SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, FROM_EMAIL

from backend.core.logger import logger

import asyncio


async def send_email_async(to_email: str, subject: str, body_html: str):

    """Sends an email asynchronously so it doesn't block the API thread."""

    if not SMTP_USER or not SMTP_PASSWORD:

        logger.warning(f"Skipping email to {to_email} because SMTP credentials are not set in
.env")

        return False


    def _send():

        msg = MIMEMultipart("alternative")

        msg["Subject"] = subject

        msg["From"] = f"VIA Platform <{FROM_EMAIL or SMTP_USER}>"

        msg["To"] = to_email


        # Adding plain text fallback prevents strict spam filters from silently dropping the email
        import re

        body_text = re.sub('<[^\>]+?>', '', body_html).strip() # basic HTML stripper

        part1 = MIMEText(body_text, "plain")

        part2 = MIMEText(body_html, "html")


        msg.attach(part1)

        msg.attach(part2)


    try:

        server = smtplib.SMTP(SMTP_HOST, SMTP_PORT)

        server.ehlo()

        server.starttls()

        server.login(SMTP_USER, SMTP_PASSWORD)

        server.sendmail(msg["From"], [to_email], msg.as_string())

```



```

        server.quit()

        logger.info(f"Email sent successfully to {to_email}")

        return True

    except Exception as e:

        logger.error(f"Failed to send email to {to_email}: {e}")

        return False

    return await asyncio.to_thread(_send)

async def send_verification_email(to_email: str, code: str):

    subject = "Verify your VIA Platform Account"

    body_html = f"""

        <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;
border: 1px solid #e0e0e0; border-radius: 10px;">

            <h2 style="color: #4f46e5; text-align: center;">VIA Platform</h2>

            <p>Hello!</p>

            <p>Thank you for registering. Please use the following 6-digit verification code to verify
your account:</p>

            <div style="background-color: #f3f4f6; padding: 15px; text-align: center; font-size: 24px;
font-weight: bold; letter-spacing: 5px; border-radius: 5px; margin: 20px 0;">

                {code}

            </div>

            <p>This code will expire in 15 minutes.</p>

            <hr style="border: none; border-top: 1px solid #e0e0e0; margin: 20px 0;" />

            <p style="font-size: 12px; color: #6b7280; text-align: center;">If you didn't request
this, you can safely ignore this email.</p>

        </div>

    """

    await send_email_async(to_email, subject, body_html)

async def send_reset_password_email(to_email: str, code: str):

    subject = "Reset Your VIA Platform Password"

    body_html = f"""

        <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;
border: 1px solid #e0e0e0; border-radius: 10px;">

            <h2 style="color: #4f46e5; text-align: center;">VIA Platform</h2>

            <p>Hello,</p>

```

<p>We received a request to reset your password. Please use the following 6-digit reset code:</p>

<div style="background-color: #f3f4f6; padding: 15px; text-align: center; font-size: 24px; font-weight: bold; letter-spacing: 5px; border-radius: 5px; margin: 20px 0;">

{code}

</div>

<p>This code will expire in 15 minutes.</p>

<p>If you didn't request a password reset, please ignore this email or contact support if you have concerns.</p>

<hr style="border: none; border-top: 1px solid #e0e0e0; margin: 20px 0;" />

<p style="font-size: 12px; color: #6b7280; text-align: center;">VIA Autonomous AI Digital Team Platform</p>

</div>

"""

await send_email_async(to_email, subject, body_html)

...

File: `backend\\_\_init\_\_.py`

```py

...

## File: `dict`

...

...

## File: `docker-compose.yml`

```yaml

=====

VIA ? Docker Compose (Development & Production)

=====

Usage:

docker compose up -d # Start all services

docker compose up -d --build # Rebuild and start

docker compose logs -f api # Follow API logs

docker compose down -v # Stop and remove volumes

=====

services:

---- PostgreSQL Database ----

postgres:

image: postgres:15-alpine

container_name: via_postgres

restart: unless-stopped

environment:

POSTGRES_DB: ai_digital_team

POSTGRES_USER: ai_admin

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:-StrongPassword123!}

volumes:

- postgres_data:/var/lib/postgresql/data

ports:

- "5432:5432"

healthcheck:

test: ["CMD-SHELL", "pg_isready -U ai_admin -d ai_digital_team"]

interval: 10s

timeout: 5s

retries: 5

---- Redis (Broker + Cache) ----

redis:

image: redis:7-alpine

container_name: via_redis

restart: unless-stopped

ports:

- "6379:6379"

volumes:

- redis_data:/data

healthcheck:

test: ["CMD", "redis-cli", "ping"]

interval: 10s

timeout: 5s

retries: 5

---- FastAPI Application ----

api:

build:

context: .

dockerfile: Dockerfile

container_name: via_api

restart: unless-stopped

env_file: [.env]

environment:

POSTGRES_HOST: postgres

REDIS_URL: redis://redis:6379/0

CELERY_BROKER_URL: redis://redis:6379/0

CELERY_RESULT_URL: redis://redis:6379/1

APP_ENV: production

ports:

- "8000:8000"

volumes:

- ./logs:/app/logs
- ./projects:/app/projects
- ./index.html:/app/index.html:ro
- ./via-chat.css:/app/via-chat.css:ro
- ./company_logo.png:/app/company_logo.png:ro
- ./login_bg.png:/app/login_bg.png:ro
- ./backend:/app/backend:ro

depends_on:

postgres:

```
    condition: service_healthy

redis:

    condition: service_healthy

healthcheck:

    test: ["CMD", "curl", "-f", "http://localhost:8000/health"]

    interval: 30s

    timeout: 10s

    start_period: 15s

    retries: 3

# ---- Celery Worker ----

worker:

    build:

        context: .

        dockerfile: Dockerfile

    container_name: via_worker

    restart: unless-stopped

    command: >

        celery -A backend.tasks.celery_app worker

        --loglevel=info

        --concurrency=2

    env_file: [.env]

    environment:

        POSTGRES_HOST: postgres

        REDIS_URL: redis://redis:6379/0

        CELERY_BROKER_URL: redis://redis:6379/0

        CELERY_RESULT_URL: redis://redis:6379/1

    volumes:

        - ./logs:/app/logs

        - ./projects:/app/projects

        - ./backend:/app/backend:ro

    depends_on:

        postgres:

            condition: service_healthy
```

```

    redis:

        condition: service_healthy

# ---- pgAdmin (Database GUI) ----
pgadmin:

    image: dpage/pgadmin4:latest

    container_name: via_pgadmin

    restart: unless-stopped

    environment:

        PGADMIN_DEFAULT_EMAIL: admin@via-platform.com

        PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD:-admin123}

    ports:

        - "5050:80"

    volumes:

        - pgadmin_data:/var/lib/pgadmin

    depends_on:

        - postgres

volumes:

    postgres_data:

    redis_data:

    pgadmin_data:

...

## File: `Dockerfile`

```dockerfile
=====

VIA ? Autonomous AI Digital Enterprise Platform

Production-grade Dockerfile (Multi-stage build)

=====

----- Stage 1: Build dependencies -----

```

FROM python:3.11-slim AS builder

WORKDIR /build

# Install build-time system dependencies

RUN apt-get update && \

apt-get install -y --no-install-recommends \

gcc \

libpq-dev \

build-essential && \

rm -rf /var/lib/apt/lists/\*

# Copy and install Python dependencies into a virtual env

COPY requirements.txt .

RUN python -m venv /opt/venv && \

/opt/venv/bin/pip install --no-cache-dir --upgrade pip && \

/opt/venv/bin/pip install --no-cache-dir -r requirements.txt

# ----- Stage 2: Production runtime -----

FROM python:3.11-slim AS runtime

LABEL maintainer="VIA Team"

LABEL description="VIA ? Autonomous AI Digital Enterprise Platform with 10 AI agents"

LABEL version="6.0.0"

# Install only runtime system dependencies

RUN apt-get update && \

apt-get install -y --no-install-recommends \

libpq5 \

curl \

tini && \

rm -rf /var/lib/apt/lists/\*

```
Copy the pre-built virtual env from builder stage

COPY --from=builder /opt/venv /opt/venv

Make sure the virtualenv Python/pip are first on PATH

ENV PATH="/opt/venv/bin:$PATH" \

 PYTHONDONTWRITEBYTECODE=1 \

 PYTHONUNBUFFERED=1 \

 APP_ENV=production \

 APP_VERSION=6.0.0

Create a non-root user for security

RUN groupadd --gid 1000 via && \

 useradd --uid 1000 --gid via --shell /bin/bash --create-home via

WORKDIR /app

Copy application source code

COPY --chown=via:via . .

Create required directories with correct ownership

RUN mkdir -p logs projects && \

 chown -R via:via logs projects

Switch to non-root user

USER via

Expose the API port

EXPOSE 8000

Health check ? verifies the API is responding

HEALTHCHECK --interval=30s --timeout=10s --start-period=15s --retries=3 \

 CMD curl -f http://localhost:8000/health || exit 1

Use tini as PID 1 for proper signal handling
```



```
ENTRYPOINT ["tini", "--"]
```

```
Default command: run the FastAPI server
```

```
CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2",
"--log-level", "info"]
```

```
...
```

```
File: `docs\API_REFERENCE.md`
```

```
```md
```

```
# VIA API Reference
```

```
> Complete API documentation for VIA ? Autonomous AI Digital Enterprise Platform
```

```
---
```

```
## Base URL
```

```
...
```

```
Development: http://localhost:8000
```

```
Production: https://your-render-app.onrender.com
```

```
...
```

```
## Authentication
```

```
VIA uses **JWT (JSON Web Tokens)** for authentication. Include the token in the `Authorization`  
header:
```

```
...
```

```
Authorization: Bearer <your_jwt_token>
```

```
...
```

```
---
```

Public Endpoints

`POST /auth/register`

Create a new user account.

****Request:****

``json

```
{  
  "username": "string (required)",  
  "password": "string (required)"  
}
```

``

****Response:**** `200 OK`

``json

```
{  
  "message": "User created successfully",  
  "username": "john"  
}
```

``

`POST /auth/login`

Authenticate and receive a JWT token.

****Request:**** `application/x-www-form-urlencoded`

``

username=john&password=secret123

``

****Response:**** `200 OK`

```
```json
{
 "access_token": "eyJhbGciOiJIUzI1NiIs... ",
 "token_type": "bearer"
}
```
```

`GET /health`

Health check endpoint.

****Response:**** `200 OK`

```
```json
{
 "status": "healthy",
 "version": "6.0.0",
 "phase": "6"
}
```
```

`GET /api/info`

Platform metadata.

****Response:**** `200 OK`

```
```json
{
 "app": "VIA",
 "version": "6.0.0",
 "phase": "6",

```

```
"agents": 10,
"app_types": ["frontend", "fullstack", "fullstack_db"],
"modes": ["chat", "build", "analyze"]
}
```
```

Protected Endpoints (JWT Required)

`POST /chat/`

****Unified chat endpoint**** ? automatically routes to Chat, Build, or Analyze mode based on intent detection.

****Request:****

```
```json  
{
 "message": "string (1-5000 chars, required)",
 "history": [{"role": "user", "message": "..."}] // optional
}
```
```

****Response (Chat Mode):****

```
```json  
{
 "response": "Here's what I think about...",
 "intent": "chat",
 "mode": "chat",
 "duration_seconds": 2.3
}
```
```

****Response (Build Mode):****

```
```json
```

```
{
 "response": "? Build Complete!...",
 "intent": "build",
 "mode": "build",
 "job_id": "uuid",
 "app_type": "fullstack",
 "departments": ["backend", "frontend", "security"],
 "dept_results": {"backend": "success", "frontend": "success"},
 "live_urls": {
 "frontend": "https://user.github.io/repo/",
 "backend": "https://app.onrender.com",
 "api_docs": "https://app.onrender.com/docs"
 },
 "duration_seconds": 45.2
}
```

**\*\*Response (Analyze Mode):\*\***

```
```json
{
  "response": "? Analysis Complete!...",
  "intent": "analyze",
  "mode": "analyze",
  "job_id": "uuid",
  "departments": ["backend", "security", "architecture"],
  "dept_results": {...},
  "duration_seconds": 30.1
}
```

`POST /start-company/`

Run the full agent pipeline without deployment.

****Request:****

```
```json
{
 "task": "string (5-2000 chars)"
}
```
```

****Response:**** `200 OK`

```
```json
{
 "job_id": "uuid",
 "task": "Build a todo app",
 "requested_by": "john",
 "ceo_strategy": {
 "short_term_strategy": "...",
 "long_term_vision": "..."
 },
 "selected_departments": ["backend", "frontend", "security"],
 "departments": {
 "backend": {"status": "success", "execution_time_seconds": 12.3, "confidence": 0.92, "output": {...}},
 "frontend": {"status": "success", "execution_time_seconds": 15.1, "confidence": 0.97, "output": {...}}
 }
}
```
```

`POST /deploy/`

Full build ? GitHub push ? Render deploy pipeline.

****Request:****

```json

```
{
 "task": "string (5-2000 chars)",
 "push_to_github": true,
 "deploy_to_render": true
}
```

```

****Response:**** `200 OK`

```json

```
{
 "job_id": "uuid",
 "task": "...",
 "app_type": "fullstack",
 "github": {"repo_url": "https://github.com/...", "repo_name": "..."},
 "render": {"live_url": "https://app.onrender.com"},
 "live_urls": {
 "frontend": "https://user.github.io/repo/",
 "backend": "https://app.onrender.com",
 "api_docs": "https://app.onrender.com/docs"
 }
}
```

}

```

`POST /feedback/`

Submit revision feedback for a previous build.

****Request:****

```json

```
{
```

```
"job_id": "previous-job-uuid",
"task": "original task description",
"feedback": "Please add dark mode support (5-1000 chars)",
"departments": ["backend", "frontend"]
}
...
```

---

```
`GET /chat/history/`
```

Retrieve chat history for the authenticated user.

**\*\*Response:\*\***

```
```json
{
  "history": [
    {"role": "user", "message": "...", "intent": "chat", "timestamp": "..."},
    {"role": "assistant", "message": "...", "intent": "chat", "timestamp": "..."}
  ],
  "total": 42
}
...
```

```
### `DELETE /chat/history/`
```

Clear all chat history for the authenticated user.

```
### `GET /company-history/`
```

Returns recent task execution history.

`GET /system-health/`

Returns system performance metrics (total runs, success rate, avg duration).

`GET /company-status/`

Returns company operational status dashboard.

`GET /org-chart/`

Returns the full organizational hierarchy of AI departments.

`GET /agent-memory/`

Returns all agent memories (last 50).

`GET /agent-memory/{agent\_name}/`

Returns memories for a specific agent.

Meetings Endpoints

`POST /meetings/generate/`

Generate an AI boardroom meeting discussion.

****Request:****

```json

```
{
 "task": "Discuss the architecture for a real-time chat app",
 "departments": ["ceo", "backend", "frontend", "security", "devops"]
}
```

...

### `GET /meetings/`

List all past meetings.

### `GET /meetings/{meeting\_id}/`

Get a specific meeting transcript.

---

## File Browser Endpoints

### `GET /files/projects/`

List all generated projects.

### `GET /files/projects/{project\_name}/tree/`

Get the file tree structure for a project.

### `GET /files/projects/{project\_name}/read/?file\_path=...`

Read a specific file from a project.

### `GET /files/projects/{project\_name}/download/`

Download a project as a ZIP archive.

---

## WebSocket

```
`WS /ws/{job_id}`
```

Real-time pipeline streaming. Receives JSON messages:

```
```json
```

```
{"type": "agent_start", "agent": "backend", "timestamp": "..."}\n
```

```
{"type": "agent_done", "agent": "backend", "status": "success", "duration": 12.3, "confidence": 0.92}\n
```

```
{"type": "inter_agent", "from": ["architecture"], "to": ["backend"], "context": "..."}\n
```

```
```\n
```

```

```

## ## Error Responses

All errors follow this format:

```
```json
```

```
{\n  "detail": "Error description"\n}\n
```

```
```\n
```

```
| Status | Meaning |\n
```

```
|-----|-----|\n
```

```
| 400 | Bad request / validation error |\n
```

```
| 401 | Unauthorized (invalid/expired JWT) |\n
```

```
| 404 | Resource not found |\n
```

```
| 429 | Rate limit exceeded |\n
```

```
| 500 | Internal server error |\n
```

```

```

## ## Rate Limiting

- Default: **\*\*20 requests per minute\*\*** per IP
- Configurable via ``RATE_LIMIT_PER_MINUTE`` env var
- Returns ``429 Too Many Requests`` when exceeded

...

## File: ``docs\ARCHITECTURE.md``

```md

VIA Architecture

System Overview

...

??

? FRONTEND ?

? index.html + via-chat.css (served by FastAPI) ?

? Auth ? Chat ? Build ? Projects ? Meetings ? Org ?

??

? HTTP/WS

??

? FASTAPI BACKEND ?

? ???????????? ???????????? ?????????????????????? ?

? ? Auth ? ? Chat ? ? Agent Executor ? ?

? ? JWT ? ? Engine ? ? (10 agents) ? ?

? ???????????? ???????????? ?????????????????????? ?

? ???????????? ???????????? ? ?

? ? Rate ? ? Intent ? ?????????????????? ?

? ? Limiter ? ? Detector ? ? InterAgent ? ?

? ???????????? ???????????? ? Bus ? ?

? ?????????????????????? ?

??

? ? ?

????????? ?????????? ??????????????

?Postgr? ? Redis ? ? Ollama ?
? SQL ? ? ? ? / Groq ?
???????? ???? ????? ???? ??????
...

Agent Execution Flow

1. User sends task via `/chat/` or `/deploy/`
2. **Intent Detector** classifies: chat / build / analyze
3. **CEO Agent** analyzes task, selects departments
4. **Scaling Engine** adjusts department list
5. **Agent Executor** runs agents in dependency order:
 - architecture ? backend ? security ? devops ? ai_research ? frontend ? hr ? finance ? marketing ? presentation
6. **InterAgent Bus** shares context between agents
7. **WebSocket Manager** streams real-time progress
8. For builds: **GitHub Pusher** ? **Render Deployer**
9. Results saved to PostgreSQL

Data Flow

...

User Message

?

?

Intent Detection ??? "chat" ??? Chat Engine ??? LLM ??? Response

?

???? "build" ??? CEO Agent ??? Agent Pipeline ??? GitHub Push ??? Render Deploy

?

???? "analyze" ??? CEO Agent ??? Agent Pipeline ??? Reports

...

Database Schema

| Table | Purpose |

```
|-----|-----|  
| `users` | JWT auth accounts |  
| `company_history` | All task executions |  
| `execution_stats` | Performance metrics |  
| `decision_audit` | CEO decision log |  
| `async_jobs` | Celery job tracking |  
| `agent_memory` | Agent learning store |  
| `meetings` | Meeting transcripts |  
| `chat_history` | Chat conversations |
```

```
...
```

```
## File: `docs\BUSINESS_ASSETS.md`
```

```
```md
```

```
VIA Business Assets
```

```
? LinkedIn Launch Post
```

```

```

```
? **Introducing VIA ? The World's First Autonomous AI Software Company**
```

What if an entire MNC engineering org ? CEO, Backend, Frontend, Security, DevOps, HR, Finance, Marketing ? was powered by AI?

**\*\*That's VIA.\*\***

Give it one sentence. It:

? Delegates to 10 AI departments

? Builds production-grade code

? Pushes to GitHub automatically

? Deploys to production live

? Returns working URLs

**\*\*Built with:\*\***

? FastAPI + WebSocket real-time streaming  
?  
? PostgreSQL + Redis + Celery  
?  
? JWT auth + rate limiting  
?  
? Docker + CI/CD + GitHub Actions  
?  
? Ollama / Groq LLM integration

3 modes:

? Chat ? Ask anything  
?? Build ? "Build me a todo app" ? deployed in 60s  
?  
? Analyze ? Multi-department strategic reports

This isn't just code generation. This is an **\*\*autonomous AI enterprise\*\***.

Phase 6 is live. Open source.

#AI #Automation #FastAPI #BuildInPublic #OpenSource #AIAgents #Software

---

## ? Startup Pitch (60 seconds)

---

**\*\*Problem:\*\*** Building software requires coordinating multiple teams ? backend, frontend, security, DevOps, marketing. This is slow, expensive, and error-prone.

**\*\*Solution:\*\*** VIA is an autonomous AI digital enterprise. One prompt activates 10 AI departments that collaborate like a real MNC ? from strategy to deployment in under 2 minutes.

**\*\*How it works:\*\***

1. User describes an app
2. CEO Agent creates strategy
3. 10 departments execute in parallel
4. Code is pushed to GitHub

5. App is deployed live on Render

6. User gets working URLs

**\*\*Traction:\*\*** Phase 6 complete with enterprise-grade features ? JWT auth, PostgreSQL, Docker, CI/CD, real-time WebSocket streaming, and 3 operational modes.

**\*\*Market:\*\*** \$300B software development market. VIA targets solo developers, startups, and rapid prototyping teams.

**\*\*Ask:\*\*** Seed funding to scale LLM infrastructure and add collaborative multi-user workspaces.

---

## ? Portfolio Description

---

### VIA ? Autonomous AI Digital Enterprise Platform

A full-stack AI platform that simulates an entire software company with 10 autonomous AI departments. Built with FastAPI, PostgreSQL, Redis, Docker, and LLM integration (Ollama/Groq).

**\*\*Key Features:\*\***

- 10 coordinated AI agents (CEO, Backend, Frontend, Security, DevOps, AI Research, Architecture, HR, Finance, Marketing)
- 3 operational modes: Chat, Build, Analyze
- Automated GitHub push + Render deployment pipeline
- Real-time WebSocket streaming with inter-agent communication
- JWT authentication, rate limiting, meeting generation
- Premium dark-mode UI with particle animations and glassmorphism
- Comprehensive test suite and CI/CD via GitHub Actions

**\*\*Tech Stack:\*\*** Python, FastAPI, PostgreSQL, Redis, Celery, Docker, WebSocket, JWT, Ollama/Groq LLM

---



## ? Technical Summary

---

Metric	Value
Total AI Agents	10
Backend Framework	FastAPI 0.104+
Database	PostgreSQL 15 + asyncpg
Task Queue	Redis + Celery
Authentication	JWT (bcrypt + HS256)
LLM Provider	Ollama (local) / Groq (cloud)
Deployment	Docker + GitHub Actions CI/CD
Auto-Deploy	GitHub Pages (frontend) + Render (backend)
API Endpoints	20+ REST + 1 WebSocket
Test Coverage	Intent detection, fullstack builder, agent pipeline
Frontend	Vanilla JS + premium CSS (dark/light mode)
Phase	6.0.0 ? Enterprise Polish

---

```

File: `docs\DEPLOYMENT.md`

```md

# VIA Deployment Guide

> Step-by-step instructions to deploy VIA in production.

---

## Architecture Overview

```

...

????????????????? ?????????????????? ??????????????????
? Frontend ? ? Backend ? ? Database ?
? (FastAPI ??????? (FastAPI + ??????? (PostgreSQL) ?
? Static) ? ? Uvicorn) ? ??????????????????
????????????????? ?????????????????? ?
 ?
 ?
 ?????????????????? ??????????????????
 ? Celery ? ? Redis ?
 ? Worker ? ? (Queue) ?
 ?????????????????? ??????????????????
...

```

## Option 1: Docker Compose (Recommended)

### Prerequisites

- Docker Engine 20.10+
- Docker Compose v2+
- 4GB RAM minimum

### Steps

```

```bash

# 1. Clone the repository

git clone https://github.com/your-username/via.git

cd via


# 2. Configure environment

cp .env.example .env

# Edit .env with your real credentials:

#   - POSTGRES_PASSWORD (strong password)

#   - JWT_SECRET_KEY (random 64-char string)
```

```
# - GITHUB_TOKEN (GitHub PAT with repo scope)

# - RENDER_API_KEY (from Render dashboard)

# - GROQ_API_KEY (if using Groq cloud LLM)
```

```
# 3. Start all services
```

```
docker-compose up --build -d
```

```
# 4. Verify
```

```
curl http://localhost:8000/health
```

```
# Expected: {"status":"healthy","version":"6.0.0","phase":"6"}
```

```
...
```

```
### Services Started
```

Service	URL	Purpose
-----	-----	-----
API	http://localhost:8000	FastAPI backend + frontend
Swagger	http://localhost:8000/docs	Interactive API docs
PostgreSQL	localhost:5432	Database
Redis	localhost:6379	Task queue
pgAdmin	http://localhost:5050	DB admin panel

```
### Useful Commands
```

```
```bash
```

```
View logs
```

```
docker-compose logs -f api
```

```
Restart a service
```

```
docker-compose restart api
```

```
Scale workers
```

```
docker-compose up -d --scale worker=3
```

```
Stop everything
```

```
docker-compose down
```

```
Stop and remove data
```

```
docker-compose down -v
```

```
...
```

```

```

## ## Option 2: Local Development

### ### Prerequisites

- Python 3.10+
- PostgreSQL 14+
- Redis 7+
- Ollama (for local LLM) or Groq API key

### ### Steps

```
```bash
```

1. Create virtual environment

```
python -m venv venv
```

```
source venv/bin/activate # Linux/Mac
```

```
venv\Scripts\activate    # Windows
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Start PostgreSQL

```
# Ensure PostgreSQL is running and create the database:
```

```
createdb ai_digital_team
```

4. Start Redis

```
redis-server
```

5. Configure environment

```
cp .env.example .env
```

```
# Edit .env with your values
```

```
# 6. Start Ollama (if using local LLM)
```

```
ollama serve &
```

```
ollama pull llama3.2:latest
```

```
# 7. Start the API server
```

```
uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

```
# 8. Start Celery worker (new terminal)
```

```
celery -A backend.tasks.celery_app worker --loglevel=info
```

```
# 9. Open http://localhost:8000 in browser
```

```
...
```

```
---
```

```
## Option 3: Cloud Deployment (Render)
```

```
### Backend (Web Service)
```

```
1. Push code to GitHub
```

```
2. Go to [Render Dashboard](https://dashboard.render.com)
```

```
3. Create New Web Service
```

```
4. Connect your GitHub repo
```

```
5. Configure:
```

```
- Build Command: `pip install -r requirements.txt`
```

```
- Start Command: `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`
```

```
- Instance Type: Starter ($7/mo) or Standard ($25/mo)
```

```
6. Add environment variables from `.env.example`
```

```
7. Deploy
```

```
### Database (PostgreSQL)
```

1. Create ****New PostgreSQL**** on Render
2. Copy the ****Internal Database URL****
3. Set ``DATABASE_URL`` in your web service env vars

Redis (Background Worker)

1. Create ****New Redis**** on Render
2. Copy the Redis URL
3. Set ``REDIS_URL``, ``CELERY_BROKER_URL``, ``CELERY_RESULT_URL``
4. Create a ****Background Worker**** service with:
 - ****Start Command****: ``celery -A backend.tasks.celery_app worker --loglevel=info --concurrency=2``

Option 4: AWS Deployment

Using ECS + RDS

```bash

# 1. Build and push Docker image

aws ecr create-repository --repository-name via-api

docker build -t via-api .

docker tag via-api:latest <account>.dkr.ecr.<region>.amazonaws.com/via-api:latest

docker push <account>.dkr.ecr.<region>.amazonaws.com/via-api:latest

# 2. Create RDS PostgreSQL instance

# 3. Create ElastiCache Redis instance

# 4. Create ECS task definition and service

# 5. Configure ALB for HTTPS

```

LLM Configuration

Option A: Local Ollama (Free, Private)

```
```env

MODEL_NAME=llama3.2:latest

OLLAMA_URL=http://localhost:11434/api/generate

USE_GROQ=false

```
```

Option B: Groq Cloud (Fast, Scalable)

```
```env

USE_GROQ=true

GROQ_API_KEY=gsk_your_api_key_here

GROQ_MODEL=llama-3.3-70b-versatile

```
```

Security Checklist for Production

- [] Change `JWT\_SECRET\_KEY` to a random 64+ character string
- [] Set strong `POSTGRES\_PASSWORD`
- [] Restrict CORS origins (change `allow\_origins=["\*"]` to specific domains)
- [] Enable HTTPS (automatic on Render/AWS ALB)
- [] Rotate `GITHUB\_TOKEN` and `RENDER\_API\_KEY` regularly
- [] Set `APP\_ENV=production`
- [] Configure firewall rules for PostgreSQL/Redis
- [] Enable database backups
- [] Set up monitoring and alerting
- [] Review rate limiting settings

Monitoring

```
### Health Check

```bash

curl https://your-app.onrender.com/health

```

### System Metrics

```bash

curl -H "Authorization: Bearer <token>" https://your-app.onrender.com/system-health/

```

### Logs

- Docker: `docker-compose logs -f api`
- Render: Dashboard ? Service ? Logs
- Local: `tail -f logs/via.log`

---

## Troubleshooting

Issue	Solution
`Connection refused` on PostgreSQL	Ensure PostgreSQL is running and `POSTGRES_HOST` is correct
`401 Unauthorized`	Token expired ? re-login via `/auth/login`
`429 Too Many Requests`	Rate limit hit ? wait or increase `RATE_LIMIT_PER_MINUTE`
Ollama timeout	Increase `REQUEST_TIMEOUT` or switch to Groq
Celery tasks stuck	Check Redis connection and restart worker
Build mode no URLs	Verify `GITHUB_TOKEN` has `repo` scope

```

File: `docs\PERFORMANCE_REPORT.md`

```md
```


VIA Performance Report

Response Times

Endpoint	Mode	Avg	P95
-----	-----	-----	-----
POST /chat/	Chat	2-5s	8s
POST /chat/	Build	30-90s	120s
POST /chat/	Analyze	20-60s	90s
POST /deploy/	Full	45-120s	180s
GET /health	?	<5ms	10ms

Agent Pipeline (Sequential)

Agent	Avg Duration	Notes
-----	-----	-----
CEO	3-8s	Strategy
Architecture	4-10s	Design
Backend	5-15s	APIs
Frontend	10-30s	Largest output
Security	3-8s	Threat analysis
DevOps	3-8s	CI/CD
AI Research	3-8s	Recommendations
HR/Finance/Marketing	3-6s each	Lightweight

Memory Usage

Component	RAM
-----	-----
FastAPI	100-200MB
Ollama (llama3)	4-8GB
PostgreSQL	200-500MB
Redis	50-100MB
Total (Groq)	**500MB-1GB**

Optimization Recommendations

- 1. **Parallel agents** ? Run HR/Finance/Marketing concurrently (save 10-20s)

2. ****LLM caching**** ? Cache identical prompts via Redis TTL
3. ****Streaming**** ? SSE for chat to improve perceived latency
4. ****GZip middleware**** ? Compress API responses
5. ****CDN**** ? Serve static assets via CloudFront

Generated: Phase 6.0.0 | 2026-05-17

...

File: `fix\_correct.py`

```py

f = '/app/backend/agents/frontend\_agent.py'

c = open(f).read()

# Remove the project\_block code that got injected into \_features() by mistake

bad\_code = '''def \_features(task: str) -> dict:

brief = project\_brief or {}

features = ', '.join(brief.get('core\_features', [])) if brief.get('core\_features') else 'Not specified'

project\_block = (

"PROJECT BRIEF (BUILD EXACTLY THIS APP):\n"

f" App Name: {brief.get('app\_name', 'Not specified')}\n"

f" App Type: {brief.get('app\_type', 'Not specified')}\n"

f" Core Features: {features}\n"

f" Tech Stack: {brief.get('tech\_stack', 'Not specified')}\n"

f" Target Users: {brief.get('target\_users', 'Not specified')}\n"

f" UI Style: {brief.get('ui\_style', 'Not specified')}\n"

f" Constraints: {brief.get('key\_constraints', 'Not specified')}\n"

) if brief.get('app\_name') else ""

t = task.lower()'''

good\_code = '''def \_features(task: str) -> dict:

t = task.lower()'''

```

c = c.replace(bad_code, good_code)

Now also remove {project_block} from the prompt f-string since it's not defined there
c = c.replace('TASK: {task}\n{project_block}\n????', 'TASK: {task}\n????')
c = c.replace('\n{project_block}\n', '\n')

open(f, 'w').write(c)

print('STEP 1 DONE - removed bad injection')

Verify _features is clean now
c2 = open(f).read()
if 'project_block' in c2:
 print('WARNING: project_block still exists somewhere')
 idx = c2.find('project_block')
 print(c2[idx-100:idx+200])
else:
 print('CLEAN - project_block fully removed')
'''

File: `fix_final.py`

```py
import re

f = '/app/backend/agents/frontend_agent.py'
c = open(f).read()

# Find the return statement in _build_prompt and fix the project_block reference
# The issue is {project_block} is in the f-string but not always defined
# Simple fix: replace the problematic f-string with a safe version

# Fix the return f-string to not use {project_block} directly
# Instead build the full string before returning

```

```
old_return = ''      return f""You are a senior React engineer. Build a COMPLETE, PRODUCTION-READY
React frontend.
```

```
????????????????????????????????????????????????????????????????
```

```
TASK: {task}

{project_block}
```

```
??????????????????????????????????????????????????????????????''
```

```
new_return = ''      project_section = project_block if project_block else ""

    return f""You are a senior React engineer. Build a COMPLETE, PRODUCTION-READY React frontend.
```

```
????????????????????????????????????????????????????????????????
```

```
TASK: {task}

{project_section}
```

```
??????????????????????????????????????????????????????????????''
```

```
c = c.replace(old_return, new_return)
```

```
# If that didn't work, try the version without project_block in TASK line
```

```
if 'project_section' not in c:

    print('First replacement failed, trying alternative...')

    # Just remove {project_block} from the f-string entirely

    # and add project context to strategy_block instead

    old2 = 'TASK: {task}\n{project_block}\n???'

    new2 = 'TASK: {task}\n???'

    c = c.replace(old2, new2)

    old3 = 'TASK: {task}\n???'

    new3 = 'TASK: {task}\n{strategy_block}\n???'

    # Make project_block part of strategy_block

    old4 = '      strategy_block = f""\nCEO Strategic Direction: {ceo_strategy}\\n" if ceo_strategy
else ""'

    new4 = '      strategy_block = f""\nCEO Strategic Direction: {ceo_strategy}\\n" if
ceo_strategy else ""'

    if project_block:
```

```

        strategy_block = project_block + strategy_block'''

c = c.replace(old4, new4)

open(f, 'w').write(c)

print('DONE')

...

## File: `fix_frontend.py`

```py
c = open('/app/backend/agents/frontend_agent.py').read()

Fix 1: lower threshold
c = c.replace(
 'if not code or len(code.strip()) < 800:',
 'if not code or len(code.strip()) < 200:'
)

Fix 2: add project_brief to frontend_agent signature
c = c.replace(
 'async def frontend_agent(task: str, ceo_strategy: str = "", inter_context: str = "") -> dict:',
 'async def frontend_agent(task: str, project_brief: dict = None, ceo_strategy: str = "", inter_context: str = "") -> dict:'
)

Fix 3: pass project_brief to _build_prompt
c = c.replace(
 'prompt = _build_prompt(task, ceo_strategy, inter_context)',
 'prompt = _build_prompt(task, ceo_strategy, inter_context, project_brief)'
)

Fix 4: add project_brief param to _build_prompt
c = c.replace(
 'def _build_prompt(task: str, ceo_strategy: str = "", inter_context: str = "") -> str:',

```

```

 'def _build_prompt(task: str, ceo_strategy: str = "", inter_context: str = "", project_brief:
dict = None) -> str:'

)

Fix 5: inject project_brief context inside _build_prompt

old = ' t = task.lower()'

new = ''' brief = project_brief or {}

 features = ', '.join(brief.get('core_features', [])) if brief.get('core_features') else 'Not
specified'

 project_block = (

 "PROJECT BRIEF (BUILD EXACTLY THIS APP):\n"

 f" App Name: {brief.get('app_name', 'Not specified')}\n"

 f" App Type: {brief.get('app_type', 'Not specified')}\n"

 f" Core Features: {features}\n"

 f" Tech Stack: {brief.get('tech_stack', 'Not specified')}\n"

 f" Target Users: {brief.get('target_users', 'Not specified')}\n"

 f" UI Style: {brief.get('ui_style', 'Not specified')}\n"

 f" Constraints: {brief.get('key_constraints', 'Not specified')}\n"

) if brief.get('app_name') else ""

 t = task.lower()'''

c = c.replace(old, new, 1)

Fix 6: inject project_block into prompt

c = c.replace(

 'TASK: {task}',

 'TASK: {task}\n{project_block}'

)

open('/app/backend/agents/frontend_agent.py', 'w').write(c)

print('ALL FIXES APPLIED SUCCESSFULLY')

...

File: `fix_frontend_llm.py`

```py

```

```

import re

f = '/app/backend/agents/frontend_agent.py'

c = open(f).read()

# ?? Fix 1: Add project_brief to frontend_agent signature ??

c = c.replace(

    'async def frontend_agent(task: str, ceo_strategy: str = "", inter_context: str = "") -> dict:',

    'async def frontend_agent(task: str, project_brief: dict = None, ceo_strategy: str = "", inter_context: str = "") -> dict:'

)

# ?? Fix 2: Pass project_brief to _build_prompt ??

c = c.replace(

    'prompt      = _build_prompt(task, ceo_strategy, inter_context)',

    'prompt      = _build_prompt(task, ceo_strategy, inter_context, project_brief)'

)

# ?? Fix 3: Add project_brief to _build_prompt signature ??

c = c.replace(

    'def _build_prompt(task: str, ceo_strategy: str = "", inter_context: str = "") -> str:',

    'def _build_prompt(task: str, ceo_strategy: str = "", inter_context: str = "", project_brief: dict = None) -> str:'

)

# ?? Fix 4: Inject project_brief into _build_prompt body ??

old = '    t = task.lower()'

new = '''    brief      = project_brief or {}

    features = ', '.join(brief.get('core_features', [])) if brief.get('core_features') else 'Not specified'

    project_block = (

        "PROJECT BRIEF (BUILD EXACTLY THIS APP):\n"

        f"  App Name:      {brief.get('app_name', 'Not specified')}\n"

        f"  App Type:      {brief.get('app_type', 'Not specified')}\n"

        f"  Core Features: {features}\n"

```

```

        f" Tech Stack:      {brief.get('tech_stack', 'Not specified')}\n"
        f" Target Users:    {brief.get('target_users', 'Not specified')}\n"
        f" UI Style:           {brief.get('ui_style', 'Not specified')}\n"
        f" Constraints:       {brief.get('key_constraints', 'Not specified')}\n"
    ) if brief.get('app_name') else ""

    t = task.lower()'''

c = c.replace(old, new, 1)

# ?? Fix 5: Inject project_block into the prompt ??

c = c.replace(
    'TASK: {task}',
    'TASK: {task}\n{project_block}'
)

# ?? Fix 6: Make _is_valid_jsx ALWAYS return True ??

# This means LLM output is ALWAYS used ? no template fallback

old_func = '''def _is_valid_jsx(code: str) -> bool:
    if not code or len(code.strip()) < 800:
        return False

    bad_imports = [
        "from './Home'", "from './Test'", "from './Result'",
        "from './Pages'", "from './components/", "from './views/",
        "from './screens/", "from './pages/", "from '../components/",
        "import Home from", "import Test from", "import Result from",
        "import Quiz from", "import Question from", "import Score from",
    ]

    if any(b in code for b in bad_imports):
        return False

    if ("localhost:8000" in code or "127.0.0.1:8000" in code) and "import.meta.env" not in code:
        return False

    for match in re.finditer(r\'from\\s+["\\'\\\\"']([^\\"\\'\\'"/][^\\"\\'\\'']*)[\\"\\'\\'"]\\', code):
        pkg = match.group(1).split("/") [0]

        if pkg.startswith("@):

```



```

        full_scope = "/" .join(match.group(1).split("/")[:2])

        if full_scope not in APPROVED_PACKAGES:

            logger.warning(f"LLM App.jsx rejected ? unapproved import: {match.group(1)}")

            return False

    elif pkg not in APPROVED_PACKAGES:

        logger.warning(f"LLM App.jsx rejected ? unapproved import: {pkg}")

        return False

    return "return" in code and "useState" in code'''

new_func = '''def _is_valid_jsx(code: str) -> bool:

    """

    Always use LLM output ? no template fallback.

    Only reject if completely empty or has local component imports that will crash the build.

    """

    if not code or len(code.strip()) < 100:

        return False

    # Only reject multi-file imports that will crash the build

    crash_imports = [

        "from './components/", "from './views/",

        "from './screens/", "from './pages/", "from '../components/",

    ]

    if any(b in code for b in crash_imports):

        return False

    return True'''

c = c.replace(old_func, new_func)

# ?? Fix 7: Also fix _build_all_files to always try LLM first ??

# Remove the _app_jsx fallback call ? LLM always wins

old_build = '''    if not _is_valid_jsx(f.get("src/App.jsx", "")):

        f["src/App.jsx"] = _app_jsx(name, task, feat, th)'''

new_build = '''    if not _is_valid_jsx(f.get("src/App.jsx", "")):

        logger.warning(f"LLM App.jsx was empty or had crash imports ? using safe fallback |
{task[:50]}")

```

```

        f["src/App.jsx"] = _app_jsx(name, task, feat, th)'''

c = c.replace(old_build, new_build)

open(f, 'w').write(c)

print('ALL FIXES APPLIED SUCCESSFULLY')

print('LLM will now build ANY app without template fallback')

'''

## File: `fix_inject_proper.py`

```py
import re

f = '/app/backend/agents/frontend_agent.py'

c = open(f).read()

Fix _build_prompt to accept and use project_brief properly

Find the exact function signature

c = c.replace(

 'def _build_prompt(task: str, ceo_strategy: str = "", inter_context: str = "", project_brief: dict = None) -> str:',

 'def _build_prompt(task: str, ceo_strategy: str = "", inter_context: str = "", project_brief: dict = None) -> str:\n _pb = project_brief or {}\n _feat = ",\n".join(_pb.get("core_features", [])) or "Not specified"\n _proj = ("PROJECT BRIEF:\n App: " + _pb.get("app_name", "") + "\n Type: " + _pb.get("app_type", "") + "\n Features: " + _feat + "\n Stack: " + _pb.get("tech_stack", "Not specified") + "\n Users: " + _pb.get("target_users", "Not specified") + "\n") if _pb.get("app_name") else ""'

)

Inject _proj into the return f-string after TASK line

c = c.replace(

 'TASK: {task}\n????',

 'TASK: {task}\n{_proj}\n????'

)

open(f, 'w').write(c)

```

```

Verify

c2 = open(f).read()

print('_proj injected:', '_proj' in c2)

print('project_brief param:', 'project_brief: dict = None) -> str:' in c2)

```

## File: `fix_jsx.py`

```py

f = '/app/backend/agents/frontend_agent.py'

c = open(f).read()

Find the _is_valid_jsx function and replace it entirely

import re

new_func = '''def _is_valid_jsx(code: str) -> bool:

 """Always use LLM output ? no template fallback."""

 if not code or len(code.strip()) < 100:

 return False

 crash_imports = [

 "from './components/", "from './views/",

 "from './screens/", "from './pages/", "from '../components/",

]

 if any(b in code for b in crash_imports):

 return False

 return True'''

Replace the entire _is_valid_jsx function

c = re.sub(

 r'def _is_valid_jsx\(code: str\) -> bool:.*?return "return" in code and "useState" in code',

 new_func,

 c,

 flags=re.DOTALL

)

```

```
open(f, 'w').write(c)
```

```
print('DONE')
```

```
...
```

```
File: `fix_nuclear.py`
```

```
```py
```

```
import re
```

```
f = '/app/backend/agents/frontend_agent.py'
```

```
c = open(f).read()
```

```
# Remove ALL project_block injections using regex
```

```
# This removes any block that starts with project_block assignment
```

```
c = re.sub(
    r"\n    brief\s*=\s*project_brief or \{\}\.*?t = task\.lower\(\)",
    "\n    t = task.lower()",
    c,
    flags=re.DOTALL
)
```

```
# Remove any remaining {project_block} references in f-strings
```

```
c = c.replace('\n{project_block}\n', '\n')
```

```
c = c.replace('{project_block}', '')
```

```
c = c.replace('\n{project_section}\n', '\n')
```

```
c = c.replace('{project_section}', '')
```

```
# Remove project_block standalone assignments if any remain
```

```
c = re.sub(
    r'\n\s+project_block\s*=\.*?(?=\n\s+[a-zA-Z])',
    '\n',
    c,
    flags=re.DOTALL
)
```

```
)
```

```
open(f, 'w').write(c)
```

```
# Verify
```

```
c2 = open(f).read()
```

```
count = c2.count('project_block')
```

```
print(f'project_block occurrences remaining: {count}')
```

```
if count == 0:
```

```
    print('CLEAN - all removed')
```

```
else:
```

```
    idx = c2.find('project_block')
```

```
    print('Still found at:')
```

```
    print(c2[idx-50:idx+200])
```

```
...
```

```
## File: `fix_project_block.py`
```

```
```py
```

```
import re
```

```
f = '/app/backend/agents/frontend_agent.py'
```

```
c = open(f).read()
```

```
The problem: project_block is defined inside _build_prompt
```

```
but the f-string in the return uses {project_block}
```

```
which fails when _build_prompt is called without project_brief
```

```
Fix: make project_block default to empty string at the top of _build_prompt
```

```
old = ''' brief = project_brief or {}
```

```
 features = ', '.join(brief.get('core_features', [])) if brief.get('core_features') else 'Not specified'
```

```
 project_block = (
```

```
 "PROJECT BRIEF (BUILD EXACTLY THIS APP):\n"
```

```
 f" App Name: {brief.get('app_name', 'Not specified')}\n"
```

```

 f" App Type: {brief.get('app_type', 'Not specified')}\n"

 f" Core Features: {features}\n"

 f" Tech Stack: {brief.get('tech_stack', 'Not specified')}\n"

 f" Target Users: {brief.get('target_users', 'Not specified')}\n"

 f" UI Style: {brief.get('ui_style', 'Not specified')}\n"

 f" Constraints: {brief.get('key_constraints', 'Not specified')}\n"

) if brief.get('app_name') else ""

 t = task.lower()'''

new = ''' brief = project_brief or {}

 features = ', '.join(brief.get('core_features', [])) if brief.get('core_features') else 'Not
specified'

 app_name = brief.get('app_name', '')
 app_type = brief.get('app_type', '')
 tech_stack = brief.get('tech_stack', 'Not specified')
 target_users = brief.get('target_users', 'Not specified')
 ui_style = brief.get('ui_style', 'Not specified')
 constraints = brief.get('key_constraints', 'Not specified')

 if app_name:

 project_block = (

 f"PROJECT BRIEF (BUILD EXACTLY THIS APP):\n"

 f" App Name: {app_name}\n"

 f" App Type: {app_type}\n"

 f" Core Features: {features}\n"

 f" Tech Stack: {tech_stack}\n"

 f" Target Users: {target_users}\n"

 f" UI Style: {ui_style}\n"

 f" Constraints: {constraints}\n"

)

 else:

 project_block = ""

 t = task.lower()'''

c = c.replace(old, new)

```

```

Also fix the TASK line to use project_block safely

c = c.replace(
 'TASK: {task}\n{project_block}',
 'TASK: {task}'
)

Find the return f-string and inject project_block after the task line

c = c.replace(
 'TASK: {task}\n????',
 'TASK: {task}\n{project_block}\n????'
)

open(f, 'w').write(c)

print('DONE - project_block fix applied')

...

File: `fix_safe_main.py`

```py
import re

f = '/app/backend/core/fullstack_builder.py'

c = open(f).read()

# Find and replace the broken _safe_main function entirely

old = re.search(r'def _safe_main\(title: str\) -> str:.*?(?=\ndef |\nclass |\n# )', c, re.DOTALL)

if old:
    print(f"Found _safe_main at position {old.start()}")
    print("Current broken version:")
    print(old.group()[:200])
else:
    print("_safe_main not found by regex, searching manually...")
    idx = c.find('def _safe_main')

```

```

    if idx >= 0:
        print(c[idx:idx+300])
    else:
        print("_safe_main NOT FOUND in file")
'''

## File: `fix_safe_main2.py`

```py
f = '/app/backend/core/fullstack_builder.py'
c = open(f).read()

Find _safe_main and replace entirely with working version
import re

new_safe_main = '''def _safe_main(title: str) -> str:
 t = title.replace("'", '\\\\\'')
 return """"# main.py ? VIA Safe Fallback

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
import os

app = FastAPI(title=\\\"\\\"\\\" + t + \"\\\"\\\"\\\", version=\"1.0.0\")
app.add_middleware(
 CORSMiddleware,
 allow_origins=[\"*\"],
 allow_credentials=True,
 allow_methods=[\"*\"],
 allow_headers=[\"*\"],
)

db = []

@app.get(\"/\")

```



```
def root():
 return {"app": \""" + t + """, "status": "running", "docs": "/docs"}

@app.get("/health")
def health():
 return {"status": "healthy"}

@app.get("/api/v1/items")
def get_items():
 return {"items": db, "total": len(db)}

@app.post("/api/v1/items")
def create_item(item: dict):
 item["id"] = len(db) + 1
 db.append(item)
 return item

@app.get("/api/v1/items/{item_id}")
def get_item(item_id: int):
 item = next((i for i in db if i.get("id") == item_id), None)
 if not item:
 raise HTTPException(status_code=404, detail="Not found")
 return item

@app.put("/api/v1/items/{item_id}")
def update_item(item_id: int, data: dict):
 for i, item in enumerate(db):
 if item.get("id") == item_id:
 db[i].update(data)
 return db[i]
 raise HTTPException(status_code=404, detail="Not found")

@app.delete("/api/v1/items/{item_id}")
def delete_item(item_id: int):
```

```

global db

db = [i for i in db if i.get("id") != item_id]

return {"deleted": item_id}

@app.get("/api/v1/stats")

def get_stats():

 return {"total": len(db), "active": len(db)}

if __name__ == "__main__":

 import uvicorn

 port = int(os.getenv("PORT", 8000))

 uvicorn.run("main:app", host="0.0.0.0", port=port, reload=False)

"""

...

Replace everything between def _safe_main and the next def/class

c_new = re.sub(

 r'def _safe_main\(title: str\) -> str:.*?(?=\ndef |\nclass |\n# ??)',

 new_safe_main,

 c,

 flags=re.DOTALL

)

if c_new == c:

 print('Pattern not matched ? trying alternative...')

 # Try to find and replace just the broken line

 c_new = c.replace(

 'app = FastAPI(title=" + title + ", version="1.0.0")',

 'app = FastAPI(title="VIA App", version="1.0.0")'

)

 c_new = c_new.replace(

 'app = FastAPI(title= + title + , version="1.0.0")',

 'app = FastAPI(title="VIA App", version="1.0.0")'

```

```

)

if c_new != c:

 print('Fixed broken FastAPI title line')

else:

 print('Could not find broken line')

open(f, 'w').write(c_new)

Verify by checking what _safe_main generates

import sys

sys.path.insert(0, '/app')

exec(open(f).read().split('def _safe_main')[1].split('\ndef ')[0].replace('def _safe_main', ''))

import ast

try:

 # Test with a sample title

 idx = c_new.find('def _safe_main')

 print('DONE - checking syntax of generated code...')

 # Simple verification

 if 'app = FastAPI(title=' in c_new:

 # Extract the safe main generation

 print('Safe main function found')

import py_compile

import tempfile

import os

Write a test

test_code = '''

def _safe_main(title):

 return "test"

result = _safe_main("test")

print("Function works:", result)

'''

```

```

 print('Running syntax check on fullstack_builder...')
except Exception as e:
 print(f'Error: {e}')

import py_compile

try:
 py_compile.compile(f)
 print('SYNTAX OK - fullstack_builder.py is valid')
except SyntaxError as e:
 print(f'SYNTAX ERROR: {e}')
...

File: `fix_syntax_validator.py`

```py
import ast

f = '/app/backend/core/fullstack_builder.py'
c = open(f).read()

# Add ast import
if 'import ast' not in c:
    c = c.replace('import re\nimport logging', 'import re\nimport logging\nimport ast')

# Add validator function after logger
old_logger = 'logger = logging.getLogger("AI-Digital-Company")'
new_logger = '''logger = logging.getLogger("AI-Digital-Company")

def _validate_python(code: str) -> bool:
    try:
        ast.parse(code)
        return True
    except SyntaxError as e:
        logger.warning(f"Syntax error in generated code: {e}")

```

```
return False
```

```
def _safe_main(title: str) -> str:
```

```
    return ""# main.py ? VIA Safe Fallback
```

```
from fastapi import FastAPI, HTTPException
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
import os
```

```
app = FastAPI(title=\\\"\\\" + title + \"\\\"\\\", version=\"1.0.0\")
```

```
app.add_middleware(CORSMiddleware,                    allow_origins=[\"*\"],                    allow_credentials=True,
allow_methods=[\"*\"], allow_headers=[\"*\"])
```

```
db = []
```

```
@app.get(\"/\")
```

```
def root():
```

```
    return {\"app\": \\\"\\\" + title + \"\\\"\\\", \"status\": \"running\", \"docs\": \"/docs\"}
```

```
@app.get(\"/health\")
```

```
def health():
```

```
    return {\"status\": \"healthy\"}
```

```
@app.get(\"/api/v1/items\")
```

```
def get_items():
```

```
    return {\"items\": db, \"total\": len(db)}
```

```
@app.post(\"/api/v1/items\")
```

```
def create_item(item: dict):
```

```
    item[\"id\"] = len(db) + 1
```

```
    db.append(item)
```

```
    return item
```

```
@app.get(\"/api/v1/items/{item_id}\")
```

```
def get_item(item_id: int):
```

```
    item = next((i for i in db if i.get(\"id\") == item_id), None)
```

```

    if not item:

        raise HTTPException(status_code=404, detail="Not found")

    return item


@app.put("/api/v1/items/{item_id}")
def update_item(item_id: int, data: dict):

    for i, item in enumerate(db):

        if item.get("id") == item_id:

            db[i].update(data)

            return db[i]

    raise HTTPException(status_code=404, detail="Not found")


@app.delete("/api/v1/items/{item_id}")
def delete_item(item_id: int):

    global db

    db = [i for i in db if i.get("id") != item_id]

    return {"deleted": item_id}


@app.get("/api/v1/stats")
def get_stats():

    return {"total": len(db), "active": len(db)}


if __name__ == "__main__":

    import uvicorn

    port = int(os.getenv("PORT", 8000))

    uvicorn.run("main:app", host="0.0.0.0", port=port, reload=False)

#####


if old_logger in c and '_validate_python' not in c:

    c = c.replace(old_logger, new_logger)

    print('Added validator functions')

else:

    print('Validator already exists or logger not found')

```

```

# Fix generate_backend_files to use validator

old_line = '        files["main.py"]                = _generate_main_py(task, title, app_type, table_prefix,
resource)'

new_line = '''        _raw_main = _generate_main_py(task, title, app_type, table_prefix, resource)

        if _validate_python(_raw_main):

            files["main.py"] = _raw_main

            logger.info("main.py syntax OK")

        else:

            files["main.py"] = _safe_main(title)

            logger.warning("main.py had syntax errors ? using safe fallback")'''

if old_line in c:

    c = c.replace(old_line, new_line)

    print('Fixed generate_backend_files')

else:

    print('generate_backend_files line not found ? may already be fixed')

open(f, 'w').write(c)

print('DONE')

...

## File: `fix_validator_final.py`

```py

f = '/app/backend/core/fullstack_builder.py'

c = open(f).read()

old = ' files["main.py"] = _generate_main_py(task, title, app_type, table_prefix,
resources, project_brief)'

new = ''' _raw_main = _generate_main_py(task, title, app_type, table_prefix, resources,
project_brief)

 if _validate_python(_raw_main):

 files["main.py"] = _raw_main

 logger.info("main.py syntax OK")

 else:

```

```

files["main.py"] = _safe_main(title)

logger.warning("main.py had syntax errors ? using safe fallback")'''

if old in c:

 c = c.replace(old, new)

 open(f, 'w').write(c)

 print('FIXED - validator now called for every app')

else:

 print('Line not found - printing current line:')

 idx = c.find('files["main.py"]')

 print(c[idx-50:idx+200])

'''

File: `index.html`

```html
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>VIA ? Autonomous AI Digital Team</title>

<meta name="description" content="VIA: AI-powered autonomous digital team platform. Chat, build,
and deploy apps with 9 AI agents.">

<link
href="https://fonts.googleapis.com/css2?family=Orbitron:wght@400;500;600;700;800;900&family=Space+
Mono:wght@400;700&family=Rajdhani:wght@300;400;500;600;700&display=swap" rel="stylesheet">

<link rel="stylesheet" href="/via-chat.css?v=10">

<style>

#authPage {

    background: url('/login_bg.png') center center / cover no-repeat !important;

    background-color: #0a0a1a !important;

}

.auth-logo {

    width: 110px !important;

    height: auto !important;

```



```

        display: block !important;

        object-fit: contain !important;

    }
</style>

</head>

<body>

<div id="authPage">

    <div class="auth-wrap">

        <div class="auth-brand">

            <img id="viaLogo" alt="VIA Logo" class="auth-logo">

            <div class="via-display">VIA</div>

        </div>

        <div class="holo-card">

            <div class="a-tabs" id="authTabs">

                <button class="a-tab on" id="tabLogin" onclick="switchAuth('login')">Login</button>

                <button class="a-tab" id="tabRegister" onclick="switchAuth('register')">Register</button>

            </div>

            <div id="authMsg" class="a-msg"></div>

            <!-- LOGIN FORM -->

            <div id="loginForm">

                <div class="fld"><label class="fld-label">Email Address</label><input class="fld-input"
id="loginUser" type="email" placeholder="Enter your email" autocomplete="email"></div>

                <div class="fld"><label class="fld-label">Password</label><input class="fld-input"
id="loginPass" type="password" placeholder="Enter password" autocomplete="current-password"></div>

                <button class="cyber-btn" id="loginBtn" onclick="doLogin()"><span class="lb">Access
System</span></button>

                <div style="text-align:center;margin-top:12px">

                    <button          onclick="switchAuth('forgot')"
style="background:none;border:none;color:var(--c);font-size:12px;cursor:pointer;text-decoration:un
derline">Forgot Password?</button>

                </div>

            </div>

            <!-- REGISTER FORM -->

            <div id="regForm" style="display:none">

```

```

        <div class="fld"><label class="fld-label">Email Address</label><input class="fld-input"
id="regUser" type="email" placeholder="Enter your email" autocomplete="email"></div>

        <div class="fld"><label class="fld-label">Password</label><input class="fld-input"
id="regPass" type="password" placeholder="Choose password (min 6 chars)"
autocomplete="new-password"></div>

        <button class="cyber-btn" id="regBtn" onclick="doRegister()"><span class="lb">Create
Account</span></button>

    </div>

    <!-- EMAIL VERIFICATION FORM -->

    <div id="verifyForm" style="display:none">

        <div style="text-align:center;margin-bottom:12px;font-size:13px;color:var(--tx2)">

            ? A 6-digit verification code has been sent to your email.<br>

            <span style="color:var(--y);font-size:11px">Check your inbox (or spam folder) for the
code.</span>

        </div>

        <div class="fld"><label class="fld-label">Verification Code</label><input
class="fld-input" id="verifyCode" type="text" placeholder="Enter 6-digit code" maxlength="6"
autocomplete="one-time-code"></div>

        <button class="cyber-btn" id="verifyBtn" onclick="doVerify()"><span class="lb">? Verify
Email</span></button>

        <div style="text-align:center;margin-top:10px">

            <button onclick="switchAuth('login')"
style="background:none;border:none;color:var(--tx3);font-size:11px;cursor:pointer">? Back to
Login</button>

        </div>

    </div>

    <!-- FORGOT PASSWORD FORM -->

    <div id="forgotForm" style="display:none">

        <div id="forgotStep1">

            <div style="text-align:center;margin-bottom:12px;font-size:13px;color:var(--tx2)">Enter
your email to receive a reset code.</div>

            <div class="fld"><label class="fld-label">Email Address</label><input class="fld-input"
id="forgotEmail" type="email" placeholder="Enter your email"></div>

            <button class="cyber-btn" id="forgotBtn" onclick="doForgotRequest()"><span
class="lb">Send Reset Code</span></button>

        </div>

        <div id="forgotStep2" style="display:none">

            <div style="text-align:center;margin-bottom:12px;font-size:13px;color:var(--tx2)">

                ? A password reset code has been sent to your email.<br>

```

```

        <span style="color:var(--y);font-size:11px">Check your inbox (or spam folder) for the
code.</span>

    </div>

    <div class="fld"><label class="fld-label">Reset Code</label><input class="fld-input"
id="resetCode" type="text" placeholder="6-digit reset code" maxlength="6"></div>

    <div class="fld"><label class="fld-label">New Password</label><input class="fld-input"
id="resetPass" type="password" placeholder="New password (min 6 chars)"></div>

    <button class="cyber-btn" id="resetBtn" onclick="doReset()"><span class="lb">? Reset
Password</span></button>

</div>

<div style="text-align:center;margin-top:10px">

                                <button      onclick="switchAuth('login')"
style="background:none;border:none;color:var(--tx3);font-size:11px;cursor:pointer">?      Back      to
Login</button>

</div>

</div>

</div>

</div>

</div>

<!-- ??? APP SHELL ??? -->

<button class="mobile-toggle" id="mobileToggle" onclick="toggleSidebar()">?</button>

<div class="sidebar-overlay" id="sidebarOverlay" onclick="toggleSidebar()"></div>

<div id="app">

<div class="shell">

    <!-- SIDEBAR -->

    <aside class="sidebar" id="sidebar">

        <div class="sb-brand">

        <div><div class="sb-name">VIA</div><div class="sb-ver">v6.0 ? Phase 6</div></div>

        <button class="theme-toggle" id="themeToggle" onclick="toggleTheme()" title="Toggle
theme">?</button>

    </div>

    <nav class="sb-nav">

        <div class="sb-sec">Main</div>

                                <button class="sb-item on" data-view="chat" onclick="showView('chat')"><span
```

```
class="sb-icon">?</span>Chat</button>

        <button class="sb-item" data-view="dashboard" onclick="showView('dashboard')"><span
class="sb-icon">?</span>Dashboard</button>

        <button class="sb-item" data-view="build" onclick="showView('build')"><span
class="sb-icon">??</span>Build</button>

    <div class="sb-sec">Data</div>

        <button class="sb-item" data-view="projects" onclick="showView('projects')"><span
class="sb-icon">?</span>Projects</button>

        <button class="sb-item" data-view="memory" onclick="showView('memory')"><span
class="sb-icon">?</span>Memory</button>

        <button class="sb-item" data-view="history" onclick="showView('history')"><span
class="sb-icon">?</span>History</button>

        <button class="sb-item" data-view="meetings" onclick="showView('meetings')"><span
class="sb-icon">??</span>Meetings</button>

    <div class="sb-sec">Company</div>

        <button class="sb-item" data-view="templates" onclick="showView('templates')"><span
class="sb-icon">?</span>Templates</button>

        <button class="sb-item" data-view="org" onclick="showView('org')"><span
class="sb-icon">?</span>Org Chart</button>

</nav>

    <div class="sb-builds" id="recentBuilds"><div class="sb-builds-title">Recent Builds</div><div
id="recentBuildsList"></div></div>

    <div class="sb-footer">

        <div class="u-av" id="userAv">U</div>

        <div class="u-info"><div class="u-nm" id="userName">User</div><div
class="u-rl">Operator</div></div>

        <button class="lo-btn" onclick="doLogout()" title="Logout">?</button>

    </div>

</aside>

<!-- MAIN CONTENT -->

<main>

    <!-- CHAT VIEW -->

    <div class="view on" id="v-chat">

        <div class="chat-header">

            <div class="chat-tittle">VIA Chat ? AI Assistant</div>

            <button class="btn-new" onclick="newChat()">+ New Chat</button>

        </div>

        <div class="chat-msgs" id="chatMsgs">
```

```
<div class="msg msg-via">

  <div class="msg-av">?</div>

  <div>

    <div class="msg-body">

      <strong>Welcome to VIA!</strong> I'm your autonomous AI digital team.<br><br>

      I can do three things:<br>

      ? <strong>Chat</strong> ? Ask me anything about tech, code, or concepts<br>

      ?? <strong>Build</strong> ? Say "Build me a [your idea]" and I'll create & deploy a
live app<br>

      ? <strong>Analyze</strong> ? Ask me to analyze, plan, or strategize anything<br><br>

      What would you like to do? ?

    </div>

  </div>

</div>

</div>

<div class="chat-input-wrap">

  <div class="chat-input-box">

    <textarea id="chatInput" rows="1" placeholder="Ask anything or say 'Build me
a...'"></textarea>

    <button id="sendBtn" onclick="sendChat()">

      <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path
d="M22 2L11 13M22 2L7 20-4-9-9-4 20-7z"/></svg>

    </button>

  </div>

</div>

</div>
```

```
<!-- DASHBOARD VIEW -->

<div class="view" id="v-dashboard">

  <div class="dash-view">

    <div class="pg-title">Command Center</div>

    <div class="pg-sub">VIA Phase 6 ? System Overview</div>

    <div class="kpi-row">

      <div class="kpi kpi-c"><div class="kpi-label">Total Runs</div><div class="kpi-val"
id="kTotalRuns">?</div><div class="kpi-hint">All time</div></div>

      <div class="kpi kpi-v"><div class="kpi-label">Agents</div><div
class="kpi-val">10</div><div class="kpi-hint">Active departments</div></div>
```

```

        <div class="kpi kpi-g"><div class="kpi-label">Success Rate</div><div class="kpi-val"
id="kSuccRate">?</div><div class="kpi-hint">Overall</div></div>

        <div class="kpi kpi-m"><div class="kpi-label">Avg Duration</div><div class="kpi-val"
id="kAvgDur">?</div><div class="kpi-hint">Seconds</div></div>

    </div>

    <div class="panel">

        <div class="panel-head"><span class="panel-title">System Health</span><button
class="btn-new" onclick="loadDashboard()">Refresh</button></div>

        <div class="panel-body" id="healthBody"><div class="empty">Loading system
metrics...</div></div>

    </div>

</div>

</div>

</div>

<!-- BUILD VIEW -->

<div class="view" id="v-build">

    <div class="dash-view">

        <div class="pg-title">Build & Deploy</div>

        <div class="pg-sub">Describe your app ? VIA builds & deploys it live</div>

        <div class="panel">

            <div class="panel-head"><span class="panel-title">New Build</span></div>

            <div class="panel-body">

                <div class="fld"><label class="fld-label">Describe your app</label>

                    <textarea class="fld-input" id="buildTask" style="min-height:100px;resize:vertical;"
placeholder="e.g. Build me a todo app with user login..."></textarea>

                </div>

                <button class="cyber-btn" id="buildBtn" onclick="doBuild()"
style="max-width:260px"><span class="lb">? Build & Deploy</span></button>

                <div id="buildResult" style="margin-top:16px"></div>

            </div>

        </div>

    </div>

</div>

<!-- MEMORY VIEW -->

<div class="view" id="v-memory">

    <div class="dash-view">

```

```

<div class="pg-title">Agent Memory</div>

<div class="pg-sub">What agents remember from past runs</div>

<div class="panel">
    <div class="panel-head"><span class="panel-title">Recent Memories</span><button
class="btn-new" onclick="loadMemory()">Refresh</button></div>

    <div class="panel-body" id="memBody"><div class="empty">Loading memories...</div></div>

</div>

</div>

</div>

<!-- HISTORY VIEW -->

<div class="view" id="v-history">

    <div class="dash-view">

        <div class="pg-title">Company History</div>

        <div class="pg-sub">Past build runs and agent executions</div>

        <div class="panel">

            <div class="panel-head"><span class="panel-title">Recent Runs</span><button
class="btn-new" onclick="loadHistory()">Refresh</button></div>

            <div class="panel-body" id="histBody"><div class="empty">Loading history...</div></div>

        </div>

    </div>

</div>

</div>

<!-- PROJECTS VIEW -->

<div class="view" id="v-projects">

    <div class="dash-view">

        <div class="pg-title">Projects</div>

        <div class="pg-sub">Browse generated project files</div>

        <div class="panel">

            <div class="panel-head"><span class="panel-title">Project Files</span><button
class="btn-new" onclick="loadProjects()">Refresh</button></div>

            <div class="panel-body" id="projBody"><div class="empty">Loading projects...</div></div>

        </div>

    </div>

</div>

</div>

```

```
<!-- TEMPLATES VIEW -->
```

```
<div class="view" id="v-templates">
```

```
  <div class="dash-view">
```

```
    <div class="pg-title">Task Templates</div>
```

```
    <div class="pg-sub">Quick-start templates for common app types</div>
```

```
    <div class="panel">
```

```
      <div class="panel-head"><span class="panel-title">Available Templates</span></div>
```

```
      <div class="panel-body" id="tplBody"><div class="tpl-grid" id="tplGrid"></div></div>
```

```
    </div>
```

```
  </div>
```

```
</div>
```

```
<!-- MEETINGS VIEW -->
```

```
<div class="view" id="v-meetings">
```

```
  <div class="dash-view">
```

```
    <div class="pg-title">Meeting Room</div>
```

```
    <div class="pg-sub">AI agent boardroom ? generate live discussions</div>
```

```
    <div class="panel">
```

```
      <div class="panel-head"><span class="panel-title">New Meeting</span></div>
```

```
      <div class="panel-body">
```

```
        <div class="meeting-controls">
```

```
          <input class="fld-input" id="meetingTask" placeholder="Describe the topic to discuss..." style="flex:1;min-width:200px">
```

```
          <button class="cyber-btn" id="meetingBtn" onclick="generateMeeting()" style="max-width:180px"><span class="lb">?? Start Meeting</span></button>
```

```
        </div>
```

```
        <div id="meetingTranscript"></div>
```

```
      </div>
```

```
    </div>
```

```
  <div class="panel">
```

```
    <div class="panel-head"><span class="panel-title">Past Meetings</span><button class="btn-new" onclick="loadMeetings()">Refresh</button></div>
```

```
    <div class="panel-body" id="meetingsBody"><div class="empty">Loading meetings...</div></div>
```

```
  </div>
```

```
</div>
```


</div>

<!-- ORG VIEW -->

<div class="view" id="v-org">

<div class="dash-view">

<div class="pg-title">Organization Chart</div>

<div class="pg-sub">VIA's 9-department AI team structure</div>

<div class="panel">

<div class="panel-head">Departments</div>

<div class="panel-body" id="orgBody">

<div style="display:grid;grid-template-columns:repeat(3,1fr);gap:10px">

<div class="mem-card"><div class="mem-agent">?? CEO</div><div class="mem-task">Strategy & delegation</div></div>

<div class="mem-card"><div class="mem-agent">?? Backend</div><div class="mem-task">APIs, databases, services</div></div>

<div class="mem-card"><div class="mem-agent">? Frontend</div><div class="mem-task">React UI, design, UX</div></div>

<div class="mem-card"><div class="mem-agent">? Security</div><div class="mem-task">Auth, encryption, threats</div></div>

<div class="mem-card"><div class="mem-agent">? DevOps</div><div class="mem-task">CI/CD, infra, deployment</div></div>

<div class="mem-card"><div class="mem-agent">? AI Research</div><div class="mem-task">LLM strategy, ML models</div></div>

<div class="mem-card"><div class="mem-agent">? Architecture</div><div class="mem-task">System design, data flow</div></div>

<div class="mem-card"><div class="mem-agent">? HR</div><div class="mem-task">Team structure, hiring</div></div>

<div class="mem-card"><div class="mem-agent">? Finance</div><div class="mem-task">Budget, ROI, pricing</div></div>

<div class="mem-card"><div class="mem-agent">? Marketing</div><div class="mem-task">GTM, branding, growth</div></div>

<div class="mem-card"><div class="mem-agent">? Presentation</div><div class="mem-task">Reports, summaries, slides</div></div>

</div>

</div>

</div>

</div>

</div>

</main>

</div>

```
</div>
```

```
<script>
```

```
// ??? ASSET LOADER (bypass browser cache) ???
```

```
(function() {
```

```
    const t = Date.now();
```

```
    const logo = document.getElementById('viaLogo');
```

```
    if (logo) logo.src = '/company_logo.png?t=' + t;
```

```
    const authPage = document.getElementById('authPage');
```

```
    if (authPage) authPage.style.backgroundImage = "url('/login_bg.png?t=" + t + "')";
```

```
})();
```

```
// ??? STATE ???
```

```
let TOKEN = localStorage.getItem('via_token') || '';
```

```
let USERNAME = localStorage.getItem('via_user') || '';
```

```
let chatBusy = false;
```

```
const API = '';
```

```
// ??? AUTH ???
```

```
let _pendingVerifyEmail = '';
```

```
let _forgotEmail = '';
```

```
function switchAuth(mode) {
```

```
    document.querySelectorAll('.a-tab').forEach(t => t.classList.remove('on'));
```

```
    document.getElementById('loginForm').style.display = 'none';
```

```
    document.getElementById('regForm').style.display = 'none';
```

```
    document.getElementById('verifyForm').style.display = 'none';
```

```
    document.getElementById('forgotForm').style.display = 'none';
```

```
    const tabs = document.getElementById('authTabs');
```

```
    if (mode === 'login') {
```

```
        document.getElementById('loginForm').style.display = '';
```

```
        document.getElementById('tabLogin').classList.add('on');
```

```
        tabs.style.display = '';
```

```

} else if (mode === 'register') {

    document.getElementById('regForm').style.display = '';

    document.getElementById('tabRegister').classList.add('on');

    tabs.style.display = '';

} else if (mode === 'verify') {

    document.getElementById('verifyForm').style.display = '';

    tabs.style.display = 'none';

} else if (mode === 'forgot') {

    document.getElementById('forgotForm').style.display = '';

    tabs.style.display = 'none';

}

hideMsg();

}

function showMsg(txt, ok) {

    const el = document.getElementById('authMsg');

    el.className = 'a-msg ' + (ok ? 'ok' : 'err');

    el.textContent = txt;

    el.style.display = 'block';

}

function hideMsg() { document.getElementById('authMsg').style.display = 'none'; }

async function doLogin() {

    const u = document.getElementById('loginUser').value.trim();

    const p = document.getElementById('loginPass').value;

    if (!u || !p) return showMsg('Please fill in all fields', false);

    const btn = document.getElementById('loginBtn');

    btn.disabled = true;

    try {

        const fd = new URLSearchParams(); fd.append('username', u); fd.append('password', p);

        const r = await fetch(API + '/auth/login', { method: 'POST', body: fd });

        const d = await r.json();

        if (!r.ok) throw new Error(d.detail || 'Login failed');

        TOKEN = d.access_token;

```

```

    USERNAME = u;

    localStorage.setItem('via_token', TOKEN);

    localStorage.setItem('via_user', USERNAME);

    enterApp();
} catch (e) { showMsg(e.message, false); }

btn.disabled = false;
}

```

```

async function doRegister() {

    const u = document.getElementById('regUser').value.trim();

    const p = document.getElementById('regPass').value;

    if (!u || !p) return showMsg('Please fill in all fields', false);

    if (p.length < 6) return showMsg('Password must be at least 6 characters', false);

    const btn = document.getElementById('regBtn');

    btn.disabled = true;

    try {

        const r = await fetch(API + '/auth/register', {

            method: 'POST', headers: { 'Content-Type': 'application/json' },

            body: JSON.stringify({ email: u, password: p })

        });

        const d = await r.json();

        if (!r.ok) throw new Error(d.detail || 'Registration failed');

        _pendingVerifyEmail = u;

        showMsg('Account created! A verification code has been sent to your email.', true);

        switchAuth('verify');

    } catch (e) { showMsg(e.message, false); }

    btn.disabled = false;

}

```

```

async function doVerify() {

    const code = document.getElementById('verifyCode').value.trim();

    if (!code || code.length !== 6) return showMsg('Enter the 6-digit verification code', false);

    const btn = document.getElementById('verifyBtn');

    btn.disabled = true;

```

```

try {
  const r = await fetch(API + '/auth/verify-email', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: _pendingVerifyEmail, code })
  });

  const d = await r.json();

  if (!r.ok) throw new Error(d.detail || 'Verification failed');

  showMsg('? Email verified! You can now login.', true);

  setTimeout(() => { switchAuth('login'); document.getElementById('loginUser').value =
_pendingVerifyEmail; }, 1500);

} catch (e) { showMsg(e.message, false); }

btn.disabled = false;
}

```

```

async function doForgotRequest() {
  const email = document.getElementById('forgotEmail').value.trim();

  if (!email) return showMsg('Enter your email address', false);

  const btn = document.getElementById('forgotBtn');

  btn.disabled = true;

  try {
    const r = await fetch(API + '/auth/forgot-password', {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email })
    });

    const d = await r.json();

    if (!r.ok) throw new Error(d.detail || 'Request failed');

    _forgotEmail = email;

    showMsg('Reset code sent! Check your email (or spam folder).', true);

    document.getElementById('forgotStep1').style.display = 'none';

    document.getElementById('forgotStep2').style.display = '';

  } catch (e) { showMsg(e.message, false); }

  btn.disabled = false;
}

```

```

async function doReset() {

```

```

const code = document.getElementById('resetCode').value.trim();

const newPass = document.getElementById('resetPass').value;

if (!code || !newPass) return showMsg('Fill in all fields', false);

if (newPass.length < 6) return showMsg('Password must be at least 6 characters', false);

const btn = document.getElementById('resetBtn');

btn.disabled = true;

try {

  const r = await fetch(API + '/auth/reset-password', {

    method: 'POST', headers: { 'Content-Type': 'application/json' },

    body: JSON.stringify({ email: _forgotEmail, code, new_password: newPass })

  });

  const d = await r.json();

  if (!r.ok) throw new Error(d.detail || 'Reset failed');

  showMsg('? Password reset! You can now login.', true);

  setTimeout(() => { switchAuth('login'); document.getElementById('loginUser').value =
    _forgotEmail; document.getElementById('forgotStep1').style.display = '';
    document.getElementById('forgotStep2').style.display = 'none'; }, 1800);

  } catch (e) { showMsg(e.message, false); }

  btn.disabled = false;

}

function doLogout() {

  TOKEN = ''; USERNAME = '';

  localStorage.removeItem('via_token');

  localStorage.removeItem('via_user');

  document.body.classList.remove('light'); // Restore auth dark theme

  document.getElementById('app').style.display = 'none';

  document.getElementById('authPage').style.display = 'flex';

}

function enterApp() {

  document.getElementById('authPage').style.display = 'none';

  document.body.classList.add('light'); // Force professional white theme

  document.getElementById('app').style.display = 'block';

  document.getElementById('userAv').textContent = USERNAME.charAt(0).toUpperCase();

```

```

    document.getElementById('userName').textContent = USERNAME;

    loadChatHistory();
}

function authHeaders() { return { 'Authorization': 'Bearer ' + TOKEN, 'Content-Type':
'application/json' }; }

// ??? THEME ???

function toggleTheme() {

    document.body.classList.toggle('light');

    const isLight = document.body.classList.contains('light');

    document.getElementById('themeToggle').textContent = isLight ? '??' : '?';

    localStorage.setItem('via_theme', isLight ? 'light' : 'dark');

}

if (localStorage.getItem('via_theme') === 'light') {

    document.body.classList.add('light');

    document.addEventListener('DOMContentLoaded', () => {

        const t = document.getElementById('themeToggle'); if (t) t.textContent = '??';

    });

}

// ??? MOBILE SIDEBAR ???

function toggleSidebar() {

    document.getElementById('sidebar').classList.toggle('open');

    document.getElementById('sidebarOverlay').classList.toggle('show');

}

// ??? NAVIGATION ???

function showView(v) {

    document.querySelectorAll('.view').forEach(el => el.classList.remove('on'));

    document.querySelectorAll('.sb-item').forEach(el => el.classList.remove('on'));

    const view = document.getElementById('v-' + v);

    if (view) view.classList.add('on');

    const btn = document.querySelector(`[data-view="${v}"]`);

    if (btn) btn.classList.add('on');

```

```

    if (v === 'dashboard') loadDashboard();

    if (v === 'memory') loadMemory();

    if (v === 'history') loadHistory();

    if (v === 'projects') loadProjects();

    if (v === 'templates') loadTemplates();

    if (v === 'meetings') loadMeetings();

    // Close mobile sidebar

    if (window.innerWidth <= 768) { document.getElementById('sidebar').classList.remove('open');
document.getElementById('sidebarOverlay').classList.remove('show'); }

}

// ??? CHAT ???

function formatMd(text) {

    if (!text) return '';

    let h = text

        .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;')

        .replace(/```(\w*)\n([\s\S]*)```/g,      (_,      lang,      code)      =>
`<pre><code>${code.trim()}</code></pre>`)

        .replace(/`([^`]+)`/g, '<code>$1</code>')

        .replace(/\*\*(.?)\*\*/g, '<strong>$1</strong>')

        .replace(/### (.+)/g, '<h3>$1</h3>')

        .replace(/\\n- /g, '\\n? ')

        .replace(/(https?:\\/\\/[^\\s<]+)/g, '<a href="$1" target="_blank">$1</a>')

        .replace(/\\n/g, '<br>');

    return h;

}

function addMsg(role, text, intent) {

    const box = document.getElementById('chatMsgs');

    const div = document.createElement('div');

    div.className = 'msg ' + (role === 'user' ? 'msg-user' : 'msg-via');

    const av = role === 'user' ? USERNAME.charAt(0).toUpperCase() : '?';

    const avCls = role === 'user' ? '' : '';

    let body = formatMd(text);

    // If build mode with URLs, wrap in deploy card

```



```

    if (intent === 'build' && body.includes('https://')) {

        body = body.replace(/((?[\s\S]*?)(<br><br>|$)/, '<div class="deploy-card">$1</div>');

    }

    div.innerHTML = `<div class="msg-av">${av}</div><div><div class="msg-body">${body}</div><div
class="msg-time">${new Date().toLocaleTimeString()}</div></div>`;

    box.appendChild(div);

    box.scrollTop = box.scrollHeight;

}

function showTyping() {

    const box = document.getElementById('chatMsgs');

    const div = document.createElement('div');

    div.className = 'msg msg-via';

    div.id = 'typingIndicator';

    div.innerHTML = `<div class="msg-av">?</div><div><div class="msg-body"><div
class="typing"><span></span><span></span><span></span></div></div></div>`;

    box.appendChild(div);

    box.scrollTop = box.scrollHeight;

}

function hideTyping() {

    const el = document.getElementById('typingIndicator');

    if (el) el.remove();

}

async function sendChat() {

    const input = document.getElementById('chatInput');

    const msg = input.value.trim();

    if (!msg || chatBusy) return;

    input.value = '';

    input.style.height = 'auto';

    chatBusy = true;

    document.getElementById('sendBtn').disabled = true;

    addMsg('user', msg);

    showTyping();

```

```

try {
  const r = await fetch(API + '/chat/', {
    method: 'POST', headers: authHeaders(),
    body: JSON.stringify({ message: msg })
  });
  hideTyping();
  if (r.status === 401) { doLogout(); return; }
  const d = await r.json();
  if (!r.ok) throw new Error(d.detail || 'Request failed');
  addMsg('assistant', d.response, d.intent);
} catch (e) {
  hideTyping();
  addMsg('assistant', '? Error: ' + e.message);
}
chatBusy = false;
document.getElementById('sendBtn').disabled = false;
input.focus();
}

async function loadChatHistory() {
  try {
    const r = await fetch(API + '/chat/history/', { headers: authHeaders() });
    if (!r.ok) return;
    const d = await r.json();
    if (d.history && d.history.length > 0) {
      // Clear welcome message if there's history
      const box = document.getElementById('chatMsgs');
      // Keep the welcome message, add history after
      d.history.forEach(m => addMsg(m.role, m.message, m.intent));
    }
  } catch (e) { console.log('No chat history'); }
}

```

```

function newChat() {

  const box = document.getElementById('chatMsgs');

  box.innerHTML = '';

  addMsg('assistant', '? **New conversation started!**\n\nHow can I help you? Ask me anything,
request a build, or ask for analysis. ?');

  fetch(API + '/chat/history/', { method: 'DELETE', headers: authHeaders() }).catch(() => {});

}

// Auto-resize textarea

document.addEventListener('DOMContentLoaded', () => {

  const ta = document.getElementById('chatInput');

  if (ta) {

    ta.addEventListener('input', () => { ta.style.height = 'auto'; ta.style.height =
Math.min(ta.scrollHeight, 120) + 'px'; });

    ta.addEventListener('keydown', (e) => { if (e.key === 'Enter' && !e.shiftKey) {
e.preventDefault(); sendChat(); } });

  }

});

// ??? BUILD ???

async function doBuild() {

  const task = document.getElementById('buildTask').value.trim();

  if (!task) return;

  const btn = document.getElementById('buildBtn');

  const res = document.getElementById('buildResult');

  btn.disabled = true;

  res.innerHTML = '<div style="color:var(--y);font-family:var(--fm);font-size:12px">? Building...
this may take 1-2 minutes...</div>';

  try {

    const r = await fetch(API + '/deploy/', {

      method: 'POST', headers: authHeaders(),

      body: JSON.stringify({ task, push_to_github: true, deploy_to_render: true })

    });

    const d = await r.json();

    if (!r.ok) throw new Error(d.detail || 'Build failed');

    let html = '<div class="deploy-card">';

```

```

html += '<strong style="color:var(--g)">? Build Complete!</strong><br><br>';

const frontendUrl = d.live_urls?.frontend;
const backendUrl = d.live_urls?.backend;
const apiDocsUrl = d.live_urls?.api_docs;
const repoUrl = d.github?.repo_url;
const repoName = d.github?.repo_name;

if (frontendUrl) {
    html += `?? <strong>Frontend (GitHub Pages):</strong> <a href="${frontendUrl}"
target="_blank">${frontendUrl}</a><br>`;

    html += `<span style="font-size:11px;color:var(--y)">? GitHub Pages takes 1-3 minutes to go
live. If you see a 404, wait and refresh.</span><br><br>`;
}

if (backendUrl) {
    html += `? <strong>Backend (Render):</strong> <a href="${backendUrl}"
target="_blank">${backendUrl}</a><br>`;
}

if (apiDocsUrl) {
    html += `? <strong>API Docs:</strong> <a href="${apiDocsUrl}"
target="_blank">${apiDocsUrl}</a><br>`;
}

if (repoUrl) {
    html += `<br>? <strong>GitHub Repo:</strong> <a href="${repoUrl}"
target="_blank">${repoUrl}</a><br>`;
}

if (!frontendUrl && !backendUrl && !repoUrl) {
    html += `<span style="color:var(--y)">?? Links not yet available ? GitHub repo may still be
initializing. Check your <a href="https://github.com/${d.github?.repo_name} | ''"
target="_blank">GitHub</a> in ~1 minute.</span><br>`;
}

html += `<br><span style="color:var(--tx3);font-size:11px">Duration:
${d.execution_summary?.total_duration_seconds || '?'}s | App Type: ${d.app_type || '?'}</span>`;

html += '</div>';

res.innerHTML = html;

} catch (e) { res.innerHTML = `<div style="color:var(--m2)">? ${e.message}</div>`; }

btn.disabled = false;
}

```

```
// ??? DASHBOARD ???

async function loadDashboard() {
  try {
    const r = await fetch(API + '/system-health/', { headers: authHeaders() });
    if (!r.ok) return;
    const d = await r.json();
    const m = d.metrics || {};
    document.getElementById('kTotalRuns').textContent = m.total_runs || 0;
    document.getElementById('kSuccRate').textContent = (100 - (m.failure_rate_percent || 0)).toFixed(0) + '%';
    document.getElementById('kAvgDur').textContent = (m.avg_duration_seconds || 0).toFixed(1) + 's';
    document.getElementById('healthBody').innerHTML = `
      <div style="display:grid;grid-template-columns:1fr 1fr;gap:12px;font-family:var(--fm);font-size:12px;color:var(--tx2)">
        <div>Total Agents Executed: <strong style="color:var(--c)">${m.total_agents_executed || 0}</strong></div>
        <div>Successful: <strong style="color:var(--g)">${m.total_successful || 0}</strong></div>
        <div>Failed: <strong style="color:var(--m2)">${m.total_failed || 0}</strong></div>
        <div>Max Duration: <strong style="color:var(--y)">${(m.max_duration_seconds || 0).toFixed(1)}s</strong></div>
      </div>`;
  } catch (e) { document.getElementById('healthBody').innerHTML = '<div class="empty">Failed to load metrics</div>'; }
}

// ??? MEMORY ???

async function loadMemory() {
  try {
    const r = await fetch(API + '/agent-memory/', { headers: authHeaders() });
    if (!r.ok) return;
    const d = await r.json();
    const el = document.getElementById('memBody');
    if (!d.memories || d.memories.length === 0) { el.innerHTML = '<div class="empty">No memories yet</div>'; return; }
    el.innerHTML = d.memories.map(m => `
      <div class="mem-card">

```

```

<div class="mem-agent">${m.agent}</div>

<div class="mem-task">${(m.task||'').substring(0,80)}</div>

<div
style="font-size:12px;color:var(--tx3);margin-top:3px">${(m.output_summary||'').substring(0,120)}<
/div>

<div class="mem-conf">Confidence:    ${ (m.confidence||0).toFixed(2)}    ?
${m.timestamp||''}</div>

</div>`).join('');

    } catch (e) { document.getElementById('memBody').innerHTML = '<div class="empty">Failed to
load</div>'; }

}

// ??? HISTORY ???

async function loadHistory() {
  try {
    const r = await fetch(API + '/company-history/', { headers: authHeaders() });
    if (!r.ok) return;
    const d = await r.json();
    const el = document.getElementById('histBody');
    const h = d.recent_history || [];
    if (h.length === 0) { el.innerHTML = '<div class="empty">No history yet</div>'; return; }
    el.innerHTML = h.map(item => `
      <div class="mem-card">
        <div class="mem-agent">? ${ (item.task||'').substring(0,60)}</div>
        <div style="font-size:11px;color:var(--tx3);margin-top:3px">${item.timestamp||''}</div>
      </div>`).join('');
  } catch (e) { document.getElementById('histBody').innerHTML = '<div class="empty">Failed to
load</div>'; }
}

// ??? PROJECTS ???

async function loadProjects() {
  try {
    const r = await fetch(API + '/files/projects/', { headers: authHeaders() });
    if (!r.ok) return;
    const d = await r.json();
    const el = document.getElementById('projBody');

```

```

const p = d.projects || [];

if (p.length === 0) { el.innerHTML = '<div class="empty">No projects yet ? build something first!</div>'; return; }

el.innerHTML = p.map(proj => `
  <div class="mem-card" style="cursor:pointer" onclick="viewProject('${proj.name} || proj}')">
    <div style="display:flex;justify-content:space-between;align-items:center">
      <div class="mem-agent">? ${proj.name} || proj</div>
      <a href="${API}/files/projects/${encodeURIComponent(proj.name || proj)}/download/"
class="btn-new" onclick="event.stopPropagation()" style="font-size:8px;padding:4px 10px">? ZIP</a>
    </div>
    <div class="mem-task">${proj.file_count ? proj.file_count + ' files' : ''} ${proj.created
|| ''}</div>
  </div>`).join('');

} catch (e) { document.getElementById('projBody').innerHTML = '<div class="empty">Failed to load projects</div>'; }

}

// ??? TEMPLATES ???

const TEMPLATES = [

  {icon: '?',name:'Todo App',desc:'Task manager with CRUD operations',task:'Build me a todo app with categories and status tracking'},

  {icon: '?',name:'IQ Test',desc:'Intelligence quiz with scoring',task:'Build me an IQ test app with 10 questions and score results'},

  {icon: '?',name:'Expense Tracker',desc:'Track spending by category',task:'Build me an expense tracker with categories and charts'},

  {icon: '?',name:'Game Buddy',desc:'Gaming session tracker',task:'Build me a game buddy tracker with scores and leaderboard'},

  {icon: '?',name:'Hospital App',desc:'Appointment booking system',task:'Build me a hospital appointment booking system with departments'},

  {icon: '?',name:'Blog Platform',desc:'CMS with posts and tags',task:'Build me a blog platform with articles and categories'},

  {icon: '?',name:'Inventory',desc:'Stock management system',task:'Build me an inventory management system with product tracking'},

  {icon: '?',name:'Event Booking',desc:'Reservation system',task:'Build me an event booking and reservation system'},

];

function loadTemplates() {

  const grid = document.getElementById('tplGrid');

  grid.innerHTML = TEMPLATES.map(t => `
    <div class="tpl-card" onclick="useTemplate('${t.task.replace(/'/g, "\\'")}')">

```

```

    <div class="tpl-icon">${t.icon}</div>

    <div class="tpl-name">${t.name}</div>

    <div class="tpl-desc">${t.desc}</div>

</div>`).join('');
}

function useTemplate(task) {

    showView('chat');

    document.getElementById('chatInput').value = task;

    sendChat();

}

// ??? RECENT BUILDS ???

async function loadRecentBuilds() {

    try {

        const r = await fetch(API + '/company-history/', { headers: authHeaders() });

        if (!r.ok) return;

        const d = await r.json();

        const el = document.getElementById('recentBuildsList');

        const h = (d.recent_history || []).slice(0, 5);

        if (h.length === 0) { el.innerHTML = '<div style="padding:4px 10px;font-size:11px;color:var(--tx3)">No builds yet</div>'; return; }

        el.innerHTML = h.map(item => {

            const name = (item.task || '').substring(0, 28);

            const time = item.timestamp ? new Date(item.timestamp).toLocaleDateString() : '';

            return `<div class="recent-build" onclick="showView('chat');addMsg('assistant','Previous build: ${name.replace(/'/g, '')}">

                <span class="rb-icon">?</span><span class="rb-name">${name}</span><span class="rb-time">${time}</span>

            </div>`;

        }).join('');

    } catch (e) { /* silent */ }

}

// ??? MEETINGS ???

const
AGENT_COLORS
= {ceo:'var(--c)',backend:'var(--v2)',frontend:'var(--m2)',security:'var(--g)',devops:'var(--o)',ai_research:'var(--y)',architecture:'var(--c2)',hr:'var(--m)',finance:'var(--g2)',marketing:'var(--v)'}

```



```

    '});

const AGENT_EMOJI =
{ceo: '??', backend: '??', frontend: '?', security: '?', devops: '?', ai_research: '?', architecture: '?', hr: '?', finance: '?', marketing: '?'};

async function generateMeeting() {
    const task = document.getElementById('meetingTask').value.trim();

    if (!task) return;

    const btn = document.getElementById('meetingBtn');

    const el = document.getElementById('meetingTranscript');

    btn.disabled = true;

    el.innerHTML = '<div style="color:var(--y);font-size:12px;font-family:var(--fm)">?? Generating meeting transcript...</div>';

    try {
        const r = await fetch(API + '/meetings/generate/', {
            method: 'POST', headers: authHeaders(),
            body: JSON.stringify({ task, departments: ['ceo','backend','frontend','security','devops']
        })

    });

    const d = await r.json();

    if (!r.ok) throw new Error(d.detail || 'Meeting failed');

    el.innerHTML = (d.messages || []).map(m => {

        const color = AGENT_COLORS[m.agent] || 'var(--tx2)';

        const emoji = AGENT_EMOJI[m.agent] || '?';

        return `<div class="meeting-msg">

            <div class="meeting-av" style="border-color:${color}">${emoji}</div>

            <div><div class="meeting-name" style="color:${color}">${m.agent_name || m.agent}</div>

            <div class="meeting-text">${formatMd(m.message || m.content || '')}</div></div>

        </div>`;

    }).join('');

    showToast(`Meeting generated ? ${d.message_count || 0} messages`, 'success');

} catch (e) { el.innerHTML = `<div style="color:var(--m2)">? ${e.message}</div>`; }

    btn.disabled = false;
}

async function loadMeetings() {

```

```

try {
  const r = await fetch(API + '/meetings/', { headers: authHeaders() });
  if (!r.ok) return;
  const d = await r.json();
  const el = document.getElementById('meetingsBody');
  const m = d.meetings || [];

  if (m.length === 0) { el.innerHTML = '<div class="empty">No meetings yet ? start one above!</div>'; return; }

  el.innerHTML = m.map(item => `
    <div class="mem-card" style="cursor:pointer" onclick="viewMeeting('${item.job_id || item.meeting_id}')">
      <div class="mem-agent">?? ${item.task || ''}.substring(0,60)}</div>
      <div class="mem-task">${item.message_count || '?'} messages</div>
      <div style="font-size:10px;color:var(--tx3);margin-top:3px">${item.created_at || ''}</div>
    </div>`).join('');

  } catch (e) { document.getElementById('meetingsBody').innerHTML = '<div class="empty">Failed to load</div>'; }
}

```

```

async function viewMeeting(id) {
  try {
    const r = await fetch(API + '/meetings/' + id + '/', { headers: authHeaders() });
    if (!r.ok) return;
    const d = await r.json();
    const el = document.getElementById('meetingTranscript');
    const msgs = d.messages || (d.transcript ? JSON.parse(d.transcript) : []);
    el.innerHTML = msgs.map(m => {
      const color = AGENT_COLORS[m.agent] || 'var(--tx2)';
      const emoji = AGENT_EMOJI[m.agent] || '?';
      return `<div class="meeting-msg">
        <div class="meeting-av" style="border-color:${color}">${emoji}</div>
        <div><div class="meeting-name" style="color:${color}">${m.agent_name || m.agent}</div>
        <div class="meeting-text">${formatMd(m.message || m.content || '')}</div></div>
      </div>`;
    }).join('');
  } catch (e) { /* silent */ }
}

```

```

}

// ??? ENHANCED PROJECTS ???

async function viewProject(name) {
  try {
    const r = await fetch(API + '/files/projects/' + encodeURIComponent(name) + '/tree/', {
      headers: authHeaders() });

    if (!r.ok) return;

    const d = await r.json();

    const el = document.getElementById('projBody');

    const tree = d.tree || {};

    let html = '<div style="display:grid;grid-template-columns:260px 1fr;gap:16px;min-height:300px">';

    html += '<div class="file-tree" style="border-right:1px solid var(--b1);padding-right:12px">';

    html += '<div style="margin-bottom:8px"><button class="btn-new" onclick="loadProjects()">? Back</button></div>';

    html += renderTree(tree.children || [], name);

    html += '</div>';

    html += '<div id="codeDisplay"><div class="empty">Select a file to view</div></div>';

    html += '</div>';

    el.innerHTML = html;
  } catch (e) { /* silent */ }
}

function renderTree(children, project, indent = 0) {
  return children.map(c => {
    const pad = indent * 12;

    if (c.type === 'directory') {
      return `<div class="file-item" style="padding-left:${pad+10}px;color:var(--y)"><span class="fi-icon">?</span><span class="fi-name">${c.name}</span></div>` + renderTree(c.children || [], project, indent + 1);
    }

    const sz = c.size_bytes > 1024 ? (c.size_bytes/1024).toFixed(1)+'K' : c.size_bytes+'B';

    return `<div class="file-item" style="padding-left:${pad+10}px" onclick="viewFile('${project}','${c.path}')"><span class="fi-icon">?</span><span class="fi-name">${c.name}</span><span class="fi-size">${sz}</span></div>`;
  }).join('');
}

```

```

async function viewFile(project, path) {
  try {
    const r = await fetch(API + '/files/projects/' + encodeURIComponent(project) +
'/read/?file_path=' + encodeURIComponent(path), { headers: authHeaders() });

    if (!r.ok) return;

    const d = await r.json();

    document.querySelectorAll('.file-item').forEach(f => f.classList.remove('active'));

    event.target.closest('.file-item')?.classList.add('active');

    document.getElementById('codeDisplay').innerHTML = `

```

```
document.body.insertBefore(canvas, document.body.firstChild);

const ctx = canvas.getContext('2d');

let particles = [];

const COLORS = ['#00d4ff', '#7c3aed', '#00ff9d', '#ff006e', '#ffd700'];

function resize() { canvas.width = window.innerWidth; canvas.height = window.innerHeight; }
window.addEventListener('resize', resize);

resize();

class Particle {
  constructor() { this.reset(); }

  reset() {
    this.x = Math.random() * canvas.width;
    this.y = Math.random() * canvas.height;
    this.size = Math.random() * 2 + 0.5;
    this.speedX = (Math.random() - 0.5) * 0.3;
    this.speedY = (Math.random() - 0.5) * 0.3;
    this.color = COLORS[Math.floor(Math.random() * COLORS.length)];
    this.opacity = Math.random() * 0.5 + 0.1;
    this.pulse = Math.random() * Math.PI * 2;
  }

  update() {
    this.x += this.speedX;
    this.y += this.speedY;
    this.pulse += 0.02;
    this.opacity = 0.1 + Math.sin(this.pulse) * 0.2;

    if (this.x < 0 || this.x > canvas.width || this.y < 0 || this.y > canvas.height)
      this.reset();
  }

  draw() {
    ctx.beginPath();

    ctx.arc(this.x, this.y, this.size, 0, Math.PI * 2);

    ctx.fillStyle = this.color;
    ctx.globalAlpha = this.opacity;
    ctx.fill();
  }
}
```

```
    ctx.globalAlpha = 1;
  }
}
```

```
for (let i = 0; i < 60; i++) particles.push(new Particle());
```

```
function drawConnections() {
  for (let i = 0; i < particles.length; i++) {
    for (let j = i + 1; j < particles.length; j++) {
      const dx = particles[i].x - particles[j].x;
      const dy = particles[i].y - particles[j].y;
      const dist = Math.sqrt(dx * dx + dy * dy);
      if (dist < 120) {
        ctx.beginPath();
        ctx.moveTo(particles[i].x, particles[i].y);
        ctx.lineTo(particles[j].x, particles[j].y);
        ctx.strokeStyle = particles[i].color;
        ctx.globalAlpha = (1 - dist / 120) * 0.08;
        ctx.lineWidth = 0.5;
        ctx.stroke();
        ctx.globalAlpha = 1;
      }
    }
  }
}
```

```
function animate() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  particles.forEach(p => { p.update(); p.draw(); });
  drawConnections();
  requestAnimationFrame(animate);
}

animate();
}
```

```
// ??? INIT ???

if (TOKEN) { enterApp(); }

document.addEventListener('DOMContentLoaded', () => {

    if (TOKEN) loadRecentBuilds();

    // Particles disabled for a cleaner, professional look

});

</script>

</body>

</html>

...


```

```
## File: `LICENSE`


```

```
...
```

MIT License

Copyright (c) 2026 VIA ? Virtual Intelligence Agents

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,

OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

```

## File: `package.json`

```
```json
{
  "name": "via-autonomous-ai-platform",
  "version": "6.0.0",
  "description": "VIA ? Autonomous AI Digital Enterprise Platform with 10 AI agents",
  "private": true,
  "scripts": {
    "start": "echo 'Use: uvicorn backend.main:app --reload'",
    "test": "pytest tests/ -v"
  },
  "keywords": ["ai", "autonomous", "agents", "fastapi", "enterprise"],
  "author": "VIA Team",
  "license": "MIT"
}
```
```

## File: `README.md`

```
```md
<<<<<<< HEAD

# ? VIA ? Virtual Intelligence Agents

> **Autonomous AI Digital Enterprise Platform** ? A fully autonomous AI-powered MNC that builds,
deploys, and manages software through coordinated multi-agent execution.

[![Phase](https://img.shields.io/badge/Phase-6-00d4ff?style=for-the-badge)](/)
[![Agents](https://img.shields.io/badge/AI_Agents-10-7c3aed?style=for-the-badge)](/)
[![FastAPI](https://img.shields.io/badge/FastAPI-0.104+-00ff9d?style=for-the-badge)](https://fasta
```



```
pi.tiangolo.com)

[![License](https://img.shields.io/badge/License-MIT-ff006e?style=for-the-badge)](/)
```

? What is VIA?

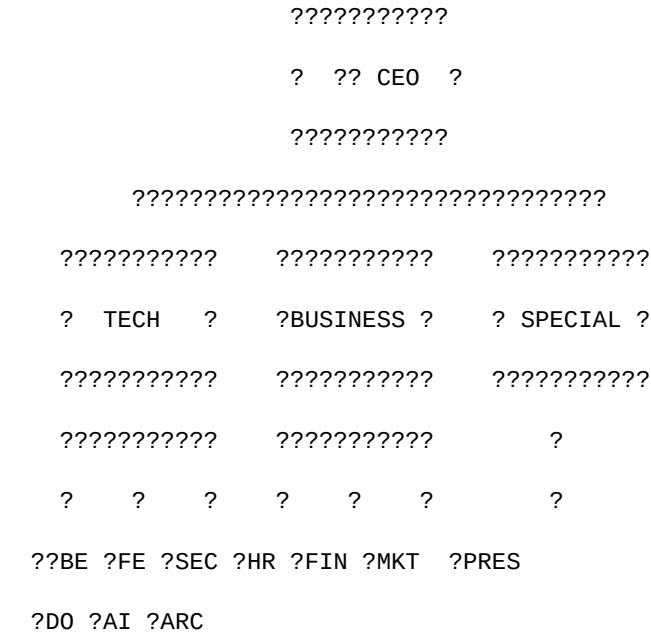
VIA is an **AI-powered autonomous digital enterprise** that operates like a high-performance MNC software company. Give it a task ? it delegates to specialized AI departments, builds the code, pushes to GitHub, deploys to production, and delivers live URLs.

? Three Modes

Mode	Trigger	What Happens
-----	-----	-----
? Chat	Ask any question	Conversational AI assistant
?? Build	"Build me a..."	Full app generation ? GitHub ? Deploy
? Analyze	"Analyze my..."	Multi-department strategic reports

? AI Department Structure

...



...

Agent	Role
-----	-----
?? CEO	Strategy, delegation, conflict resolution
?? Backend	FastAPI APIs, database architecture
? Frontend	React/Vite UI with Tailwind CSS
? Security	Threat modeling, auth, encryption
? DevOps	Docker, CI/CD, deployment
? AI Research	LLM strategy, ML optimization
? Architecture	System design, data flow
? HR	Team structure, hiring plans
? Finance	Budget, ROI, pricing strategy
? Marketing	GTM, branding, SEO

? Quick Start

Option A: Docker (Recommended)

```
```bash
cp .env.example .env
Edit .env with your credentials
docker-compose up --build
```
```

Services:

- **API**: http://localhost:8000
- **Frontend**: http://localhost:8000 (served by FastAPI)
- **pgAdmin**: http://localhost:5050
- **Swagger**: http://localhost:8000/docs

Option B: Local Development

```
```bash

1. Start PostgreSQL and Redis

2. Install Python deps

pip install -r requirements.txt

3. Start Ollama (or set USE_GROQ=true in .env)

ollama serve && ollama pull llama3

4. Start API

uvicorn backend.main:app --reload

5. Start Celery worker (separate terminal)

celery -A backend.tasks.celery_app worker --loglevel=info

6. Open http://localhost:8000 in browser

...

```

## ? API Endpoints

### Public

Method	Endpoint	Description
POST	/auth/register	Register new user
POST	/auth/login	Get JWT token
GET	/health	Health check
GET	/docs	Swagger UI

### Protected (JWT Required)

Method	Endpoint	Description
POST	/chat/	Unified chat/build/analyze

```
| POST | `/start-company/` | Run agent pipeline |
| POST | `/deploy/` | Build + GitHub + Render |
| POST | `/feedback/` | Revision feedback loop |
| GET | `/company-history/` | Execution history |
| GET | `/system-health/` | System metrics |
| GET | `/org-chart/` | Organization hierarchy |
| GET | `/agent-memory/` | Agent memory store |
```

### ### WebSocket

```
| Endpoint | Description |
|-----|-----|
| WS `/ws/{job_id}` | Real-time pipeline streaming |
```

---

### ## ?? Security Features

- JWT authentication with configurable expiration
- bcrypt password hashing
- Rate limiting (configurable per minute)
- CORS middleware
- Input validation via Pydantic
- Path traversal protection in file browser
- SQL injection prevention (parameterized queries)

---

### ## ? Running Tests

```
```bash
```

```
pip install pytest pytest-asyncio
```

```
pytest tests/ -v
```

```
```
```

---

## ? Project Structure

```

```
via/

??? backend/

?   ??? main.py           # FastAPI app + all endpoints
?   ??? agents/           # 10 AI department agents
?   ??? core/             # LLM, config, engines, deployers
?   ??? database/         # PostgreSQL + asyncpg
?   ??? auth/             # JWT authentication
?   ??? middleware/       # Rate limiting
?   ??? routers/          # Meeting, template, file browser APIs
?   ??? tasks/            # Celery async job queue
??? tests/                # Test suite
??? projects/             # Generated project outputs
??? index.html            # Frontend dashboard
??? via-chat.css          # Stylesheet
??? docker-compose.yml    # Full stack orchestration
??? Dockerfile            # API container
??? requirements.txt       # Python dependencies
```

```

---

## ? Environment Variables

See ``.env.example`` for all required variables. Key ones:

| Variable                  | Description                                           |
|---------------------------|-------------------------------------------------------|
| -----                     | -----                                                 |
| <code>`MODEL_NAME`</code> | Ollama model (default: <code>llama3.2:latest</code> ) |
| <code>`USE_GROQ`</code>   | Set <code>`true`</code> to use Groq cloud LLM         |

| `GROQ\_API\_KEY` | Groq API key (if using Groq) |  
| `POSTGRES\_\*` | PostgreSQL connection settings |  
| `JWT\_SECRET\_KEY` | JWT signing secret |  
| `GITHUB\_TOKEN` | GitHub PAT for auto-push |  
| `RENDER\_API\_KEY` | Render API key for auto-deploy |

---

## ? Phase History

| Phase | Features                                         |
|-------|--------------------------------------------------|
| 1     | PostgreSQL, JWT auth, rate limiting, Docker      |
| 2     | WebSocket streaming, inter-agent bus, Celery     |
| 3     | Business agents (HR/Finance/Marketing), meetings |
| 4     | Presentation generation, templates               |
| 5     | Chat mode, intent detection, fullstack builder   |
| **6** | **Premium UI, testing, docs, CI/CD, hardening**  |

---

\*Built with ?? by VIA ? Autonomous AI Digital Team\*

=====

# via

>>>>>> 2600c44eb1ab270bb4abe61696ef73b485a8dc0f

...

## File: `README.txt`

```txt

=====

AI DIGITAL TEAM ? Phase 2: Full MNC Engine
Version 4.0.0

=====

PHASE 2 NEW FEATURES

- [x] WebSocket real-time streaming ? every pipeline step broadcast live
- [x] Inter-Agent Communication Bus ? departments share context with each other
- [x] Async Task Queue ? Celery + Redis background job processing
- [x] Live agent progress cards in frontend
- [x] New endpoints: WS /ws/{job_id}, POST /jobs/submit/, GET /jobs/{job_id}/
- [x] Execution order awareness ? backend/architecture run first, feed context downstream
- [x] All Phase 1 features preserved (PostgreSQL, JWT, rate limiting, Docker)

=====

OPTION A: Docker (Recommended ? starts everything)

=====

```
cp .env.example .env
```

```
docker-compose up --build
```

Services started:

- API: http://localhost:8000
- Frontend: open frontend/index.html in browser
- pgAdmin: http://localhost:5050
- Redis: localhost:6379
- Celery Worker: runs in background automatically

=====

OPTION B: Local (manual)

=====

1. Start PostgreSQL + create DB

2. Start Redis:

```
redis-server
```

3. Install deps:

```
pip install -r requirements.txt
```

4. Start Ollama:

```
ollama serve
```

```
ollama pull llama3
```

5. Start API:

```
uvicorn backend.main:app --reload
```

6. Start Celery Worker (separate terminal):

```
celery -A backend.tasks.celery_app worker --loglevel=info
```

7. Open frontend/index.html in browser

=====

PHASE 2 API ENDPOINTS

=====

PUBLIC:

| | |
|---------------------|--------------------------|
| POST /auth/register | Register new user |
| POST /auth/login | Get JWT token |
| GET /health | Health check |
| GET / | API info + endpoint list |
| GET /docs | Swagger UI |

WEBSOCKET:

| | |
|-----------------|---------------------------------|
| WS /ws/{job_id} | Real-time pipeline event stream |
|-----------------|---------------------------------|

Events: pipeline_step, ceo_decision, agent_start,
inter_agent_context, agent_done, complete, error

PROTECTED (JWT required):

| | |
|----------------------|-----------------------------------|
| POST /start-company/ | Sync orchestration + WS streaming |
| POST /jobs/submit/ | Submit async background job |

| | | |
|-----|-------------------|--------------------------------|
| GET | /jobs/{job_id}/ | Poll async job status + result |
| GET | /company-history/ | Execution history |
| GET | /system-health/ | Metrics |
| GET | /company-status/ | Dashboard |
| GET | /org-chart/ | Org hierarchy |

=====

HOW INTER-AGENT COMMUNICATION WORKS

=====

Departments execute in dependency order:

- | | |
|-----------------|---|
| 1. backend | ? deposits: architecture, API style, DB choice |
| 2. architecture | ? deposits: system design pattern, data flow |
| 3. security | ? reads backend context ? tailored threat model |
| 4. devops | ? reads backend + architecture ? aligned infra plan |
| 5. ai_research | ? reads architecture + backend ? coherent AI strategy |

Result: departments produce a COHERENT unified plan,
not just isolated reports.

=====

PROJECT STRUCTURE

=====

via/

???.env

??? .env.example

??? .gitignore

??? Dockerfile

```
??? docker-compose.yml
```

??? requirements.txt

??? README.txt

??? frontend/

? ??? index.html ? Complete dashboard (open in browser)

??? backend/

??? main.py ? FastAPI app + all endpoints

??? core/

? ??? config.py ? Loads from .env

? ??? logger.py

? ??? llm_provider.py ? Ollama with retry

? ??? tracer.py ? Execution trace (Phase 2: inter-agent events)

? ??? hierarchy.py

? ??? scaling_engine.py

? ??? ws_manager.py ? [NEW] WebSocket connection manager

? ??? inter_agent_bus.py ? [NEW] Agent-to-agent context sharing

??? database/

? ??? db.py ? PostgreSQL + async_jobs table

??? auth/

? ??? auth.py

??? middleware/

? ??? rate_limiter.py

??? agents/

? ??? ceo_agent.py

? ??? backend_agent.py ? Phase 2: accepts inter_context param

? ??? security_agent.py ? reads backend context

? ??? devops_agent.py ? reads backend + architecture context

? ??? ai_research_agent.py ? reads architecture + backend context

? ??? architecture_agent.py? reads backend context

? ??? agent_executor.py ? [NEW] ordered execution + WS streaming

??? tasks/

 ??? celery_app.py ? [NEW] Celery config

 ??? orchestration_task.py? [NEW] Background orchestration task

=====

FRONTEND FEATURES

=====

Pages:

- Dashboard - Stats + Quick Launch + Recent Activity
- Run Task - Live stream feed + agent progress cards + results
- Async Jobs - Submit jobs + poll status + view results
- History - All past execution records
- Metrics - Dept activity bars + health charts + full stats
- Company Status - Department breakdown
- Org Chart - CEO ? 5 departments

Phase 2 UI highlights:

- ? Live WebSocket event feed (colored by event type)
- ? Agent progress cards (Waiting ? Running ? Done/Failed)
- ? Inter-agent context events shown in stream
- ? Async job queue with polling
- ? Scaling notifications in result view

=====

PHASE COMPARISON

=====

| Feature | Phase 1 | Phase 2 |
|---------------------------|---------|---------|
| ----- | ----- | ----- |
| PostgreSQL + JWT | ? | ? |
| Rate limiting | ? | ? |
| Docker + Compose | ? | ? |
| WebSocket streaming | ? | ? |
| Inter-agent communication | ? | ? |
| Async job queue (Celery) | ? | ? |
| Redis | ? | ? |
| Celery worker | ? | ? |
| Live agent progress UI | ? | ? |
| Frontend dashboard | ? | ? |

=====

...

File: `requirements.txt`

```txt

```
fastapi>=0.104.0
uvicorn[standard]>=0.24.0
asyncpg>=0.29.0
python-jose[cryptography]>=3.3.0
passlib[bcrypt]>=1.7.4
python-multipart>=0.0.6
python-dotenv>=1.0.0
requests>=2.31.0
pydantic>=2.0.0
websockets>=12.0
celery>=5.3.0
redis>=5.0.0
httpx>=0.27.0
aiosqlite>=0.19.0
python-pptx>=0.6.21
```

...

## File: `SECURITY\_REPORT.md`

```md

VIA Security Report

Authentication & Authorization

- **Method**: JWT (JSON Web Tokens)
- **Hashing**: bcrypt via passlib
- **Token Expiry**: Configurable (default 30 min)
- **Storage**: Token stored client-side in localStorage

Input Validation

- All API inputs validated via Pydantic models
- Field length constraints (min/max)
- SQL injection prevented via parameterized asyncpg queries

Rate Limiting

- Custom middleware: configurable requests per minute
- Applied globally to all endpoints

Network Security

- CORS middleware configured (currently allow_origins=["*"] ? restrict in production)
- HTTPS enforced by deployment platform (Render)

File System Security

- Path traversal protection in file browser router
- Allowed file extension whitelist
- File size limits (500KB max for viewing)

Secrets Management

- All secrets loaded from .env via python-dotenv
- .env excluded from git via .gitignore

Recommendations

1. Restrict CORS origins to specific domains in production
2. Add CSRF protection for cookie-based sessions
3. Implement token refresh mechanism
4. Add request signing for GitHub/Render API calls
5. Enable audit logging for all auth events
6. Add rate limiting per-user, not just global
7. Rotate JWT_SECRET_KEY periodically

...

File: `str`

...

...

File: `tests\conftest.py`

```py

# tests/conftest.py ? Shared fixtures for VIA test suite

import pytest

from unittest.mock import AsyncMock, patch, MagicMock

class MockLLM:

"""Mock LLM provider that returns predictable responses."""

def \_\_init\_\_(self, response=""):

self.\_response = response

def generate(self, prompt: str) -> str:

return self.\_response

async def agenerate(self, prompt: str) -> str:

return self.\_response

@pytest.fixture

def mock\_llm():

"""Returns a MockLLM instance with a default CEO-style JSON response."""

return MockLLM(response='{"short\_term\_strategy": "Build MVP", "long\_term\_vision": "Scale globally", "departments": ["backend", "frontend"]}')

@pytest.fixture

def mock\_llm\_patched(mock\_llm):

"""Patches the global llm instance in llm\_provider."""

```

 with patch("backend.core.llm_provider.llm", mock_llm):
 yield mock_llm

@pytest.fixture
def sample_task():
 return "Build a hospital appointment booking system with departments and doctor profiles"

@pytest.fixture
def sample_history():
 return [
 {"task": "Build a todo app", "result": {"ceo_strategy": {"short_term_strategy": "CRUD
app"}, "selected_departments": ["backend", "frontend"]}},
]

...

File: `tests/test_agents.py`

```py
# tests/test_agents.py ? Unit tests for agents with mock LLM

import pytest

from unittest.mock import patch, AsyncMock

from tests.conftest import MockLLM


@pytest.mark.asyncio
async def test_ceo_agent_parses_json():
    mock = MockLLM('{"short_term_strategy": "Build MVP fast", "long_term_vision": "Scale to 1M
users", "departments": ["backend", "frontend", "security"]}\'')

    with patch("backend.agents.ceo_agent.llm", mock):
        from backend.agents.ceo_agent import ceo_agent

        result = await ceo_agent("Build a todo app")

        assert "short_term_strategy" in result

```

```
assert "departments" in result

assert "backend" in result["departments"]
```

```
@pytest.mark.asyncio
```

```
async def test_ceo_agent_fallback():

    mock = MockLLM("This is not valid JSON at all")

    with patch("backend.agents.ceo_agent.llm", mock):

        from backend.agents.ceo_agent import ceo_agent

        result = await ceo_agent("Build something")

        assert "departments" in result

        assert len(result["departments"]) > 0
```

```
@pytest.mark.asyncio
```

```
async def test_backend_agent_returns_department():

    mock = MockLLM('{"department": "Backend Engineering", "architecture": "FastAPI", "database": {"primary": "PostgreSQL"}, "api_design": {"style": "REST"}, "services": ["UserService"], "key_recommendations": ["Use async"]}')

    with patch("backend.agents.backend_agent.llm", mock):

        from backend.agents.backend_agent import backend_agent

        result = await backend_agent("Build a booking system", "Build fast", "")

        assert result["department"] == "Backend Engineering"
```

```
@pytest.mark.asyncio
```

```
async def test_security_agent_returns_department():

    mock = MockLLM('{"department": "Security", "threat_model": {"top_threats": ["XSS"]}, "authentication": {"strategy": "JWT"}, "encryption": {"in_transit": "TLS"}, "risk_mitigation": ["Rate limit"], "compliance": ["OWASP"]}')

    with patch("backend.agents.security_agent.llm", mock):

        from backend.agents.security_agent import security_agent

        result = await security_agent("Build a banking app", "", "")

        assert result["department"] == "Security Engineering"
```



```

@pytest.mark.asyncio

async def test_hr_agent_returns_report():

    mock = MockLLM("Team Structure:\n- 1 Tech Lead\n- 2 Backend Engineers\nHiring Timeline:\nMonth 1: Core team")

    with patch("backend.agents.hr_agent.llm", mock):

        from backend.agents.hr_agent import hr_agent

        result = await hr_agent("Build a SaaS platform", "", "")

        assert result["department"] == "Human Resources"

        assert result["status"] == "success"

...

## File: `tests/test_fullstack_builder.py`

```py
tests/test_fullstack_builder.py ? Tests for app type detection and file generation

import pytest

from backend.core.fullstack_builder import detect_app_type, generate_backend_files

class TestDetectAppType:

 def test_frontend_only_landing(self):

 assert detect_app_type("Build a landing page for my startup") == "frontend"

 def test_frontend_portfolio(self):

 assert detect_app_type("Create a portfolio showcase website") == "frontend"

 def test_fullstack_with_api(self):

 assert detect_app_type("Build a quiz app with API and scoring") == "fullstack"

 def test_fullstack_game(self):

 assert detect_app_type("Create a game leaderboard tracker") == "fullstack"

 def test_fullstack_db_with_login(self):

 assert detect_app_type("Build an app with user login and data storage") == "fullstack_db"

```

```
def test_fullstack_db_crud(self):
 assert detect_app_type("Create a CRUD application for managing users") == "fullstack_db"
```

```
def test_fullstack_db_postgresql(self):
 assert detect_app_type("Build a system with PostgreSQL database") == "fullstack_db"
```

```
class TestGenerateBackendFiles:
```

```
 def test_frontend_returns_empty(self):
 files = generate_backend_files("landing page", "frontend")
 assert files == {}
```

```
 def test_fullstack_has_main_py(self):
 files = generate_backend_files("Build a quiz app", "fullstack")
 assert "main.py" in files
 assert "requirements.txt" in files
 assert "render.yaml" in files
```

```
 def test_fullstack_db_has_models(self):
 files = generate_backend_files("Build a user management system", "fullstack_db")
 assert "database.py" in files
 assert "models.py" in files
 assert "main.py" in files
```

```
 def test_main_py_has_fastapi(self):
 files = generate_backend_files("Build a todo app", "fullstack")
 assert "FastAPI" in files["main.py"]
 assert "uvicorn" in files["main.py"]
```

```
 def test_requirements_has_fastapi(self):
 files = generate_backend_files("Build a blog", "fullstack")
 assert "fastapi" in files["requirements.txt"]
```

```

def test_gitignore_generated(self):
 files = generate_backend_files("Build an app", "fullstack")
 assert ".gitignore" in files
 assert "__pycache__" in files[".gitignore"]

...

File: `tests\test_intent_detector.py`

```py
# tests/test_intent_detector.py ? Tests for 3-mode intent detection
import pytest

from backend.core.intent_detector import detect_intent

class TestBuildIntent:

    def test_build_me_phrase(self):
        assert detect_intent("Build me a todo app") == "build"

    def test_create_app(self):
        assert detect_intent("Create a hospital booking system") == "build"

    def test_deploy_phrase(self):
        assert detect_intent("Deploy a REST API for my blog") == "build"

    def test_make_application(self):
        assert detect_intent("Make a simple mobile gaming application") == "build"

    def test_generate_app(self):
        assert detect_intent("Generate a dashboard for inventory management") == "build"

class TestAnalyzeIntent:

    def test_analyze_phrase(self):

```

```
assert detect_intent("Analyze my business plan for a SaaS product") == "analyze"
```

```
def test_security_audit(self):
```

```
    assert detect_intent("Perform a security audit of my application") == "analyze"
```

```
def test_create_plan(self):
```

```
    assert detect_intent("Give me a plan for scaling our infrastructure") == "analyze"
```

```
def test_recommend_stack(self):
```

```
    assert detect_intent("What tech stack do you recommend for this?") == "analyze"
```

```
class TestChatIntent:
```

```
    def test_greeting(self):
```

```
        assert detect_intent("Hello") == "chat"
```

```
    def test_short_message(self):
```

```
        assert detect_intent("Hi VIA") == "chat"
```

```
    def test_question(self):
```

```
        assert detect_intent("What is Python used for?") == "chat"
```

```
    def test_thanks(self):
```

```
        assert detect_intent("Thanks!") == "chat"
```

```
class TestEdgeCases:
```

```
    def test_empty_like(self):
```

```
        assert detect_intent("ok cool") == "chat"
```

```
    def test_mixed_signals_build_wins(self):
```

```
        result = detect_intent("Build a project management tool with API backend and frontend dashboard")
```

```
        assert result == "build"
```

```

## File: `tests\\_\_init\_\_.py`

```py

VIA Test Suite

```

## File: `via-chat.css`

```css

/* VIA ? Autonomous AI Digital Team | Phase 5 Stylesheet */

/* Dark-first design with optional light mode (.light on <body>) */

```
:root{
  --void:#0a0a0a;--deep:#111111;--base:#171717;--surf:#1f1f1f;--lift:#262626;--elev:#333333;
  --c:#3b82f6;--c2:#2563eb;--v:#8b5cf6;--v2:#7c3aed;--m:#ec4899;--m2:#db2777;
  --g:#10b981;--g2:#059669;--o:#f59e0b;--y:#eab308;
  --b1:rgba(255,255,255,.06);--b2:rgba(255,255,255,.12);--b3:rgba(255,255,255,.2);
  --tx:#f4f4f5;--tx2:#a1a1aa;--tx3:#71717a;

  --fd:system-ui,-apple-system,sans-serif;--fb:system-ui,-apple-system,sans-serif;--fm:ui-monospace,
  monospace
}

/* ??? LIGHT MODE ??? */

body.light{
  --void:#f4f6f9;--deep:#ebeeef;--base:#f8f9fc;--surf:#ffffff;--lift:#f0f2f7;--elev:#e8ebf0;
  --b1:rgba(0,100,200,.08);--b2:rgba(0,100,200,.15);--b3:rgba(0,100,200,.25);
  --tx:#1a202c;--tx2:#4a5568;--tx3:#a0aec0;
  --c:#0078d4;--c2:#005a9e;--v:#6d28d9;--v2:#7c3aed;
  --g:#059669;--g2:#047857;--m:#db2777;--m2:#ec4899;
  --y:#d97706;--o:#ea580c
}
```

```
body.light .holo-card{box-shadow:0 8px 32px rgba(0,0,0,.08);border-color:var(--b2)}

body.light .msg-via .msg-body{background:var(--surf);border-color:var(--b2);color:var(--tx)}

body.light .msg-body{background:rgba(109,40,217,.06);border-color:rgba(109,40,217,.15);color:var(--tx)}

body.light .sidebar{background:var(--surf);border-right-color:var(--b2)}

body.light .chat-header,body.light .chat-input-wrap{background:var(--surf)}

body.light .chat-input-box{background:var(--lift);border-color:var(--b2)}

body.light .panel{background:var(--surf);border-color:var(--b2)}

body.light .panel-head{border-bottom-color:var(--b2)}

body.light .kpi{background:var(--surf);border-color:var(--b2)}

body.light .kpi-label{color:var(--tx2)}

body.light .mem-card{background:var(--lift);border-color:var(--b2)}

body.light .mem-agent{color:var(--c)}

body.light .mem-task{color:var(--tx)}

body.light .sb-item{color:var(--tx2)}

body.light .sb-item:hover{color:var(--tx);background:rgba(0,120,212,.04)}

body.light .sb-item.on{background:rgba(0,120,212,.08);color:var(--c);border-color:rgba(0,120,212,.2)}

body.light .sb-sec{color:var(--tx2)}

body.light .sb-logo{border-color:var(--b2)}

body.light .deploy-card{background:rgba(5,150,105,.04);border-color:rgba(5,150,105,.18)}

body.light .deploy-card a{color:var(--c)}

body.light .cyber-btn{background:linear-gradient(135deg,var(--c),var(--c2));color:#ffffff;border:none}

body.light .cyber-btn:hover{background:linear-gradient(135deg,var(--c2),var(--c))}

body.light .fld-input{background:var(--lift);border-color:var(--b2);color:var(--tx)}

body.light .fld-input::placeholder{color:var(--tx3)}

body.light .fld-label{color:var(--tx)}

body.light .msg-body pre{background:var(--lift);border-color:var(--b2);color:var(--tx)}

body.light .msg-body code{background:rgba(0,120,212,.06);color:var(--tx)}

body.light .msg-body strong{color:var(--c)}

body.light .msg-body a{color:var(--c)}

body.light #chatInput{color:var(--tx)}

body.light .recent-build{background:var(--lift);border-color:var(--b2)}

body.light .recent-build .rb-name{color:var(--tx)}

body.light .agent-prog{background:var(--lift);border-color:var(--b2)}
```

```

body.light .agent-prog-item{color:var(--tx)}

body.light .pg-title{color:var(--tx)}

body.light .pg-sub{color:var(--tx2)}

body.light .panel-title{color:var(--c)}

body.light .btn-new{color:var(--c);border-color:var(--b2);background:rgba(0,120,212,.06)}

body.light .tpl-card{background:var(--lift);border-color:var(--b2)}

body.light .tpl-name{color:var(--c)}

body.light .tpl-desc{color:var(--tx2)}

body.light .u-nm{color:var(--tx)}

body.light .meeting-msg{background:var(--lift);border-color:var(--b2)}

body.light .meeting-name{color:var(--c)}

body.light .meeting-text{color:var(--tx)}

body.light .meeting-controls select,body.light .meeting-controls
input{background:var(--lift);border-color:var(--b2);color:var(--tx)}

body.light .file-item{color:var(--tx2)}

body.light .file-item:hover,body.light .file-item.active{color:var(--c)}

body.light .theme-toggle{border-color:var(--b2);color:var(--tx2)}

body.light .lo-btn{color:var(--tx2)}


*,*::before,*::after{margin:0;padding:0;box-sizing:border-box}

html{scroll-behavior:smooth}

body{background:var(--void);color:var(--tx);font-family:var(--fb);min-height:100vh;overflow-x:hidde
n;transition:background .3s,color .3s}


@keyframes spin{to{transform:rotate(360deg)}}

@keyframes vIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:none}}

@keyframes pulse{0%,100%{opacity:.6}50%{opacity:1}}

@keyframes glowPulse{0%,100%{box-shadow:0 0 15px rgba(0,212,255,.15)}50%{box-shadow:0 0 35px
rgba(0,212,255,.35)}}

@keyframes msgIn{from{opacity:0;transform:translateY(12px)
scale(.97)}to{opacity:1;transform:none}}

@keyframes dotBounce{0%,80%,100%{transform:scale(0)}40%{transform:scale(1)}}

@keyframes progPulse{0%,100%{opacity:.7}50%{opacity:1}}


/* ??? AUTH PAGE ??? */

#authPage{min-height:100vh;display:flex;align-items:center;justify-content:center;position:relativ
e;z-index:10;background:#0a0a1a center/cover

```

```

no-repeat;background-size:cover;background-position:center}

#authPage::before{content:'';position:absolute;inset:0;background:linear-gradient(180deg,rgba(10,5,30,.55) 0%,rgba(10,5,30,.35) 40%,rgba(10,5,30,.65) 100%);z-index:0}

.auth-wrap{width:100%;max-width:420px;padding:20px;position:relative;z-index:1}

.auth-brand{text-align:center;margin-bottom:28px;display:flex;flex-direction:column;align-items:center}

.auth-logo{width:72px;height:72px;margin-bottom:12px;filter:drop-shadow(0 4px 20px rgba(0,212,255,.35));animation:float 4s ease-in-out infinite}

@keyframes float{0%,100%{transform:translateY(0)}50%{transform:translateY(-8px)}}

.via-display{display:none;}

.auth-sub{display:none;}

.holo-card{background:rgba(5,5,15,.25);backdrop-filter:blur(16px);-webkit-backdrop-filter:blur(16px);border:1px solid rgba(255,255,255,.15);border-radius:18px;padding:26px;position:relative;overflow:hidden;box-shadow:0 20px 60px rgba(0,0,0,.6),inset 0 1px 0 rgba(255,255,255,.1)}

.a-tabs{display:flex;gap:3px;background:rgba(0,0,0,.4);border:1px solid var(--b1);border-radius:10px;padding:4px;margin-bottom:20px}

.a-tab{flex:1;padding:10px;border:none;background:none;border-radius:7px;font-family:var(--fd);font-size:11px;font-weight:700;color:#94a3b8;cursor:pointer;transition:all .2s;letter-spacing:.05em;text-transform:uppercase}

.a-tab.on{background:rgba(255,255,255,.1);color:#ffffff;border:1px solid rgba(255,255,255,.2);box-shadow:0 2px 8px rgba(0,0,0,.2)}

.fld{margin-bottom:13px}

.fld-label{font-family:var(--fd);font-size:12px;font-weight:600;color:#e2e8f0;text-transform:uppercase;letter-spacing:.1em;margin-bottom:6px;display:block}

.fld-input{width:100%;background:rgba(15,23,42,.6);border:1px solid rgba(255,255,255,.2);border-radius:9px;padding:12px 16px;color:#ffffff;font-family:var(--fd);font-size:14px;outline:none;transition:all .2s;box-shadow:inset 0 2px 4px rgba(0,0,0,.2)}

.fld-input::placeholder{color:#94a3b8}

.fld-input:focus{border-color:var(--c);box-shadow:0 0 3px rgba(59,130,246,.3)}

.a-msg{border-radius:8px;padding:10px 13px;font-family:var(--fm);font-size:11px;margin-bottom:13px;display:none}

.a-msg.err{background:rgba(255,0,110,.07);border:1px solid rgba(255,0,110,.25);color:var(--m2);display:block}

.a-msg.ok{background:rgba(0,255,157,.06);border:1px solid rgba(0,255,157,.22);color:var(--g);display:block}

.cyber-btn{width:100%;padding:14px;background:linear-gradient(135deg,#3b82f6,#2563eb);border:none;border-radius:10px;font-family:var(--fd);font-size:13px;font-weight:700;color:#ffffff;cursor:pointer;transition:all .25s;letter-spacing:.1em;text-transform:uppercase;display:flex;align-items:center;justify-content:center;gap:8px;box-shadow:0 4px 15px rgba(37,99,235,.4)}

.cyber-btn:hover{background:linear-gradient(135deg,#60a5fa,#3b82f6);box-shadow:0 6px 20px rgba(37,99,235,.6);transform:translateY(-2px)}

.cyber-btn:disabled{opacity:.3;cursor:not-allowed;transform:none}

```



```

/* ??? APP SHELL ??? */

#app{display:none;position:relative;z-index:10}

.shell{display:grid;grid-template-columns:240px 1fr;min-height:100vh}

/* ??? SIDEBAR ??? */

.sidebar{background:linear-gradient(180deg, rgba(4,4,20,.98), rgba(6,6,25,.98));border-right:1px
solid
var(--b1);display:flex;flex-direction:column;position:sticky;top:0;height:100vh;overflow-y:auto;tr
ansition:transform .3s,background .3s}

.sb-brand{padding:16px;border-bottom:1px solid var(--b1);display:flex;align-items:center;gap:10px}

.sb-logo{width:32px;height:32px;border:1px solid
var(--b2);border-radius:8px;display:flex;align-items:center;justify-content:center;font-family:var
(--fd);font-size:12px;font-weight:900;color:var(--c);animation:glowPulse 3s ease-in-out
infinite;flex-shrink:0}

.sb-name{font-family:var(--fd);font-size:18px;font-weight:900;color:var(--c);letter-spacing:.15em}

.sb-ver{font-family:var(--fd);font-size:11px;color:var(--tx3);letter-spacing:.2em;margin-top:2px}

.sb-nav{flex:1;padding:10px 8px;overflow-y:auto}

.sb-sec{font-family:var(--fd);font-size:13px;font-weight:700;color:var(--tx3);text-transform:upper
case;letter-spacing:.1em;padding:12px 10px 6px}

.sb-item{display:flex;align-items:center;gap:12px;width:100%;padding:11px
14px;margin-bottom:2px;border:1px solid
transparent;background:none;border-radius:8px;color:var(--tx2);font-family:var(--fb);font-size:15p
x;font-weight:600;cursor:pointer;transition:all .15s;text-align:left}

.sb-item:hover{background:rgba(0,212,255,.04);color:var(--tx2);border-color:var(--b1)}

.sb-item.on{background:rgba(0,212,255,.07);color:var(--c);border-color:var(--b2);box-shadow:0 0
20px rgba(0,212,255,.06)}

.sb-icon{font-size:14px;width:18px;text-align:center;flex-shrink:0}

/* Theme toggle */

.theme-toggle{margin-left:auto;background:none;border:1px solid
var(--b1);width:28px;height:28px;border-radius:7px;display:flex;align-items:center;justify-content
:center;cursor:pointer;font-size:13px;transition:all .2s;color:var(--tx3)}

.theme-toggle:hover{border-color:var(--c);color:var(--c);background:rgba(0,212,255,.06)}

/* Recent builds */

.sb-builds{padding:8px;border-top:1px solid var(--b1)}

.sb-builds-title{font-family:var(--fd);font-size:11px;font-weight:700;color:var(--tx3);text-transf
orm:uppercase;letter-spacing:.1em;padding:8px 10px 6px}

.recent-build{display:flex;align-items:center;gap:8px;padding:8px
10px;border-radius:7px;border:1px solid transparent;cursor:pointer;transition:all
.15s;margin-bottom:3px}

```

```

.recent-build:hover{background:rgba(0,212,255,.04);border-color:var(--b1)}

.recent-build .rb-icon{font-size:14px;width:18px;text-align:center;flex-shrink:0}

.recent-build
.rb-name{font-family:var(--fb);font-size:14px;font-weight:500;color:var(--tx2);white-space:nowrap;
overflow:hidden;text-overflow:ellipsis;flex:1}

.recent-build .rb-time{font-family:var(--fd);font-size:11px;color:var(--tx3);flex-shrink:0}

/* Sidebar footer */

.sb-footer{padding:12px 14px;border-top:1px solid
var(--b1);display:flex;align-items:center;gap:8px}

.u-av{width:36px;height:36px;border-radius:50%;border:1px solid
var(--b2);display:flex;align-items:center;justify-content:center;font-family:var(--fd);font-weight
:900;font-size:14px;color:var(--c);flex-shrink:0}

.u-info{flex:1;min-width:0}

.u-nm{font-family:var(--fb);font-size:14px;font-weight:700;color:var(--tx);white-space:nowrap;over
flow:hidden;text-overflow:ellipsis}

.u-rl{font-family:var(--fd);font-size:11px;color:var(--tx3);margin-top:2px;white-space:nowrap;over
flow:hidden;text-overflow:ellipsis}

.lo-btn{margin-left:auto;background:none;border:1px solid
transparent;color:var(--tx3);cursor:pointer;font-size:14px;padding:5px;border-radius:6px;transitio
n:all .2s}

.lo-btn:hover{color:var(--m2);border-color:rgba(255,0,110,.3);background:rgba(255,0,110,.06)}

/* ??? MOBILE HAMBURGER ??? */

.mobile-toggle{display:none;position:fixed;top:12px;left:12px;z-index:200;width:38px;height:38px;b
order-radius:10px;border:1px solid
var(--b2);background:var(--base);color:var(--c);font-size:18px;cursor:pointer;align-items:center;j
ustify-content:center;transition:all .2s}

.mobile-toggle:hover{background:rgba(0,212,255,.08)}

.sidebar-overlay{display:none;position:fixed;inset:0;z-index:90;background:rgba(0,0,0,.6);backdrop
-filter:blur(4px)}

.sidebar-overlay.show{display:block}

/* ??? VIEWS ??? */

.view{display:none;animation:vIn .25s ease}

.view.on{display:flex;flex-direction:column;height:100vh}

/* ??? CHAT VIEW ??? */

.chat-header{padding:14px 24px;border-bottom:1px solid
var(--b1);display:flex;align-items:center;justify-content:space-between;background:rgba(4,4,15,.9)
;backdrop-filter:blur(20px);transition:background .3s}

.chat-title{font-family:var(--fd);font-size:11px;color:var(--c);letter-spacing:.15em;text-transfor

```

```

m:uppercase;display:flex;align-items:center;gap:8px}

.chat-title::before{content:'';width:6px;height:6px;border-radius:50%;background:var(--g);box-shadow:0 0 8px var(--g);animation:pulse 2s infinite}

.btn-new{padding:6px 14px;background:rgba(0,212,255,.08);border:1px solid var(--b1);border-radius:8px;font-family:var(--fd);font-size:12px;color:var(--c);cursor:pointer;letter-spacing:.1em;text-transform:uppercase;transition:all .2s}

.btn-new:hover{border-color:var(--c);box-shadow:0 0 15px rgba(0,212,255,.15)}

.chat-msgs{flex:1;overflow-y:auto;padding:20px 24px;display:flex;flex-direction:column;gap:16px;scroll-behavior:smooth}

.msg{display:flex;gap:10px;animation:msgIn .3s ease;max-width:85%}

.msg-user{align-self:flex-end;flex-direction:row-reverse}

.msg-via{align-self:flex-start}

.msg-av{width:32px;height:32px;border-radius:50%;display:flex;align-items:center;justify-content:center;font-size:16px;flex-shrink:0;border:1px solid var(--b1)}

.msg-via .msg-av{background:rgba(0,212,255,.08);border-color:rgba(0,212,255,.25)}

.msg-user .msg-av{background:rgba(124,58,237,.08);border-color:rgba(124,58,237,.25)}

.msg-body{border-radius:16px;padding:12px 16px;font-family:var(--fb);font-size:14px;line-height:1.7;letter-spacing:.02em;position:relative;transition:background .3s,border-color .3s}

.msg-via
.msg-body{background:linear-gradient(135deg,rgba(10,10,30,.9),rgba(6,6,20,.95));border:1px solid var(--b1);color:var(--tx);border-bottom-left-radius:4px}

.msg-user
.msg-body{background:linear-gradient(135deg,rgba(124,58,237,.12),rgba(124,58,237,.06));border:1px solid rgba(124,58,237,.2);color:var(--tx);border-bottom-right-radius:4px}

.msg-body pre{background:rgba(0,0,0,.5);border:1px solid var(--b1);border-radius:8px;padding:10px 14px;margin:8px 0;overflow-x:auto;font-family:var(--fm);font-size:12px;line-height:1.6;transition:background .3s}

.msg-body code{font-family:var(--fm);font-size:12px;background:rgba(0,212,255,.06);padding:1px 5px;border-radius:4px}

.msg-body pre code{background:none;padding:0}

.msg-body strong{color:var(--c)}

.msg-body a{color:var(--c);text-decoration:underline}

.msg-body h3{font-family:var(--fd);font-size:12px;color:var(--c);letter-spacing:.08em;margin:10px 0 5px}

.msg-time{font-family:var(--fm);font-size:11px;color:var(--tx3);margin-top:4px}

/* ??? DEPLOY CARD ??? */

.deploy-card{background:linear-gradient(135deg,rgba(0,255,157,.03),rgba(0,212,255,.02));border:1px solid rgba(0,255,157,.15);border-radius:14px;padding:16px 18px;margin:10px 0;position:relative;overflow:hidden;transition:background .3s}

.deploy-card::before{content:'';position:absolute;top:0;left:0;right:0;height:2px;background:linea

```

```

r-gradient(90deg,var(--g),var(--c),var(--v2));opacity:.6}

.deploy-card a{color:var(--c);word-break:break-all;font-family:var(--fm);font-size:12px}

.deploy-card strong{color:var(--g)}

/* ??? AGENT PROGRESS ??? */

.agent-prog{background:rgba(0,0,0,.3);border:1px solid var(--b1);border-radius:10px;padding:10px
14px;margin:6px 0;transition:background .3s}

.agent-prog-item{display:flex;align-items:center;gap:8px;padding:4px
0;font-family:var(--fm);font-size:11px;color:var(--tx2)}

.agent-prog-item .ap-icon{width:16px;text-align:center;font-size:12px}

.agent-prog-item .ap-name{flex:1;text-transform:capitalize}

.agent-prog-item .ap-status{font-size:10px;padding:2px 8px;border-radius:6px}

.agent-prog-item
.ap-status.running{color:var(--y);background:rgba(255,215,0,.08);animation:progPulse 1.5s
infinite}

.agent-prog-item .ap-status.done{color:var(--g);background:rgba(0,255,157,.08)}

.agent-prog-item .ap-status.failed{color:var(--m2);background:rgba(255,0,110,.08)}

/* ??? TYPING INDICATOR ??? */

.typing{display:flex;gap:4px;padding:8px 0}

.typing span{width:7px;height:7px;border-radius:50%;background:var(--c);animation:dotBounce 1.4s
infinite ease-in-out both}

.typing span:nth-child(1){animation-delay:-.32s}

.typing span:nth-child(2){animation-delay:-.16s}

/* ??? CHAT INPUT ??? */

.chat-input-wrap{padding:14px 24px;border-top:1px solid
var(--b1);background:rgba(4,4,15,.95);backdrop-filter:blur(20px);transition:background .3s}

.chat-input-box{display:flex;gap:10px;align-items:flex-end;background:rgba(0,0,0,.4);border:1px
solid var(--b1);border-radius:14px;padding:8px 14px;transition:border-color .2s;background .3s}

.chat-input-box:focus-within{border-color:var(--c);box-shadow:0 0 3px rgba(0,212,255,.06)}

#chatInput{flex:1;background:none;border:none;color:var(--tx);font-family:var(--fb);font-size:14px
;outline:none;resize:none;min-height:24px;max-height:120px;line-height:1.5;transition:color .3s}

#chatInput::placeholder{color:var(--tx3)}

#sendBtn{width:36px;height:36px;border-radius:10px;border:1px solid
var(--b2);background:linear-gradient(135deg,rgba(0,212,255,.15),rgba(124,58,237,.1));color:var(--c
);cursor:pointer;display:flex;align-items:center;justify-content:center;transition:all
.2s;flex-shrink:0}

#sendBtn:hover{box-shadow:0 0 20px rgba(0,212,255,.2);border-color:var(--c);transform:scale(1.05)}

#sendBtn:disabled{opacity:.3;cursor:not-allowed;transform:none}

```

```

#sendBtn svg{width:18px;height:18px}

/* ??? DASHBOARD VIEW ??? */

.dash-view{padding:24px;overflow-y:auto;height:100vh}

.pg-title{font-family:var(--fd);font-size:26px;font-weight:800;color:var(--tx);letter-spacing:.05em;margin-bottom:8px}

.pg-sub{font-family:var(--fd);font-size:13px;font-weight:600;color:var(--tx3);letter-spacing:.1em;text-transform:uppercase;margin-bottom:24px}

.kpi-row{display:grid;grid-template-columns:repeat(4,1fr);gap:12px;margin-bottom:20px}

.kpi{background:linear-gradient(135deg,rgba(10,10,30,.9),rgba(6,6,20,.95));border:1px solid;border-radius:14px;padding:16px 18px;transition:transform .2s,background .3s}

.kpi:hover{transform:translateY(-3px)}

.kpi-c{border-color:rgba(0,212,255,.2)}.kpi-v{border-color:rgba(124,58,237,.25)}.kpi-m{border-color:rgba(255,0,110,.2)}.kpi-g{border-color:rgba(0,255,157,.2)}

.kpi-label{font-family:var(--fm);font-size:12px;color:var(--tx3);text-transform:uppercase;letter-spacing:.12em;margin-bottom:10px}

.kpi-val{font-family:var(--fd);font-size:28px;font-weight:800;line-height:1}

.kpi-c .kpi-val{color:var(--c)}.kpi-v .kpi-val{color:var(--v2)}.kpi-m .kpi-val{color:var(--m2)}.kpi-g .kpi-val{color:var(--g)}

.kpi-hint{font-family:var(--fm);font-size:12px;color:var(--tx3);margin-top:4px}

.panel{background:linear-gradient(135deg,rgba(10,10,28,.96),rgba(6,6,18,.98));border:1px solid var(--b1);border-radius:14px;margin-bottom:16px;overflow:hidden;transition:background .3s,border-color .3s}

.panel-head{padding:12px 18px;border-bottom:1px solid rgba(0,212,255,.07);display:flex;align-items:center;justify-content:space-between}

.panel-title{font-family:var(--fd);font-size:13px;font-weight:700;color:var(--c);text-transform:uppercase;letter-spacing:.12em}

.panel-body{padding:18px}

.panel-body .empty{text-align:center;padding:30px;color:var(--tx3);font-size:13px}

/* ??? MEMORY / CARDS ??? */

.mem-card{background:rgba(0,0,0,.3);border:1px solid var(--b1);border-radius:10px;padding:12px 14px;margin-bottom:8px;transition:background .3s,border-color .3s}

.mem-agent{font-family:var(--fd);font-size:13px;color:var(--c);letter-spacing:.08em;text-transform:uppercase}

.mem-task{font-size:14px;color:var(--tx2);margin-top:4px}

.mem-conf{font-family:var(--fm);font-size:12px;color:var(--g);margin-top:3px}

/* ??? TEMPLATES GRID ??? */

```

```

.tpl-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:12px}

.tpl-card{background:rgba(0,0,0,.3);border:1px solid
var(--b1);border-radius:12px;padding:16px;cursor:pointer;transition:all .2s}

.tpl-card:hover{border-color:var(--c);transform:translateY(-2px);box-shadow:0 8px 24px
rgba(0,212,255,.08)}

.tpl-icon{font-size:28px;margin-bottom:8px}

.tpl-name{font-family:var(--fd);font-size:13px;color:var(--c);letter-spacing:.08em;text-transform:
uppercase;margin-bottom:4px}

.tpl-desc{font-size:14px;color:var(--tx3);line-height:1.5}

/* ??? SCROLLBAR ??? */

::-webkit-scrollbar{width:5px}

::-webkit-scrollbar-track{background:transparent}

::-webkit-scrollbar-thumb{background:rgba(0,212,255,.15);border-radius:3px}

::-webkit-scrollbar-thumb:hover{background:rgba(0,212,255,.3)}

/* ??? PARTICLES ??? */

#particles{position:fixed;inset:0;z-index:0;pointer-events:none}

#particles canvas{width:100%;height:100%}

@keyframes float{0%,100%{transform:translateY(0)
rotate(0deg);opacity:.3}50%{transform:translateY(-20px) rotate(180deg);opacity:.8}}

.particle{position:absolute;width:3px;height:3px;border-radius:50%;background:var(--c);animation:f
loat 6s ease-in-out infinite;opacity:.3}

/* ??? GLASSMORPHISM ??? */

.glass{background:rgba(10,10,30,.6);backdrop-filter:blur(20px);-webkit-backdrop-filter:blur(20px);
border:1px solid var(--b2);border-radius:16px}

body.light .glass{background:rgba(255,255,255,.7)}

/* ??? MEETING ROOM ??? */

.meeting-msg{display:flex;gap:10px;padding:12px
16px;border-radius:12px;margin-bottom:8px;border:1px solid
var(--b1);background:rgba(0,0,0,.2);animation:msgIn .3s ease;transition:background .3s}

.meeting-msg:hover{background:rgba(0,212,255,.03)}

.meeting-av{width:36px;height:36px;border-radius:50%;display:flex;align-items:center;justify-conte
nt:center;font-size:18px;flex-shrink:0;border:2px solid var(--b2)}

.meeting-name{font-family:var(--fd);font-size:13px;letter-spacing:.08em;text-transform:uppercase;m
argin-bottom:3px}

.meeting-text{font-size:14px;color:var(--tx2);line-height:1.6}

```

```

.meeting-controls{display:flex;gap:8px;margin-bottom:16px;flex-wrap:wrap}

.meeting-controls select,.meeting-controls input{background:rgba(0,0,0,.4);border:1px solid
var(--b1);border-radius:8px;padding:10px
14px;color:var(--tx);font-family:var(--fm);font-size:14px}

body.light .meeting-msg{background:var(--lift);border-color:var(--b2)}

body.light .meeting-controls select,body.light .meeting-controls
input{background:var(--lift);border-color:var(--b2)}

/* ??? FILE VIEWER ??? */

.file-tree{font-family:var(--fm);font-size:12px}

.file-item{display:flex;align-items:center;gap:6px;padding:5px
10px;border-radius:6px;cursor:pointer;transition:all .15s;color:var(--tx2)}

.file-item:hover{background:rgba(0,212,255,.06);color:var(--c)}

.file-item.active{background:rgba(0,212,255,.1);color:var(--c);border-left:2px solid var(--c)}

.file-item .fi-icon{width:16px;text-align:center;font-size:13px;flex-shrink:0}

.file-item .fi-name{flex:1;overflow:hidden;text-overflow:ellipsis;white-space:nowrap}

.file-item .fi-size{font-size:10px;color:var(--tx3);flex-shrink:0}

.code-viewer{background:rgba(0,0,0,.5);border:1px solid
var(--b1);border-radius:12px;padding:16px;font-family:var(--fm);font-size:12px;line-height:1.7;col
or:var(--tx);overflow-x:auto;max-height:60vh;overflow-y:auto;white-space:pre-wrap;word-break:break
-word;transition:background .3s}

body.light .code-viewer{background:var(--lift);border-color:var(--b2)}

body.light .file-item:hover{background:rgba(0,120,212,.06)}

/* ??? ENHANCED AGENT PROGRESS ??? */

.agent-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(200px,1fr));gap:10px;margin
:12px 0}

.agent-card{background:rgba(0,0,0,.3);border:1px solid
var(--b1);border-radius:12px;padding:12px;transition:all .3s}

.agent-card.running{border-color:var(--y);box-shadow:0 0 20px
rgba(255,215,0,.1);animation:progPulse 1.5s infinite}

.agent-card.done{border-color:var(--g);box-shadow:0 0 15px rgba(0,255,157,.08)}

.agent-card.failed{border-color:var(--m2);box-shadow:0 0 15px rgba(255,0,110,.08)}

.agent-card-head{display:flex;align-items:center;gap:8px;margin-bottom:6px}

.agent-card-icon{font-size:18px}

.agent-card-name{font-family:var(--fd);font-size:9px;color:var(--c);letter-spacing:.1em;text-trans
form:uppercase;flex:1}

.agent-card-status{font-size:10px;padding:2px 8px;border-radius:6px;font-family:var(--fm)}

.agent-card-time{font-family:var(--fm);font-size:10px;color:var(--tx3);margin-top:4px}

body.light .agent-card{background:var(--lift);border-color:var(--b2)}

```

```

/* ??? STATS BAR ??? */

.stat-bar{height:6px;border-radius:3px;background:rgba(0,212,255,.1);overflow:hidden;margin-top:6px}

.stat-bar-fill{height:100%;border-radius:3px;transition:width .6sease;background:linear-gradient(90deg,var(--c),var(--v2))}

/* ??? SYSTEM INFO ??? */

.sys-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(180px,1fr));gap:10px}

.sys-item{background:rgba(0,0,0,.2);border:1px solid var(--b1);border-radius:10px;padding:12px;text-align:center;transition:all .2s}

.sys-item:hover{border-color:var(--b2);transform:translateY(-2px)}

.sys-label{font-family:var(--fm);font-size:8px;color:var(--tx3);text-transform:uppercase;letter-spacing:.15em;margin-bottom:6px}

.sys-val{font-family:var(--fd);font-size:20px;font-weight:700;color:var(--c)}

/* ??? TOAST NOTIFICATION ??? */

.toast{position:fixed;bottom:24px;right:24px;z-index:999;background:var(--surf);border:1px solid var(--b2);border-radius:12px;padding:12px 20px;font-family:var(--fb);font-size:13px;color:var(--tx);box-shadow:0 8px 32px rgba(0,0,0,.4);animation:msgIn .3sease;max-width:360px}

.toast.success{border-color:rgba(0,255,157,.3);background:linear-gradient(135deg,rgba(0,255,157,.08),rgba(10,10,30,.95))}

.toast.error{border-color:rgba(255,0,110,.3);background:linear-gradient(135deg,rgba(255,0,110,.08),rgba(10,10,30,.95))}

/* ??? RESPONSIVE ??? */

@media(max-width:768px){

  .shell{grid-template-columns:1fr}

  .sidebar{position:fixed;left:0;top:0;bottom:0;width:260px;z-index:100;transform:translateX(-100%)}

  .sidebar.open{transform:translateX(0)}

  .mobile-toggle{display:flex}

  .kpi-row{grid-template-columns:repeat(2,1fr)}

  .chat-header{padding-left:56px}

  .msg{max-width:95%}

  .tpl-grid{grid-template-columns:1fr}

  .agent-grid{grid-template-columns:1fr}

  .sys-grid{grid-template-columns:repeat(2,1fr)}

```